

Григорис Антониоу, Паул Грос,
Фрэнк ван Хармелен, Ринке Хоекстра

Семантический веб.

Учебник

ТРЕТЬЕ ИЗДАНИЕ

Grigoris Antoniou, Paul Groth,
Frank van Harmelen, Rinke Hoekstra

A Semantic Web Primer

THIRD EDITION

The MIT Press
Cambridge, Massachusetts
London, England

Григорис Антониоу, Паул Грос,
Фрэнк ван Хармелен, Ринке Хоекстра

Семантический веб.

Учебник

ТРЕТЬЕ ИЗДАНИЕ



Москва, 2016

УДК 004.738.5:004.42
ББК 32.973.4
A72

А72 **Антониоу Г., Грос П., Хармелен ван Ф., Хоекстра Р.**
Семантический веб. Учебник. 3-е изд. / пер. с англ. Т. Шульга. –
М.: ДМК Пресс, 2016. – 240 с.: ил.
ISBN 978-5-94074-333-8

УДК 004.738.5:004.42
ББК 32.973.4

Все права защищены. Любая часть этой книги не может быть воспроизведена в какой бы то ни было форме и какими бы то ни было средствами без письменного разрешения владельцев авторских прав.

Материал, изложенный в данной книге, многократно проверен. Но поскольку вероятность технических ошибок все равно существует, издательство не может гарантировать абсолютную точность и правильность приводимых сведений. В связи с этим издательство не несет ответственности за возможные ошибки, связанные с использованием книги.

All rights reserved. No part of this book may be reproduced or transmitted in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without prior written permission the publisher. For information on getting permission for reprints and excerpts, contact permission@peachpit.com. RUS-
SIAN language edition published by ДМК PUBLISHERS, Copyright © 2015.

ISBN 978-0-262-01828-9 (анг.)

© 2012 Massachusetts Institute
of Technology

ISBN 978-5-94074-333-8 (рус.)

© Оформление, ДМК Пресс, 2016

Содержание

Предисловие к серии	9
Глава 1. Концепция семантического веба	13
1.1. Введение	13
1.2. Технологии семантического веба	18
1.3. Многоуровневый подход	26
1.4. Обзор книги	29
1.5. Выводы.....	30
Рекомендуемая литература	30
Глава 2. Описание веб-ресурсов: RDF	32
2.1. Введение	32
2.2. RDF: модель данных.....	34
2.3. Синтаксисы RDF	39
2.4. RDFS: добавление семантики	47
2.5. RDF Schema: язык.....	51
2.6. Формальные определения языков RDF и RDF Schema.....	56
2.7. Аксиоматическая семантика языков RDF и RDF Schema.....	59
2.8. Системы прямого вывода для языков RDF и RDFS	64
2.9. Выводы.....	65
Рекомендуемая литература	66
Упражнения и проекты	67
Глава 3. Запросы в семантическом вебе	70
3.1. SPARQL-инфраструктура	71
3.2. Основы SPARQL: сравнение с шаблоном.....	71
3.3. Фильтры	75
3.4. Конструкции, учитывающие предположение об открытости мира	78
3.5. Представление результатов запроса	80
3.6. Другие формы SPARQL-запросов.....	81
3.7. Запросы к схемам данных	82
3.8. Добавление информации с помощью протокола SPARQL Update.....	83
3.9. Принцип «Следуй за своим носом»	85
3.10. Выводы	86
Рекомендуемая литература	86
Упражнения и проекты	86

Глава 4. Язык веб-онтологий: OWL2	88
4.1. Введение	88
4.2. Требования к языкам онтологий	89
4.3. Совместимость OWL2 с RDF/RDFS	93
4.4. Язык OWL	96
4.5. Профили языка OWL2	116
4.6. Выводы	118
Рекомендуемая литература	119
Упражнения и проекты	120
 Глава 5. Логика и логический вывод: правила	122
5.1. Введение	122
5.2. Пример монотонных правил: родственные связи	127
5.3. Монотонные правила: синтаксис	128
5.4. Монотонные правила: семантика	131
5.5. OWL2 RL: дескрипционные логики и правила	134
5.6. Формат обмена правилами: RIF	137
5.7. Язык правил сематического веба SWRL	142
5.8. Правила в языке SPARQL: SPIN	143
5.9. Немонотонные правила: назначение и синтаксис	145
5.10. Пример немонотонных правил: посреднические сделки	147
5.11. Язык разметки правил RuleML	151
5.12. Выводы	153
Рекомендуемая литература	153
Упражнения и проекты	155
 Глава 6. Приложения	158
6.1. GoodRelations	158
6.2. BBC Artists	162
6.3. Сайт BBC World Cup 2010	164
6.4. Правительственные данные	168
6.5. New York Times	170
6.7. OpenCalais	172
6.8. Schema.org	173
6.9. Выводы	174
 Глава 7. Онтологический инжиниринг	175
7.1. Введение	175
7.2. Ручная разработка онтологий	176
7.3. Повторное использование существующих онтологий	180
7.4. Полуавтоматическое построение онтологий	183

7.5. Отображение онтологий	187
7.6. Использование реляционных баз данных	189
7.7. Архитектура приложений семантического веба	191
Рекомендуемая литература	195
Упражнения и проекты	196
Глава 8. Заключение	201
8.1. Принципы	201
8.2. Что дальше?.....	203
Приложение А. Основы XML	205
А.1. Язык XML.....	205
А.2. Структурирование информации.....	209
А.3. Пространства имен.....	222
А.4. Адресация и запросы к XML-документам.....	223
А.5. Обработка.....	229
Предметный указатель	236

Посвящается Константине, Вангелису, Гиоргосу
– Григорис Антониоу

Посвящается профессору Томасу Гроту
– Паул Грос

Предисловие к серии

Традиционный взгляд на информационные системы как на дорогостоящие, сделанные на заказ программные приложения, основанные на базах данных, быстро меняется. Это изменение обусловлено, с одной стороны, развитием индустрии программного обеспечения, в которой все чаще используются готовые типовые компоненты и стандартные программные решения, а с другой – натиском информационной революции. В свою очередь, эти изменения привели к формированию новых требований к информационным услугам: однородность с точки зрения представления и взаимодействия их моделей, открытость с точки зрения архитектуры программного обеспечения и глобальность с точки зрения масштаба использования. Эти требования пришли в основном из прикладных областей, таких как электронная коммерция, банковское дело, производство (в том числе собственно индустрия программного обеспечения), профессиональное обучение, образование и управление природоохранной деятельностью; и здесь перечислены лишь немногие из этих областей.

Информационные системы будущего должны поддерживать бесперебойное взаимодействие с большим количеством независимых друг от друга источников данных различных производителей и унаследованных приложений, работающих на разнородных платформах в распределенных информационных сетях. При описании содержания таких источников данных и при их интеграции решающую роль будут играть метаданные.

Кроме того, информационные системы следующего поколения должны поддерживать больше разнообразных социально ориентированных моделей взаимодействия. Эти модели могут включать модели навигации по данным, запросов к данным, извлечения данных и должны работать в сочетании с механизмами персонализированных уведомлений, аннотирования и профилирования. Кроме того, такие модели взаимодействия должны эффективно поддерживаться прикладными программными продуктами и быть динамически интегрированы в единые совместно используемые среды, ориентированные на пользователей. Кроме того, нужны крупные инвестиции как правительств, так и бизнеса в информационные ресурсы, подразумеваются конкретные меры по обеспечению безопасности, соблюдению принципа неприкосновенности частной жизни и правильности контента этих ресурсов.

Все перечисленные задачи должны быть решены в информационных системах следующего поколения. Мы называем такие системы *совместными информационными системами* (англ. *cooperative information systems*), и именно они находятся в центре внимания этой серии.

Говоря простым языком, совместные информационные системы удовлетворяют разнообразным сочетаниям требований, определяемых *контентом – обществом – коммерцией*. Эти требования порождаются текущими тенденциями в области готовых программных решений, таких как ERP-системы и системы электронной коммерции.

Основной задачей при создании совместных информационных систем является разработка технологий, что предполагает непрерывное совершенствование и увеличение массовых вложений сил и средств в информационные ресурсы и системы. Такие технологии должны обеспечивать соответствующую инфраструктуру, которая поддерживает не только разработку, но и развитие программного обеспечения.

Результаты ранних исследований в области совместных информационных систем становятся основой технологий, используемых для разработки социально ориентированных информационных порталов или шлюзов. Информационный портал обеспечивает реализацию принципа «одного окна» для доступа к разнообразным информационным ресурсам и услугам, тем самым создавая сообщество постоянных пользователей.

Научно-исследовательские результаты, которые послужат базой для реализации совместных информационных систем, не могут быть получены в рамках только одного научного направления в области информационных технологий. Представляется перспективным использование таких достаточно зрелых технологий, как базы данных, системы, основанные на знаниях, распределенные системы, программное обеспечение поддержки коллективной работы и графические пользовательские интерфейсы. Несмотря на то что отдельные технологии еще требуют усовершенствования, предполагается, что основным фактором успеха в этой области будет эволюция существующих технологий в единую технологию для разработки и управления совместными информационными системами.

Серия MIT Press «Совместные информационные системы» включает в себя учебники и научные издания, посвященные данной области и предназначенные для исследователей и специалистов, которые хотят быть в курсе текущих событий и будущих тенденций.

Серия будет включать в себя три типа книг:

- учебники и справочные пособия, предназначенные для студентов старших курсов и аспирантов;
- научно-исследовательские монографии, в которых собраны и обобщены результаты исследований и опыт разработок последних лет;
- сборники научных трудов, включающие статьи на конкретные темы.

Мы ждем предложений от авторов. Для этого надо предоставить в редакцию оглавление предлагаемой книги и образец глав книги. Все предоставленные предложения будут формально рассмотрены, и авторы будут уведомлены о результатах этого рассмотрения.

Данные в источнике данных являются полезными, так как они моделируют некоторую часть реального мира, объект изучения (или приложение, или область интерпретации). Проблема *семантики данных* заключается в установлении и поддержании соответствия между источником данных, который в дальнейшем будем называть моделью, и предполагаемым объектом моделирования. Моделью могут являться база данных о сотрудниках в компании, схема базы данных, описывающей детали, проекты и поставщиков, веб-сайт, представляющий информацию об университете, или текстовый файл, описывающий битву при Ватерлоо. Проблема семантики данных появилась с момента разработки первых баз данных. Однако эта проблема может оставаться под контролем, пока базы данных функционируют в закрытой и относительно стабильной среде. В этих условиях смысл данных определяется правильно определенной структурой базы данных, и предполагается, что он будет интерпретироваться небольшим количеством постоянных пользователей и прикладных программ.

Появление веба все изменило. Базы данных сегодня стали в некоторой форме доступными в вебе, который является открытой средой: пользователи, прикладные программы, способы использования данных постоянно меняются. В этих условиях семантика данных должна быть доступна непосредственно вместе с самими данными. Доступность семантики для пользователей можно обеспечить за счет выбора соответствующего формата представления информации. Однако для прикладных программ семантика должна быть представлена в формальной, машинно обрабатываемой форме. Именно этот факт обусловил появление инициативы под названием «семантический веб»¹.

¹ Tim Berners-Lee and Mark Fischetti, *Weaving the Web: The Original Design and Ultimate Destiny of the World Wide Web by Its Inventor*. San Francisco: HarperCollins, 1999.

Неудивительно, что это инициатива Тима Бернерса-Ли привлекла огромное внимание как исследователей, так и практиков. В настоящее время регулярно проводится международная конференция, посвященная семантическому вебу «International Semantic Web Conference»¹, издательством «Elsevier» выпускается журнал «Journal of Web Semantics»², действуют отраслевые комитеты, которые рассматривают стандарты первого поколения для семантического веба³.

Данная книга представляет собой своевременную публикацию, учитывая стремительный характер развития концепций, технологий и стандартов семантического веба. Книга предлагает постепенное введение в концепцию и технологии семантического веба, в том числе XML, DTD, XML-схемы, RDF, RDFS, OWL, логику и логический вывод. Все материалы книги сопровождаются примерами и приложениями, иллюстрирующими способы использования этих технологий.

Мы рады включить эту книгу, посвященную семантическому вебу, в серию «Совместные информационные системы». Надеемся, что для читателей она будет интересной, познавательной и полезной.

Джон Милопулос
(John Mylopoulos)
jm@cs.toronto.edu
Кафедра компьютерных наук
Университет Торонто
Торонто, Онтарио
Канада

Майкл Папазоглоу
(Michael Papazoglou)
M. P. Papazoglou@kub.nl
INFOLAB
P. O. Box 90153
Университет Тилбурга
Нидерланды

¹ <http://swsa.semanticweb.org/content/international-semantic-web-conference-iswc>.

² www.semanticwebjournal.org.

³ На момент перевода книги уже утвержден целый ряд стандартов семантического веба второго поколения. См. <http://www.w3.org/standards/semantic-web/>. – *Прим. перев.*

Концепция семантического веба

1.1. Введение

1.1.1. Цели семантического веба

Общая концепция «семантического веба» может быть сведена к одной фразе: «сделать веб более доступным для компьютеров». На данный момент веб – это веб текста и изображений. Такие мультимедийные материалы полезны для людей, но компьютеры играют очень ограниченную роль в вебе: они индексируют сайты по ключевым словам, передают информацию от серверов к клиентам, но этим и ограничиваются их функции. Вся интеллектуальная работа (выбор информации, ее объединение, агрегирование и т. д.) выполняется человеком. А что, если сделать веб более информативным для машин, наполнить его машиночитаемыми, то есть «понятными» для машин, данными? Такой веб позволит реализовать функции, осуществление которых невозможно на данный момент. *Поиск* информации не будет больше ограничиваться просто поиском по ключевым словам, он может стать более осмысленным, учитывать синонимы, исключать омонимы, а также учитывать контекст и цель поисковых запросов. Сайты могут стать более *персонализированными*, если браузеры смогут понимать содержимое веб-страницы и адаптировать его к личным интересам пользователя в соответствии с его профилем. *Связывание* страниц может стать более осмысленным, если реализовать динамическое создание полезных ссылок на основе действий конкретного пользователя, вместо того заранее создать одинаковые ссылки для всех пользователей. Станет возможна *интеграция* информации с разных веб-сайтов, в отличие от настоящего времени, когда пользователю для объединения найденной на сайтах информации приходится использовать «интеллектуальную» технологию копипаста.

1.1.2. Проектные решения семантического веба

Существуют различные подходы к построению более «семантического» веба. Один из них – разработка проекта «Giga Google», основанного на «чрезмерно высокой эффективности данных»¹ для поиска правильных взаимосвязей между словами, между терминами и контекстом и т. д. Отсутствие прогресса в производительности поисковых машин, свидетелями чего мы являемся в течение нескольких последних лет, позволяет предположить, что такой подход имеет ограничения: ни один из поисковых гигантов не смог предложить решение, позволяющее в результате поиска возвращать нечто большее, чем простой список несвязанных страниц.

Семантический веб, также известный в последние годы как веб данных (англ. *Web of Data*)², основан на различных принципах проектирования, которые можно обобщить следующим образом:

- 1) обеспечение доступности в вебе структурированных и полуструктурированных данных, представленных в стандартных форматах;
- 2) обеспечение доступности в вебе не только наборов данных, но и отдельных элементов данных и отношений между ними;
- 3) описание предполагаемой семантики таких данных с помощью формализма, обеспечивающего возможность машинной обработки этой семантики.

Решение использовать структурированные и полуструктурированные данные основано на результатах исследований, а именно на том факте, что на сегодняшний день основой неструктурированного «веба текста и изображений» является на самом деле очень большое количество структурированных и полуструктурированных документов.

В основном содержание современных веб-страниц генерируется на основе баз данных и систем управления контентом, содержащих хорошо структурированные наборы данных. Однако структурные связи, существующие в таких наборах данных, полностью теряются при публикации их в виде человекочитаемых веб-страниц на языке разметки гипертекста HTML (Hypertext Markup Language) (см.

¹ The Unreasonable Effectiveness of Data Alon Halevy, Peter Norvig, and Fernando Pereira, IEEE Intelligent Systems, March/April 2009, pgs 8-12, http://static.googleusercontent.com/external_content/untrusted_dlcp/research.google.com/en/pubs/archive/35179.pdf.

² http://www.readwriteweb.com/archives/web_of_data_machine_accessible_information.php.

рис. 1.1). Для того чтобы сделать веб более «семантическим», необходимо реализовать следующую ключевую идею: публиковать и связывать друг с другом структурированные наборы данных (вместо того чтобы публиковать и связывать между собой HTML-страницы, после того как потеряна большая часть структурных связей данных).

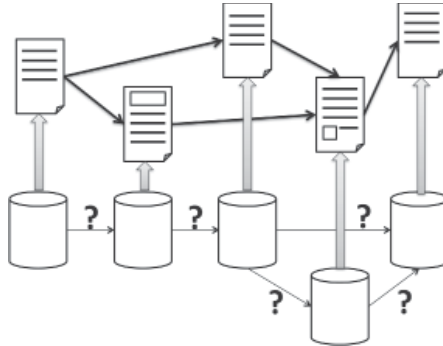


Рис. 1.1 ❖ Структурированные и неструктурированные данные в вебе

1.1.3. Основные технологии семантического веба

Три вышеупомянутых принципа проектирования реализованы с помощью конкретных технологий, описанию которых и посвящена большая часть данной книги:

1. В качестве модели данных для описания объектов и отношений между ними используются *помеченные графы*. Объекты представляются как вершины графа, а отношения между ними – как дуги. В качестве формализма для представления таких графов используется язык с не очень удачным названием – фреймворк описания ресурсов «Resource Description Framework» RDF¹.
2. Для идентификации отдельных элементов данных и отношений между ними, которые включаются в наборы данных, используются *веб-идентификаторы* URI (Uniform Resource Identifier)².

¹ Возможно, более удачным названием для языка было бы название «Rich Data Format» (многофункциональный формат данных).

² В последних версиях стандартов семантического веба для идентификации ресурсов используется IRI, а не URI. IRI (Internationalized Resource Identifier) – интернационализированный идентификатор ресурса, представляет собой Unicode-строку и соответствует синтаксису, определённом в стандарте URI. – *Прим. перев.*

Использование веб-идентификаторов отражено в стандарте языка RDF.

3. В качестве модели данных, позволяющей формально представить предполагаемую семантику этих данных, используются *онтологии* (если говорить кратко: иерархически организованные словари терминов – типов объектов и отношений между ними – свойств). Данная модель представляется с помощью таких формализмов, как язык RDF Schema и язык веб-онтологий OWL (Web Ontology Language), в которых URI используются для идентификации терминов и их свойств.

1.1.4. От данных к знаниям

Важно понимать, что такие формализмы, как RDF Schema и OWL, являются не только языками описания данных, но и фактически простыми языками *представления знаний*, поскольку они обеспечивают представление предполагаемой семантики. Эти языки являются «логикой», которая позволяет на основе явно заданной информации получать (выводить) дополнительную информацию. Язык RDF Schema представляет собой логику с очень низкой выразительностью, которая позволяет получать только очень простые выводы, такие как наследование свойств в соответствии с иерархией терминов и ограничениями на область значения и область определения свойств. Язык OWL представляет собой более выразительную (но все еще относительно простую) логику, которая дает возможность осуществлять более сложные выводы, например о равенстве и неравенстве, определенных ограничениях, о существовании объектов и т. д. Возможность осуществлять подобные выводы в языках RDF Schema и OWL позволяет разработчикам сайтов публиковать минимально возможное количество фактов, которые читатели должны знать. Кроме того, язык OWL дает возможность описать публикуемую информацию таким образом, чтобы читатель не мог предполагать (додумывать) определенные факты об опубликованных данных (имеется в виду – не нарушая согласованности опубликованной онтологии).

Таким образом, возможность реализации логического вывода для этих языков позволяет задавать как нижнюю, так и верхнюю границу предполагаемой семантики публикуемых данных. По мере уточнения онтологии эти нижние и верхние границы могут сближаться, тем самым все более точно обозначая предполагаемую семантику. Близость такого сближения определяется только точностью онтологии, требуемой для конкретного варианта ее использования.

1.1.5. Веб-архитектура сематического веба

Одна из основных характеристик традиционного веба заключается в том, что веб-контент является распределенным как в смысле расположения, так и в смысле владения им: веб-страницы, которые ссылаются друг на друга, часто размещены на разных веб-серверах, и эти серверы находятся в разных местах и принадлежат разным лицам. Развитие веба основано на принципе «кто угодно может сказать что угодно о чем угодно»¹, или точнее: любой может сослаться на любую веб-страницу без какого-либо разрешения владельца этой страницы и не заботясь о правильности ее адреса. Аналогичный механизм работает и в сематическом вебе (см. рис. 1.2): одна сторона может опубликовать набор данных в вебе (левая часть диаграммы), вторая сторона – независимо опубликовать словарь терминов (правая часть диаграммы), а третья сторона может проаннотировать данные первой стороны с помощью терминов из словаря второй стороны, не спрашивая разрешения ни у одной из сторон. Более того, первые две стороны могут даже не знать об этом. Именно подобное ослабление связей является сутью паутинообразной природы сематического веба.



Рис. 1.2 ❖ Веб-архитектура связанных данных

1.1.6. Из настоящего в будущее

Для того чтобы реализовать вышеизложенную концепцию семантического веба и соответствующие архитектурные решения, требуется осуществление следующих существенных шагов:

- 1) необходимо договориться о стандартном синтаксисе для представления данных и метаданных;
- 2) нужно иметь соглашение о словарях метаданных, для того чтобы совместно использовать предполагаемую семантику данных;

¹ <http://www.w3.org/DesignIssues/RDFnot.html>.

- 3) следует опубликовать большие объемы данных в форматах, предложенных на шаге 1, с использованием словарей, разработанных на шаге 2.

За последние десять лет (самые ранние проекты семантического веба датируются последними годами XX века) в реализации всех трех шагов был достигнут существенный прогресс: языки RDF, RDF Schema и OWL (и их вариации, такие как RDFa, OWL2 и т. д.) получили официальную поддержку консорциума World Wide Web Consortium (W3C) и приобрели статус стандартов де-факто. В этих форматах опубликованы тысячи словарей¹, и уже начинается сближение между ними за счет развития автоматизированных технологий отображения онтологий и под давлением социальных и экономических требований (например, разработки словаря schema.org)². Постоблака связанных данных LOD (Linked Open Data)³ привел к тому, что миллиарды объектов и отношений между ними становятся доступными в вебе на основе общего синтаксиса и словарей.

1.1.7. Где мы сейчас

По сравнению с ситуацией на момент публикации первого издания этой книги в 2003 году, многие из основных элементов семантического веба уже реализованы. Разработаны быстро развивающиеся технологии для поддержки всех этапов развертывания приложений семантического веба, стремительно растет количество значительных проектов семантического веба как для коммерческих, так и для общественных организаций. Однако основные проблемы пока остаются неразрешенными: всевозрастающие объемы данных, преодоление барьеров, возникающих при внедрении, и, конечно, борьба с вездесущим врагом информационных систем: семантической гетерогенностью (неоднородностью).

1.2. Технологии семантического веба

1.2.1. Явное описание метаданных

В настоящее время веб-контент форматируется в расчете на то, что он будет анализироваться человеком, а не программами. Большинство веб-станций разработано на языке HTML (непосредственно или

¹ <http://swoogle.umbc.edu/>.

² <http://schema.org>.

³ <http://linkeddata.org>.

с помощью соответствующих инструментов). Часть типичной веб-страницы, содержащей информацию о физиотерапевтическом центре, может быть представлена следующим образом:

```
<h1> Физиотерапевтический центр "Гибкость" </ h1>
```

Добро пожаловать на главную страницу физиотерапевтического центра "Гибкость". Вы чувствуете боль? У Вас были травмы? Наши сотрудники Лиза Дэвенпорт, Келли Таунсенд (наш очаровательный секретарь) и Стив Мэтьюз позаботятся о Вашем теле и душе.

```
<h2> Приемные часы </ h2>
```

```
Пн. 11.00 - 19.00 <br>
```

```
Вт. 11.00 - 19.00 <br>
```

```
Ср. 15.00 - 19.00 <br>
```

```
Чт. 11.00 - 19.00 <br>
```

```
Пт. 11.00 - 13.00 <p>
```

Но обратите внимание, что прием не ведется во время проведения ежегодного соревнования

```
<a href = ...> State of Origin </a>.
```

Для людей эта информация представлена в удобочитаемом виде, но при ее анализе машинами могут возникнуть проблемы. Поиск на основе ключевых слов идентифицирует слова «физиотерапевтический» и «Приемные часы». Интеллектуальный агент может даже идентифицировать персонал центра. Но при этом могут возникнуть проблемы с тем, чтобы отличить врачей от секретаря, и еще большие проблемы – с определением точного времени приема (для решения которых необходимо перейти по соответствующей ссылке, чтобы найти информацию о днях проведения соревнования State of Origin).

Подход к решению этих проблем в рамках концепции сематического веба не подразумевает разработку суперинтеллектуальных агентов. Вместо этого предлагается решать данные проблемы со стороны веб-страниц. Веб-страницы смогут представлять структуру и смысл их контента, если заменить язык HTML на языки, более подходящие для этих целей. В этом случае, кроме информации о форматировании, предназначенной для отображения страниц, пригодных для чтения людьми, страницы могли бы содержать информацию об их контенте.

Первым шагом в этом направлении является создание расширяемого языка разметки XML (eXtensible Markup Language), который позволяет определять структуру информации веб-страниц. В нашем примере информация на языке XML может быть представлена следующим образом:

```
<company>
```

```
<treatmentOffered>Физиотерапия</treatmentOffered>
```

```
<companyName> Физиотерапевтический центр "Гибкость" </companyName>
<staff>
  <therapist> Лиза Дэвенпорт </therapist>
  <therapist>Стив Мэтьюз</therapist>
  <secretary>Келли Таунсенд</secretary>
</staff>
</company>
```

Это представление более пригодно для машинной обработки. В частности, это может быть использовано для обмена информацией в вебе, который является одной из самых известных областей применения технологии XML.

Однако язык XML по-прежнему представляет уровень синтаксиса, так как он позволяет описать структуру информации, а не ее смысл. Основным языком семантического веба является язык RDF, который позволяет представить информацию в виде утверждений. Для нашего примера подобные утверждения могут быть записаны следующим образом:

Компания А предлагает физиотерапевтическое лечение.
Название компании А "Физиотерапевтический центр Гибкость".
Лиза Давенпорт врач.
Лиза Дэвенпорт работает в компании А.

...

Для человека, просматривающего веб-страницу, разница между представлением информации на языке XML и списком RDF-утверждений может показаться минимальной, но эти два формата отличаются по своей природе: XML позволяет описать структуру данных, а RDF позволяет формулировать утверждения от этих данных¹.

Термин «*метаданные*» означает «данные о данных». Метаданные позволяют частично представить значение, смысл данных, а следовательно, и слово «*семантический*» в словосочетании «*семантический веб*».

1.2.2. Онтологии

Термин «онтология» происходит из философии. В этом контексте он означает название одной из областей философского знания, а именно «учение о бытии» или «учение о сущем» (буквальный перевод греческого слова *Οντολογία*). Учение о бытии – это раздел метафизики, изучающий наиболее общие категории сущего и способы их описа-

¹ Человек, читающий XML-страницу, интерпретирует значение ее элементов на основе имен тегов; очевидно, что машина не может этого сделать.

ния. Приведем пример типичного онтологического суждения: мир состоит из определенных объектов, которые могут быть сгруппированы в абстрактные классы на основании их общих свойств.

Однако в последние годы термин «онтология», как и многие другие слова из различных областей, стал использоваться в информатике и приобрел техническое значение, которое сильно отличается от первоначального. Для наших целей будем использовать определение Т. Р. Грубера (T. R. Gruber), позже уточненное Р. Штудером (R. Studer): онтология – это явная и формальная спецификация концептуализации.

Онтология формально описывает некоторую предметную область. Обычно онтология состоит из конечного списка терминов и отношений между этими терминами. Термины обозначают важные понятия – *концепты* (классы объектов) рассматриваемой предметной области. Например, при описании университета в качестве концептов онтологии могут выступать сотрудники, студенты, курсы, лекционные аудитории, дисциплины.

К отношениям онтологии относят, как правило, иерархическое отношение классов. Иерархическое отношение «класс–подкласс» определяется следующим образом: класс C является подклассом класса C' , если каждый объект класса C также является объектом класса C' . Например, все преподаватели являются сотрудниками. На рис. 1.3 изображена иерархия концептов для онтологии, описывающей деятельность университета.

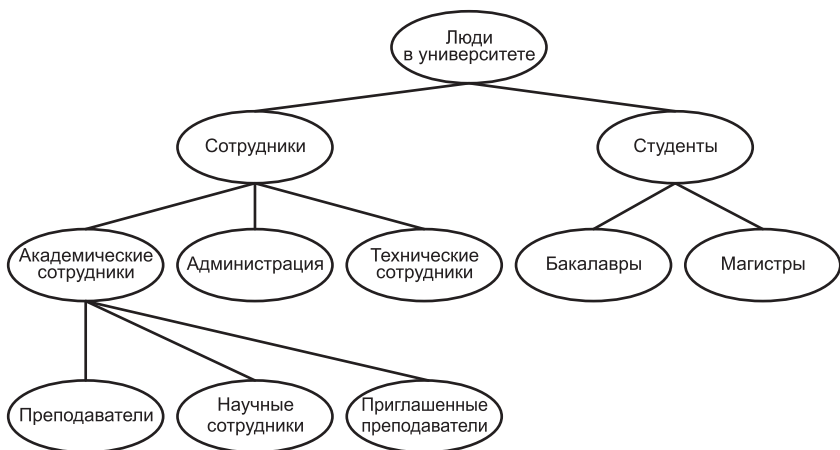


Рис. 1.3 ❖ Иерархия

Помимо отношений «класс–подкласс», онтология может включать в себя следующие типы информации:

- свойства¹ (например, свойство «преподает»: X преподает Y);
- ограничения на значения (например, только преподаватели могут преподавать курсы);
- взаимоисключающие утверждения (например, сотрудник университета не может быть одновременно и преподавателем, и техническим работником);
- характеристики логических связей между объектами (например, на каждой кафедре работает, по крайней мере, десять преподавателей).

В вебе онтологии используются для обеспечения *общего понимания предметной области*. Такое общее понимание необходимо для решения проблемы терминологического разнообразия. Например, в одном приложении для обозначения почтового индекса используется термин «Почтовый индекс», а в другом – просто «Индекс». Еще одна проблема заключается в том, что два приложения могут использовать один тот же термин, но с разными значениями. Например, в университете А под термином «курс» может пониматься программа подготовки бакалавров в целом (курс «Компьютерные науки»), а в университете В – конкретная дисциплина (курс «Программирование на языке C#»). Такие различия могут быть преодолены путем сопоставления терминологии конкретного приложения и терминологии, используемой в некоторой общей онтологии рассматриваемой предметной области, либо путем определения прямого отображения между конкретными онтологиями. В любом случае, онтологии обеспечивают семантическую совместимость.

Онтологии можно использовать для организации навигации по веб-сайтам. На многих современных веб-сайтах в левой части страниц представлены термины верхнего уровня иерархии понятий. Пользователь может «кликнуть» на любой из них и выбрать нужную подкатегорию.

Онтологии также применяются для повышения точности поиска в вебе. Поисковые системы, вместо того чтобы выдавать пользователю все страницы, в которых встречаются заданные (как правило, неоднозначные) ключевые слова, могут искать только те страницы, которые содержат точные концепты онтологии. Таким образом, могут быть преодолены различия в терминологии веб-страниц и терминологии запросов.

Кроме того, поисковые машины могут использовать обобщенную/уточненную информацию на основе онтологии. Если не были най-

¹ Свойство в онтологии – это отношение между классами. – *Прим. перев.*

дены документы, соответствующие заданному запросу, то поисковая система может предложить пользователю более общий запрос. Даже возможно, что поисковая система заблаговременно выполнит такой запрос, чтобы уменьшить время реакции в случае, если пользователь принимает это предложение. Если же, наоборот, в ответ на запрос будет возвращено очень много страниц, то можно предложить пользователю варианты уточняющего запроса.

В области искусственного интеллекта (ИИ) уже давно разрабатываются и используются языки онтологий. Эти разработки можно использовать в качестве основы при исследованиях в области сематического веба. В настоящее время к наиболее значимым языкам онтологий относят следующие языки:

- RDF Schema – язык представления онтологий, позволяющий описывать свойства и классы RDF-ресурсов, иерархии свойств и классов, а также область определения (домен) и область значения (диапазон) для свойств;
- OWL – более выразительный язык представления онтологий, который тоже позволяет описывать свойства и классы, а также отношения между классами (например, непересекаемость), ограничения кардинальности¹ (например, «точно один»), отношения эквивалентности для классов и свойств, различные типы свойств, характеристики свойств (например, симметричность) и классы, задаваемые перечислением.

1.2.3. Логика

Логика – это дисциплина, изучающая принципы рассуждения; она восходит своими корнями к Аристотелю. Логика прежде всего предлагает *формальные языки* для представления знаний. Еще логика предоставляет *широко распространенную формальную семантику*: в большинстве логик смысл высказывания определяется без необходимости дополнительных пояснений. Часто мы говорим о декларативных знаниях: описываем то, что уже известно, не заботясь о том, каким образом получены эти знания.

И в-третьих, автоматизированные машины логического вывода из заданных знаний могут получить (выводить) новые знания, тем самым делая неявные знания явными. Такие машины логического вывода широко изучаются в области ИИ. Ниже приведен пример ло-

¹ Ограничения кардинальности также называют ограничениями мощности. Например, лекции по дисциплине могут читаться только одним преподавателем, или многодетная мать имеет трех и более детей. – *Прим. перев.*

гического вывода. Предположим, известно, что все профессора являются преподавателями, все преподаватели являются сотрудниками и что Михаил – профессор. В логике предикатов информация выражается следующим образом:

профессор(X) $\rightarrow$ *преподаватель*(X),
преподаватель(X) $\rightarrow$ *сотрудник*(X),
профессор(Михаил).

Тогда можно вывести следующее:

преподаватель(Михаил),
сотрудник(Михаил),
профессор(X) $\rightarrow$ *сотрудник*(X).

Заметим, что знания из данного примера представляют собой типичные знания, которые хранятся в онтологиях. Таким образом, логика может быть использована для того, чтобы получать неявные знания на основе явных онтологических знаний. Благодаря возможности применять логический вывод в онтологии можно также выявлять в онтологических знаниях непредусмотренные отношения и противоречивые утверждения.

Но логика является более общей моделью, чем онтология. Она может быть использована интеллектуальными агентами для принятия решений и выбора вариантов действий. Например, агент интернет-магазина может принять решение о предоставлении скидки клиенту, основанное на правиле

постоянныйКлиент(X) $\rightarrow$ *скидка*(X , 5%),

где постоянность клиентов определяется на основе данных, хранящихся в корпоративной базе данных.

В общем случае существует компромисс между выразительной мощностью и вычислительной эффективностью логик. Более выразительная логика является менее эффективной, с точки зрения поддержки вывода. Иногда в выразительных логиках вывод вообще не может быть завершен. К счастью, большинство знаний для семантического веба можно представить в относительно простой форме. Например, наши предыдущие примеры описывают *правила* вида: «Если условия, то вывод», — где условия и вывод являются простыми высказываниями, и для того чтобы сделать вывод о возможности применения данных правил, необходимо рассмотреть только конечное число объектов. Такое подмножество логики, называемое логикой Хорна, является разрешимым и поддерживается эффективными инструментами логического вывода.

Важным преимуществом логики является то, что она позволяет получить объяснения логического вывода: последовательность шагов вывода можно проследить. Кроме того, исследователи ИИ разработали способы представления объяснений в удобном для восприятия человеком виде: доказательство представляется как естественный вывод, а шаги вывода низкого уровня группируются в меташаги, каждый из которых обычно расценивается человеком как один шаг доказательства. В конечном счете объяснение позволяет проследить, каким образом был получен ответ, то есть какие существующие факты и правила были использованы для его получения и в каком порядке.

Подобные объяснения важны для сематического веба, поскольку они увеличивают доверие пользователей к агентам сематического веба (см. предыдущий пример про физиотерапевтический центр). Тим Бернерс-Ли говорит о необходимости кнопки «Да?», с помощью которой пользователь может запросить объяснения.

Объяснения также необходимы для осуществления взаимодействия между агентами. В то время как одни агенты поддерживают логический вывод, другие могут только осуществлять валидацию доказательств, то есть проверять, обоснованы ли утверждения, сделанные первыми агентами. Приведем простой пример. Пусть агент 1, представляющий интернет-магазин, отправляет сообщение «Вы должны мне \$80» (конечно, не на естественном, а на формальном, машинно обрабатываемом языке) агенту 2, представляющему человека. Затем агент 2 может попросить объяснения, и агент 1 предоставит ответ в виде следующей последовательности:

Веб-журнал покупки на \$80

Доказательство поставки (например, с идентификационным номером 2341)

Правило с терминами и условиями магазина:

покупка(Х, товар) ∧ цена(товар, Цена) ∧ доставлен(товар, Х) → должен(Х, Цена).

Таким образом, факты обычно отслеживаются на некоторых веб-адресах (надежность которых может быть проверена агентами), а правила могут быть частью общей онтологии по продажам или политике интернет-магазина.

Для того чтобы применять логику в вебе, необходимо, во-первых, чтобы ее можно было использовать в сочетании с другими данными, во-вторых, чтобы она была машинно обрабатываемой. Поэтому идет непрерывающаяся работа, направленная на возможность представления логических знаний и доказательств в веб-языках. Первона-

чальные предложенные подходы работают на уровне языка XML, но в будущем правила и доказательства должны быть представлены на уровне языка RDF и языков онтологий.

1.2.4. Сематический веб и искусственный интеллект

Как уже отмечалось, большинство технологий, необходимых для реализации концепции сематического веба, основано на результатах, полученных в области искусственного интеллекта. Учитывая, что ИИ имеет долгую историю, но не всегда коммерчески успешные результаты, можно опасаться, что в худшем случае сематический веб повторит ошибки ИИ: большие обещания, которые вызывают высокие ожидания, оказывающиеся неоправданными (по крайней мере, в обещанный срок).

Это беспокойство является необоснованным. Реализация концепции сематического веба не предполагает достижения веб-агентами человеческого уровня интеллекта; на самом деле, как мы пытались объяснить, используется другой подход к решению проблем. Задача ИИ глубоко научна и, возможно, сопоставима с основными задачами физики (объяснить физический мир) или биологии (объяснить живой мир). Поэтому трудности в достижении искусственным интеллектом уровня человеческого интеллекта в течение десяти или двадцати лет (как это было обещано в некоторых работах прошлых лет) вполне объяснимы.

Но в сематическом вебе работают частичные решения. Даже если интеллектуальный агент не в состоянии сделать все выводы, которые на основании этих же данных может сделать человек, он способствует усовершенствованию веба. Это позволит сформулировать принципиальное различие между ИИ и семантическим вебом. Если конечная цель ИИ заключается в создании интеллектуального агента, обладающего интеллектом человеческого уровня (и выше), то цель сематического веба – оказание помощи веб-пользователям в их повседневной онлайн-деятельности.

Очевидно, что в сематическом вебе активно используются современные технологии ИИ, и прогресс в этих технологиях приведет к развитию семантического веба. Но нет необходимости ждать, пока в области ИИ будут получены достижения еще более высокого уровня; для реализации концепции сематического веба вполне достаточно существующих современных технологий ИИ.

1.3. Многоуровневый подход

Развитие сематического веба происходит пошагово, при этом на каждом шаге строится уровень (слой) поверх другого, уже существую-

щего уровня. Этот поход имеет прагматическое обоснование: легче достичь согласованности, продвигаясь небольшими шагами, а не решая все проблемы сразу. Обычно одновременно работает несколько исследовательских групп, движущихся в разных направлениях; такая конкуренция идей является основной движущей силой научного прогресса. Однако, с технической точки зрения, существует потребность в стандартизации. Так, если большинство исследователей пришло к согласию по определенным вопросам, имеет смысл это зафиксировать, даже если не достигнуто согласие в других вопросах. Таким образом, даже если более амбициозные научно-исследовательские проекты потерпят неудачу, то могут использоваться, по крайней мере, частичные положительные результаты.

После того как разработан стандарт, многие группы и компании могут приступить к его внедрению, а не ждать завершения исследований в альтернативных направлениях. Природа сематического веба такова, что компании и отдельные пользователи должны разрабатывать инструменты, создавать контент и использовать этот контент. Мы не можем ждать, пока концепция сематического веба будет полностью реализована – это может занять еще десять лет (конечно, имеется в виду реализация текущего видения этой концепции).

При разработке одного слоя сематического веба поверх другого следует придерживаться двух основных принципов:

- нисходящая совместимость. Агенты, предназначенные для работы с информацией одного уровня, должны также быть в состоянии интерпретировать и использовать информацию, записанную на более низких уровнях. Например, агенты, предназначенные для работы с семантикой языка OWL, могут полноценно обрабатывать информацию, представленную на языках в RDF и RDF Schema;
- восходящая частичная согласованность. Агенты, предназначенные для работы с информацией одного уровня, должны также быть в состоянии, по крайней мере частично, использовать информацию, записанную на более высоких уровнях. Например, агент, предназначенный для работы исключительно с семантикой языков RDF и RDF Schema, может частично интерпретировать знания, записанные на языке OWL, игнорируя элементы, не входящие в RDF и RDF Schema. Конечно, необязательно, чтобы все инструменты имели такую функциональность. Важно, чтобы эта функциональность в принципе была доступна.

Эти идеи привлекательны с теоретической точки зрения и используются в качестве основных принципов для развития сематического веба.

Однако их реализация на практике оказалась достаточно сложной, и поэтому приходится идти на некоторые компромиссы. Это станет понятно из главы 4, в которой обсуждается совместимость слоев RDF и OWL.

На рис. 1.4 изображен «слоеный пирог» семантического веба, который представляет основные уровни концепции семантического веба и его развития. В его нижней части находится язык XML, позволяющий создавать структурированные веб-документы с помощью определяемых пользователем терминов. XML особенно удобен для пересылки документов в вебе. Кроме того, идентификаторы URI, используемые в XML, могут быть сгруппированы в *пространства имен* (англ. *namespace*), которые обозначены на рис. 1.4 как NS.

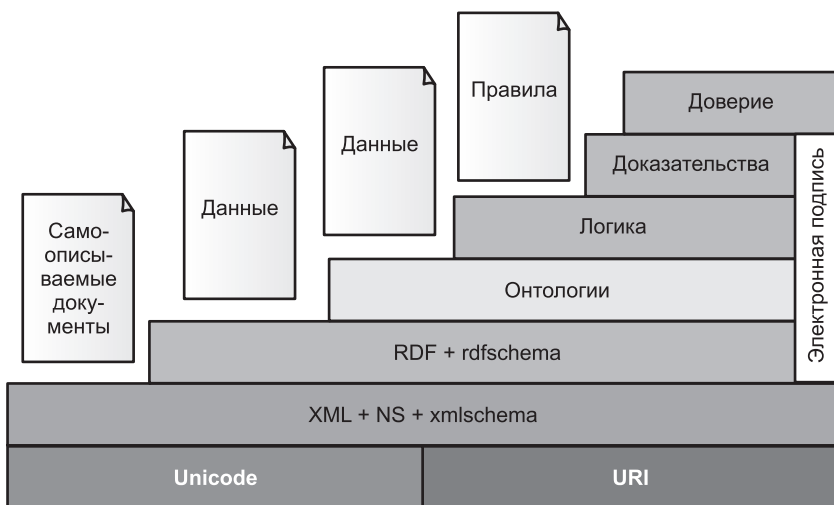


Рис. 1.4 ❖ Многоуровневый подход к развитию семантического веба

RDF является основной моделью данных в семантическом вебе. Эта модель, подобная модели «сущность–связь», используется для записи простых утверждений о веб-объектах (ресурсах). Модель данных RDF не основана на языке XML, но язык RDF имеет синтаксис на основе XML¹. Поэтому на рис. 1.4 слой RDF расположен поверх слоя XML.

Язык RDF Schema предоставляет примитивы моделирования для организации веб-объектов в иерархии. Основными примитивами яв-

¹ На самом деле только один из синтаксисов языка RDF основан на XML. См. главу 2. – *Прим. перев.*

ляются классы, свойства, отношения «класс–подкласс», «свойство–подсвойство», ограничения на область значений и область определения свойств. Язык RDF Schema основан на языке RDF.

Язык RDF Schema можно рассматривать как примитивный язык для написания онтологий. Но существует потребность в более выразительных *языках онтологий*, которые расширяют язык RDF Schema и позволяют представлять более сложные отношения между веб-объектами. Слой «Логика» используется для расширения возможностей языков онтологий и позволяет описывать декларативные знания для конкретных приложений.

Слой «Доказательства» включает в себя представление процесса дедуктивного вывода, а также представление доказательств в веб-языках (из более низких уровней) и валидацию доказательств.

Наконец, слой «Доверие» реализуется через использование *цифровых подписей* и других видов знаний, основанных на рекомендациях надежных агентов или на рейтингах и сертификации различных организаций и групп потребителей. Иногда термин «веб-доверия» используется для обозначения того, что доверие будет организовано таким же распределенным и хаотичным образом, как и сам веб. Находясь на вершине пирамиды, доверие представляет крайне важную концепцию высокого уровня: потенциал веба может полностью реализоваться только тогда, когда пользователи будут доверять веб-приложениям (их безопасности) и качеству предоставляемой информации.

Рассмотренный классический «слоеный пирог» был принят как основная концепция развития сематического веба, однако в настоящее время такое его представление является довольно устаревшим. В частности, появились альтернативы для слоя онтологий. Кроме того, поверх слоя RDF, минуя слой онтологий, были определены языки правил. Это особенно важно в условиях перехода от выразительных семантических структур к обработке огромных количеств (семантических) данных. Таким образом, «слоеный пирог» представлен здесь в иллюстративных целях, чтобы продемонстрировать видение концепции сематического веба в историческом аспекте.

1.4. Обзор книги

В этой книге представлены технологии семантического веба, которые достигли достаточной степени зрелости.

В главе 2 описываются языки RDF и RDF Schema. RDF является языком, с помощью которого можно выразить утверждения об объ-

ектах (ресурсах); это стандартная модель данных для машинно обрабатываемой семантики. Язык RDF Schema содержит целый ряд примитивов моделирования для организации RDF-терминов в типизированные иерархии.

Глава 3 посвящена языку запросов SPARQL, который играет ту же роль в мире RDF-данных, что и язык SQL в мире реляционных данных.

В главе 4 рассматривается язык OWL2, текущая версия языка веб-онтологий OWL. Он предлагает больше примитивов моделирования, чем язык RDF Schema, и имеет понятную, формальную семантику.

Глава 5 посвящена правилам в семантическом вебе. Несмотря на то что правила в семантическом вебе еще не находятся на таком же уровне стандартизации и принятия веб-сообществом, как языки RDF, SPARQL или OWL, принципы, на основе которых происходит их внедрение, вполне понятны, поэтому имеет смысл их представить.

В главе 6 рассматривается несколько приложений семантического веба в конкретных предметных областях и объясняются преимущества, которые можно получить от реализации концепции семантического веба.

В главе 7 рассматриваются различные ключевые вопросы, связанные с разработкой веб-приложений, основанных на онтологиях.

Наконец, в главе 8 кратко освещаются несколько вопросов, которые являются актуальными для современного сообщества семантического веба.

1.5. Выводы

- Семантический веб является инициативой, которая направлена на улучшение текущего состояния мировой паутины.
- Ключевая идея заключается в использовании в вебе информации, доступной для машинной обработки.
- Основные технологии семантического веба: публикация данных с явно заданными метаданными, онтологии, логика и логический вывод.
- Развитие семантического веба происходит в соответствии с многоуровневым подходом.

Рекомендуемая литература

Превосходная вступительная статья, посвященная концепции семантического веба:

- Berners-Lee T., Hendler J., and Lassila O. The Semantic Web. *Scientific American* 284 (May 2001): 34–43.

Вдохновляющая книга об истории (и будущем) веба:

- Berners-Lee T., with Fischetti M. *Weaving the Web*. San Francisco: Harper, 1999.

Веб-сайты, представляющие обновляемую информацию о сематическом вебе и связанных с ним темах:

- www.semanticweb.org/;
- www.w3.org/2001/sw/.

Существует целый ряд научно-исследовательских работ, в которых можно найти техническую информацию по вопросам, связанным с сематическим вебом:

- Domingue J., Fensel D., and Hendler J. *Handbook of Semantic Web Technologies*. Springer, 2011.

Сборники трудов конференций и журналы, в которых публикуются последние достижения – новые идеи, технологии и описание приложений сематического веба:

- The conference series of the *International Semantic Web Conference* // <http://www.semanticweb.org/>;
- The conference series of the *Extended (European) Semantic Web Conference* // <http://www.eswc.org/>;
- Journal of Web Semantics // <http://www.journals.elsevier.com/journal-of-web-semantics/>.

В последние годы выпущен целый ряд книг, посвященных сематическому вебу, например книга о проблемах онтологического моделирования:

- Allemang D. and Hendler J. *Semantic Web for the Working Ontologist: Effective Modeling in RDFS and OWL*. New York, NY.: Morgan Kaufmann, 2008,

или книга, посвященная логическим основам сематического веба:

- Hitzler P., Kroetzsch M., and Rudolph S. *Foundations of Semantic Web Technologies*. Boca Raton, FL.: Chapman and Hall, 2009.

Глава 2

Описание веб-ресурсов: RDF

2.1. Введение

Успех Всемирной путины WWW продемонстрировал значение использования стандартизованных механизмов для обмена информацией. Стандартным языком для написания веб-страницы является язык HTML. Использование данного стандарта позволяет публиковать документ с уверенностью, что он будет правильно отображен в любом веб-браузере.

Язык HTML, как и любой другой язык, предназначенный для обмена информацией, имеет три составляющие: синтаксис, модель данных и семантику. Синтаксис предоставляет способ записи данных. Модель данных задает структуру или организацию данных. Семантика позволяет интерпретировать эти данные. Проиллюстрируем каждый из этих компонентов языка с помощью фрагмента HTML-документа:

```
<html>
  <head>
    <title>Квартиры для аренды</title>
  </head>
  <body>
    <ol>
      <li> Квартира-студия на Флорида Аве.
      <li> Квартира на Барон Вэй с 3 спальнями
    </ol>
  </body>
</html>
```

Синтаксис языка HTML представлен текстом, размеченным тегами в угловых скобках (например, <title>). Модель данных в языке HTML, известная как объектная модель документа (Document Object

Model), определяет организацию элементов документа, представленных тегами, в виде иерархической древовидной структуры. Например, тег `<head>` должен находиться перед тегом `<body>`, а теги `<li>` – внутри элемента, обозначенного тегом `<ol>`. Наконец, семантика языка HTML определяет, каким образом браузер должен интерпретировать веб-страницу. Например, браузер должен отобразить содержимое тела веб-страницы в окне браузера, а элементы `<ol>` должны быть отображены в виде упорядоченного списка. Синтаксис, модель данных и семантика HTML определяются в рамках стандарта этого языка.

Язык HTML обеспечивает передачу информации о структуре документов, предназначенных для человеческого восприятия. Для реализации концепции семантического веба нужен более богатый язык. Необходима модель данных, которая может быть использована несколькими приложениями и позволяет описывать не только человекочитаемые документы, но и информацию для конкретных приложений. Эта модель данных должна быть независимой от предметной области, для того чтобы ее могли использовать совершенно различные приложения, начиная от сайтов о недвижимости и заканчивая социальными сетями. В дополнение к гибкой модели данных необходим механизм присвоения семантики данным, представленным с использованием этой модели, что позволит пользователям описывать, каким образом данные будут интерпретироваться приложениями, например как приложение должно интерпретировать термин «друг» в описании социальной сети или термин «город» в географическом описании. Наконец, как и в языке HTML, нужен способ записи всей этой информации, то есть синтаксис.

Язык RDF (Resource Description Framework) предоставляет именно такую гибкую модель данных, независимую от предметной области. Основным элементом языка RDF является триплет (тройка) вида «сущность – атрибут – значение», называемый утверждением. Приведем примеры RDF-утверждений: «Квартира Барон Вэй – это квартира», «Квартира Барон Вэй является частью здания Барон Вэй», «Здание Барон Вэй находится в Амстердаме». Так как язык RDF не специфицирован для какой-либо определенной предметной области, то пользователям необходимо определить термины, которые они используют в RDF-утверждениях. Для этого служит язык RDF Schema (RDFS). RDFS позволяет пользователям точно определить, каким образом следует интерпретировать их *словарь* (vocabulary), то есть используемые ими термины.

Эти технологии в сочетании определяют компоненты стандартного языка для обмена произвольными данными между машинами:

- RDF – модель данных;
- RDFS – семантика;
- Turtle / RDFa / RDF/XML – синтаксис.

Заметим, что термин RDF изначально обозначает модель данных языка, но он часто используется и в качестве названия всего языка (в том числе и в данной книге).

Обзор главы

- В разделе 2.2 описывается модель данных RDF.
- Различные синтаксисы, используемые для записи RDF, представлены в разделе 2.3.
- Основы языка RDF Schema вводятся в разделе 2.4, а полное описание языка представлено в разделах 2.5 и 2.6.
- Формальная семантика языка RDFS задается двумя различными способами в разделах 2.7 и 2.8.

2.2. RDF: модель данных

Основными понятиями в языке RDF являются понятия «ресурс», «свойство», «утверждение» и «граф».

2.2.1. Ресурсы

Ресурс можно рассматривать как объект, сущность или «вещь», о которой мы хотим сделать какое-либо высказывание. Ресурсами могут быть авторы, квартиры, теннисисты, места, люди, гостиницы, поисковые запросы и т. д. Каждый ресурс имеет URI¹. URI может представлять из себя веб-адрес URL (Uniform Resource Locator) или другой вид уникального идентификатора. Схемы URI определены не только для ресурсов, размещенных в вебе, но и для телефонных номеров, номеров ISBN и географических мест. URI обеспечивают механизм однозначной идентификации «вещей», о которых мы хотим что-то сказать. Таким образом, если необходимо сделать утверждение о плавательном бассейне (англ. *swimming pool*), то можно использовать URI, идентифицирующий именно плавательный бассейн, что не позволит перепутать данный ресурс с видом бильярда (англ. *pool*) или

¹ Напомним, что в последних версиях стандартов сематического веба для идентификации ресурсов используется IRI, а не URI. – *Прим. перев.*

группой людей, например программистов (англ. *pool of programmers*). Эта проблема известна как проблема омонимов¹.

Использование URI не обязательно обеспечивает доступ к ресурсу. Однако в качестве идентификаторов ресурсов рекомендуется применять URL, которые возможно разыменовывать (dereferencable URI). Разыменование URI позволяет пользователям получить либо сам ресурс (например, изображение), либо более подробное описание этого ресурса (например, описание человека)². Все ресурсы в данной книге будут идентифицироваться с помощью URL. Использование URI представляет собой одно из ключевых проектных решений, лежащих в основе языка RDF. Именно эта идея позволяет имя любого ресурса сделать уникальным в глобальном масштабе. Использование подобной схемы именования ресурсов существенно снижает остроту проблемы омонимов, которая до настоящего момента являлась препятствием для представления распределенных данных.

2.2.2. Свойства

Свойства представляют собой особый вид ресурсов; они описывают отношения между другими ресурсами. Приведем примеры свойств: «является другом», «является автором» «расположен в». Как и все ресурсы, свойства идентифицируются с помощью URI. URL свойств, так же как и URL любых ресурсов, можно разыменовывать, чтобы найти их описания.

2.2.3. Утверждения

Утверждения задают определенные свойства для определенных ресурсов. Утверждение представляет собой триплет вида «сущность – атрибут – значение», состоящий из ресурса, свойства и значения этого свойства соответственно. В качестве значения свойства может выступать некоторый ресурс или *литерал*. Литералы представляют собой атомарные величины, например числа, строки или даты. Для обозначения сущности в RDF-утверждении часто используется термин «субъект», а для обозначения значения – термин «объект».

¹ Омонимы – разные по значению, но одинаковые по звучанию и написанию единицы языка. Например, ключ – от двери, ключ – природный источник воды, ключ – музыкальный знак, ключ – инструмент, ключ – регистрационный номер программного обеспечения. – *Прим. перев.*

² Иными словами, разыменование URI – это предоставление доступа к ресурсу, идентифицируемому данным URI. – *Прим. перев.*



Рис. 2.1 ❖ Представление RDF-утверждения в виде графа

Рассмотрим пример утверждения «Здание Барон Вэй находится в Амстердаме». На языке RDF оно может быть записано следующим образом:

```

<http://www.semanticwebprimer.org/ontology/apartments.ttl#ЗданиеБаронВэй>1
<http://dbpedia.org/ontology/location>
<http://dbpedia.org/resource/Amsterdam>.
  
```

Обратите внимание, что для идентификации субъекта, свойства и объекта в данном утверждении были использованы URL.

2.2.4. Графы

Приведенное выше утверждение может быть представлено в виде *графа* (рис. 2.1). При этом для повышения читабельности в данном графе не используются URI.

На рис. 2.1 представлен граф, в котором помеченные узлы соединены помеченными дугами. При этом любая дуга в графе направлена от субъекта утверждения к объекту утверждения, а метка на дуге представляет собой свойство утверждения. Метки узлов представляют собой идентификаторы субъектов и объектов утверждений. Объект утверждения может являться субъектом другого утверждения. Например, можно сделать утверждение, что «Амстердам является городом». Данное утверждение представлено в виде графа на рис. 2.2.

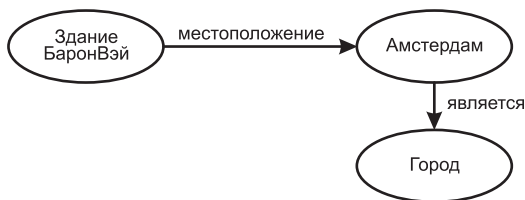


Рис. 2.2 ❖ RDF-граф

¹ Для повышения степени усвоения материала русскоязычными читателями все названия объектов, свойств и субъектов RDF-утверждений в примерах приведены на русском языке. Это не относится к идентификаторам ресурсов из реально существующих в вебе наборов данных, например проекта DBpedia.org. – *Прим. перев.*

Данное графическое представление показывает, что RDF является графоориентированной моделью данных. Действительно, модель RDF напоминает так называемую семантическую сеть из теории искусственного интеллекта. Можно продолжить расширение графа, описывающего здание Барон Вэй. На рис. 2.3 изображена расширенная версия этого RDF-графа.

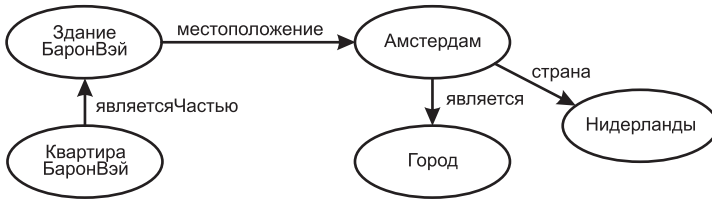


Рис. 2.3 ❖ Расширенный RDF-граф

Важно отметить, что RDF-граф может быть создан в распределенном режиме несколькими различными участниками исключительно за счет использования URL. Это способствует созданию веба данных, который позволяет повторно использовать существующие знания. Например, если мы найдем в вебе описание города Амстердама в RDF-формате, то сможем повторно использовать эту информацию, только за счет указания URL данного ресурса. В целях создания глобального RDF-графа рекомендуется применять так называемые *принципы связанных данных* (Linked Data principles), которые способствуют повторному использованию и доступности информации:

- 1) применение URI в качестве имен сущностей;
- 2) использование HTTP URI для реализации возможности обращения пользователей к этим сущностям;
- 3) предоставление полезной информации тому, кто обращается по URI, с помощью стандартов (RDF);
- 4) включение ссылок на другие URI, позволяющие найти дополнительную информацию.

Несмотря на то что сама модель данных RDF не подразумевает следование этим принципам, их выполнение позволит нам получить большое преимущество – пользоваться знаниями других людей. Обратите внимание на то, как была повторно использована информация из проекта DBpedia.org в наших примерах. Вы можете перейти по указанным URL, чтобы получить больше информации о соответствующих ресурсах (<http://dbpedia.org/ontology/location> и <http://dbpedia.org/resource/Amsterdam>).

2.2.5. Выделение утверждений и графов

Иногда необходимо выделить конкретные утверждения и части графов, например для того, чтобы указать степень доверия к этим утверждениям или определить их источник. Предположим, нужно зафиксировать тот факт, что утверждение о местоположении здания Барон Вэй создано человеком – Фрэнком. Для этого язык RDF предоставляет два механизма.

Один из этих механизмов называется *реификация* (или материализация) утверждений. Основная идея реификации заключается во введении вспомогательного объекта, например объекта *Локальное Утверждение*, который связывается с каждой из трех частей заданного утверждения посредством RDF-свойств *subject* (субъект), *predicate* (предикат) и *object* (объект). В предыдущем примере субъектом для *Локальное Утверждение* является *Здание БаронВэй*, предикатом – *местоположение*, а объектом – *Амстердам*. Теперь мы можем указать *Локальное Утверждение* в качестве субъекта в другом триplete, который определяет его создателя. На рис. 2.4 изображен полученный таким образом граф. Как и в предыдущем случае, для улучшения читабельности в графе не указаны полные URI.

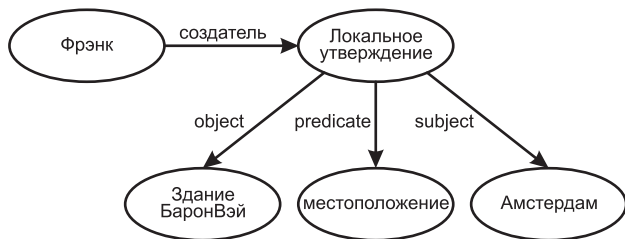


Рис. 2.4 ❖ Пример реификации

Этот довольно громоздкий подход необходим, так как в языке RDF нет ничего, кроме триплетов, и мы не можем непосредственно создать идентификатор для триплета (тогда это был бы не триплет (тройка), а четверка). Из-за затрат вычислительных ресурсов, которых требует реификация, в новых версиях стандарта языка RDF было введено понятие «именованный граф». При данном подходе утверждению или набору утверждений явным образом присваивается идентификатор (URL). Этот идентификатор может быть использован в обычных триплетах. Это более простой механизм для идентификации утверждений и графов. Говоря другими словами, именованный граф позво-

ляет выделить множество RDF-утверждений и присвоить этому множеству идентификатор. В разделе 2.3.1.3 приводится пример записи рассмотренного выше реифицированного утверждения с помощью именованных графов.

2.2.6. Предикаты с несколькими аргументами

Триплет (x, P, y) можно рассматривать как логическую формулу $P(x, y)$, где P – бинарный предикат, который связывает объект x с объектом y . Фактически с помощью языка RDF можно записывать только *бинарные предикаты* (свойства). Однако в некоторых случаях необходимы предикаты, которые имеют более двух аргументов. К счастью, такие предикаты могут быть смоделированы с помощью последовательности бинарных предикатов. Проиллюстрируем эту методику на примере предиката **брокер** с тремя аргументами. Интуитивное значение предиката *брокер*(X, Y, Z) заключается в следующем: X является брокером в сделке по продаже дома между продавцом Y и покупателем Z .

Введем новый вспомогательный ресурс *дом-на-продажу*¹ и три бинарных предиката *брокер*, *продавец* и *покупатель*. Тогда предикат *брокер*(X, Y, Z) может быть предоставлен следующим образом:

брокер(*дом-на-продажу*, X),
продавец(*дом-на-продажу*, Y),
покупатель(*дом-на-продажу*, Z).

Несмотря на то что предикаты с тремя аргументами записываются более компактно, использование бинарных предикатов позволяет упростить модель данных в целом.

2.3. Синтаксисы RDF

В предыдущем разделе был введен один из синтаксисов языка RDF, а именно графический синтаксис. Однако этот синтаксис не стандартизирован. Кроме того, он не пригоден для машинной интерпретации. В данном разделе вводятся стандартный машинно интерпретируемый синтаксис, называемый *Turtle*, а также дается краткий обзор нескольких других синтаксисов.

¹ Подобный вспомогательный ресурс может быть задан в RDF-утверждении как пустой узел (blank node). В соответствии со спецификацией языка RDF пустые узлы представляют собой множество, непересекающееся со множеством URI и литералов. Пустые узлы могут указываться в качестве субъекта и объекта RDF-триплета. – *Прим. перев.*

2.3.1. Turtle

Turtle (Terse RDF Triple Language, лаконичный язык RDF-триплетов) представляет собой текстовый синтаксис для языка RDF. Текстовые файлы, содержащие данные в формате Turtle, имеют расширение «.ttl». Выше уже был приведен пример утверждения, записанного с помощью синтаксиса Turtle:

```
<http://www.semanticwebprimer.org/ontology/apartments.ttl#ЗданиеБаронВэй>
<http://dbpedia.org/ontology/location>
<http://dbpedia.org/resource/Amsterdam>.
```

Адреса URL заключаются в угловые скобки. RDF-утверждение записывается в следующем порядке: **предмет**, свойство, объект – и заканчивается точкой. Таким образом можно записать целый RDF-граф:

```
<http://www.semanticwebprimer.org/ontology/apartments.ttl#>
  <http://www.semanticwebprimer.org/ontology/apartments.ttl#являетсяЧастью>
  <http://www.semanticwebprimer.org/ontology/apartments.ttl#ЗданиеБаронВэй>.
<http://www.semanticwebprimer.org/ontology/apartments.ttl#ЗданиеБаронВэй>
  <http://dbpedia.org/ontology/location>
  <http://dbpedia.org/resource/Amsterdam>.
```

2.3.1.1. Литералы

Как было показано выше, RDF-утверждения связывают ресурсы. Однако они могут содержать и литералы, то есть атомарные значения. В синтаксисе Turtle литералы записываются как некоторые значения, заключенные в кавычки, после которых указывается *тип данных*. Тип данных позволяет интерпретировать указанное значение как строку, дату, целое число или как величину некоторых других типов. Типы данных, так же как и ресурсы, идентифицируются с помощью URL. Рекомендуется использовать типы данных, определенные с помощью языка XML Scheme. При применении этих типов данных указанные значения должны соответствовать определению языка XML Scheme. Если для литерала тип данных не задан, то предполагается, что тип этого литерала – строковый. Покажем, как некоторые общие типы данных записываются в синтаксисе Turtle:

```
строки (string) - "Барон Вэй"
целые числа (integer) - "1"^^<http://www.w3.org/2001/XMLSchema#integer>
дробные числа (decimal) - "1.23"^^<http://www.w3.org/2001/XMLSchema#decimal>
даты (date) - "1982-08-30"^^<http://www.w3.org/2001/XMLSchema#date>
время (time) - "11:24:00"^^<http://www.w3.org/2001/XMLSchema#time>
дата-время (dateTime) - "1982-08-30T11:24:00"^^<http://www.w3.org/2001/
```



```
XMLSchema#dateTime>
```

Предположим, что мы хотим добавить к нашему RDF-графу утверждение о том, что квартира БаронВэй имеет три спальни. С помощью синтаксиса Turtle это утверждение записывается следующим образом:

```
<http://www.semanticwebprimer.org/ontology/apartments.  
ttl#КвартираБаронВэй> <http://www.semanticwebprimer.org/ontology/apartments.  
ttl#имеетКоличествоСпален> "3"^^<http://www.w3.org/2001/XMLSchema#integer>.  
  
<http://www.semanticwebprimer.org/ontology/apartments.ttl#КвартираБаронВэй>  
<http://www.semanticwebprimer.org/ontology/apartments.ttl#являетсяЧастью>  
<http://www.semanticwebprimer.org/ontology/apartments.ttl#ЗданиеБаронВэй>.  
  
<http://www.semanticwebprimer.org/ontology/apartments.ttl#ЗданиеБаронВэй>  
<http://dbpedia.org/ontology/location>  
<http://dbpedia.org/resource/Amsterdam>.
```

Приведенные выше примеры являются довольно громоздкими. Для упрощения записи в синтаксисе Turtle используется целый ряд конструкций, рассмотренных в следующем разделе.

2.3.1.2. Сокращения

Часто для определения словарей используются одинаковые URI. В нашем примере ресурсы Квартира Барон Вэй и Здание Барон Вэй описаны с использованием одного и того же URL – <http://www.semanticwebprimer.org/ontology/apartments.ttl>. Этот URL определяет так называемое *пространство имен* для данных ресурсов. Синтаксис Turtle позволяет вводить аббревиатуры (сокращения) для таких URL. Для этого используется ключевое слово `@prefix`, определяющее короткое имя для конкретного пространства имен. Например, для пространства имен <http://www.semanticwebprimer.org/ontology/apartments.ttl> можно ввести сокращенное имя `swp`. Такие сокращенные имена называются также *уточненными*. Перепишем приведенный выше пример с использованием префиксов.

```
@prefix swp: <http://www.semanticwebprimer.org/ontology/apartments.ttl#>.  
@prefix dbpedia: <http://dbpedia.org/resource/>.  
@prefix dbpedia-owl: <http://dbpedia.org/ontology/>.  
@prefix xsd: <http://www.w3.org/2001/XMLSchema#>.  
  
swp:КвартираБаронВэй swp:имеетКоличествоСпален "3"^^<xsd:integer>.  
swp:КвартираБаронВэй swp:являетсяЧастью swp:ЗданиеБаронВэй.
```

```
swp:ЗданиеБаронВэй dbpedia-owl:location dbpedia:Amsterdam.
```

Обратите внимание, в данном примере ресурсы описываются с помощью уточненных имен, при этом угловые скобки не указываются. Заметим, что при описании ресурсов можно сочетать обычные URL и учтенные имена.

Кроме того, синтаксис Turtle позволяет не повторять имя субъекта, если о нем имеется несколько утверждений.

В приведенном выше примере ресурс swp:КвартираБаронВэй используется в качестве субъекта в двух триплетах. Данные утверждения можно записать более компактно, применяя точку с запятой после первого утверждения. Например:

```
@prefix swp:      <http://www.semanticwebprimer.org/ontology/apartments.ttl#>.
@prefix dbpedia:  <http://dbpedia.org/resource/>.
@prefix dbpedia-owl: <http://dbpedia.org/ontology/>.
@prefix xsd:      <http://www.w3.org/2001/XMLSchema#>.
```

```
swp:КвартираБаронВэй swp:имеетКоличествоСпален "3"^^<xsd:integer>;
swp:являетсяЧастью swp:ЗданиеБаронВэй.
swp:ЗданиеБаронВэй dbpedia-owl:location dbpedia:Amsterdam.
```

Если повторяются и субъект, и предикат утверждения, то в конце утверждения можно использовать запятую. Например, расширим наш пример, указав, что здание Барон Вэй находится не только в Амстердаме, но и в Нидерландах. С помощью синтаксиса Turtle это можно записать следующим образом:

```
@prefix swp:      <http://www.semanticwebprimer.org/ontology/apartments.ttl#>.
@prefix dbpedia:  <http://dbpedia.org/resource/>.
@prefix dbpedia-owl: <http://dbpedia.org/ontology/>.
@prefix xsd:      <http://www.w3.org/2001/XMLSchema#>.
```

```
swp:КвартираБаронВэй swp:имеетКоличествоСпален "3"^^<xsd:integer>;
                        swp:являетсяЧастью swp:ЗданиеБаронВэй.
swp:ЗданиеБаронВэй dbpedia-owl:location dbpedia:Amsterdam,
dbpedia:Netherlands.
```

Кроме того, синтаксис Turtle позволяет сокращать общие типы данных. Например, числа могут записываться без кавычек. Если число содержит дробную часть (например, 14.3), то оно интерпретируется как десятичная дробь (типа *decimal*). Если число не содержит дробной части (например, 1), оно интерпретируется как целое (типа *integer*). Это еще немного сокращает наш пример:

```
@prefix swp: <http://www.semanticwebprimer.org/ontology/apartments.ttl#>.
@prefix dbpedia: <http://dbpedia.org/resource/>.
@prefix dbpedia-owl: <http://dbpedia.org/ontology/>.
```

```
swp:КвартираБаронВэй swp:имеетКоличествоСпален 3;
                        swp:являетсяЧастью swp:ЗданиеБаронВэй.
swp:ЗданиеБаронВэй dbpedia-owl:location dbpedia:Amsterdam,
                        dbpedia:Netherlands.
```

2.3.1.3. Именованные графы

Выше была рассмотрена возможность выделения конкретных утверждений. Она может быть реализована с помощью языка TriG¹, который является расширением синтаксиса Turtle. Например, укажем, что утверждения о квартире Барон Вэй созданы человеком по имени Фрэнк, который идентифицируется следующим URL: <http://www.cs.vu.nl/~frankh>. Для этого нужен набор утверждений, которому присваивается URL, заключить в фигурные скобки. Данный пример будет записан следующим образом:

```
@prefix swp: <http://www.semanticwebprimer.org/ontology/apartments.ttl#>.
@prefix dbpedia: <http://dbpedia.org/resource/>.
@prefix dbpedia-owl: <http://dbpedia.org/ontology/>.
@prefix dc: <http://purl.org/dc/terms/>2.
```

```
{
  <http://www.semanticwebprimer.org/ontology/apartments.ttl#>
  dc:creator <http://www.cs.vu.nl/~frankh>
}
```

```
<http://www.semanticwebprimer.org/ontology/apartments.ttl#>
{
  swp:КвартираБаронВэй swp:имеетКоличествоСпален 3;
    swp:являетсяЧастью swp:ЗданиеБаронВэй.
  swp:ЗданиеБаронВэй dbpedia-owl:location dbpedia:Amsterdam,
    dbpedia:Netherlands.
}
```

При таком подходе утверждения, которые не являются частью определенного графа, также заключаются в фигурные скобки, но пе-

¹ См. <http://www.w3.org/TR/trig/>. – *Прим. перев.*

² Словарь «Дублинское ядро», содержащий термины для описания метаданных любых ресурсов в соответствии с инициативой Dublin Core Metadata Initiative (DCMI). – *Прим. перев.*

ред ними не указывается URL. Данный граф называется *графом по умолчанию*.

2.3.2. Другие синтаксисы RDF

Для записи RDF, кроме синтаксиса Turtle, можно использовать и ряд других синтаксисов, среди которых два являются стандартизованными: RDF/XML и RDFa¹.

2.3.2.1. RDF/XML

Синтаксис RDF/XML представляет собой кодирование модели данных RDF на языке XML, что позволяет использовать существующие инструменты языка XML. Первоначально RDF/XML был единственным стандартизованным синтаксисом для RDF. Однако синтаксис Turtle был принят в качестве дополнительного стандарта, так как является более читабельным. Ниже приведен синтаксис RDF/XML. Субъекты утверждений в данном синтаксисе обозначаются с помощью атрибута `rdf:about` термина `rdf:Description`, заключенного в угловые скобки. Предикаты и объекты, связанные с субъектом, указываются внутри элемента `rdf:Description`. Используемые пространства имен указываются с помощью XML-конструкции `xmlns:..`. Все содержимое документа в формате RDF/XML должно быть указано внутри элемента `rdf:RDF`.

```
<?xml version="1.0" encoding="utf-8"?>
<rdf:RDF xmlns:dbpedia-owl="http://dbpedia.org/ontology/"
  xmlns:dbpedia="http://dbpedia.org/resource/"
  xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  xmlns:swp="http://www.semanticwebprimer.org/ontology/apartments.ttl#">
  <rdf:Description rdf:about="http://www.semanticwebprimer.org/ontology/
    apartments.ttl#КвартираБаронВэй">
    <swp:имеетКоличествоСпален
      rdf:datatype="http://www.w3.org/2001/XMLSchema#integer">
        3
      </swp:имеетКоличествоСпален>
    </rdf:Description>
  <rdf:Description
    rdf:about="http://www.semanticwebprimer.org/ontology/apartments.ttl#
    КвартираБаронВэй ">
```

¹ На момент перевода книги стандартизирован уже целый ряд синтаксисов, помимо двух указанных, а именно: Turtle, N-Quads, N-Triples, TriG, JSON-LD 1.0. См. http://www.w3.org/standards/techs/rdf#w3c_all. — Прим. перев.

```

<swp: являетсяЧастью
  rdf:resource="http://www.semanticwebprimer.org/ontology/apartments.ttl#
    КвартираБаронВэй"/>
</rdf:Description>
<rdf:Description
  rdf:about="http://www.semanticwebprimer.org/ontology/apartments.
    ttl#ЗданиеБаронВэй">
  <dbpedia-owl:location
    rdf:resource="http://dbpedia.org/resource/Amsterdam"/>
</rdf:Description>
<rdf:Description
  rdf:about="http://www.semanticwebprimer.org/ontology/apartments.ttl#
    ЗданиеБаронВэй">
  <dbpedia-owl:location
    rdf:resource="http://dbpedia.org/resource/Netherlands"/>
</rdf:Description>
</rdf:RDF>

```

2.3.2.2. RDFa

Один из вариантов использования языка RDF заключается в описании и разметке содержимого веб-страниц в формате HTML. Для этого и введен синтаксис RDFa. Синтаксис RDFa предполагает встраивание RDF-элементов в атрибуты HTML-тегов. Приведем пример веб-страницы, содержащей рекламу квартиры Барон Вэй.

```

<html>
<body>
<H1> Продается квартира Барон Вэй</H1>
Квартира Барон Вэй имеет три спальни. Рассчитана на семью. Расположена в зда-
нии Барон Вэй, на севере Амстердама.
</body>
</html>

```

Эта страница не содержит какого-либо машиночитаемого описания ресурсов. Можно разметить данную страницу с использованием синтаксиса RDFa следующим образом:

```

<html
  xmlns:dbpedia="http://dbpedia.org/resource/"
  xmlns:dbpediaowl="http://dbpedia.org/ontology/"
  xmlns:swp="http://www.semanticwebprimer.org/ontology/apartments.ttl#"
  xmlns:geo="http://www.geonames.org/ontology#">
<body>

```

```

<h1> Продается квартира Барон Вэй </h1>

<div about="[swp:КвартираБаронВэй]">
Квартира Барон Вэй имеет <span property="swp:имеетКоличествоСпален">3</span>
спальни. Рассчитана на семью. Расположена в <span
rel="swp:являетсяЧастью" resource="[swp:ЗданиеБаронВэй]">здании Барон
Вэй</span>

<div about="[swp:ЗданиеБаронВэй]">
Здание расположено на севере Амстердама.
  <span rel="dbpediaowl:location" resource="[dbpedia:Amsterdam]"></span>
  <span rel="dbpediaowl:location" resource="[dbpedia:Netherlands]"></span>
</div>

</div>
</body>
</html>

```

Эта разметка содержит такое же RDF-описание ресурсов, как и описание в синтаксисе Turtle, приведенное выше. RDF-утверждения кодируются в тегах, таких как `<span>`, `<p>` и `<link>`. RDF-элементы не воспроизводятся браузерами при отображении HTML-страницы. Пространства имен так же, как и в синтаксисе RDF/XML, кодируются с использованием объявления `xmlns`. В некоторых случаях нужно использовать скобки для информирования парсера о том, что используются префиксы. Субъекты определяются с помощью атрибута `about`. Свойства идентифицируются с помощью либо атрибута `rel`, либо атрибута `property`. Атрибут `rel` используется в тех случаях, когда объект утверждения является ресурсом, а атрибут `property` – когда объект утверждения является литералом. Свойства связываются с субъектами утверждений с использованием иерархической структуры HTML-документа.

Каждый из синтаксисов языка RDF, представленных выше, полезен для решения различных задач. Однако важно понимать, что все эти синтаксисы, несмотря на различия, реализуют одну и ту же модель данных и семантику. Таким образом, были рассмотрены способы записи утверждений о сущностях, определяемых с помощью URL. Но что означают эти утверждения? Как компьютер должен интерпретировать эти утверждения? Эти вопросы обсуждаются в следующем разделе, где будет введен язык схем для модели данных RDF.

2.4. RDFS: добавление семантики

RDF является универсальным языком, который позволяет пользователям описывать ресурсы, используя свои собственные словари. RDF не рассчитан на использование в какой-то конкретной предметной области и не определяет семантику какой-либо предметной области. Для того чтобы задать семантику предметной области, разработчик или пользователь RDF должен определить, что именно означают используемые им термины. Это осуществляется посредством основных предметно-независимых структур, определённых в языке RDF Schema.

2.4.1. Классы и свойства

Каким образом можно описать конкретную предметную область? В качестве предметной области рассмотрим предметную область – аренда квартир. Прежде всего необходимо определить сущности, о которых мы хотим делать высказывания. И здесь нужно сделать первое фундаментальное различие таких сущностей. С одной стороны, необходимо описывать конкретные квартиры, такие как квартира Барон Вэй, и их конкретные места расположения, такие как город Амстердам. И это можно сделать с помощью языка RDF.

Но также необходимо описать квартиры, здания, страны, города и т. д. В чем же заключается разница? В первом случае мы говорим об *отдельных объектах* (ресурсах), а во втором мы говорим о *классах*, определяющих типы этих объектов.

Класс можно рассматривать как множество элементов. Отдельные объекты, которые принадлежат к классу, называются *экземплярами* этого класса. Язык RDF позволяет определить отношение между экземплярами классов и классами с помощью специального свойства `rdf:type`.

Использование классов имеет очень важное значение с точки зрения создания ограничений на утверждения в RDF-документе за счет использования схем. Здесь можно провести аналогию с языками программирования, в которых *типизирование* используется как механизм предотвращения бессмысленных операторов (например, таких как $A + 1$, где A – массив; данное выражение некорректно, поскольку предполагается, что аргументы операции $+$ должны быть числами). Подобный механизм нужен и для языка RDF. Необходимо запретить такие утверждения, как:

Квартира Барон Вэй арендует Джеффа Мейера,
Амстердам имеет количество спален 3.

Первое утверждение бессмысленно, так как квартира не может арендовать человека. Это накладывает ограничение на значения свойства «арендует». Говоря математическим языком, мы ограничиваем область значения – *диапазон* свойства (*range*).

Второе утверждение бессмысленно, потому что города не имеют спален. Это накладывает ограничение на объекты, к которым может быть применено свойство. Говоря математическим языком, мы ограничиваем область значений – *домен* свойства (*domain*).

2.4.2. Иерархии классов и наследование

После определения классов следует установить отношения между ними. Например, предположим, что у нас имеются классы для обозначения следующих типов сущностей:

- помещение;
- жилое помещение;
- производственное помещение;
- дом и квартира;
- офис.

Эти классы связаны друг с другом. Например, каждое жилое помещение является помещением. В этом случае можно утверждать, что класс «жилое помещение» является *подклассом* (subclass of) класса «помещение», или, что эквивалентно, класс «помещение» является *надклассом* (superclass of) класса «жилое помещение». Отношения типа «класс–подкласс» определяют иерархию классов, представленную на рис. 2.5. В общем случае класс *A* является подклассом класса *B*, если каждый экземпляр класса *A* является также и экземпляром класса *B*. Язык RDF Schema не требует, чтобы классы образовывали строгую иерархию. Другими словами, граф, отображающий отношения «класс–подкласс», например граф, представленный на рис. 2.5, не должен быть деревом. Класс может иметь несколько надклассов. Если класс *A* является подклассом как класса *B*₁, так и класса *B*₂, это просто означает, что каждый экземпляр класса *A* является одновременно экземпляром и класса *B*₁, и класса *B*₂.

Иерархическая организация классов имеет очень важное практическое значение. Рассмотрим следующее ограничение на диапазон свойства:

Люди могут арендовать только жилые помещения.

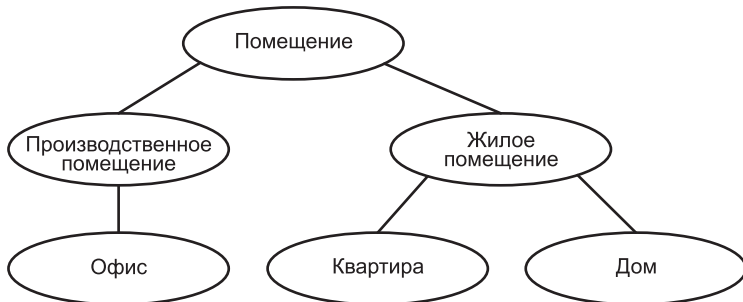


Рис. 2.5 ❖ Иерархия классов

Пусть квартира Барон Вэй определена как экземпляр класса «квартира». Она не квалифицируется как жилое помещение, так как отсутствует явное утверждение о том, что квартира Барон Вэй является жилым помещением, и, следовательно, не подпадает под указанное ограничение. Для решения данной проблемы было бы нелогично добавлять утверждение о том, что квартира Барон Вэй является экземпляром класса «жилое помещение». Вместо этого следует указать, что квартира Барон Вэй *наследует* способность быть арендованной от класса «жилое помещение». Именно это и происходит при использовании языка RDF Schema.

Таким образом, язык RDF Schema фиксирует семантику отношения «класс–подкласс», и именно эта семантика используется всеми приложениями, работающими с RDF-документами. Так как язык RDFS позволяет записывать подобные семантические определения, то можно говорить о том, что он является языком для определения семантики конкретных предметных областей (хотя и достаточно ограниченным). Другим словами, RDF Schema – это примитивный *язык онтологий*.

Термины «класс», «наследование», «свойство» известны и в других областях компьютерных наук, например в объектно-ориентированном программировании. Но, несмотря на сходство значений данных терминов, есть и отличия. В объектно-ориентированном программировании объект класса характеризуется набором свойств, которые определены в этом классе. Добавление новых свойств к классу означает его модификацию.

А в языке RDFS свойства определяются в глобальном масштабе. То есть они не инкапсулируются в качестве атрибутов в определениях классов. Допускается определение новых свойств, которые

могут применяться к существующему классу без изменения самого класса.

С одной стороны, это мощный механизм, применение которого имеет важные следствия: мы можем использовать классы, определенные другими пользователями, и адаптировать их к нашим требованиям с помощью создания новых свойств. С другой стороны, такая трактовка свойств отличается от стандартного подхода, который сложился в области моделирования и объектно-ориентированного программирования. Это еще одна отличительная особенность языков RDF/RDFS.

2.4.3. Иерархии свойств

В предыдущем разделе было показано, что между классами могут быть определены иерархические отношения. То же самое можно делать и для свойств. Например, свойство «арендует» является *подсвойством* (suprproperty) свойства «проживает в». Если человек p арендует жилое помещение r , то человек p также и проживает в жилом помещении r . Обратное не всегда верно. Например, человек p может быть ребенком, проживающим в квартире с семьей, или просто гостем, который не платит арендную плату.

В общем случае если некоторое свойство P является подсвойством Q , то если выполняется $P(x, y)$, тогда выполняется и $Q(x, y)$.

2.4.4. Различие слоев RDF и RDFS

В заключение проиллюстрируем разницу слоев RDF и RDF Schema на простом примере. Рассмотрим следующее RDF-утверждение:

Джефф Мейер арендует Квартиру Барон Вэй.

Схема для этого утверждения может содержать такие классы, как «человек», «квартира», «дом», «помещение», и такие свойства, как «арендует», «проживает в», «адрес». Рисунок 2.6 иллюстрирует слои языков RDF и RDF Schema для данного примера. На этом рисунке в прямоугольных блоках указаны свойства, в овалах над пунктирной линией – классы, а в овалах под пунктирной линией – экземпляры классов.

На рис. 2.6 схема представлена с использованием элементов формального языка RDF Schema, таких как `subClassOf`, `Class`, `Property`, `subPropertyOf`, `Resource` и т. д. Ниже язык RDF Schema будет описан более подробно.

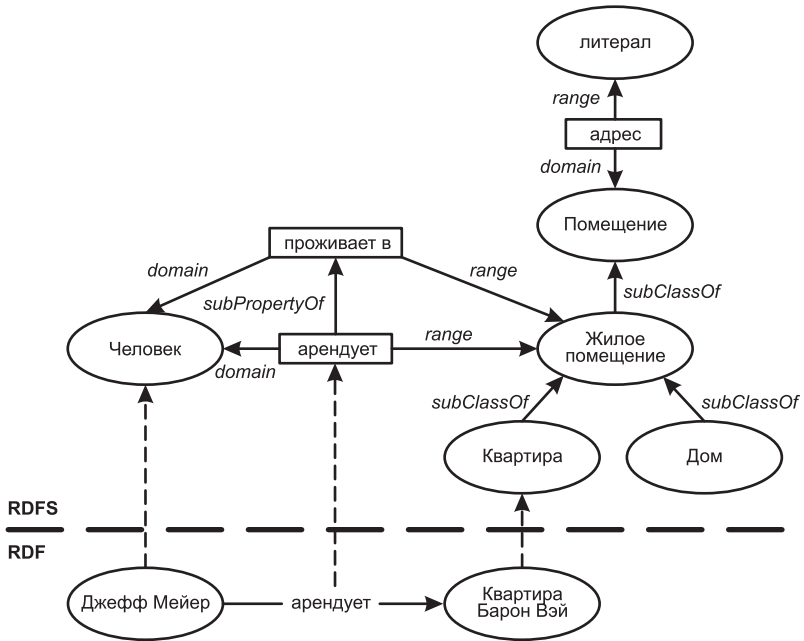


Рис. 2.6 ❖ Слои RDF и RDFS

2.5. RDF Schema: язык

Язык RDF Schema предоставляет различные примитивы моделирования для выражения информации, описанной в разделе 2.4. Однако важно понимать, как именно используются эти примитивы. Вас не должно удивлять, что для записи выражений на языке RDF Schema будет использоваться модель данных RDF: примитивы моделирования языка определяются с помощью ресурсов и свойств. Такой подход становится понятен, если еще раз посмотреть на рис. 2.6.

Мы описали этот рисунок как изображение иерархии классов и свойств, а также экземпляров классов, но фактически он представляет собой помеченный граф, который может быть закодирован на языке RDF. Напомним, что язык RDF позволяет выразить любое утверждение о любом ресурсе и что любая сущность, идентифицируемая URI, может рассматриваться как ресурс. Таким образом, если требуется выразить тот факт, что класс «квартира» является подклассом класса «жилое помещение», то это можно сделать следующим образом:

- 1) определить необходимые ресурсы: квартира, жилое помещение и `subClassOf`;
- 2) определить `subClassOf` как свойство;
- 3) записать триплет: квартира `subClassOf` жилое помещение.

Все эти шаги можно реализовать средствами языка RDF. Таким образом, RDFS-документ представляет собой RDF-документ, который может быть записан с помощью одного из известных синтаксисов RDF.

Ниже определяются примитивы моделирования языка RDF Schema.

2.5.1. Основные классы

К основным классам языка RDFS относятся следующие классы:

- `rdfs:Class` – класс всех классов;
- `rdfs:Resource` – класс всех ресурсов;
- `rdfs:Literal` – класс всех литералов;
- `rdfs:Datatype` – ~~это~~ класс типов данных;
- `rdf:Property` – класс всех свойств;
- `rdf:Statement` – класс всех реифицированных утверждений.

2.5.2. Основные свойства для определения отношений

К основным свойствам языка RDFS, которые предназначены для определения отношений между ресурсами, относятся следующие свойства.

Свойство `rdf:type` связывает ресурс и класс, к которому он относится (см. раздел 2.4.1), то есть с помощью этого свойства ресурс объявляется как экземпляр конкретного класса.

Свойство `rdfs:subClassOf` связывает класс с одним из его надклассов. Все экземпляры класса являются также и экземплярами его надкласса. Подчеркнем, что класс может быть подклассом более чем одного класса. Например, класс ПрофессорЖенщина может быть подклассом и класса Женщина, и класса Профессор.

Свойство `rdfs:subPropertyOf` связывает свойство с одним из своих надсвойств.

В качестве примера приведем утверждение о том, что любая квартира является жилым помещением:

```
swp:Квартира rdfs:subClassOf swp:ЖилоеПомещение.
```

Заметим, что свойства `rdfs:subClassOf` и `rdfs:subPropertyOf` являются по определению транзитивными. Кроме того, подчеркнем интересные факты: класс `rdfs:Class` является подклассом класса `rdfs:Resource` (то есть каждый класс представляет собой ресурс), а класс `rdfs:Resource` является экземпляром класса `rdfs:Class` (`rdfs:Resource` – это класс всех ресурсов, то есть это класс!). По той же причине каждый класс является экземпляром класса `rdfs:Class`.

2.5.3. Основные свойства для определения ограничений

К основным свойствам языка RDFS, которые используются для определения ограничений на другие свойства, относятся следующие.

Свойство `rdfs:domain` определяет домен свойства P и означает, что любой ресурс, имеющий свойство P , является экземпляром класса, указанного в качестве домена.

Свойство `rdfs:range` определяет диапазон свойства P и означает, что значения свойства P являются экземплярами класса, указанного в качестве диапазона.

В качестве примера приведем утверждения о том, что если некоторый ресурс имеет адрес, то этот ресурс является помещением (что может быть заключено путем логического вывода), и что значением адреса является литерал:

```
swp:адреса rdfs:domain swp:Помещение.  
swp:адрес rdfs:range rdf:Literal.
```

2.5.4. Свойства, используемые для реификации

Ниже приведены некоторые полезные свойства для реификации RDF-утверждений:

- Свойство `rdf:subject` связывает реифицированное утверждение с его субъектом.
- Свойство `rdf:predicate` связывает реифицированное утверждение с его предикатом.
- Свойство `rdf:object` связывает реифицированное утверждение с его объектом.

2.5.5. Контейнерные классы

Кроме того, язык RDF позволяет представить контейнеры стандартным образом. К контейнерам в языке относят пакеты, последовательности и альтернативы (то есть выбор):

- `rdf:Bag` – класс пакетов;
- `rdf:Seq` – класс последовательностей;
- `rdf:Alt` – класс альтернатив;
- `rdfs:Container` – суперкласс всех контейнерных классов, в том числе трех предыдущих.

2.5.6. Вспомогательные свойства

Ресурс может быть определен и описан в вебе многократно. Следующие свойства позволяют определить ссылки на адреса ресурса:

- Свойство `rdfs:seeAlso` связывает ресурс с другим ресурсом, содержащим любую информацию о нем.
- Свойство `rdfs:isDefinedBy` является подсвойством свойства `rdfs:seeAlso` и связывает ресурс с другим ресурсом, который определяет его, как правило, с помощью RDF-схемы.

Часто бывает полезно представить более подробную информацию о ресурсе, предназначенную для чтения людьми. Это может быть сделано с помощью следующих свойств.

- Свойство `rdfs:comment` задает для ресурса комментарий, то есть относительно большой фрагмент текста.
- Свойство `rdfs:label` задает для ресурса человекочитаемую метку (имя). Кроме прочего, данная метка может использоваться в качестве имени узла в графовом представлении RDF-документа¹.

2.5.7. Пример: жилье

В качестве примера рассмотрим предметную область – жилье – и представим концептуальную модель данной предметной области, то есть онтологию².

¹ Для задания многоязычных имен в свойствах `rdfs:label` и `rdfs:comment` можно использовать теги. Например, `rdfs:label "Apartment"@en, "Appartement"@nl, "Квартира"@ru`. – *Прим. перев.*

² Хотя не существует формальных правил именования классов и свойств, рекомендуется использовать следующие походы. Называть класс следует именем существительным в единственном числе. При этом желательно, чтобы имя класса начиналось с прописной буквы и не содержало пробелов, например *Жилое Помещение*. Альтернативным способом наименования классов может быть разделение слов в имени класса знаком подчеркивания, например *Жилое Помещение*. Имя свойства рекомендуется начинать с глагола со строчной буквы, а остальные слова в имени свойства начинать с большой буквы и указывать без пробелов, например *проживаетВ*. Не важно, какую из предложенных нотаций для именования классов и свойств вы будете использовать, важно придерживаться в именовании имен одной выбранной нотации. – *Прим. перев.*

```
@prefix swp: <http://www.semanticwebprimer.org/ontology/apartments.ttl#>.
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>.
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#>.
```

```
swp:Человек rdf:type rdfs:Class.
swp:Человек rdfs:comment "Класс людей"@ru.
```

```
swp:Помещение rdf:type rdfs:Class.
swp:Помещение rdfs:comment "Часть пространства здания или другого объекта не-
движимого имущества, выделенная для самостоятельного использования"@ru.
```

```
swp:ЖилоеПомещение rdf:type rdfs:Class.
swp:ЖилоеПомещение rdfs:subClassOf swp:Помещение.
swp:ЖилоеПомещение
    rdfs:comment "Класс всех помещений, в которых живут люди"@ru.
```

```
swp:Квартира rdf:type rdfs:Class.
swp:Квартира rdfs:subClassOf swp:ЖилоеПомещение.
swp:Квартира rdfs:comments "Класс квартир"@ru.
```

```
swp:Дом rdf:type rdfs:Class.
swp:Дом rdfs:subClassOf swp:ЖилоеПомещение.
swp:Дом rdfs:comment "Класс домов"@ru.
```

```
swp:проживаетB rdf:type rdfs:Property.
swp:проживаетB rdfs:comment "Связывает человека и место его проживания"@ru.
swp:проживаетB rdfs:domain swp:Человек.
swp:проживаетB rdfs:range swp:ЖилоеПомещение.
```

```
swp:арендует rdf:type rdfs:Property.
swp:арендует rdfs:comment "Свойство наследует домен (swp:Человек)
и диапазон (swp:ЖилоеПомещение) от своего надсвойства (swp:проживаетB)"@
ru.
swp:арендует rdfs:subPropertyOf swp:проживаетB.
```

```
swp:адрес rdf:type rdfs:Property.
swp:адрес rdfs:comment "Это свойство помещений, имеющее литеральное значе-
ние"@ru.
swp:адрес rdfs:domain swp:Помещение.
swp:адрес rdfs:range rdf:Literal.
```

2.5.8. Пример: автотранспорт

В данном разделе представлена простая онтология автотранспор-та. Отношения между классами данной онтологии представлены на

рис. 2.7.

```
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
```

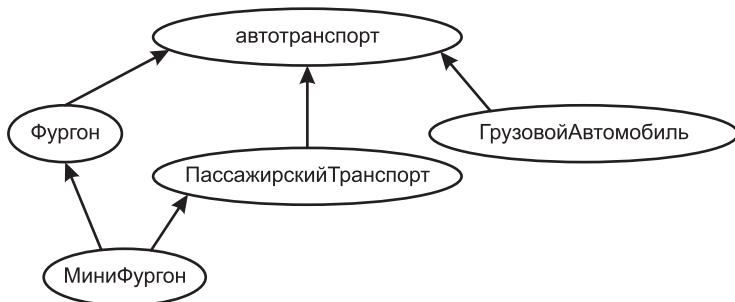
```
<#МиниФургон> a rdfs:Class ;
  rdfs:subClassOf <#ПассажирскийТранспорт >, <#Фургон> .
```

```
<#Автотранспорт> a rdfs:Class .
```

```
<#ПассажирскийТранспорт> a rdfs:Class ;
  rdfs:subClassOf <#Автотранспорт> .
```

```
<#ГрузовойАвтомобиль> a rdfs:Class ;
  rdfs:subClassOf <#Автотранспорт > .
```

```
<#Фургон> a rdfs:Class ;
  rdfs:subClassOf <#Автотранспорт > .
```

**Рис. 2.7** ❖ Иерархия классов для онтологии автотранспорта

2.6. Формальные определения языков RDF и RDF Schema

После изучения основных элементов языков RDF и RDF Schema полезно ознакомиться с их формальными определениями. Эти определения выражены на языке RDF Schema. Проанализируйте, насколько легко они читаются, за счет ~~чего~~ значение каждого из компонентов было разъяснено в предыдущих разделах.

Следующие определения представляют собой лишь часть полной спецификации языка. Остальные части можно найти в пространствах

имен, указанных в теге `rdf:RDF`. Мы предоставляем их в оригинальном синтаксисе XML.

2.6.1. RDF

```
<?xml version="1.0" encoding="UTF-16"?>
<rdf:RDF
  xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  xmlns:rdfs="http://www.w3.org/2000/01/rdf-schema#">

  <rdfs:Class rdf:ID="Statement"
    rdfs:comment="The class of triples consisting of a
      predicate, a subject and an object
      (that is, a reified statement)"/>

  <rdfs:Class rdf:ID="Property"
    rdfs:comment="The class of properties"/>

  <rdfs:Class rdf:ID="Bag"
    rdfs:comment="The class of unordered collections"/>

  <rdfs:Class rdf:ID="Seq"
    rdfs:comment="The class of ordered collections"/>

  <rdfs:Class rdf:ID="Alt"
    rdfs:comment="The class of collections of alternatives"/>

  <rdf:Property rdf:ID="predicate"
    rdfs:comment="Identifies the property used in a statement
      when representing the statement in reified form">
  <rdfs:domain rdf:resource="#Statement"/>
  <rdfs:range rdf:resource="#Property"/>
  </rdf:Property>

  <rdf:Property rdf:ID="subject"
    rdfs:comment="Identifies the resource that a statement is
      describing when representing the statement in reified form">
  <rdfs:domain rdf:resource="#Statement"/>
  </rdf:Property>

  <rdf:Property rdf:ID="object"
    rdfs:comment="Identifies the object of a statement
      when representing the statement in reified form"/>

  <rdf:Property rdf:ID="type"
```

```
    rdfs:comment="Identifies the class of a resource.  
        The resource is an instance of that class."/>  
  
</rdf:RDF>
```

2.6.2. RDF Schema

```
<rdf:RDF  
  xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"  
  xmlns:rdfs="http://www.w3.org/2000/01/rdf-schema#">  
  
  <rdfs:Class rdf:ID="Resource"  
    rdfs:comment="The most general class"/>  
  
  <rdfs:Class rdf:ID="comment"  
    rdfs:comment="Use this for descriptions">  
  <rdfs:domain rdf:resource="#Resource"/>  
  <rdfs:range rdf:resource="#Literal"/>  
</rdfs:Class>  
  
  <rdfs:Class rdf:ID="Class"  
    rdfs:comment="The concept of classes.  
        All classes are resources.">  
  <rdfs:subClassOf rdf:resource="#Resource"/>  
</rdfs:Class>  
  
  <rdf:Property rdf:ID="subClassOf">  
  <rdfs:domain rdf:resource="#Class"/>  
  <rdfs:range rdf:resource="#Class"/>  
</rdf:Property>  
  
  <rdf:Property rdf:ID="subPropertyOf">  
  <rdfs:domain rdf:resource="#rdf:Property"/>  
  <rdfs:range rdf:resource="#rdf:Property"/>  
</rdf:Property>  
  
</rdf:RDF>
```

Указанные пространства имен не обеспечивают полного определения языков RDF и RDF Schema. Рассмотрим, например, свойство `rdfs:subClassOf`. Согласно приведенному выше определению, можно сказать лишь то, что данное свойство применяется к классам и имеет класс в качестве значения. Но смысл языкового примитива «подкласс» (а именно что все экземпляры класса являются также экземплярами его надкласса) никаким образом не зафиксирован. Фактически он

и не может быть выражен в RDF-документе. Если бы это можно было сделать, то отпала бы необходимость в языке RDF Schema.

Формальная семантика языков RDF и RDF Schema будет представлена в следующем разделе. Конечно, RDF-анализаторы и другие программные инструменты для работы с RDF-документами (в том числе процессоры запросов) должны иметь средства работы с этой полной семантикой.

2.7. Аксиоматическая семантика языков RDF и RDF Schema

В этом разделе будет формализован смысл примитивов моделирования языков RDF и RDF Schema и, таким образом, будет полностью задана *семантика* этих языков.

В качестве формального языка описания семантики будем использовать *логику предикатов*, общепризнанный формализм для представления всех (символьных) знаний. Формулы, используемые в логике предикатов, называются *аксиомами*.

За счет описания семантики языков RDF и RDFS с помощью формального языка, такого как логика предикатов, она становится однозначной и пригодной для машинной интерпретации. Кроме того, создается основа для поддержки логического вывода с помощью автоматизированных средств (машин вывода), которые манипулируют логическими формулами.

2.7.1. Общий подход

Все языковые примитивы в языках RDF и RDF Schema представлены константами: *Resource*, *Class*, *Property*, *subClassOf* и т. д. Для выражения отношений между константами используется несколько предопределенных предикатов.

Также используется вспомогательная теория списков, а именно следующие функциональные символы:

- *nil* (пустой список),
- *cons*(*x*, *l*) (добавляет элемент в начало списка),
- *first*(*l*) (возвращает первый элемент списка),
- *rest*(*l*) (возвращает остаток списка), —

и предикатные символы:

- *item*(*x*, *l*) (имеет значение «истина» тогда и только тогда, когда элемент встречается в списке),

- *list(l)* (имеет значение «истина» тогда и только тогда, когда *l* является списком).

Списки используются для представления контейнеров в языке RDF. Они также необходимы для задания смысла некоторых конструкций (например, ограничений кардинальности) в более выразительных языках онтологий.

Большинство аксиом обеспечивают типизирование информации. Например, предикат

Type(subClassOf, Property)

означает, что *subClassOf* – это свойство. Мы используем логику предикатов, включающую символ равенства. Имена переменных начинаются со знака «?». Все аксиомы неявно связаны квантором общности ($\forall$).

Далее приводятся определения большинства элементов языков RDF и RDF Schema. С аксиоматической семантикой этих языков в полном объеме можно ознакомиться в онлайн-документе авторов R. Fikes, D. McGuinness. (см. список рекомендованной литературы).

2.7.2. Основные предикаты

К основным относятся следующие предикаты.

Предикат *PropVal(P, R, V)* с тремя аргументами, который используется для представления RDF-утверждения с ресурсом *R*, свойством *P* и значением свойства *V*.

Предикат *Type(R, T)* представляет сокращенную запись предиката *PropVal(type, R, V)* и определяет, что ресурс *R* имеет тип *T*:

$Type(?r, ?t) \leftrightarrow PropVal(type, ?r, ?t).$

2.7.3. RDF

RDF-утверждение, то есть триплет вида (R, P, V) , представляется как предикат *PropVal(P, R, V)*.

Классы

В языке RDF имеются константы *Class*, *Resource*, *Property* и *Literal*. Все классы являются экземплярами класса *Class*; то есть имеют тип *Class*:

Type(Class, Class)

Type(Resource, Class)

Type(Property, Class)

Type(Literal, Class)

Resource представляет собой наиболее общий класс: каждый объ-

ект является ресурсом. Таким образом, каждый класс и каждое свойство являются ресурсом:

$$Type(?p, Property) \rightarrow Type(?p, Resource)$$

$$Type(?c, Class) \rightarrow Type(?c, Resource)$$

Наконец, предикат в RDF-утверждении должен быть свойством:

$$PropVal(?p, ?r, ?v) \rightarrow Type(?p, Property)$$

Свойство *type*

type – это свойство:

$$Type(type, Property)$$

Обратите внимание, что этот предикат эквивалентен предикату $PropVal(type, type, Property)$: типом *type* является свойство. Свойство *type* может быть применено к ресурсам и имеет класс в качестве значения:

$$Type(?r, ?c) \rightarrow (Type(?r, Resource) \wedge Type(?c, Class))$$

Вспомогательное свойство *FuncProp*

Функциональное свойство – это свойство, которое является функцией: оно связывает ресурс не более чем с одним значением. Язык RDF не включает в себя понятия «функциональное свойство», однако данное понятие используется в аксиоматизации других примитивов.

Константа *FuncProp* представляет класс всех функциональных свойств. Свойство *P* является функциональным тогда и только тогда, когда не существует x, y_1 и y_2 , таких что $P(x, y_1)$, $P(x, y_2)$ и $y_1 \neq y_2$.

$$Type(?p, FuncProp) \leftrightarrow$$

$$(Type(?p, Property) \wedge \forall ?r \forall ?v1 \forall ?v2$$

$$(PropVal(?p, ?r, ?v1) \wedge PropVal(?p, ?r, ?v2) \rightarrow ?v1 = ?v2))$$

Реифицированные утверждения

Константа *Statement* представляет класс всех реифицированных утверждений. Все реифицированные утверждения являются ресурсами, а константа *Statement* является экземпляром класса *Class*:

$$Type(?s, Statement) \rightarrow Type(?s, Resource)$$

$$Type(Statement, Class)$$

Реифицированное утверждение может быть декомпозировано на три части RDF-триплета:

$$Type(?st, Statement) \rightarrow$$

$$\exists ?p \exists ?r \exists ?v (PropVal(Predicate, ?st, ?p) \wedge \\ PropVal(Subject, ?st, ?r) \wedge PropVal(Object, ?st, ?v))$$

Subject, *Predicate* и *Object* представляют собой функциональные свойства. То есть каждое утверждение имеет только один субъект, один предикат и один объект:

$$Type(Subject, FuncProp) \\ Type(Predicate, FuncProp) \\ Type(Object, FuncProp)$$

Причем их типы определяются следующим образом:

$$PropVal(Subject, ?st, ?r) \rightarrow \\ (Type(?st, Statement) \wedge Type(?r, Resource)) \\ PropVal(Predicate, ?st, ?p) \rightarrow \\ (Type(?st, Statement) \wedge Type(?p, Property)) \\ PropVal(Object, ?st, ?v) \rightarrow \\ (Type(?st, Statement) \wedge (Type(?v, Resource) \vee Type(?v, Literal)))$$

Последняя аксиома означает, что если *Object* является свойством в RDF-утверждении, то он должен применяться к реинфицированному утверждению и иметь в качестве значения либо ресурс, либо литерал.

Контейнеры

Все контейнеры являются ресурсами:

$$Type(?c, Container) \rightarrow Type(?c, Resource)$$

Контейнеры являются списками:

$$Type(?c, Container) \rightarrow list(?c)$$

Контейнеры представляют собой пакеты, последовательности или альтернативы:

$$Type(?c, Container) \leftrightarrow \\ (Type(?c, Bag) \vee Type(?c, Seq) \vee Type(?c, Alt))$$

Пакеты и последовательности не пересекаются:

$$\neg (Type(?x, Bag) \wedge Type(?x, Seq))$$

Для каждого натурального числа $n > 0$ существует селектор $_n$, который выбирает n -ый элемент контейнера. Он представляет собой функциональное свойство

$$Type(n, FuncProp)$$

и применяется только к контейнерам:

$$\text{PropVal}(n, ?c, ?o) \rightarrow \text{Type}(?c, \text{Container})$$

2.7.4. RDF Schema

Подклассы и подсвойства

`subClassOf` является свойством:

$$\text{Type}(\text{subClassOf}, \text{Property})$$

Если класс C является подклассом класса C' , то все экземпляры класса C являются также экземплярами класса C' :

$$\begin{aligned} \text{PropVal}(\text{subClassOf}, ?c, ?c') &\leftarrow \\ &(\text{Type}(?c, \text{Class}) \wedge \text{Type}(?c', \text{Class}) \wedge \\ &\forall x(\text{Type}(?x, ?c) \rightarrow \text{Type}(?x, ?c'))) \end{aligned}$$

По аналогии дается определение для свойства `subPropertyOf`: если P является подсвойством свойства P' , то если выполняется $P(x, y)$, тогда выполняется и $P'(x, y)$:

$$\begin{aligned} \text{Type}(\text{subPropertyOf}, \text{Property}) \\ \text{PropVal}(\text{subPropertyOf}, ?p, ?p') &\leftrightarrow \\ &(\text{Type}(?p, \text{Property}) \wedge \text{Type}(?p', \text{Property}) \wedge \\ &\forall ?r ?v(\text{PropVal}(?p, ?r, ?v) \rightarrow \text{PropVal}(?p', ?r', ?v))) \end{aligned}$$

Ограничения

Каждый ресурс-ограничение является ресурсом:

$$\text{PropVal}(\text{subClassOf}, \text{ConstraintResource}, \text{Resource})$$

Свойство-ограничение – это свойство, которое является ресурсом-ограничением:

$$\begin{aligned} \text{Type}(?cp, \text{ConstraintProperty}) &\leftrightarrow \\ &(\text{Type}(?cp, \text{ConstraintResource}) \wedge \text{Type}(?cp, \text{Property})) \end{aligned}$$

Домен и диапазон – это свойства-ограничения:

$$\begin{aligned} \text{Type}(\text{domain}, \text{ConstraintProperty}) \\ \text{Type}(\text{range}, \text{ConstraintProperty}) \end{aligned}$$

Свойства *domain* и *range* определяют, соответственно, домен и диапазон свойств. Напомним, что домен свойства P – это множество всех объектов, к которым применяется свойство P . Если D – домен свойства P , то для любого $P(x, y)$, $x \in D$.

$$\begin{aligned} \text{PropVal}(\text{domain}, ?p, ?d) &\rightarrow \\ \forall ?x \forall ?y(\text{PropVal}(?p, ?x, ?y) &\rightarrow \text{Type}(?x, ?d)) \end{aligned}$$

Диапазон свойства P – это множество всех значений, которые может принимать свойство P . Если R – диапазон свойства P , то для любого $P(x, y)$, $y \in R$.

$$\begin{aligned} & PropVal(range, ?p, ?r) \rightarrow \\ & \forall ?x \forall ?y (PropVal(?p, ?x, ?y) \rightarrow Type(?y, ?r)) \end{aligned}$$

Из вышеуказанных формул могут быть выведены следующие утверждения:

$$\begin{aligned} & PropVal(domain, range, Property) \\ & PropVal(range, range, Class) \\ & PropVal(domain, domain, Property) \\ & PropVal(range, domain, Class) \end{aligned}$$

Таким образом, мы формализовали семантику языков RDF и RDFS. Программное обеспечение, имеющее механизмы работы с данной семантикой, может производить интересные выводы. Например, если указано, что диапазон свойства *арендует* – это класс *ЖилоеПомещение*, а класс *ЖилоеПомещение* – это подкласс класса *Помещение* и что справедлив предикат *арендует(ДжеффМаейр, КвартираБаронВэй)*, то программа, используя семантику логики предикатов или одну из систем доказательства, основанную на логике предикатов, сделает вывод, что справедлив предикат *Помещение(КвартираБаронВэй)*.

2.8. Системы прямого вывода для языков RDF и RDFS

Как показано выше, аксиоматическая семантика, описанная в разделе 2.7, может быть использована для автоматизированного логического вывода в языках RDF и RDF Schema. Однако для этого требуется система доказательств, основанная на логике первого порядка, которая будет осуществлять такой вывод. Это трудновыполнимое требование, или даже вряд ли выполнимое, если описание данных включает в себя миллионы (или миллиарды) утверждений (например, миллионы утверждений вида *Type(?r, ?c)*).

По этой причине семантика языка RDF (и, соответственно, система логического вывода, которая работает на основе данной семантики) выражается не только в терминах логики первого порядка (то есть терминах аксиоматической семантики, представленной в разделе 2.7.), но и непосредственно в терминах RDF-триплетов.

Такая система вывода состоит из правил следующего вида:

ЕСЛИ E содержит определенные триплеты

ТО добавить в E определенные дополнительные триплеты

(где E – произвольный набор RDF-триплетов).

С полным набором правил вывода можно ознакомиться в официальных документах по языку RDF, здесь же приведем только несколько основных примеров:

ЕСЛИ E содержит триплет $(?x, ?p, ?y)$

ТО E также содержит триплет $(?p, \text{rdf:type}, \text{rdf:property})$

~~В ней говорится~~, если произвольный ресурс $?p$ указан в утверждении на позиции свойства, то можно сделать вывод, что он является членом класса `rdf:Property`.

Приведем более интересный пример правила, которое кодирует транзитивность отношения вида «класс–подкласс»:

ЕСЛИ E содержит триплеты $(?u, \text{rdfs:subClassOf}, ?v)$

и $(?v, \text{rdfs:subClassOf}, ?w)$

ТО E также содержит триплет $(?u, \text{rdfs:subClassOf}, ?w)$

С этим правилом тесно связано следующее правило, определяющее точное значение свойства `rdfs:subClassOf`:

ЕСЛИ E содержит триплеты $(?x, \text{rdf:type}, ?u)$

и $(?u, \text{rdfs:subClassOf}, ?v)$

ТО E также содержит триплет $(?x, \text{rdf:type}, ?v)$

Последний пример часто становится неожиданностью для людей, только начинающих изучение языка RDF Schema:

ЕСЛИ E содержит триплеты $(?x, ?p, ?y)$ и $(?p, \text{rdfs:range}, ?u)$

ТО E также содержит триплет $(?y, \text{rdf:type}, ?u)$

Это правило означает, что если произвольный ресурс $?y$ указан в качестве значения свойства $?p$, то можно сделать вывод, что он является членом класса, указанного в качестве диапазона свойства $?p$. Таким образом, определение диапазона в языке RDF Schema используется не для того, чтобы ~~ограничить диапазон~~ свойства, а для того, чтобы *сделать логическое заключение* о членстве указанного в качестве диапазона ресурса в определенном классе.

Общее количество подобных правил замыкания не превышает нескольких десятков. Данные правила могут эффективно выполняться без реализации сложной технологии автоматического доказательства теорем.

2.9. Выводы

- Язык RDF обеспечивает основу для представления и обработки машиночитаемых данных.
- Язык RDF поддерживает графоориентированную модель данных. Ключевые понятия этой модели: ресурс, свойство, утверждение, граф. Утверждение представляет собой триплет вида «ресурс – свойство – значение».
- Язык RDF имеет три стандартных синтаксиса (Turtle, RDF/XML и RDFa)¹ для поддержки синтаксической совместимости.
- В основе языка RDF лежит идея децентрализации знаний, то есть язык позволяет постепенно описывать знания, он рассчитан на совместное и повторное использование знаний.
- RDF является языком, независимым от предметной области. Язык RDF Schema обеспечивает механизм для описания конкретных предметных областей.
- RDF Schema представляет собой примитивный язык онтологий. Он имеет определенные примитивы моделирования с фиксированным значением. Основные понятия языка RDF Schema: класс, отношение вида «класс–подкласс», свойство, отношение вида «свойство–подсвойство», ограничения на домен и диапазон свойств.

Рекомендуемая литература

Ниже приведены некоторые официальные онлайн-документы:

- Adida B. and Birbeck M. RDFa Primer: Bridging the Human and Data Webs. W3C Working Group Note. October 4, 2008 // www.w3.org/TR/xhtml1-rdfa-primer/.
- Beckett D., ed. RDF/XML Syntax Specification (Revised). W3C Recommendation. February 10, 2004 // www.w3.org/TR/rdf-syntax-grammar/.
- Beckett D., Berners-Lee T., and Prud'hommeaux E., eds. Turtle: Terse RDF Triple Language. W3C Working Draft. August 9, 2011 // <http://www.w3.org/TR/turtle/>.

¹ Напомним, что на момент перевода книги стандартизирован уже целый ряд синтаксисов, помимо трех указанных, а именно: N-Quads, N-Triples, TriG, JSON-LD 1.0. См. http://www.w3.org/standards/techs/rdf#w3c_all. – *Прим. перев.*

- Brickley D. and Guha R.V., eds. RDF Vocabulary Description Language 1.0: RDF Schema. W3C Recommendation. February 10, 2004 // www.w3.org/TR/rdf-schema/.
- Fikes R. and McGuinness D. An Axiomatic Semantics for RDF, RDF Schema and DAML+OIL. March 2001 // www.daml.org/2001/03/axiomatic-semantics.html.
- Hayes P., ed. RDF Semantics. W3C Recommendation. February 10, 2004 // www.w3.org/TR/rdf-mt/.
- Klyne G. and Carroll J., eds. Resource Description Framework (RDF): Concepts and Abstract Syntax. W3C Recommendation. February 10, 2004 // www.w3.org/TR/rdf-concepts/.
- Manola F. and Miller E., eds. RDF Primer. W3C Recommendation. February 10, 2004 // www.w3.org/TR/rdf-primer/.
- Prud'hommeaux E. and Seaborne A., eds. SPARQL Query Language for RDF. W3C Candidate Recommendation. June 14, 2007 // www.w3.org/TR/rdf-sparql-query/.

Кроме того, рекомендуем следующий источник:

- Bizer C. and Cyganiak R. SPARQL The Trig Syntax // www4.wiwi.fu-berlin.de/bizer/trig/.

Упражнения и проекты

1. Составьте список из десяти URI, которые идентифицируют сущности в вашем окружении.
2. Приведите несколько примеров того, как можно расширить определение литералов в языке RDF. Почему это было бы полезно?
3. Прочитайте описание пространств имен языка RDFS и попытайтесь понять значение элементов, которые не были представлены в этой главе.
4. Спецификация языка RDFS позволяет для одного свойства определять более одного домена, то есть использовать в качестве домена пересечение нескольких классов. Обсудите плюсы и минусы использования в качестве домена свойств пересечения, а не объединения классов.
5. В старой версии спецификации языка RDFS не допускалось использовать свойство `rdfs:subClassOf` в циклах. Попробуйте представить ситуации, где циклические отношения между классами были бы полезными. (Подсказка: подумайте об экви-

валентности классов.)

6. Обсудите разницу между следующими утверждениями и нарисуйте их графы, чтобы продемонстрировать эту разницу:
 - X поддерживает предложение; Y поддерживает предложение; Z поддерживает предложение.
 - Группа, состоящая из X, Y, Z, поддерживает предложение.
7. Докажите формулы, **выведенные** в конце раздела 2.7, используя предыдущие аксиомы.
8. Обсудите, почему языки RDF и RDF Schema не допускают логических противоречий. Любой RDF/S-документ является непротиворечивым; таким образом, он имеет, по крайней мере, одну модель.
9. Сопоставьте реляционную модель данных (используемую в теории баз данных) и RDF-модель.
10. Сравните модель «сущность–связь» и RDF-модель данных.
11. Запишите на языке RDF Schema модель предметной области «Библиотека», содержащую сущности: книга, автор, издатель, год, копия, дата и т. д. Затем напишите несколько RDF-утверждений, используя синтаксис Turtle. С помощью валидатора (см. <http://librdf.org/parse> или <http://www.rdfabout.com/demo/validator/>) убедитесь, что ваш RDF-документ является синтаксически правильным.
12. Разработайте онтологию для предметной области «География», описывающую сущности: город, страна, столица, граница, государство и т. д.
13. Разработайте небольшую веб-страницу о себе. Определите основные понятия и отношения между ними, используемые на этой странице, и на их основе постройте небольшую онтологию. Если возможно, используйте уже существующие онтологии. Разметьте страницу, используя разработанную онтологию.

При выполнении следующих упражнений поразмышляйте об ограничениях языка RDFS. В частности, о том, что именно в действительности должно быть выражено в онтологии и может ли это быть выражено с помощью языка RDF Schema. Эти ограничения будут рассматриваться в главе 4, где будет представлен более выразительный язык моделирования.

1. Рассмотрите классы «Мужчина» и «Женщина». Назовите отношения между ними, которые должны быть включены в онтологию.
2. Рассмотрите классы «Человек», «Мужчина» и «Женщина». На-

зовите отношения между этими тремя классами, которые должны быть включены в онтологию. Какие из названных отношений могут быть выражены с помощью языка RDF Schema?

3. Предположим, что существуют два утверждения: «Боб это отец Марии» и «Питер это отец Марии». Очевидно, что здесь присутствует семантическая ошибка. Каким образом семантическая модель может предотвратить появление такой ошибки?
4. Как связаны между собой свойства «является ребенком» и «является родителем»?
5. Рассмотрите свойство «ест» с доменом «Животное» и диапазоном «Животное» или «Растение». Допустим, что мы определяем новый класс «Вегетарианец». Назовите ограничение на свойство «ест», которое должно выполняться для этого класса. Как вы думаете, это ограничение может быть выражено с помощью свойства `rdfs:range` языка RDF Schema?

Запросы в семантическом вебе

В предыдущей главе были рассмотрены способы представления знаний с помощью языка RDF. После того как информация представлена в формате RDF, необходимо получить доступ к конкретным частям данной информации с целью осуществления логического вывода и дальнейшей разработки соответствующих приложений. В этой главе будет рассмотрен язык запросов, называемый SPARQL¹, который позволяет выбирать, извлекать знания и осуществлять другие операции с информацией, представленной в RDF-формате. Язык SPARQL разработан специально для запросов к RDF-данным и предполагает использование известных веб-технологий. Если вы знакомы с языками запросов к базам данных, такими как SQL, то заметите много общего между этими языками и языком SPARQL. Однако материал данной главы не предполагает обязательного знакомства читателя с языком SQL и позволяет изучить основы SPARQL «с нуля».

Обзор главы

Данная глава начинается с обсуждения инфраструктуры (то есть программного обеспечения), необходимой для выполнения SPARQL-запросов. Затем описываются основы языка SPARQL и постепенно вводятся более сложные элементы языка. В заключение рассматриваются способы сбора RDF-данных в семантическом вебе.

¹ Фактически SPARQL (The SPARQL Protocol and RDF Query Language) представляет собой и язык запроса к данным в формате RDF, и протокол доступа к этим данным (см. последние версии стандартов <http://www.w3.org/TR/2013/REC-sparql11-query-20130321/> и <http://www.w3.org/TR/2013/REC-sparql11-protocol-20130321/> соответственно). – *Прим. перев.*

3.1. SPARQL-инфраструктура

Для выполнения SPARQL-запросов необходимо соответствующее программное обеспечение. Наиболее распространенным программным обеспечением данного класса являются так называемые хранилища триплетов. По сути, хранилище триплетов представляет собой базу данных в формате RDF. Целый ряд хранилищ триплетов доступен для скачивания онлайн. В спецификации языка SPARQL хранилища триплетов также называются хранилищами графов (Graph Store).

Прежде чем осуществлять запросы к хранилищу триплетов, его необходимо наполнить данными в формате RDF. Большинство хранилищ предоставляют возможность блочной загрузки данных. Существует также механизм, называемый SPARQL Update, с помощью которого можно осуществлять вставку, загрузку и удаление RDF-триплетов в хранилище. Протокол SPARQL Update будет рассмотрен ниже в этой главе.

После того как данные загружены в хранилище триплетов, можно выполнять запросы к ним с использованием протокола SPARQL. Каждое хранилище триплетов обеспечивает SPARQL-точку доступа (SPARQL-endpoint), которая позволяет пользователю отправлять запросы и просматривать результаты их выполнения. Подчеркнем, что клиенты отправляют запросы к SPARQL-точке доступа с использованием протокола HTTP. Фактически можно создать запрос к SPARQL-точке доступа, введя его в адресной строке браузера! Тем не менее мы рекомендуем использовать клиентские приложения, разработанные специально для SPARQL-запросов. Некоторые из таких приложений доступны онлайн.

Так как язык SPARQL использует стандартные веб-технологии, в Сети существует множество SPARQL-точек доступа. Эти точки предоставляют доступ к большим объемам данных. Например, dbpedia.org/sparql обеспечивает точку доступа для запросов к данным проекта Википедия, представленным в RDF-формате. С полным списком SPARQL-точек доступа можно ознакомиться на сайте CKAN.org.

При наличии базовой инфраструктуры можно перейти к написанию SPARQL-запросов.

3.2. Основы SPARQL: сравнение с шаблоном

Напомним, что в предыдущей главе на языке RDF были описаны квартира БаронВэй и ее местоположение:

```
@prefix swp: <http://www.semanticwebprimer.org/ontology/apartments.ttl#>.
@prefix dbpedia: <http://dbpedia.org/resource/>.
@prefix dbpedia-owl: <http://dbpedia.org/ontology/>.
```

```
swp:КвартираБаронВэй swp:имеетКоличествоСпален 3;
                        swp:являетсяЧастью swp:ЗданиеБаронВэй.
swp:ЗданиеБаронВэй dbpedia-owl:location dbpedia:Amsterdam,
                        dbpedia:Netherlands.
```

К этим данным можно делать запросы, например найти местоположение здания. Как подобный запрос выразить с помощью языка SPARQL? Это можно сделать следующим образом. Пусть результат запроса должен соответствовать триплету .

```
swp:ЗданиеБаронВэй dbpedia-owl:location dbpedia:Amsterdam.
```

На языке SPARQL можно просто заменить любой элемент данного триплета переменной. Для обозначения переменных используется знак «?» в качестве первого символа их имени. Вводя переменную для местоположения, можно записать запрос следующим образом:

```
swp:ЗданиеБаронВэй dbpedia-owl:location ?местоположение.
```

Хранилище триплетов получит этот *графовый шаблон* и попытается найти наборы соответствующих ему триплетов. Таким образом, в результате сравнения данного шаблона с данными в RDF-формате будут возвращены значения DBpedia:Amsterdam и DBpedia:Netherlands. Фактически будут найдены все триплеты, где в качестве субъекта указан ресурс swp:ЗданиеБаронВэй, а в качестве предиката – dbpedia-owl:location.

Для написания законченного SPARQL-запроса необходимо сделать несколько дополнений. Во-первых, должны быть определены все префиксы. Во-вторых, необходимо явно указать, какая именно переменная интересует нас в качестве результата запроса. Таким образом, завершённый SPARQL-запрос будет записан следующим образом:

```
PREFIX swp: <http://www.semanticwebprimer.org/ontology/apartments.ttl#>.
PREFIX dbpedia: <http://dbpedia.org/resource/>.
PREFIX dbpedia-owl: <http://dbpedia.org/ontology/>.
```

```
SELECT ?местоположение
WHERE {
    swp:ЗданиеБаронВэй dbpedia-owl:location ?местоположение.
}
```


Как и в синтаксисе Turtle, ключевое слово PREFIX позволяет задать различные аббревиатуры для адресов URL. Ключевое слово SELECT указывает, какие именно переменные необходимо представить как результат запроса. Графовые шаблоны, с которыми будут сравниваться данные, указываются в скобках после ключевого слова WHERE. Результаты запроса возвращаются в виде набора данных, называемых *связками* (binding), которые содержат элементы, соответствующие заданной переменной. Каждая строка приведенной ниже таблицы представляет собой единичный результат запроса или связку. Таким образом, в результате выполнения вышеприведенного запроса мы получим:

?местоположение
http://dbpedia.org/resource/Amsterdam
http://dbpedia.org/resource/Netherlands

Базовая идея SPARQL-запросов очень проста – найти наборы триплетов, которые соответствуют заданному графовому шаблону. При этом язык SPARQL обеспечивает возможность формирования более сложных шаблонов и предоставления результатов в различных форматах; но независимо от того, насколько сложным является графовый шаблон, процедура выполнения запроса основана на той же идее сопоставления данных с шаблоном. Приведем другой пример запроса: найти, где находится КвартираБаронВэй. На языке SPARQL этот запрос записывается следующим образом:

```
PREFIX swp: <http://www.semanticwebprimer.org/ontology/apartments.ttl#>.
PREFIX dbpedia: <http://dbpedia.org/resource/>.
PREFIX dbpedia-owl: <http://dbpedia.org/ontology/>.
SELECT ?местоположение
WHERE {
    swp:ЗданиеБаронВэй swp:являетсяЧастью ?здание.
    ?здание dbpedia-owl:location ?местоположение.
}
```

В данном примере мы расширили графовый шаблон. Сделаем несколько пояснений к данному запросу. Во-первых, имя переменной здесь появляется в качестве субъекта триплета. В общем случае в SPARQL-запросе переменные могут встречаться в любой позиции шаблона. Во-вторых, в запросе дважды используется имя переменной ?building. Таким образом, указывается, что необходимо найти триплеты, в которых объект первого утверждения совпадает с субъектом второго утверждения. Мы оставляем читателю самостоятельно определить, что является результатом вышеприведенного запроса.

В запросе можно указывать несколько переменных. Например, если необходимо в хранилище триплетов найти всю информацию о квартире БаронВэй, можно использовать следующий SPARQL-запрос:

```
PREFIX swp: <http://www.semanticwebprimer.org/ontology/apartments.ttl#>.
PREFIX dbpedia: <http://dbpedia.org/resource/>.
PREFIX dbpedia-owl: <http://dbpedia.org/ontology/>.
SELECT ?p ?o
WHERE {
    swp:КвартираБаронВэй ?p ?o.
}
```

Результат выполнения данного запроса будет представлен следующим образом:

?p	?o
swp:имеетКоличествоСпален	3
swp:являетсяЧастью	swp:ЗданиеБаронВэй

Каждая строка в данной таблице представляет собой отдельный результат запроса, соответствующий графовому шаблону. Для рассматриваемого набора данных, имеющего небольшой объем, легко вернуть все возможные ответы. Однако для больших по объему наборов данных трудно предсказать количество возвращаемых результатов, и возможно в качестве результата запроса получить весь набор данных. На [рис. 3.1](#) составляет труда написать запрос, который может вернуть миллионы триплетов. Таким образом, хорошей практикой является ограничение количества возвращаемых результатов запроса, особенно при использовании публичных SPARQL-точек доступа. Ограничить количество результатов запроса можно с помощью ключевого слова `LIMIT`, как показано в листинге 3.1. В данном листинге количество результатов ограничивается числом 10.

Листинг 3.1 ❖ SPARQL-запрос с ключевым словом `LIMIT`

```
PREFIX swp:<http://www.semanticwebprimer.org/ontology/apartments.ttl#>.
PREFIX dbpedia: <http://dbpedia.org/resource/>.
PREFIX dbpedia-owl: <http://dbpedia.org/ontology/>.
SELECT ?p ?o
WHERE {
    swp:КвартираБаронВэй ?p ?o.
}
LIMIT 10
```

Выше было показано, каким образом можно составлять отдельные графовые шаблоны и цепочки графовых шаблонов. Однако SPARQL обеспечивает и более лаконичный способ записи цепочек свойств. Это средство называется *пути свойств*. Рассмотрим следующий пример: найти все квартиры, которые являются частью зданий, расположенных в Амстердаме.

```
PREFIX swp: <http://www.semanticwebprimer.org/ontology/apartments.ttl#>.
PREFIX dbpedia: <http://dbpedia.org/resource/>.
PREFIX dbpedia-owl: <http://dbpedia.org/ontology/>.
SELECT ?квартира
WHERE {
  ?квартира swp:являетсяЧастью ?здание.
  ?здание dbpedia-owl:location dbpedia:Amsterdam.
}
```

Данный запрос можно записать следующим образом:

```
PREFIX swp: <http://www.semanticwebprimer.org/ontology/apartments.ttl#>.
PREFIX dbpedia: <http://dbpedia.org/resource/>.
PREFIX dbpedia-owl: <http://dbpedia.org/ontology/>.

SELECT ? квартира
WHERE {
  ?квартира swp:являетсяЧастью/dbpedia-owl:location dbpedia:Amsterdam..
}
```

Существуют и другие пути свойств, которые используются для построения длинных запросов и запросов с атрибутами. Большинство из них будет рассмотрено в этой главе. Читатель сможет оценить пользу сокращенных путей свойств при написании более сложных SPARQL-запросов.

Таким образом, с помощью сопоставления графовых шаблонов можно осуществлять простой поиск данных в хранилищах триплетов. Однако иногда необходимо наложить более сложные ограничения на результаты запросов. В следующем разделе будет рассмотрено, каким образом выразить подобные ограничения с помощью фильтров.

3.3. Фильтры

Продолжим рассмотрение примера с квартирами и составим запрос для поиска всех квартир, имеющих три спальни. До сих пор во всех примерах в графовых шаблонах использовались только ресурсы.

Однако в графовом шаблоне можно указывать и литералы. Вышеприведенный запрос на языке SPARQL будет записан следующим образом:

```
PREFIX swp: <http://www.semanticwebprimer.org/ontology/apartments.ttl#>.
PREFIX dbpedia: <http://dbpedia.org/resource/>.
PREFIX dbpedia-owl: <http://dbpedia.org/ontology/>.
SELECT ?квартира
WHERE {
    ?квартира swp:имеетКоличествоСпален 3.
}
```

Обратите внимание, что, как и в синтаксисе Turtle, в SPARQL-запросах допускается указание укороченных форм общих литералов. В данном примере литерал 3 представляет собой сокращенную запись литерала "3"_{xsd:integer}. Различные синтаксические сокращения литералов в синтаксисе Turtle и языке SPARQL совпадают.

Заметим, что этот запрос довольно надуманный. Более вероятно, что потребуются найти квартиры с количеством спален больше или меньше заданного числа. Можно сформулировать подобный запрос на языке SPARQL с использованием ключевого слова FILTER:

```
PREFIX swp: <http://www.semanticwebprimer.org/ontology/apartments.ttl#>.
PREFIX dbpedia: <http://dbpedia.org/resource/>.
PREFIX dbpedia-owl: <http://dbpedia.org/ontology/>.

SELECT ?квартира
WHERE {
    ?квартира swp:имеетКоличествоСпален ?спальни.
    FILTER (?спальни > 2).
}
```

Результат выполнения данного запроса:

?квартира
swp:КвартираБаронВэй

Отношения «меньше, чем», «больше, чем» и «равно» допустимо использовать для числовых типов данных (то есть целых чисел, десятичных дробей), а также для типа дата/время. Язык SPARQL также позволяет накладывать фильтр на строки.

Например, предположим, что наш набор данных содержит триплет:

```
swp:КвартираБаронВэй swp:адрес "4 Барон Вэй Серкл".
```

Пусть требуется составить запрос для поиска всех ресурсов, которые имеют адрес, содержащий строку «4 Барон Вэй». Это можно сделать с помощью поддерживаемых в языке SPARQL регулярных выражений. Регулярные выражения являются мощным инструментом для поиска строк. Подробное описание регулярных выражений выходит за рамки этой книги, но авторы рекомендуют читателям самостоятельно ознакомиться с данной темой. Регулярное выражение для поиска строки «4 Барон Вэй» в начале другой строки записывается как «^4 Барон Вэй». В SPARQL-запросе это будет выражено следующим образом:

```
PREFIX swp: <http://www.semanticwebprimer.org/ontology/apartments.ttl#>.
PREFIX dbpedia: <http://dbpedia.org/resource/>.
PREFIX dbpedia-owl: <http://dbpedia.org/ontology/>.
```

```
SELECT ?квартира
WHERE {
    ?квартира swp:адрес ?адрес.
    FILTER regex(?адрес, "^4 Барон Вэй").
}
```

В данном примере после ключевого слова FILTER указывается имя специальной функции фильтра — `regex`. Параметры этой функции перечислены в круглых скобках. В языке SPARQL можно использовать и другие типы фильтров, полезные для решения частных задач. Однако числовые и строковые фильтры используются чаще всего. Примером еще одной полезной функции является функция `str`. Она преобразует ресурс или литерал в строку, которая затем может быть использована в регулярном выражении. Например, можно искать строку «Барон» непосредственно в URL ресурса, а не в его метке:

```
PREFIX swp: <http://www.semanticwebprimer.org/ontology/apartments.ttl#>.
PREFIX dbpedia: <http://dbpedia.org/resource/>.
PREFIX dbpedia-owl: <http://dbpedia.org/ontology/>.
SELECT ?квартира ?адрес
WHERE {
    ?квартира swp:адрес ?адрес.
    FILTER regex(str(?квартира), "Барон").
}
```

Таким образом, фильтры обеспечивают гибкость при составлении запросов. Кроме того, язык SPARQL содержит конструкции для обработки различной, часто противоречивой информации, которая размещена в семантическом вебе.

3.4. Конструкции, учитывающие предположение об открытости мира

В отличие от традиционных баз данных, однотипные ресурсы в семантическом вебе не обязательно описываются с помощью однотипных схем и могут иметь разные наборы свойств. Данный подход соответствует *предположению об открытости мира* (open world assumption)¹. Например, одни квартиры могут быть описаны более подробно, чем другие. Более того, разные квартиры могут быть описаны с помощью разных словарей. Рассмотрим следующий пример описания на языке RDF:

```
@prefix swp: <http://www.semanticwebprimer.org/ontology/apartments.ttl#>.
@prefix dbpedia: <http://dbpedia.org/resource/>.
@prefix dbpedia-owl: <http://dbpedia.org/ontology/>.
@prefix xsd: <http://www.w3.org/2001/XMLSchema#>.
```

```
swp:КвартираБаронВэй swp:имеетКоличествоСпален 3.
swp:КвартираБаронВэй dbpedia-owl:location dbpedia:Amsterdam.
swp:КвартираБаронВэй refs:label "Квартира Барон Вэй для сдачи в аренду".
```

```
swp:СтудияФлоридаАве swp: имеетКоличествоСпален 1.
swp:СтудияФлоридаАве dbpedia-owl:locationCity dbpedia:Amsterdam.
```

В этом примере ресурс СтудияФлоридаАве не имеет человекочитаемой метки (label), а его местоположение описывается с помощью предиката DBpedia-owl:locationCity, а не DBpedia owl: location. Несмотря на такую несогласованность данных, возможно построить запрос для поиска квартир, расположенных в Амстердаме, и их меток (при их наличии). В языке SPARQL для записи такого запроса могут быть использованы две конструкции. Рассмотрим их на следующем примере ~~запроса~~:

```
PREFIX swp: <http://www.semanticwebprimer.org/ontology/apartments.ttl#>.
PREFIX geo: <http://www.geonames.org/ontology#>.
PREFIX dbpedia: <http://dbpedia.org/resource/>.
PREFIX dbpedia-owl: <http://dbpedia.org/ontology/>.
```

```
SELECT ?квартира ?метка
WHERE {
```

¹ В общем случае предположение об открытости мира означает, что мы не можем предполагать, что чего-то не существует, пока явно не указано, что этого не существует. Другими словами, если нет явного утверждения, что что-либо истинно, это не значит, что это ложно. — Прим. перев.

```
{?квартира dbpedia-owl:location dbpedia:Amsterdam.}
UNION
{?квартира dbpedia-owl:locationCity dbpedia:Amsterdam.}
OPTIONAL
{?квартира rdfs:label ?метка.}
}
```

Результаты выполнения данного запроса:

?квартира	?метка
swp: КвартираБаронВэй	Квартира Барон Вэй для сдачи в аренду
swp: СтудияФлоридаАве	

Ключевое слово UNION означает, что в качестве результата запроса необходимо вернуть данные из хранилища, соответствующие одному или более графовым шаблонам. Ключевое слово OPTIONAL означает, что в качестве результатов запроса могут быть возвращены триплеты, соответствующие указанному графовому шаблону, при их наличии. Другими словами, возвращаемый триплет не обязательно должен соответствовать графовому шаблону, указанному после ключевого слова OPTIONAL. Таким образом, если в приведенном примере не указать это ключевое слово, то ресурс СтудияФлоридаАве не будет включен в результаты выполнения запроса.

Кроме того, для создания более компактных SPARQL-запросов могут быть использованы пути свойств. Использование оператора | позволяет выразить выбор одного или нескольких атрибутов. Таким образом, вышеприведенный SPARQL-запрос может быть переписан следующим образом:

```
PREFIX swp: <http://www.semanticwebprimer.org/ontology/apartments.ttl#>.
PREFIX dbpedia: <http://dbpedia.org/resource/>.
PREFIX dbpedia-owl: <http://dbpedia.org/ontology/>.

SELECT ?квартира ?метка
WHERE {
  {?квартира dbpedia-owl:location|dbpedia-owl:locationCity
   dbpedia:Amsterdam.}
  OPTIONAL
  {?квартира rdfs:label ?метка.}
}
```

Здесь приведены лишь некоторые примеры того, как SPARQL-запросы могут быть использованы для поиска знаний из разных источников.

3.5. Представление результатов запроса

Очень часто возникает потребность в том, чтобы результаты запросов были возвращены особым образом, например сгруппированы, подсчитаны, упорядочены. Язык SPARQL поддерживает ряд функций, которые позволяют таким образом представить результаты запросов. Уже было показано, как можно ограничить количество результатов запроса, используя ключевое слово `LIMIT`. Также можно удалить повторяющиеся триплеты из результатов запроса за счет использования ключевого слова `DISTINCT`, указав его после ключевого слова `SELECT` (например, `SELECT DISTINCT ?name WHERE`). Данная запись запроса гарантирует, что будут возвращены только уникальные связки переменных.

Язык SPARQL также позволяет упорядочить возвращаемый набор результатов с помощью ключевого слова `ORDER BY`. Например, можно запросить список квартир, упорядоченный по количеству спален.

```
PREFIX swp: <http://www.semanticwebprimer.org/ontology/apartments.ttl#>.
```

```
PREFIX dbpedia: <http://dbpedia.org/resource/>.
```

```
PREFIX dbpedia-owl: <http://dbpedia.org/ontology/>.
```

```
SELECT ?квартира ?спальни
```

```
WHERE {
```

```
    ?квартира swp:имеетКоличествоСпален ?спальни.
```

```
}
```

```
ORDER BY DESC(?спальни)
```

Результаты выполнения данного запроса:

?квартира	?спальни
swp:КвартираБаронВэй	3
swp:СтудияФлоридаАве	1

Ключевое слово `DESC` обозначает, что результаты запроса будут представлены в порядке убывания. Для того чтобы представить результаты запроса в порядке возрастания, используется ключевое слово `ASC`. Заметим, что упорядочивание строк или URL происходит в соответствии с алфавитным порядком.

Кроме того, возможно группировать результаты запросов с помощью *агрегатных* функций. В частности, можно подсчитать количество результатов запроса (`COUNT`), сумму значений (`SUM`), а также вычислить минимальное, максимальное и среднее значения (`MIN`, `MAX`, `AVG`). Приведем пример запроса, в котором вычисляется среднее число спален для квартир в нашем наборе данных:


```

PREFIX swp: <http://www.semanticwebprimer.org/ontology/apartments.ttl#>.
PREFIX dbpedia: <http://dbpedia.org/resource/>.
PREFIX dbpedia-owl: <http://dbpedia.org/ontology/>.
SELECT (AVG(?спальни) AS ?среднееКоличествоКомнат)
WHERE {
    ?квартира swp:имеетКоличествоСпален ?спальни.
}

```

Выполнение данного запроса даст следующий результат:

?-среднееКоличествоКомнат
2

Агрегатные функции используются в сочетании с ключевым словом **AS**, после которого указывается имя переменной, обозначающие множество результатов запроса. Группировку результатов запроса можно осуществлять не только с помощью этих агрегатных функций, которые применяются ко всему множеству результатов. Выполнять агрегирование конкретных групп результатов можно также с помощью ключевого слова **GROUP BY**.

Таким образом, SPARQL обеспечивает мощные механизмы для представления результатов запросов определенным образом, удобным для дальнейшего использования в соответствующих приложениях.

3.6. Другие формы SPARQL-запросов

До сих пор рассматривались запросы, позволяющие найти определенные значения в наборе RDF-данных. Но язык SPARQL поддерживает и другие формы запросов. Кроме запроса с ключевым словом **SELECT**, часто используются еще две формы запросов с ключевыми словами **ASK** и **CONSTRUCT**.

Запрос с ключевым словом **ASK** просто проверяет, существуют ли в наборе данных триплеты, соответствующие графовому шаблону, но не возвращает их. Например, нижеприведенный запрос вернет значение «true» (истина).

```

PREFIX swp: <http://www.semanticwebprimer.org/ontology/apartments.ttl#>.
PREFIX dbpedia: <http://dbpedia.org/resource/>.
PREFIX dbpedia-owl: <http://dbpedia.org/ontology/>.

```

```

ASK ?квартира
WHERE {
    ?квартира swp:имеетКоличествоСпален 3.
}

```

Запросы с ключевым словом `ASK` выполняются быстрее, чем аналогичные запросы с ключевым словом `SELECT`.

Форма запроса с ключевым словом `CONSTRUCT` используется для получения RDF-графа из большого набора RDF-данных. Таким образом, можно создавать запрос к хранилищу триплетов не с целью получения связей переменных, а с целью получения RDF-графа. Например, создадим новый граф, который описывает большие квартиры, имеющие более чем 2 спальни:

```
PREFIX ex: <http://www.example.org/>
PREFIX dbpedia: <http://dbpedia.org/resource/>
PREFIX geo: <http://www.geonames.org/ontology#>

CONSTRUCT {?квартира swp:имеетКоличествоСпален ?спальни.
?квартира swp:являетсяБольшойКвартирой true.}
WHERE{
    ?квартира swp:имеетКоличествоСпален ?спальни.
}
FILTER (?спальни > 2)
```

В результате выполнения запроса будет возвращен следующий граф.

```
@prefix swp: <http://www.semanticwebprimer.org/ontology/apartments.ttl#>.
@prefix dbpedia: <http://dbpedia.org/resource/>.
@prefix dbpedia-owl: <http://dbpedia.org/ontology/>.
@prefix xsd: <http://www.w3.org/2001/XMLSchema#>.
```

```
swp:КвартираБаронВэй swp:имеетКоличествоСпален 3.
swp:КвартираБаронВэй swp:являетсяБольшойКвартирой true.
```

Запросы с ключевым словом `CONSTRUCT` часто используются для преобразования между схемами, соответствующими конкретным шаблонам, и подстановками в свойства из целевой схемы.

3.7. Запросы к схемам данных

Важной особенностью языка SPARQL является возможность написания запросов к схемам данным. Эта возможность реализуется за счет того, что схема данных представлена в RDF-формате. Например, следующая схема является частью онтологии домов, приведенной в предыдущей главе:

```
@prefix swp: <http://www.semanticwebprimer.org/ontology/apartments.ttl#>.
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>.
```

```
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#>.
```

```
swp:Помещение rdfs:type rdfs:Class.
swp:ЖилоеПомещение rdfs:type rdfs:Class.
swp:ЖилоеПомещение rdfs:subClassOf swp:Помещение.
swp:Квартира rdfs:type rdfs:Class.
swp:Квартира rdfs:subClassOf swp:ЖилоеПомещение.
```

Язык SPARQL позволяет определить ~~жилое помещение~~ в нашем наборе данных одновременно с помощью запросов как к самим данным, так и к их схеме:

```
PREFIX swp: <http://www.semanticwebprimer.org/ontology/apartments.ttl#>.
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
```

```
SELECT ?квартира
WHERE{
    ?квартира a ?типПомещения.
    ?типПомещения rdfs:subClassOf swp:ЖилоеПомещение.
}
```

Обратите внимание, что для обозначения свойства `rdfs:type` в приведенном запросе, как и в синтаксисе Turtle, использована сокращенная форма записи `a`. Возможность делать запросы к схеме является очень важной особенностью языков SPARQL и RDF, так как это позволяет получить не только данные из хранилища триплетов, но и их семантику.

3.8. Добавление информации с помощью протокола SPARQL Update

Как упоминалось в разделе 3.1, язык SPARQL определяет протокол для обновления контента хранилища триплетов. Этот протокол называется SPARQL Update. Фактически он добавляет несколько новых ключевых слов в язык SPARQL для вставки, загрузки и удаления триплетов. Приведем примеры запросов каждого из этих типов.

Вставка и загрузка триплетов. Следующий запрос вставляет утверждение о том, что `ЭксклюзивнаяКвартира` — это подкласс класса `Квартира`. При этом данный триплет будет добавлен в хранилище триплетов вне зависимости от его текущего контента.

```
PREFIX swp: <http://www.semanticwebprimer.org/ontology/apartments.ttl#>.
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
```

```
INSERT DATA
{
  swp:ЭксклюзивнаяКвартира rdfs:subClassOf swp:Квартира.
}
```

Обратите внимание, что сами данные в запросе записаны с помощью синтаксиса Turtle, который был введен в главе 2.

Большой файл в RDF-формате, доступный через Интернет, можно загрузить в хранилище триплетов с помощью следующей команды:

```
LOAD <http://example.com/apartment.rdf>
```

Удаление триплетов. Существуют два способа удаления триплетов из хранилища. Первый способ заключается в явном указании триплетов, которые требуется удалить, с помощью ключевого слова DELETE DATA. Удаление триплета, добавленного в хранилище выше, выглядит следующим образом:

```
PREFIX swp: <http://www.semanticwebprimer.org/ontology/apartments.ttl#>.
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
```

```
DELETE DATA
{
  swp:ЭксклюзивнаяКвартира rdfs:subClassOf swp:Квартира.
}
```

Обратите внимание, что в данной форме запроса указание переменных не допускается, все элементы триплетов должны быть полностью определены.

Для построения более гибкого запроса на удаление триплетов можно использовать конструкцию DELETE WHERE. В этом случае удаляются триплеты, соответствующие заданному графовому шаблону. Приведенный ниже запрос позволит удалить все триплеты, содержащие информацию о квартирах с более чем двумя спальнями:

```
PREFIX swp: <http://www.semanticwebprimer.org/ontology/apartments.ttl#>.
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
```

```
WHERE{
  ?квартира swp:имеетКоличествоСпален ?спальни.
  FILTER (?спальни > 2)
}
```

При выполнении как первого, так и второго запроса на удаление, если не будут найдены указанные триплеты или триплеты, соответствующие графовому шаблону, ничего не произойдет.

Наконец, чтобы удалить все содержимое хранилища триплетов, может быть использована конструкция CLEAR следующим образом:

```
CLEAR ALL
```

Протокол SPARQL Update поддерживает также несколько конструкций для управления отдельными частями хранилища триплетов. Эти операции особенно полезны, если необходимо постепенно добавлять данные в хранилище. В следующем разделе будет рассмотрен частный случай, для которого удобно применять подобные операции обновления.

3.9. Принцип «Следуй за своим носом»

Язык SPARQL предоставляет возможности для построения запросов и обновления данных в хранилище триплетов. Но как заполнить эти хранилища? Как уже упоминалось ранее, многие поставщики данных предоставляют доступ к данным через SPARQL-точки доступа. Однако некоторые поставщики просто публикуют данные в виде наборов связанных данных. То есть данные в формате RDF могут быть доступны в вебе либо в виде файлов, либо в виде данных, встроенных в веб-страницы. И эти опубликованные триплеты можно вставлять в наше собственное хранилище триплетов. Концепция семантического веба позволяет любому поставщику данных описывать собственную информацию, используя любые другие ресурсы и информацию в вебе. Для этого можно применять принцип «Следуй за своим носом»: пусть известен URL, идентифицирующий некоторый RDF-ресурс, можно получить данные по этому URL и загрузить их в собственное хранилище триплетов. Эти действия можно повторять до тех пор, пока хранилище не заполнится настолько, чтобы предоставлять нужные данные в ответ на определенные запросы.

Некоторые процессоры запросов, такие как SQUIN, реализуют данную функцию (см. <http://squ.in.sourceforge.net/>). Кроме того, новая версия языка SPARQL включает в себя команды для таких федеративных запросов¹. Однако заметим, что выполнение федеративных запросов требует достаточно много времени, поскольку данные собираются из внешних источников именно в момент выполнения запроса.

¹ Под федеративным запросом понимается запрос одновременно к нескольким источникам данных. — *Прим. перев.*

Реализация принципа «Следуй за своим носом» является одним из способов сбора информации, необходимой для обработки запросов в семантическом вебе.

3.10. Выводы

В этой главе были даны основы языка SPARQL, а именно приведены способы составления запросов и способы обновления данных, а также описана базовая инфраструктура, необходимая для реализации SPARQL-запросов.

- SPARQL-запрос выбирает информацию из хранилищ триплетов на основе сопоставления триплетов с заданными в запросе графовыми шаблонами. Язык SPARQL обеспечивает возможность фильтрации данных на основе сравнения числовых и строковых литералов.
- Синтаксис, используемый в SPARQL-запросах, аналогичен синтаксису Turtle.
- С помощью языка SPARQL можно строить запросы как к данным, так и к схемам данных.
- Конструкции языка **UNION** и **OPTIONAL** позволяют строить запросы, учитывающие предположение об открытости мира.
- Протокол SPARQL Update обеспечивает механизмы для обновления и удаления информации в хранилище триплетов.

Рекомендуемая литература

- ARQ SPARQL Tutorial // <http://jena.sourceforge.net/ARQ/Tutorial/>.
- Harris S. and Seaborne A., eds. SPARQL 1.1 Query Language. W3C Working Draft (work in progress), 12 May 2011 // <http://www.w3.org/TR/sparql11-query/>.
- Polleres, Gearon P., and Passant A., eds. SPARQL 1.1 Update. W3C Working Draft (work in progress), 12 May 2011 // www.w3.org/TR/sparql11-update/.
- Torres E., Feigenbaum L., and Clark K., eds. SPARQL Protocol for RDF. W3C Recommendation, 15 January 2008 // www.w3.org/TR/rdf-sparql-protocol/.

Упражнения и проекты

1. Выберите один из запросов, приведенных в этой главе. Нарисуйте схему данных, к которым делается этот запрос, в виде гра-

фа. Нарисуйте вторую диаграмму, помеченную переменными из выбранного запроса. Можно ли выразить запрос также в виде диаграммы?

2. Подумайте о том, в чем заключаются трудности выбора информации из хранилища триплетов в ответ на SPARQL-запросы. Обсудите факторы, влияющие на эти трудности.
3. Сравните языки SPARQL и SQL. Выделите различия и сходные черты этих языков.
4. Выполните несколько запросов к данным проекта <http://www.dbpedia.org>, используя один из предоставленных веб-интерфейсов для выполнения запросов: <http://dbpedia.org/snorql/> или <http://dbpedia.org/sparql>. Обсудите трудности, с которыми вы столкнулись при построении запросов.
5. Загрузите и установите хранилище триплетов. Примерами такого типа программного обеспечения являются 4store, Virtuoso, Sesame и OWLIM. Загрузите в хранилище RDF-данные, доступные на сайте этой книги (www.semanticwebprimer.org), и проверьте выполнение всех запросов, приведенных в этой главе. Если хранилище триплетов поддерживает логический вывод, проверьте, меняются ли ответы на запросы в зависимости от его результатов.
6. Обсудите, каким образом SPARQL использует другие веб-стандарты (например, HTTP).
7. Какое преимущество дает использование онтологий при построении SPARQL-запросов?
8. Разработайте веб-приложение командой в составе от 2 до 4 человек. В качестве основы для данного приложения используйте онтологию из предыдущей главы. Например, можно разработать приложение для поиска и бронирования квартир. Для хранения данных вместо стандартной базы данных используйте хранилище триплетов. Мы рекомендуем использовать JavaScript-библиотеку `rdqQuery` (<http://code.google.com/p/rdq-query/>) для организации взаимодействия между веб-страницей и хранилищем триплетов. Кроме того, для этих целей можно использовать процессор PHP (Hypertext Preprocessor), например RAP (<http://www4.wiwiw.fu-berlin.de/bizer/rdqapi/>). После завершения разработки приложения напишите отчет с указанием достоинств и недостатков использования онтологий и хранилищ триплетов для веб-приложений.

Глава 4

Язык веб-онтологий: OWL2

4.1. Введение

Выразительные возможности языков RDF и RDF Schema, которые обсуждались в предыдущих главах, весьма ограничены с точки зрения представления наших знаний о мире. Упрощенно говоря, возможности языка RDF ограничиваются бинарными предикатами, а возможности языка RDF Schema – иерархиями классов, иерархиями свойств и механизмом доменов и диапазонов свойств. Эти языки разрабатывались прежде всего с учетом требования гибкости.

Однако часто возникает потребность в более выразительном языке для представления более «сложных» знаний, например для представления знаний о том, что каждый человек имеет только одну дату рождения или что человек не может одновременно являться и мужчиной, и женщиной.

Рабочие группы консорциума W3C – Web Ontology Working Group и OWL Working Group¹ – выявили ряд характерных вариантов использования для семантического веба, которые требуют использования языка с гораздо более мощными выразительными возможностями, по сравнению с языками RDF и RDF Schema. В результате был разработан язык веб-онтологий OWL2, основанный на дескрипционных логиках (Description Logics, DL). *Дескрипционные логики* – это семейство логик, созданное специально для представления терминологических знаний, которое имеет собственную богатую историю и особенности, хорошо известные в соответствующем профессиональном сообществе. OWL2 является второй версией языка OWL.

¹ См. www.w3.org/2001/sw/WebOnt/ и http://www.w3.org/2007/OWL/wiki/OWL_Working_Group соответственно.

Обзор главы

В этой главе рассматриваются требования к языку онтологий OWL2 (раздел 4.2) и его взаимосвязь с языками RDF и RDFS (раздел 4.3). В разделе 4.4 подробно описываются различные элементы языка OWL2, а в разделе 4.5 обсуждаются три профиля языка OWL2.

4.2. Требования к языкам онтологий

Как вы уже знаете из предыдущих глав, языки RDF и RDF Schema позволяют описывать классы, или основные понятия – концепты, присущие некоторой предметной области, и обеспечивают совместное использование этих описаний в вебе. Явную формальную спецификацию основных понятий в некоторой предметной области называют *онтологией*¹. Языки, которые служат для представления онтологий, называются, соответственно, *языками онтологий*. Перечислим основные требования к этим языкам: четко определенный синтаксис, формальная семантика, достаточная выразительная мощность, удобство выражения знаний и эффективная поддержка логического вывода.

4.2.1. Синтаксис

Важность четко определенного синтаксиса очевидна и известна из области языков программирования, это необходимое условие для машинной обработки информации. Синтаксис языка является четко определенным, если любое утверждение на этом языке может быть записано однозначным способом. Все языки, представленные ранее в этой книге, имеют четко определенный синтаксис. Язык OWL2 основан на языках на RDF и RDFS и использует расширение их синтаксиса.

Четко определённый синтаксис не обязательно является удобочитаемым. Например, синтаксис RDF/XML, как известно, не обладает свойством читабельности. Однако этот недостаток нельзя считать существенным, так как большинство разработчиков онтологий пользуется не текстовыми редакторами, а специализированными средствами разработки онтологий.

4.2.2. Формальная семантика

Формальная семантика точно описывает значение языка, то есть не допускает субъективно-интуитивных толкований и различных ин-

¹ Напомним, что, согласно определению, данному Томом Грубером в 1993 году, «онтология – явная спецификация концептуализации». См. <http://tomgruber.org/writing/ontolingua-kaj-1993.pdf>. – *Прим. перев.*

терпретаций выражений на этом языке разными людьми (или машинами). Важность формальной семантики хорошо известна, например, в области математической логики.

Сочетание формальной семантики с четко определенным синтаксисом позволяет интерпретировать предложения, записанные с помощью этого синтаксиса, то есть понимать их смысл. Кроме того, формальная семантика позволяет делать выводы о знаниях, выраженных в предложениях. Например, формальная семантика языка RDFS позволяет делать выводы о принадлежности экземпляра к классу. На основе утверждений

```
:x rdf:type :C.  
:C rdfs:subClassOf: D.
```

можно сделать вывод, что `:x` является экземпляром класса `:D`. Свойства `rdfs:domain` и `rdfs:range` также позволяют делать подобные выводы, например утверждения

```
:p rdfs:range :D.  
:x :p :y.
```

дают возможность сделать вывод о том, что справедливо утверждение `:y rdf:type :D`.

4.2.3. Выразительность

К сожалению, выразительная мощность языков RDF и RDFS очень ограничена и часто не позволяет сделать утверждения о чем-либо с необходимой точностью. При разработке онтологий нужно иметь возможность оперировать следующими категориями знаний.

Членство в классе. Как было показано в предыдущих главах, язык RDFS имеет некоторые простые механизмы для определения принадлежности экземпляра к классу, такие как механизм подклассов, домены и диапазоны свойств. Однако более точное описание условий, при которых экземпляр можно считать членом определенного класса, позволит точнее осуществлять логический вывод. Например, если мы заявили, что некоторые пары «свойство–значение» представляют собой достаточное условие для членства в классе `:A`, тогда если экземпляр `:x` удовлетворяет этим условиям, можно сделать вывод, что `:x` является экземпляром класса `:A`. Приведем пример достаточного условия: что-то является теннисным матчем, если это что-то включает в себя, по крайней мере, игроков, ракетки и т. д.

Классификация. Кроме того, необходима возможность использовать условия членства в классе для того, чтобы делать выводы об от-

ношениях между самими классами. Например, простое определение теннисного матча может быть повторно использовано для определения бадминтонного матча.

Эквивалентность и равенство. Очень полезно иметь возможность выразить эквивалентность между классами. Например, класс :Черепаха имеет точно такие же экземпляры, как и класс :Земная_Черепаха, следовательно, эти классы эквивалентны. Кроме того, важна возможность выразить тот факт, что два экземпляра являются одной и той же сущностью, например экземпляры :утренняя_звезда и :вечерняя_звезда представляют собой имена одной и той планеты :венера, и поэтому эти экземпляры являются одним и тем же объектом. Следовательно, язык должен предоставлять возможность выразить подобные утверждения об эквивалентности классов и идентичности сущностей непосредственно с помощью применения формальной семантики к описанию этих классов и экземпляров класса.

Непересекаемость и различие. Также язык должен предоставлять возможность выразить тот факт, что два класса не имеют общих экземпляров (то есть они не пересекаются) или что два экземпляра не являются одной и той же сущностью. Например, классы :Победитель и :Проигравший не пересекаются, а экземпляры :роджер_федерер и :rafaэль_надаль являются разными индивидами.

Логические комбинации классов. Иногда классы необходимо определить через некоторую комбинацию других классов. Очевидно, что отношение «класс–подкласс» не подходит для этих целей. Например, класс :Человек может быть определен как непересекающееся объединение классов :Мужчина и :Женщина.

Локальная область свойств. Свойство `rdfs:range` определяет, что все экземпляры, указанные в качестве диапазона некоторого свойства, например свойства :играют_в, принадлежат к определенному классу. Таким образом, с помощью языка RDFS нельзя объявить ограничения диапазонов свойств в зависимости от контекста. Например, мы не можем выразить тот факт, что теннисисты играют только в теннис, а бадминтонисты – только в бадминтон.

Специальные характеристики свойств. Иногда полезно указать явным образом, что свойство является транзитивным (например, транзитивное свойство :старше_чем), уникальным (например, уникальное свойство :является_матерью_для) или обратным (инверсивным) для другого свойства (например, обратные свойства :ест и :съедается).

Ограничения кардинальности свойств. Иногда необходимо наложить ограничения на количество различных значений, которые

могут принимать свойства. Например, каждый человек имеет ровно двух родителей, а дисциплина преподается, по крайней мере, одним преподавателем.

Согласованность (непротиворечивость). Если существует возможность определять различные отношения между классами, то желательно иметь и возможность определения конфликтов между этими определениями. Предположим, что мы объявили, что класс :Рыба и класс :Млекопитающее являются непересекающимися. В этом случае утверждение о том, что :дельфин является экземпляром и того, и другого класса, очевидно, является ошибкой. Достаточно выразительный язык онтологий позволит обнаружить противоречия такого рода.

Наконец, язык онтологий должен быть максимально удобным для построения утверждений. Например, язык нельзя считать удобным, если необходимо повторять полное определение классов каждый раз, когда мы хотим объявить эквивалентность этих классов.

4.2.4. Поддержка логического вывода

Наличие формальной семантики языка является необходимым условием для *поддержки логического вывода*. Выявление различного рода противоречий в онтологиях может осуществляться автоматически. Наличие автоматического вывода очень важно, так как позволяет проверить правильность онтологии, а именно:

- проверить согласованность онтологии;
- выявить непредусмотренные отношения между классами;
- выявить непредусмотренные классификации сущностей.

Автоматизированная поддержка вывода, по сравнению с «ручным» выводом, позволяет осуществить намного больше проверок такого рода. Это особенно важно при проектировании больших онтологий, в котором участвуют несколько разработчиков, а также при интеграции онтологий и обмене онтологиями из различных источников.

Обеспечение формальной семантики и поддержки логического вывода для языка онтологий возможно за счет его отображения в известные логические формализмы и использования уже существующих для этих формализмов автоматизированных машин вывода.

Очевидно, что необходим язык онтологий с большей выразительной мощностью, чем язык RDF Schema, обладающий перечисленными выше возможностями. При разработке такого языка нужно найти компромисс между выразительной мощностью языка и эффективной поддержкой логического вывода. Вообще говоря, чем богаче логиче-

ский формализм, менее эффективна поддержка логического вывода. При этом выразительно мощный язык может вообще не гарантировать разрешимости логического вывода, другими словами, не гарантировать, что логический вывод будет завершен. Таким образом, необходим компромисс – язык, который, с одной стороны, обладает достаточной выразительной мощностью для представления различного рода знаний, а с другой – может поддерживать эффективный логический вывод.

4.3. Совместимость OWL2 с RDF/RDFS

Язык OWL2 является расширением языка RDF Schema в том смысле, что OWL2 поддерживает определения и значения классов и свойств, приведенные в RDFS (`rdfs:Class`, `rdfs:subClassOf` и т. д.), и добавляет собственные языковые примитивы для поддержки требуемой выразительности. Такой подход в определении языка соответствует идеологии многоуровневой архитектуры семантического веба (см. рис. 1.4).

К сожалению, простое расширение языка RDF Schema не может дать желаемого компромисса между выразительной мощностью и эффективностью логического вывода. Язык RDF Schema имеет несколько очень мощных примитивов моделирования. Такие конструкции этого языка, как `rdfs:Class` (класс всех классов) и `rdfs:Property` (класс всех свойств), очень выразительны, но могут привести к неконтролируемым вычислительным свойствам, если включить эти примитивы в логику языка OWL2.

4.3.1. Две семантики

Полный набор требований к языку онтологий кажется нереализуемым: эффективная поддержка логического вывода невозможна для языка, обладающего выразительностью языка RDF Schema в комбинации с полной логикой. В соответствии с этими соображениями рабочие группы консорциума W3C разделили язык OWL2 на два различных диалекта, каждый из которых имеет собственную семантику. Данные диалекты реализуют различные аспекты полного набора требований к языку онтологий¹.

¹ Первая версия языка OWL включала три диалекта. Третий диалект назывался OWL Lite. Однако этот язык был заменен профилями, которые будут рассмотрены в разделе 4.5.

4.3.1.1. OWL2 Full: семантика, основанная на RDF

Полный язык веб-онтологий называется OWL2 Full. Он использует все языковые примитивы языка OWL2 и позволяет произвольным способом комбинировать примитивы языка OWL с примитивами языков RDF и RDF Scheme. Язык предоставляет возможность изменения смысла предопределенных RDF- или OWL2-примитивов путем применения примитивов языка друг к другу. Например, в языке OWL2 Full возможно наложить ограничение кардинальности на класс всех классов, тем самым ограничивая число классов, которые могут быть описаны в любой онтологии.

Преимущество языка OWL2 Full заключается в том, что он базируется на семантике языка RDF, что позволяет говорить о полной как структурной, так и семантической совместимости с RDF: любой правильный документ на языке RDF является правильным документом на языке OWL2 Full, а любой правильный вывод на языке RDF Schema является правильным выводом и на языке OWL2 Full. Недостаток языка OWL2 Full заключается в том, что его большая выразительная мощность не гарантирует полной (эффективной) поддержки логического вывода.

4.3.1.2. OWL2 DL: прямая семантика

Для обеспечения вычислительной эффективности был предложен второй диалект языка OWL2 – OWL2 DL, который является отображением на дескрипционные логики (description logic, DL). Дескрипционные логики представляют собой подмножество логики предикатов, для которого возможна эффективная поддержка логического вывода. OWL2 DL ограничивает способ использования примитивов языков OWL2, RDF и RDFS. Приведем некоторые из таких ограничений:

- OWL2 DL не позволяет применения примитивов языка OWL2 друг к другу;
- OWL2 DL позволяет определить классы только для нелитеральных ресурсов. Все классы OWL2 DL являются экземплярами класса `owl:Class`, а не класса `rdfs:Class`;
- OWL2 DL строго разделяет свойства, для которых диапазон включает в себя нелитеральные ресурсы, и свойства, для которых диапазон включает в себя литеральные значения. Соответственно, каждое свойство в языке OWL2 DL является экземпляром либо свойства `owl:ObjectProperty`, либо свойства `owl:DatatypeProperty`, но не обоих одновременно;

- в OWL2 DL ресурс не может быть одновременно классом, свойством или экземпляром. Ресурсы могут иметь одинаковые имена (это называется «каламбур»), но при этом они всегда будут рассматриваться как отдельные сущности.

Вышеуказанные ограничения показывают, что язык OWL2 DL эквивалентен по выразительной мощности хорошо изученной дескрипционной логике. На рис. 4.1 изображены иерархические отношения между некоторыми примитивами моделирования языков OWL2 и RDF/RDFS.

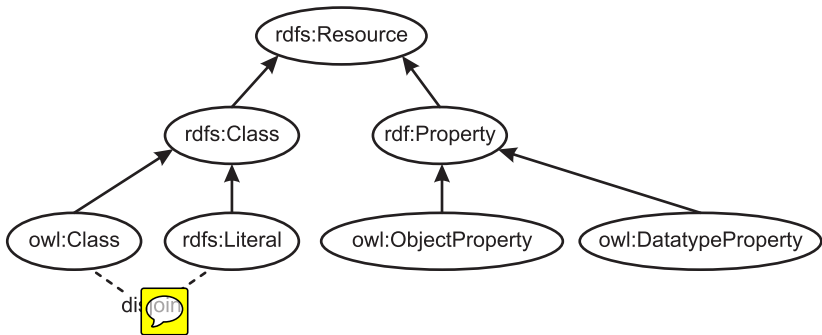


Рис. 4.1 ❖ Иерархические отношения между некоторыми примитивами моделирования языков OWL2 и RDF/RDFS

Преимущество ограниченной выразительности языка OWL2 DL заключается в возможности эффективной поддержки логического вывода. OWL2 DL может использовать многочисленные уже существующие машины логического вывода, такие как Pellet, FaCT, RACER, HermiT. Недостаток языка заключается в потере полной совместимости с RDF. Чтобы документ на языке RDF считался правильным документом на языке OWL2 DL, он должен быть, с одной стороны, расширен, а в другой – удовлетворять некоторым ограничениям. Однако каждый правильный документ на языке OWL2 DL является правильным документом на языке RDF.

Одним из основных преимуществ многоуровневой архитектуры семантического веба (см. рис. 1.4) является нисходящая совместимость и соответствующая возможность повторного использования программного обеспечения для различных слоев. Однако преимущество полной нисходящей совместимости (в том смысле, что любой OWL2-совместимый процессор обеспечит правильные интерпрета-

ции любого документа на языке RDF Schema) достигается только для языка OWL2 Full за счет потери вычислительной эффективности.

В разделе 4.5 будет дан обзор трех дополнительных профилей языка OWL2, представляющих собой различные компромиссы между выразительностью языка и эффективностью логического вывода.

4.4. Язык OWL

В этом разделе вводятся языковые примитивы языка OWL2. Так как язык OWL2 очень близок языку формальной логики, то удобно использовать следующую соответствующую терминологию.

- В языке OWL2 члены классов обычно называются *индивидами*, а не *экземплярами*, но в данной книге эти два термина будут использоваться как взаимозаменяемые.
- Высказывание о том, что некоторый ресурс имеет определённый тип, называется *утверждением*. Например,

```
:роджер_федерер rdf:type :Человек
```

представляет собой утверждение о принадлежности к классу, а именно о том, что индивид :роджер_федерер является членом класса Человек.

- Комбинации классов, свойств и экземпляров записываются в форме *выражений*. Например,

```
_:x rdf:type owl:Class;  
owl:unionOf ( :Мужчина :Женщина ).
```

представляет собой выражение о классе, в котором определяется (анонимное) объединение классов :Мужчина и :Женщина.

- Соотнесение подобного определения с одним из классов называется *аксиомой*. Например,

```
:Человек owl:equivalentClass _:x .  
_:x rdf:type owl:Class ;  
owl:unionOf ( :Мужчина :Женщина ) .
```

представляет собой аксиому эквивалентности классов, которая утверждает, что класс :Человек эквивалентен определенному выше объединению. Аксиомы класса иногда называют *ограничениями*, так как они ограничивают множество индивидов, которые могут являться членами класса.

Подчеркнем, что OWL2, по существу, представляет собой язык для описания множеств сущностей. Эти множества называются *классами*.

ми. Любое утверждение о классе, сделанное на языке OWL2, предназначено для выделения этого класса из множества всех возможных сущностей.

4.4.1. Способы записи языка OWL

Язык OWL2 основан на языках RDF и RDF Schema и поэтому может быть записан с использованием любого допустимого синтаксиса языка RDF. Тем не менее для OWL2 существует и множество собственных синтаксисов, каждый из которых имеет свои преимущества и недостатки.

Функциональный синтаксис. Этот синтаксис тесно связан с формальной структурой онтологий. Он используется в спецификации языка, в определениях семантики OWL2-онтологий, в преобразованиях OWL-данных в RDF-синтаксис и наоборот, в различных профилях языка OWL2. Данный синтаксис является гораздо более компактным и читабельным, чем многие другие. Например, приведенное выше ограничение класса может быть записано в этом синтаксисе следующим образом:

```
EquivalentClasses( :Человек ObjectUnionOf( :Мужчина :Женщина ) )
```

OWL/XML. Это XML-синтаксис для языка OWL2, который не соответствует RDF-соглашениям, но легко преобразуется в функциональный синтаксис¹. Основное преимущество данного синтаксиса – возможность разрабатывать онтологии с использованием стандартных XML-инструментов разработки. Например, рассматриваемая аксиома эквивалентности класса в синтаксисе OWL/XML будет записана следующим образом:

```
<EquivalentClasses>
  <Class abbreviatedIRI=":Человек"/>
  <ObjectUnionOf>
    <Class IRI="#Мужчина"/>
    <Class IRI="#Женщина"/>
  </ObjectUnionOf>
</EquivalentClasses>
```

¹ OWL/XML – сериализация определена на сайте <http://www.w3.org/TR/owl-xml-serialization/>. Не следует путать ее с более ранним презентационным синтаксисом OWL/XML (определенным на сайте <http://www.w3.org/TR/owl-xmlsyntax>), который основан на абстрактном синтаксисе первой версии языка OWL.

Манчестерский синтаксис. Этот синтаксис разработан в Манчестерском университете. Основным требованием при его разработке было требование удобочитаемости. Этот синтаксис используется в пользовательском интерфейсе большинства популярных редакторов онтологий, таких как Protégé. Рассматриваемая аксиома эквивалентности класса в манчестерском синтаксисе будет записана следующим образом:

```
Class: Person
    EquivalentTo: Man or Woman
```

В дополнение к этим синтаксисам для представления языка OWL можно использовать любой синтаксис языка RDF. Поэтому в этой главе мы продолжим использовать синтаксис Turtle, введенный в предыдущих главах.

4.4.2. Документы онтологий

При использовании синтаксиса Turtle документ онтологии на языке OWL2, или просто *онтология*, представляется так же, как и любой другой документ на языке RDF. OWL2-онтология должна включать в себя как минимум следующие пространства имен:

```
@prefix owl: <http://www.w3.org/2002/07/owl#>.
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>.
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#>.
@prefix xsd: <http://www.w3.org/2001/XMLSchema#>.
```

OWL2-онтология начинается с набора вспомогательных утверждений. Эти утверждения вводят базовое пространство имен, собственно онтологию, имя онтологии, возможные комментарии, информацию о контроле версий и включении других онтологий. Например:

```
@prefix : <http://www.semanticwebprimer.org/ontologies/apartments.ttl#> .
@prefix dbpedia-owl: <http://dbpedia.org/ontology/> .
@prefix dbpedia: <http://dbpedia.org/resource/> .
@base <http://www.semanticwebprimer.org/ontologies/apartments.ttl> .
```

```
<http://www.semanticwebprimer.org/ontologies/apartments.ttl>
  rdf:type owl:Ontology ;
  rdfs:label "Онтология квартир"^^xsd:string ;
  rdfs:comment "Пример OWL2-онтологии"^^xsd:string ;
  owl:versionIRI <http://www.semanticwebprimer.org/ontologies/
    apartments.ttl#1.0> ;
  owl:imports <http://dbpedia.org/ontology/> ;
  owl:imports <http://dbpedia.org/resource/> .
```

Импорт. Только два из перечисленных в примере утверждений влияют на содержание онтологии: `owl:imports <http://dbpedia.org/ontology/>` и `owl:imports <http://dbpedia.org/resource/>`. В выражении `owl:imports` указывается онтология, аксиомы которой должны стать частью описываемой онтологии. В нашем примере онтология квартир импортирует все аксиомы, определенные в онтологии проекта DBpedia, а также все данные из проекта DBpedia. Данный пример иллюстрирует основной недостаток конструкции `owl:imports`: для того чтобы использовать часть информации из проекта DBpedia, мы должны импортировать все 672 миллиона триплетов, описанных в нем¹.

Если пространства имен используются только для устранения неоднозначности в именовании ресурсов, то импортированные онтологии обеспечивают определения, которые могут быть использованы в текущей онтологии. Обычно онтология содержит утверждения импорта для каждого используемого пространства имен, но возможно импортировать и дополнительные онтологии, например онтологии, которые обеспечивают определения без введения каких-либо новых имен. Утверждение `owl:imports` транзитивно, то есть если онтология O_i импортирует онтологию O_j , а онтология O_j импортирует онтологию O_k , это означает, что онтология O_i также импортирует онтологию O_k .

4.4.3. Типы свойств

Как упоминалось в разделе 4.3.1, в языке OWL2 различают два типа свойств: *объектные свойства* (object properties) и *свойства типа данных* (datatype properties). На самом деле в OWL2 вводится ряд дополнительных типов свойств для представления различных характеристик. В этом разделе кратко рассматривается каждый из типов свойства языка OWL.

Объектные свойства. Эти свойства связывают одних индивидов с другими. Примеры объектных свойств: `:арендует` и `:живетВ`.

```
:арендует rdf:type    owl:ObjectProperty ;
rdfs:domain           :Человек ;
rdfs:range            :Квартира ;
rdfs:subPropertyOf    :живетВ .
```

Свойства типа данных. Эти свойства связывают индивидов с литеральными значениями, которые имеют определённый тип данных. Примеры свойств типа данных: `:имя` и `:возраст`.

¹ См. текущий статус проекта <http://dbpedia.org/About>.

```
:возраст rdf:type owl:DatatypeProperty ;
rdfs:range xsd:nonNegativeInteger .
```

Так же, как и в языке RDF, в OWL2 можно использовать типы данных XML Scheme для указания типа литерала или диапазона свойств типа данных¹. Кроме того, в XML-схеме могут быть определены пользовательские типы данных, которые затем можно использовать в OWL2-онтологии (см. раздел 4.4.6).

Из описанных ниже типов свойств при описании свойств типа данных в языке OWL2 DL могут быть использованы только *функциональные свойства*. Это обусловлено прямой семантикой данного языка.

Свойства аннотации. Свойства аннотации относятся к свойствам, которые в соответствии с прямой семантикой языка OWL2 DL не имеют смысла. Другими словами, они игнорируются DL-машинами вывода. Тем не менее они будут учтены машинами вывода для языков RDF Schema и OWL2 Full.

Свойства аннотации, как правило, используются для добавления читаемых меток, комментариев или пояснений для OWL2-онтологий, классов, свойств и индивидов. Рассмотрим пример свойства аннотации.

```
:метка rdf:type owl:AnnotationProperty .
rdfs:range rdf:PlainLiteral .
rdfs:subPropertyOf rdf:label
```

```
:Квартира :метка "Apartment"@en,
            "Appartement"@nl,
            "Квартира"@ru .
```

В начале вышеприведенного примера определено свойство :метка с типом owl:AnnotationProperty и диапазоном rdf:PlainLiteral. Тип данных rdf:PlainLiteral представляет собой специальный тип данных языка RDF для обозначения текста на естественном языке, то есть литералы данного типа могут иметь языковой тег. Далее определено, что свойство :метка является подсвойством свойства rdf:label, и приведены три метки для класса :Квартира на английском, голландском и русском языках.

На практике свойства аннотации имеют литеральные значения, но они также могут быть использованы и для связи между нелитеральными ресурсами.

¹ В языке OWL2 введены два дополнительных типа данных owl:real и owl:rational как надтипы для типа **xsd:decimal**.

Свойства Top и Bottom. Все объектные свойства в языке OWL2 являются подсвойствами свойства `owl:topObjectProperty`. Это свойство определяется как свойство, которое связывает все индивиды в онтологии. И наоборот, свойство `owl:bottomObjectProperty` – это свойство, которое не связывает никаких индивидов. Кроме того, свойство `owl:topObjectProperty` связывает всех индивидов с любыми возможными литералами, а свойство `owl:bottomObjectProperty` не связывает ни одного индивида с каким-либо литералом.

Транзитивные свойства. Обсуждая свойство `rdfs:subClassOf`, мы пришли к выводу, что оно транзитивно: каждый класс является подклассом всех надклассов своего непосредственного надкласса. Очевидно, что существуют и другие свойства, которые являются транзитивными, например свойства `:являетсяЧастью` или `:дешевлеЧем`. Определить свойство как транзитивное можно следующим образом:

```
:являетсяЧастью rdf:type owl:ObjectProperty ;
                rdf:type owl:TransitiveProperty .
```

Таблица 4.1. Ограничения на композитные свойства

Какое свойство является композитным?
• Свойство <code>top</code> и свойство <code>bottom</code>. • Любое свойство, которое является транзитивным или имеет обратное свойство, являющееся транзитивным. • Любое свойство, которое имеет транзитивное подсвойство, или подсвойство, обратное к которому является транзитивным. • Любое свойство, которое является надсвойством цепочки свойств или является обратным для надсвойства цепочки свойств. • Любое свойство, которое является эквивалентным одному из вышеперечисленных свойств или является надсвойством свойства, эквивалентного одному из вышеперечисленных свойств. • Композиционные свойства иногда называют <i>сложными ролями</i>, или <i>непростыми свойствами</i>
Ограничения
Композиционные свойства не могут употребляться в следующих аксиомах: • уточненные и неуточненные ограничения кардинальности на классы; • самоограничения на классы; • аксиомы непересекающихся свойств. Кроме того, следующие типы свойств не могут являться композиционными: • функциональные и обратные функциональным свойства; • иррефлексивные свойства; • асимметричные свойства.

Транзитивные свойства являются так называемыми *композиционными* свойствами: они создаются за несколько шагов. Например, на основе утверждений

:КвартираБаронВэй :являетсяЧастью : ЗданиеБаронВэй.
 :КухняБаронВэй :являетсяЧастью : КвартираБаронВэй.

машина вывода заключит, что:

:КухняБаронВэй :являетсяЧастью : ЗданиеБаронВэй.

Последнее отношение :являетсяЧастью составлено на основе двух предыдущих утверждений. Из-за такого рода композитности транзитивные свойства имеют ряд ограничений, перечисленных в табл. 4.1.

Симметричные и асимметричные свойства. Некоторые свойства, такие как :являетсяСмежнымС, являются симметричными, то есть если справедливо утверждение a :являетсяСмежнымС b , то справедливо и обратное утверждение. Другими словами, симметричные свойства эквивалентны их обратным свойствам (см. 4.4.4). Для некоторых свойств, наоборот, симметрия невозможна. Например, отношение :занимаетБолееНизкоеМестоЧем является асимметричным, так как участник соревнований не может быть побежден участником, который ему проиграл¹. Приведем пример описания симметричного и асимметричного свойств.

```
:являетсяСмежнымС rdf:type owl:ObjectProperty ;
                    rdf:type owl:SymmetricProperty .

:занимаетБолееНизкоеМестоЧем rdf:type owl:ObjectProperty ;
                               rdf:type owl:AsymmetricProperty ;
                               rdf:type owl:TransitiveProperty .
```

Функциональные и обратно-функциональные свойства. О некоторых свойствах известно, что каждый индивид связан этим свойством не более чем с одним другим индивидом. Например, свойство :имеетКоличествоКомнат является функциональным свойством, а свойство :имеетКомнату — обратно-функциональным свойством. Приведем пример описания функционального и обратно-функционального свойств.

```
:имеетКоличествоКомнат rdf:type owl:DatatypeProperty ;
                        rdf:type owl:FunctionalProperty .

:имеетКомнату          rdf:type owl:ObjectProperty ;
                        rdf:type owl:InverseFunctionalProperty .
```

¹ Предполагается, что каждая пара участников соревнований сравнивается по результативности в рамках одного матча.

Заметим, утверждение о том, что две квартиры a_1 и a_2 связаны с помощью свойства `:имеетКомнату` с одной и той же комнатной r , необязательно является противоречивым: машина логического вывода заключит, что квартира a_1 и квартира a_2 представляют собой один и тот же индивид.

Рефлексивные и иррефлексивные свойства. Рефлексивность свойства означает, что каждый индивид связан с помощью этого свойства сам с собой. Например, все свойства вида `:являетсяЧастью` являются рефлексивными. Иррефлексивность свойства означает, что ни один индивид не связан с помощью этого свойства сам с собой. Большинство свойств с непересекающимися доменами и диапазонами являются иррефлексивными. Приведем пример описания рефлексивного и иррефлексивного свойств.

```
:являетсяЧастью    rdf:type owl:ObjectProperty ;
                  rdf:type owl:ReflexiveProperty .

:арендует          rdf:type owl:ObjectProperty ;
                  rdf:type owl:IrreflexiveProperty .
```

4.4.4. Аксиомы свойств

В дополнение к различным типам свойств, рассмотренных в предыдущем разделе, возможно указать дополнительные характеристики свойств с точки зрения отношений описываемого свойства с классами и другими свойствами. Некоторые из этих характеристик известны вам из языка RDF Schema, а некоторые являются совершенно новыми.

Домен и диапазон свойств. Как отмечено в разделе 4.4.3, в языке OWL2 домен и диапазон свойств трактуются точно так же, как и в языке RDF Schema. Если свойство описывается более чем одним диапазоном `rdfs:range` или доменом `rdfs:domain`, то фактический диапазон или домен этого свойства представляет собой пересечение классов, указанных в аксиоме свойства.

Существует распространенное заблуждение, что домен и диапазон свойства представляют собой ограничение на типы индивидов, которые могут быть связаны посредством этого свойства. На самом деле домены и диапазоны могут быть использованы только для определения принадлежности к классу этих индивидов. Учитывая приведенное выше определение свойства `:арендует`, любые два индивида $ч$ и $к$ – такие, что $ч$:арендует $к$, – будут классифицироваться как члены классов `:Человек` и `:Квартира` соответственно.

Обратные свойства. Язык OWL2 позволяет определить обратные свойства. Типичным примером обратных свойств является пара свойств `:арендует` и `:арендуется`. Например:

```
:арендуется rdf:type    owl:ObjectProperty ;
              owl:inverseOf :арендует.
```

Данная аксиома позволит машине вывода определить, что если два индивида *ч* и *к* связаны отношением `ч :арендует к`, то они связаны и отношением `к :арендуется ч`, и наоборот. Домен и диапазон свойства наследуются от обратного ему свойства, например свойство `:арендуется` имеет в качестве диапазона класс `:Человек`, а в качестве домена – класс `:Квартира`. В языке OWL2 DL только объектные свойства могут иметь обратные им свойства.

Эквивалентные свойства. Свойства также могут быть определены как эквивалентные: каждые два индивида, связанные с помощью свойства, всегда будут связаны и с помощью эквивалентного ему свойства, и наоборот. Эквивалентность является удобным механизмом для *отображения* элементов различных онтологий друг в друга. Например:

```
:являетсяЧастью rdf:type    owl:ObjectProperty ;
              owl:equivalentProperty dbpedia:partOf .
```

Непересекающиеся свойства. Для некоторых пар свойств известно, что не существует двух индивидов, связанных одновременно с помощью этих свойств. Такие свойства называются непересекающимися. Примером непересекающихся свойств являются свойства `:арендует` и `:владеет`:

```
:арендует rdf:type          owl:ObjectProperty ;
          rdfs:domain        :Человек ;
          rdfs:range          :Квартира ;
          owl:disjointProperty :владеет .
```

Очевидно, что владелец квартиры не может являться ее арендатором. Заметим, что в соответствии с прямой семантикой языка OWL2 DL свойства `owl:ObjectProperty` и `owl:DatatypeProperty` являются непересекающимися.

Цепочки свойств. Более сложной особенностью языка OWL2 является возможность определить цепочки свойств. Иногда полезно явным образом определить короткие пути в графе свойств, связывающих различных индивидов. Например, если известно, что `Паул :арендует :КвартираБаронВэй` и `:КвартираБаронВэй :являетсяЧастью :ЗданиеБаронВэй`, для которого справедливо `ЗданиеБаронВэй dbpedia-owl:location`

dbpedia:Amsterdam, то должно быть справедливо утверждение о том, что :Паул :живетВ :Амстердам. В языке OWL2 мы можем описать такие утверждения с помощью цепочек аксиом свойств:

```
:живетВ rdf:type owl:ObjectProperty ;
    owl:propertyChainAxiom ( :арендует :являетсяЧастью
                                dbpedia-owl:location ) .
```

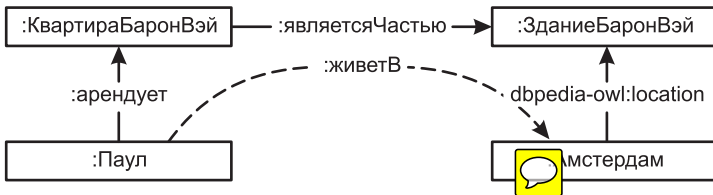


Рис. 4.2. Цепочки свойств
(пунктирной линией обозначены свойства,
полученные машиной вывода)

На рис. 4.2 показано, как из примера с квартирами может быть сделан вывод о существовании свойства :живетВ. Обратите внимание, что цепочка аксиом свойств не позволяет определить именованное свойство (:живетВ) как эквивалентное цепочке свойств; оно будет определено как подсвойство цепочки свойств. В языке OWL2 DL цепочки свойств могут включать только объектные свойства, хотя большинство машин вывода может обрабатывать цепочки, которые в качестве последнего звена содержат свойство типа данных.

Большая выразительная мощность цепочек свойств накладывает на них ряд ограничений. Во-первых, подобно транзитивным свойствам, цепочки свойств являются *композиционными*. Это означает, что они не могут быть использованы в ряде аксиом (см. табл. 4.1). Во-вторых, цепочки свойств не могут быть рекурсивными: надсвойство цепочки, подсвойство цепочки и обратные им свойства не могут использоваться в цепочке аксиом свойств. Например, язык OWL2 DL не позволяет расширить свойство :живетВ следующим образом:

```
:живетВ rdf:type owl:ObjectProperty ;
    owl:propertyChainAxiom ( :арендует :являетсяЧастью dbpedia-owl:location )
;
    owl:propertyChainAxiom ( :живетВ dbpedia-owl:country ) .
```

хотя это позволило бы на основе утверждения :Паул живет в dbpedia:Amsterdam сделать вывод о том, что он живет в dbpedia:Нидерланды.

4.4.5. Аксиомы классов

В языке OWL2 классы определяются с помощью утверждений вида: ресурс имеет тип `owl:Class`. Существуют два предопределенных класса, которые играют важную роль при логическом выводе: `owl:Thing` и `owl:Nothing`. Класс `owl:Thing` является наиболее общим классом: любой индивид в языке OWL2 является членом этого класса, и каждый экземпляр класса `owl:Class` является подклассом этого класса. Класс `owl:Nothing` представляет собой пустой класс: он не имеет членов, и каждый экземпляр класса `owl:Class` является надклассом этого класса. Противоречивые классы не могут иметь членов и, следовательно, являются эквивалентными классу `owl:Nothing`. Подчеркнем, что ограничения на класс `owl:Thing` очень важны при логическом выводе, так как они справедливы для каждого класса и индивида в онтологии.

Отношения «класс–подкласс». Отношение «класс–подкласс» определяется так же, как и в языке RDF Schema. Например, мы можем определить класс `:КвартираКлассаЛюкс` следующим образом:

```
:КвартираКлассаЛюкс rdf:type      owl:Class ;
                        rdfs:subClassOf :Квартира .
```

Эквивалентность классов. Эквивалентность классов означает, что каждый член класса должен являться и членом эквивалентного ему класса, и наоборот. Другими словами, эквивалентные классы описывают одно и то же множество индивидов. Эквивалентность классов может быть определена с использованием свойства `owl:equivalentClass`, например:

```
:Квартира      owl:equivalentClass dbpedia:Apartment .
```

В данном примере утверждается, что класс `:Квартира` из нашей онтологии квартир эквивалентен классу `dbpedia:Apartment`, импортируемому из проекта DBpedia. Утверждение об эквивалентности между двумя классами равносильно утверждению о существовании отношения «класс–подкласс» между этими классами в двух направлениях. Например, приведенное выше утверждение об эквивалентности равносильно следующим утверждениям:

```
:Квартира      rdfs:subClassOf dbpedia:Apartment .
dbpedia:Apartment rdfs:subClassOf :Квартира .
```

Каламбур, или игра слов. Как вы могли заметить, определение класса `dbpedia:Apartment` относится к пространству имен `dbpedia`, а не `dbpedia-owl`. На самом деле `dbpedia:Apartment` является индивидом, а не классом.

По сравнению с нашей онтологией, DBPedia описывает квартиры на более высоком уровне абстракции. Классы в онтологии DBPedia предназначены для классификации отдельных тематических разделов, а не для классификации отдельных сущностей (например, квартир в Амстердаме). Рассмотрение индивидов в качестве классов называется *метамоделированием*.

Хотя прямая семантика языка OWL2 не допускает метамоделирования, язык OWL2 DL обходит это ограничение с помощью синтаксического трюка под названием *каламбур (punning)*, или «игра слов». Это означает, что всякий раз, когда URI ресурса `dbpedia:Apartement` встречается в аксиоме класса, данный ресурс рассматривается как класс, а когда URI этого же ресурса встречается в отдельном утверждении, то ресурс рассматривается как индивид.

Для каламбура действует следующее правило: имена классов, имен индивидов и имена свойств могут свободно взаимозаменяться. Однако имена объектных свойств и свойств типа данных нельзя взаимозаменять.

Перечисления. Самый простой (хотя невыразительный и вычислительно дорогой) способ определения класса заключается в явном перечислении всех его индивидов, например:

```
:КомнатыБаронВэй rdf:type      owl:Class;
                    owl:oneOf  ( :КухняБаронВэй
                                   :Спальня1БаронВэй
                                   :Спальня2БаронВэй
                                   :Спальня3БаронВэй
                                   :ГостинаяБаронВэй
                                   :ВаннаяБаронВэй
                                   ... ) .
```

Данное выражение определяет класс всех квартир в Амстердаме. Такой способ определения класса представляется очень громоздким, особенно если список известных членов класса большой. Кроме того, данный способ невозможно использовать, если на момент определения класса неизвестны все его индивиды. Например, мы можем решить создать новую комнату за счет сноса стены между `:Спальня1БаронВэй` и `:Спальня2БаронВэй`.

Непересекающиеся классы. Два класса называются непересекающимися, если ни один из членов одного класса не может быть членом другого класса, другими словами, множества индивидов таких классов не пересекаются. Мы можем объявить, что класс `:КвартираКлас-`

салюкс не пересекается с классом :КвартираСХолоднойВодой с помощью свойства owl:disjointWith:

```
:КвартираКлассаЛюкс owl:disjointWith :КвартираСХолоднойВодой .
```

Это означает, что квартира класса люкс не может одновременно являться квартирой с холодной водой.

Дополнение. Дополнение С класса А — это класс всех сущностей, не принадлежащих классу А. Другими словами, объединение класса А и его дополнение С эквивалентны классу owl:Thing. Отметим, дополнение всегда является надклассом всех непересекающихся с А классов. Важно понимать, что дополнение представляет собой очень мощную конструкцию моделирования, которую нужно использовать с осторожностью. Например:

```
:КвартираМеблированная rdfs:subClassOf: Квартира.  
:КвартираНемеблированная rdfs:subClassOf: Квартира;  
owl:complementOf: :КвартираМеблированная.
```

В данном выражении говорится, что класс меблированных квартир является дополнением класса немеблированных квартир. Это может привести к проблемам, если наша онтология содержит классы, описывающие не только квартиры. Например, если мы добавим утверждение:

```
:Дуплекс owl:disjointWith :Квартира.
```

то класс :Дуплекс будет определен как пустой. Почему? Класс :Квартира покрывает классы :КвартираМеблированная, и его дополнение — класс :КвартираНемеблированная. Это означает, что класс :Квартира эквивалентен классу owl:Thing, который не может содержать каких-либо индивидов, кроме индивидов классов :КвартираМеблированная и :КвартираНемеблированная. Если мы дополнительно введем класс, который не пересекается с классом :Квартира, то этот класс будет не пересекаться и с классом owl:Thing. Таким образом, класс :Дуплекс не может содержать каких-либо индивидов и, следовательно, эквивалентен классу owl:Nothing.

Объединение и непересекающееся объединение. Часто для некоторого класса известно, что он эквивалентен двум или более другим классам: каждый член такого класса является членом, по крайней мере, одного из классов в объединении. Объединение классов может быть определено с помощью конструкции owl:unionOf. Например:

```
:Квартира rdf:type owl:Class ;  
owl:unionOf ( :КвартираСХолоднойВодой  
:КвартираКлассаЛюкс  
:КвартираПентхаус
```

```

:КвартираСтудия
:КвартираНаЦокольномЭтаже
:КвартираМеблированная
:КвартираНемеблированная
).
```

Часто классы, входящие в объединение, взаимно не пересекаются. Конечно, можно явным образом объявить отношение `owl:disjointWith` между данными классами, но удобнее непосредственно указать, что объявление является непересекающимся, например:

```

:Квартира rdf:type owl:Class ;
          owl:disjointUnionOf (
                                :КвартираМеблированная
                                :КвартираНемеблированная ).
```

Перечисление. Кроме того, можно констатировать, что класс является точным пересечением двух или более других классов: каждый член класса является членом каждого из классов в пересечении. Например:

```

:КвартираКлассаЛюкс
  rdf:type owl:Class ;
  owl:intersectionOf ( :КвартираСХорошимРасположением
                        :КвартираБольшая
                        :КвартираСХорошимВидом
                        :КвартираСЭксклюзивнойВанной ).
```

В данной аксиоме утверждается, что к классу `:КвартираКлассаЛюкс` относятся квартиры, которые одновременно имеют хорошее расположение, большие размеры, красивый вид из окна и эксклюзивную ванную комнату.

4.4.6. Аксиомы классов с ограничениями на свойства

В предыдущем разделе были описаны различные аксиомы классов, которые фактически задают определения классов. Однако язык OWL2 позволяет еще более точно определять классы за счет введения дополнительных аксиом. Эти аксиомы *ограничивают* набор индивидов, которые могут считаться членом класса, при условии что они имеют определенные свойства. Их наличие позволяет, например, автоматически делать вывод о принадлежности индивида к классу. Аксиомы ограничения класса формируются за счет связи класса `owl:Class` со специальным типом анонимного класса (который в синтаксисе Turtle обозначается как `owl:Restriction`), содержащего всех индивидов, удовлетворяющих заданным ограничениям.

Универсальные ограничения. Универсальное ограничение на класс C и свойство p означает, что для любого члена класса C все значения свойства p принадлежат к определенному классу. Другими словами, универсальное ограничение может быть использовано для определения диапазона свойств, который является локальным для ограничиваемого класса. Такой тип ограничения записывается с использованием конструкции `owl:allValuesFrom`. Например:

```
:КвартираСЭксклюзивнойВанной
  rdf:type          owl:Class;
  rdfs:subClassOf [rdf:type          owl:Restriction;
                  owl:onProperty   :имеетВанную ;
                  owl:allValuesFrom :ЭксклюзивнаяВанная
                  ] .
```

Данная аксиома определяет класс `:КвартираСЭксклюзивнойВанной` как подкласс множества индивидов, которые связаны свойством `:имеетВанную` только с индивидами класса `:ЭксклюзивнаяВанная`. Отметим, что ограничение `owl:allValuesFrom` просто указывает, что *если* член ограничиваемого класса связан указанным свойством с каким-либо индивидом, то этот индивид должен быть членом указанного класса. Однако универсальное ограничение не означает, что член ограничиваемого класса связан указанным свойством с каким-либо индивидом. Приведенное выше определение класса `:КвартираСЭксклюзивнойВанной` не означает, что квартира с эксклюзивной ванной вообще имеет ванную!

При описании универсальных ограничений могут быть использованы не только объектные свойства, но и свойства типа данных, например для того, чтобы указать, что значение свойства должно иметь определенный тип или находиться в пределах определенного диапазона значений (см. ниже).

Ограничения существования. Ограничение существования на класс C и свойство p означает, что для любого члена класса C существует, по крайней мере, одно значение свойства p , которое принадлежит определенному классу. Такой тип ограничения задается при помощи ключевого слова `owl:someValuesFrom`. Например:

```
:КвартираСЭксклюзивнойВанной
  rdf:type          owl:Class;
  rdfs:subClassOf [ rdf:type          owl:Restriction;
                  owl:onProperty   :имеетВанную ;
                  owl:someValuesFrom :ЭксклюзивнаяВанная
                  ] .
```

Необходимые и достаточные условия. Для того чтобы объявить, что ограничиваемый класс в точности совпадает с классом, опреде-

ляемым ограничением, нужно при описании ограничения вместо свойства `rdfs:subClassOf` использовать свойство `owl:equivalentClass`. Ограничение `rdfs:subClassOf` задает *необходимые условия* принадлежности к классу, а ограничение `owl:equivalentClass` – *необходимые и достаточные условия* принадлежности к классу.

Машина логического вывода может определить принадлежность индивида к классу только на основе необходимых и достаточных условий. Например, приведенное выше ограничение существования не позволит сделать вывод, что индивид, который связан свойством `:имеетВанную` с индивидом класса `:ЭксклюзивнаяВанная`, является членом класса `:КвартираСЭксклюзивнойВанной`. Класс `:КвартираСЭксклюзивнойВанной` является только подклассом соответствующего ограничения, и мы не имеем достаточно информации, чтобы определить, является ли индивид ограничения членом ограничиваемого класса. Если мы описываем класс как эквивалентный заданному ограничению, то очевидно, что любой индивид, удовлетворяющий этому ограничению, является членом класса.

Тем не менее в обоих случаях, если мы явно заявили, что некоторый индивид является членом класса `:КвартираСЭксклюзивнойВанной`, то машина вывода заключит, что существует, по крайней мере, один (неизвестный) индивид класса `:ЭксклюзивнаяВанная`, являющийся значением свойства `:имеетВанную` для данного индивида.

Ограничения на значения. Ограничения на значения используют для определения класса на основе отношений с известными индивидами (для объектных свойств) или конкретными значениями (для свойств типа данных). Например, можно определить класс всех квартир в Амстердаме следующим образом:

```
:КвартираВАмстердаме
  rdf:type          owl:Class;
  owl:equivalentClass [ rdf:type      owl:Restriction;
                        owl:onProperty dbpedia-owl:location ;
                        owl:hasValue  dbpedia:Amsterdam
                      ] .
```

Ограничения кардинальности. Ограничение кардинальности означает, что каждый член класса может иметь только определенное количество значений заданного свойства. Ограничение кардинальности, для которого дополнительно указано, что эти значения должны принадлежать определенному (уточняющему) классу, называется *уточненным*. Например:

```

:КвартираСтудия
  rdf:type          owl:Class;
  rdfs:subClassOf [ rdf:type          owl:Restriction;
                    owl:onProperty  :имеетКомнату ;
                    owl:cardinality "1"^^xsd:integer
                  ] .

```

Данная аксиома означает, что квартира-студия может иметь только одно значение для свойства :имеетКомнату. Мы можем преобразовать это ограничение в *уточненное ограничение кардинальности*, указав, что данное ограничение распространяется лишь на члены классов :Гостиная, :Кухня, :Спальня (квартиры-студии обычно имеют отдельные ванные комнаты):

```

:КвартираСтудия
  rdf:type          owl:Class;
  rdfs:subClassOf [ rdf:type          owl:Restriction;
                    owl:onProperty  :имеетКомнату ;
                    owl:qualifiedCardinality "1"^^xsd:integer ;
                    owl:onClass [ owl:unionOf (:Гостиная :Кухня :Спальня) ]
                  ] .

```

Заметим, что уточненное ограничение по-прежнему позволяет членам ограничиваемого класса иметь дополнительные значения для свойства, при условии что эти значения принадлежат дополнению уточняющего класса¹. Любое неуточненное ограничение кардинальности эквивалентно уточненному ограничению с уточняющим классом Thing². В табл. 4.2 приведены различные ограничения кардинальности языка OWL2.

Таблица 4.2. Ограничения кардинальности в языке OWL2

Тип ограничения	Уточненные ограничения	Неуточненные ограничения
Точная кардинальность	owl:qualifiedCardinality	owl:Cardinality
Минимальная кардинальность	owl:qualifiedMinCardinality	owl:MinCardinality
Максимальная кардинальность	owl:qualifiedMaxCardinality	owl:MaxCardinality

¹ В приведенном примере индивиды класса КвартираСтудия могут быть связаны свойством имеетКомнату, например, с двумя индивидами класса Ванная. — *Прим. перев.*

² Например, ограничение owl:cardinality "1"^^xsd:integer эквивалентно ограничению owl:qualifiedCardinality "1"^^xsd:integer ; owl:onClass owl:Thing. — *Прим. перев.*

Ограничения на диапазон значений и типы данных. Универсальные ограничения и ограничения существования на свойства типа данных позволяют членам ограничиваемого класса иметь любое значение свойства, при условии что оно принадлежит указанному типу данных. Однако иногда требуется более точное ограничение, например для описания класса совершеннолетних людей, которые могут арендовать квартиру, или для описания минимальной площади квартиры.

```
:СовершеннолетнийЧеловек
  rdfs:subClassOf dbpedia:Person ;
  rdfs:subClassOf [ rdf:type owl:Restriction ;
                    owl:onProperty :имеетВозраст ;
                    owl:someValuesFrom
                    [ rdf:type rdfs:Datatype ;
                      owl:onDatatype xsd:integer ;
                      owl:withRestrictions (
                        [ xsd:minInclusive "18"^^xsd:integer ]
                      )
                    ]
                  ] .
```

Данная аксиома определяет класс :СовершеннолетнийЧеловек как подкласс людей, для которых значение свойство :имеетВозраст является целым числом и попадает в диапазон «больше или равно 18». Как показано в примере, диапазон данных определяется как анонимный класс `rdfs:Datatype`. Кроме того, возможно создать новый именованный тип данных для повторного использования в онтологии:

```
:ВозрастСовершеннолетия rdf:type          rdfs:Datatype ;
                          owl:onDatatype  xsd:integer ;
                          owl:withRestrictions (
                            [ xsd:minInclusive "18"^^xsd:integer ]
                          ) .

: СовершеннолетнийЧеловек rdf:type owl:Class ;
                          rdfs:subClassOf dbpedia:Person ;
                          rdfs:subClassOf [ rdf:type owl:Restriction ;
                                              owl:onProperty :имеетВозраст ;
                                              owl:someValuesFrom :ВозрастСовершеннолетия
                                            ] .
```

Для определения новых типов данных в OWL2 можно использовать язык XML Schema. Однако в ограничениях допускается использование только тех типов данных, которые определены с помощью фасетов языка XML Schema (для получения дополнительной информации см. рекомендованную литературу).

Самоограничения. Хорошо известно, что хорошие квартиры продаются «сами собой», то есть если квартира находится в хорошем месте, имеет хороший вид из окна и площадь, то вам не придётся тратить много времени на косметический ремонт, чтобы хорошо продать ее. В OWL2 мы можем выразить подобные знания с помощью самоограничений. Например:

```
:КвартираХорошая
  rdf:type          owl:Class ;
  rdfs:subClassOf [ rdf:type      owl:Restriction ;
                    owl:onProperty :продает ;
                    owl:hasSelf   "true"^^xsd:boolean ;
                    ] .
```

Данная аксиома означает, что каждый экземпляр класса :КвартираХорошая связан сам с собой свойством :продает. Очевидно, в OWL2 DL нельзя использовать самоограничения на свойства типа данных.

Ключи. В классических базах данных для идентификации записей в таблице используются ключи. Эти ключи не обязательно представляют из себя URI, и разработка элегантной схемы преобразования таких ключей в URI является непростой задачей. Язык OWL2 позволяет указать, что для некоторых классов (аналог таблиц в теории баз данных) значение определенного свойства типа данных (или комбинации свойств) следует рассматривать в качестве уникального идентификатора. Например, комбинация почтового индекса и номера почтового адреса представляет собой уникальный идентификатор для любого жилья в Нидерландах:

```
:почтовыйИндекс  rdf:type  owl:DatatypeProperty .
:номерАдреса     rdf:type  owl:DatatypeProperty .

:Жилье
  rdf:type  owl:Class ;
  owl:hasKey ( :почтовыйИндекс :addressNumber ) .
```

Заметим, что механизм ключей позволяет определить обратно-функциональные свойства типа данных, которые являются локальными для класса. Любые два индивида класса :Жилье, которые имеют одинаковое значение для свойств :почтовыйИндекс и :номерАдреса, должны рассматриваться как один и тот же индивид. К сожалению, в языке OWL2 DL нельзя определять глобальные обратно-функциональные свойства типа данных из-за непредсказуемых последствий логического вывода.

4.4.7. Утверждения об индивидах¹

Теперь у нас есть общее представление о том, как в языке OWL2 определяются свойства и классы, и можно перейти к изучению способов описания индивидов. Многие из этих способов были представлены в предыдущих главах. Однако, используя аксиомы классов, можно получить дополнительную информацию об индивидах. Высказывания об индивидах обычно называются *утверждениями*.

Утверждения о классах и свойствах. Утверждения о принадлежности индивида к классу и его свойствах записываются в языке OWL2 таким же образом, как и в языке RDF Schema, например:

```
:Квартира      rdf:type    owl:Class .

:БаронВейКвартира rdf:type    :Квартира;
                  имеетКоличествоКомнат "4"^^xsd:integer ;
                  арендуется      :Паул .
```

Данное утверждение означает, что `:БаронВейКвартира` является экземпляром класса `:Квартира`, она имеет четыре комнаты и сдается в аренду индивиду `:Паул`. Напомним, что в соответствии с прямой семантикой языка OWL2 DL отношение `rdf:type` может устанавливаться только между сущностями разного уровня: классами и индивидами².

Утверждения об идентичности индивидов. Так как язык OWL2 поддерживает предположение об открытости мира, мы не можем утверждать, что два индивида с разными URI являются разными сущностями. Вполне возможно, что это одна сущность, имеющая несколько имен. Как уже упоминалось, в некоторых случаях возможно автоматически сделать вывод о существовании отношений идентичности, однако удобнее объявить идентичность экземпляров явным образом, например:

```
:КвартираБаронВэй owl:sameAs :КвартираПаула ;
                  owl:differentFrom :КвартираФенка .
```

Список различных индивидов может быть достаточно большим. Например, даже в небольшом городе находятся сотни квартир, и для каждой пары квартир необходимо описать отношение `owl:differentFrom`. К счастью, мы можем записать это более элегантно способом с помощью конструкции `owl:AllDifferent`:

¹ Утверждения об индивидах в онтологиях также называют фактами об индивидах. – *Прим. перев.*

² См. также понятие каламбура в разделе 4.4.5.

```
_:x rdf:type owl:AllDifferent ;
    owl:members ( :КвартираФрэнка :КвартираПаула ) .
```

Отрицательные утверждения. Иногда известно, что «нечто не верно». Выражение этого знания в явном виде очень ценно в открытом мире: исключение возможностей часто позволяет получить новые знания в результате логического вывода. Например, знание о том, что :КвартираБаронВэй не арендуется Фрэнком, позволяет сделать вывод, что :КвартираБаронВэй и :КвартираФрэнка — два разных индивида.

```
_:x rdf:type owl:NegativePropertyAssertion ;
    owl:sourceIndividual :КвартираБаронВэй ;
    owl:assertionProperty :арендуется;
    owl:targetIndividual :Фрэнк .
```

Заметим, что знание о том, что индивид не является членом определенного класса, можно выразить явно с помощью утверждения, что индивид является членом дополнения этого класса:

```
:КвартираБаронВэй rdf:type [ owl:complementOf :КвартираКлассаЛюкс ] .
```

4.5. Профили языка OWL2

Спецификация OWL2 включает в себя ряд так называемых профилей. Одни из них являются известными подмножествами языка OWL2 DL, а другие обладают большей выразительностью, чем OWL2 DL, но при этом не поддерживают полную семантику языка OWL2 Full. Многие существующие онтологии используют только ограниченный набор языковых конструкций, доступных в дескрипционных логиках. Именно это послужило поводом к разработке различных профилей языка OWL2. Использование менее выразительного языка позволяет значительно увеличить производительность машин логического вывода. Поэтому стандартная библиотека профилей языка OWL2 с различными вариантами компромиссов между выразительностью и вычислительной сложностью очень полезна на практике.

Для любого профиля справедливо следующее:

- профиль ограничен синтаксисом, причем значение синтаксиса определяется спецификацией языка OWL2 DL;
- профиль определяется логикой, которая позволяет осуществить вывод за полиномиальное время в зависимости либо от количества фактов в онтологии, либо от размера онтологии в целом.

В данном разделе приводится краткий обзор профилей, определенных в языке OWL2, и областей их применения.

OWL2 EL. Профиль $\mathcal{EL}$ является расширением дескрипционной логики EL. Его основная особенность – выполнение за полиномиальное время вывода для онтологий с большим количеством аксиом классов¹. Данный профиль разработан для поддержки выразительной мощности нескольких существующих крупных онтологий в сфере здравоохранения и наук о жизни (например, SNOMED-CT, Gene Ontology и GALEN)².

Основное преимущество профиля OWL2 EL заключается в работе с пересечениями классов и ограничениями существования. Язык считается легким и поддерживает обоснованный и полный вывод за полиномиальное время. Наиболее существенное различие профиля с языком OWL2 DL состоит в том, что он не поддерживает ограничения `rdfs:allValuesFrom`. Однако при этом язык поддерживает ограничение на диапазон свойств `rdfs:range`, реализующее, по сути, аналогичные функции.

OWL2 QL. Машины вывода, разработанные для языков OWL2 DL и OWL2 EL, оптимизированы для вывода на аксиомах классов и относительно неэффективны для работы с онтологиями, содержащими достаточно простые определения классов и большое количество утверждений об индивидах. Профиль OWL2 QL был разработан для обеспечения эффективной обработки запросов именно к таким онтологиям. Профиль использует технологии управления реляционными базами данных³. Он основан на семействе дескрипционных логик DL-Lite и расширен такими выразительными языковыми примитивами, как аксиомы включения свойств (`@powl: subPropertyOf`), функциональные и обратно-функциональные объектные свойства.

OWL2 RL. Профиль OWL2 RL основан на так называемых программах дескрипционной логики (Description Logic Programs, DLP) и обеспечивает взаимодействие между дискреционными логиками и правилами. Фактически профиль представляет собой крупнейший синтаксический фрагмент языка OWL2 DL, реализуемый с помощью правил. Это очень важная особенность, так как правила эффективно

¹ Другими словами, профиль OWL2 EL подходит для приложений, использующих онтологии с очень большим числом классов и/или свойств. – *Прим. перев.*

² См. <http://www.snomed.org>, <http://www.geneontology.org> и http://www.openclinical.org/prj_galen.html соответственно.

³ Данные (утверждения) хранятся в стандартной реляционной базе данных, и доступ к ним осуществляется через онтологию, посредством простой перезаписи запроса на язык SQL. – *Прим. перев.*

выполняются параллельно, что позволяет разрабатывать масштабируемые реализации логического вывода.

OWL2 RL отличается от профилей QL и EL тем, что он представляет собой некоторое промежуточное звено между языками OWL DL и OWL Full: машины вывода, основанные на правилах, могут легко игнорировать ограничения OWL DL (такие как различие между классами и индивидами). Это означает, что реализации правил языка OWL2 RL можно расценивать как реализации подмножества языка OWL Full. Многие из наиболее масштабируемых машин вывода для семантического веба основаны на использовании языка OWL2 RL или очень похожего языка, называемого pD^* или OWL-Horst. Набор правил, который должен быть реализован в таких приложениях, опубликован в спецификации профиля OWL2 RL.

4.6. Выводы

- Язык OWL2 расширяет выразительные возможности языков RDF и RDF Schema за счет таких языковых примитивов, как ограничения кардинальности свойств, аксиомы эквивалентности классов, аксиомы пересечения и непересечения классов.
- Формальная семантика языка OWL2 и поддержка логического вывода обеспечиваются за счет его соответствия формальным логикам.
- Существуют два диалекта языка OWL2. Язык OWL2 DL накладывает некоторые ограничения на комбинации элементов языка OWL2 и языка RDFS с целью поддержки разрешимости логического вывода. Язык OWL2 Full представляет собой расширение языка RDF Schema (полностью совместимое с RDF Schema), включает все возможности языка OWL2, но не обеспечивает разрешимости логического вывода.
- Три профиля языка OWL – OWL2 EL, OWL2 QL и OWL2 RL – представляют собой синтаксические подмножества языка с разными вычислительными свойствами. В частности, профиль OWL2 RL, реализованный с помощью технологий, основанных на правилах, стал стандартом де-факто для логического вывода в семантическом вебе.
- Существуют четыре стандартных способа записи языка OWL2: RDF/XML-синтаксис, манчестерский синтаксис, OWL/XML-синтаксис и функциональный синтаксис.

Рекомендуемая литература

Дополнительные источники по языку OWL2:

- Boris Motik, Bernardo Cuenca Grau, Ian Horrocks, Zhe Wu, Achille Fokoue, and Carsten Lutz, eds. OWL 2 Web Ontology Language Profiles // www.w3.org/TR/owl2-profiles/.
- Boris Motik, Peter F. Patel-Schneider, and Bernardo Cuenca Grau, eds. OWL 2 Web Ontology Language Direct Semantics // www.w3.org/TR/owl2-direct-semantics/.
- Boris Motik, Peter F. Patel-Schneider, and Bijan Parsia, eds. OWL 2 Structural Specification and Functional-Style Syntax // www.w3.org/TR/owl2-syntax/.
- Michael Schneider, ed. OWL 2 Web Ontology Language RDF-Based Semantics // www.w3.org/TR/owl2-rdf-based-semantics/.
- W3C OWL Working Group, eds. OWL 2 Web Ontology Language Document Overview // www.w3.org/TR/owl2-overview/.

Учебники по языку OWL2:

- Pascal Hitzler, Markus Krötzsch, Bijan Parsia, Peter F. Patel-Schneider, and Sebastian Rudolph. OWL 2 Web Ontology Language Primer // www.w3.org/TR/owl2-primer/.
- Matthew Horridge. The Manchester Pizza Tutorial // owl.cs.manchester.ac.uk/tutorials/protegeowltutorial/.

Основные публикации о логиках, используемых языком OWL2:

- Baader Franz, Brandt Sebastian, and Lutz Carsten. Pushing the EL Envelope. Proceedings of IJCAI 2005: 364–369.
- Horrocks I., Kutz O., and Sattler U. The Even More Irresistible SROIQ. In Proceedings of the 10th International Conference of Knowledge Representation and Reasoning (KR-2006, Lake District UK), 2006.
- Herman J. ter Horst. Combining RDF and Part of OWL with Rules: Semantics, Decidability, Complexity. International Semantic Web Conference 2005, 668–684.

Неполный список программного обеспечения и машин логического вывода (на момент написания книги):

- CEL, OWL-машина вывода, оптимизированная для профиля OWL2 EL, разработана в университете Дрездена. См. <http://lat.inf.tu-dresden.de/systems/cel/>.
- HermiT, быстрая OWL-машина вывода для сложных онтологий, разработанная в Оксфордском университете. См. <http://www.hermit-reasoner.com>.

- OWLIM, быстрая OWL-машина вывода для профиля OWL2 RL, разработанная компанией Ontotext. См. <http://www.ontotext.com/owlim>.
- Pellet, одна из более функциональных OWL-машин вывода, разработанная компанией Clark & Parsia. См. <http://pellet.owldl.com>.
- Protégé, фактически представляет собой среду разработки онтологий, содержащую несколько машин вывода. Разработана в Стенфордском университете. См. <http://protege.stanford.edu>.
- TopBraid Composer, среда разработки OWL-онтологий, основанная на языке RDF, разработана компанией TopQuadrant. Поддерживает SPARQL-запросы, связь с хранилищами триплетов, вывод с использованием OWLIM-машины вывода. См. http://www.topquadrant.com/products/TB_Composer.html.
- WebPIE, масштабируемая машина вывода для подмножества OWL, называемого *ter Horst fragment of* (который практически эквивалентен профилю OWL2 RL). Разработана в Амстердамском свободном университете (VU University Amsterdam). См. <http://www.few.vu.nl/~jui200/webpie.html>.

Упражнения и проекты

1. Прочитайте онлайн-спецификацию языка OWL.
2. Приведите три различных способа, с помощью которых на языке OWL можно выразить тот факт, что классы не пересекаются.
3. Выразите факт, что все математические курсы преподаются исключительно Дэвидом Биллингтоном (то есть никакие другие преподаватели не могут читать данные курсы). Также выразите тот факт, что математические курсы – это курсы, которые читаются Дэвидом Биллингтоном. Объясните разницу между данными утверждениями.
4. Строго говоря, свойство `owl:SymmetricProperty` не является необходимым в OWL, потому что оно может быть выражено через другие языковые примитивы. Объясните, как это можно сделать. (*Подсказка: рассмотрите обратные свойства.*)
5. Ответьте на аналогичный вопрос для свойств `owl:FunctionalProperty` и `owl:NegativePropertyAssertion`. Покажите, каким образом они могут быть выражены с помощью других языковых конструкций языка OWL.
6. Определите в целом, какие элементы языка OWL являются необходимыми, а какие просто удобны, но могут быть выражены с помощью других примитивов моделирования.

7. Объясните связь между следующими типами свойств: функциональное свойство, обратно-функциональное свойство и свойство, обратное к заданному.
8. Объясните, почему класс `owl:Class` объявлен как подкласс класса `rdfs:Class`.
9. В разделе 2.7 была представлена аксиоматическая семантика языка RDF. Аналогичная семантика может быть разработана для языка OWL. Определите аксиоматическую семантику элемента языка `owl:intersectionOf`.
10. Определите аксиоматическую семантику для элемента языка `owl:inverseOf`.
11. Определите аксиоматическую семантику для ограничений кардинальности:
 - а) определите `noRepeatsList` (список без повторений). L является «списком без повторений», если каждый элемент встречается в списке не более одного раза. Данное понятие не является частью языка OWL, но будет использоваться при подсчете элементов для ограничения кардинальности;
 - б) определите элементы `owl:minCardinality` и `owl:maxCardinality` как свойства с доменом `owl:Restriction` и диапазоном `⊆NonNegativeInteger`;
 - в) запишите аксиому, которая задает смысл для ограничения `minCardinality`:
 Если $onProperty(R, P)$ и $minCardinality(R, n)$, то x является экземпляром класса R тогда и только тогда, когда существует «список без повторений» L длины $\geq n$, такой, что $P(x, y)$ для всех $y \in L$;
 - г) выразите смысл элемента `owl:maxCardinality` аналогичным образом.
12. Проанализируйте некоторые онтологии на сайте <http://www.co-ode.org/ontologies/>.
13. Разработайте собственные онтологии на языке OWL2.
14. OWL2 является последней версией языка OWL. Прочитайте информацию о предыдущей версии (см. <http://www.w3.org/2004/OWL/>) и проанализируйте некоторые примеры онтологий. Сравните обе версии, обращая внимание как на общие черты, так и на различия.
15. Сравните онлайн-документацию для OWL2 и для первой версии OWL.

Логика и логический вывод: правила

5.1. Введение

5.1.1. Логика и правила

В главах 2 и 4, если рассуждать с абстрактной точки зрения, были рассмотрены способы представления знаний: знаний о содержании веб-ресурсов и знаний об основных понятиях предметной области и отношениях между ними (то есть онтологии).

Способы представления знаний изучались задолго до появления Всемирной паутины в области искусственного интеллекта, а еще раньше в философии. Фактически историю представления знаний можно проследить вплоть до Древней Греции; родоначальником логики считается Аристотель. Логика и сейчас является основой представления знаний, в частности логика предикатов (также известная как логика первого порядка). Ниже перечислены несколько причин, объясняющих популярность и важность логики:

- логика предоставляет высокоуровневый язык, с помощью которого знание может быть выражено понятным образом. И этот язык имеет высокую степень выразительности;
- логика имеет хорошо изученную формальную семантику, которая обеспечивает однозначное толкование логических утверждений;
- существует точное понятие логического следствия, которое определяет, следует ли семантически одно утверждение из множества других утверждений (посылок, антецедентов). В действительности первоначально основной задачей логики считалось изучение объективных законов логического следствия;

- существуют системы доказательства¹, которые автоматически выводят утверждения на основе множества посылок, учитывая только их синтаксические характеристики;
- существуют системы доказательства, для которых семантические логические следствия совпадают с выводами, основанными на синтаксических характеристиках посылок. Система доказательства должна быть непротиворечивой (все выведенные утверждения семантически следуют из посылок) и полной (в системе доказательства могут быть получены все логические следствия на основе посылок);
- логика предикатов является уникальной логикой в том смысле, что для нее существуют непротиворечивые и полные системы доказательства. Более выразительные логики (логики более высоких порядков) не имеют таких систем доказательства;
- системы доказательств позволяют отследить по шагам процесс логического вывода. В этом смысле логика может предоставить объяснения полученных ответов.

Язык RDF и профили языка OWL2 (за исключением языка OWL2 Full) можно рассматривать как специализации логики предикатов.

Существование таких специализированных языков оправдывается, во-первых, тем, что они обеспечивают синтаксис, подходящий для определенного целевого назначения (в нашем случае для веб-языков, основанных на тегах). Кроме того, эти специализированные языки определяют разрешимые подмножества логики. Как уже упоминалось, существует проблема компромисса между выразительной мощностью и вычислительной сложностью некоторых логик: для более выразительных языков системы доказательства являются менее эффективными. Как мы уже отмечали, большинство диалектов языка OWL соответствуют дескрипционным логикам, представляющим собой подмножество логики предикатов, для которого существуют эффективные системы доказательства.

Другим подмножеством логики предикатов, для которого возможен эффективный логический вывод, являются так называемые *системы правил* (также известные как *логика Хорна* или *четкие логические программы* (англ. *definite logic programs*)). Правило имеет следующий вид:

¹ Такие системы доказательства также называют средствами автоматического доказательства теорем, средствами автоматизированного формирования рассуждений или системами логического вывода. Данные системы позволяют из тождественно истинных формул выводить другие тождественно истинные формулы. – *Прим. перев.*

$$A_1, \dots, A_n \rightarrow B,$$

где A_i и B – атомарные формулы. На самом деле существуют два способа интуитивной трактовки такого правила:

1. Если известно, что формулы $A_1, \dots, A_n$ истинны, то формула B также истинна. Правило с такой интерпретацией называют *дедуктивным правилом*¹.
2. Если выполняются условия $A_1, \dots, A_n$, то произвести действие B . Правило с такой интерпретацией называют *реактивным правилом*.

Обе трактовки правил имеют важные приложения. Однако в этой главе будет рассматриваться только дедуктивный подход. Будут описаны язык правил, запросы на этом языке и соответствующие результаты запросов. Кроме того, будет кратко описана работа системы доказательства, которая возвращает такие результаты.

Отметим интересный факт, что дескрипционные логики и логика Хорна ортогональны в том смысле, что ни одна из них не является подмножеством другой. Например, с помощью дескрипционных логик невозможно определить класс счастливых супругов как класс состоящих в браке людей, которые при этом являются лучшими друзьями. Но это знание может быть легко представлено следующим правилом:

состоитВБраке(X, Y), *лучшийДруг*(X, Y) $\rightarrow$ *счастливыйСупруг*(X, Y).

С другой стороны, с помощью правил (в общем случае) не может быть выражено (а) отрицание/дополнение классов; (б) дизъюнкция/объединение информации (например, человек является либо мужчиной, либо женщиной); (в) использование квантора существования (например, все люди имеют отца). В отличие от систем правил, язык OWL позволяет выразить дополнение классов, объединение классов и некоторые формы экзистенциальной квантификации.

Затем будет рассмотрен другой вид правил. Приведем простой пример. Предположим, онлайн-поставщик хочет предоставлять специальную скидку в день рождения клиента. Эта бизнес-политика может быть выражена с помощью следующих правил:

- R1: Если день рождения, то предоставить специальную скидку.
 R2: Если не день рождения, то не предоставлять специальную скидку.

¹ Дедуктивные правила могут использоваться двумя способами: от тела ($A_1, \dots, A_n$) к логическому следствию B (прямой логический вывод) и от логического следствия (цели) к телу (обратный логический вывод).

Это решение работает правильно в том случае, если день рождения клиента известен. Но представьте себе клиента, который отказывается сообщить дату своего рождения из-за принципа неприкосновенности частной жизни. В этом случае указанные выше правила не могут быть применены, так как не известны их посылки. Для исправления этой ситуации следует переписать правила следующим образом:

- R1: Если день рождения, то предоставить специальную скидку.
- R2': Если день рождения не известен, то не предоставлять специальную скидку.

Однако посылка правила R2' не может быть выражена с помощью логики предикатов. Таким образом, нужен новый вид системы правил. Отметим, что решение с правилами R1 и R2 работает в тех случаях, когда имеется полная информация о ситуации (например, известно, что сегодня либо день рождения клиента, либо нет). Новый вид системы правил может применяться в тех случаях, когда имеющаяся информация является неполной.

Логика предикатов и ее подмножества обладают свойством *монотонности* в следующем смысле. Если логический вывод сделан, то он останется в силе, даже если системе станут доступны новые знания. Но если сделан вывод «не предоставлять специальную скидку» на основе правила R2', то он может оказаться недействительным, если день рождения клиента станет известен позднее и этот день рождения совпадет с датой покупки. Подобное поведение является *немонотонным*, потому что добавление новой информации приводит к потере логического следствия. Таким образом, немонотонные правила отличаются от монотонных (которые представляют частный случай логики предикатов). В этой главе будут рассматриваться как монотонные, так и немонотонные правила.

5.1.2. Правила в семантическом вебе

Технологии, основанные на правилах, развивались в течение последних десятилетий, нашли широкое практическое применение и достигли значительной зрелости. Широкое внедрение данных технологий привело к развитию различных подходов. В результате оказалось, что стандартизировать эту область в контексте (семантического) веба гораздо сложнее, чем другие его технологии. Рабочая группа консорциума W3C разработала стандарт формата обмена правилами RIF (Rule Interchange Format). Важно понимать, чем этот язык отличается от языков RDF и OWL: RDF и OWL предназначены для непо-

средственного представления знаний, а формат RIF разработан в первую очередь для обмена правилами между разными приложениями. Например, пусть данные о ценообразовании, возвратах и политики конфиденциальности некоторого интернет-магазина хранятся в виде правил, и необходимо сделать эти данные доступными для интеллектуальных агентов. Мы уже обсуждали основную идею семантического веба – представление знаний в форме, доступной для машинной обработки, с помощью одного из веб-языков.

Так как основное назначение формата RIF – обмен между различными системами правил, то он сочетает в себе многие из особенностей этих систем и является достаточно сложным языком. Поэтому есть некоторые сомнения в том, что формат RIF будет широко использоваться в качестве основного языка представления знаний. Действительно, существуют различные альтернативы для разработки систем правил в семантическом вебе:

- правила на основе RDF могут быть выражены элегантным способом с помощью конструкций языка SPARQL; одно из недавних предложений в этом направлении – нотация SPIN¹;
- если требуется применять правила в сочетании с выразительными семантическими структурами, можно использовать язык SWRL, который сочетает функциональность профиля OWL DL с определенными типами правил;
- те, кто хочет осуществлять моделирование знаний с помощью языка OWL, но для реализации приложений применять технологию правил, могут использовать профиль OWL2 RL.

Такое множество существующих подходов объясняет, почему данная глава очень отличается от предыдущих, в которых описан один стандарт или семейство стандартов, являющихся стабильными и получивших общее признание. Разработка стандарта RIF представляет шаг в этом направлении, но данный стандарт находится пока на другом уровне принятия веб-сообществом. Таким образом, в этой главе представляются идеи на общем уровне и ряд конкретных подходов.

Обзор главы

Читателям, которые незнакомы с обозначениями и основными понятиями логики предикатов, рекомендуется прочитать вводные раз-

¹ Однако SPARQL не является языком правил, по сути, он обеспечивает однократное выполнение правила. Система правил должна быть разработана на основе, например, языка SPIN.

дела книг по логике, которые перечислены в конце данной главы. В противном случае можно частично пропустить разделы 5.3 и 5.4.

- В разделе 5.2 представлен пример использования монотонных правил (то есть подмножество логики предикатов, называемое логикой Хорна).
- В разделах 5.3 и 5.4 описываются синтаксис и семантика логики Хорна.
- В разделе 5.5 рассматривается взаимосвязь между профилем OWL2 RL и правилами.
- Раздел 5.6 представляет описание семейства диалектов RIF, при этом основное внимание уделяется языкам, основанным на логике.
- В разделе 5.7 описывается язык SWRL как способ объединения правил и дескрипционных логик.
- В разделе 5.8 кратко описывается моделирование правил с помощью конструкций языка SPARQL.
- В разделе 5.9 представлен синтаксис немонотонных правил, а в разделе 5.10 приведен пример использования немонотонных правил.
- И наконец, в разделе 5.11 кратко описываются язык RuleML и продолжающаяся деятельность по разметке веб-страниц на основе правил с открытой и экспериментальной проблематикой, которая в будущем может быть введена в новые стандарты.

5.2. Пример монотонных правил: родственные связи

Пусть имеется база данных, содержащая факты о некоторых родственных связях. Предположим, что эта база данных содержит факты о следующих базовых предикатах:

<i>мать</i> (X, Y)	X является матерью Y ,
<i>отец</i> (X, Y)	X является отцом Y ,
<i>мужчина</i> (X)	X это мужчина,
<i>женщина</i> (X)	X это женщина.

На основе этих предикатов и соответствующих правил вывода можно получить предикаты, описывающие другие родственные связи. Во-первых, можно определить предикат *родитель*: родитель – это либо отец, либо мать:

$мать(X, Y) \rightarrow родитель(X, Y),$
 $отец(X, Y) \rightarrow родитель(X, Y).$

Затем можно определить, что брат – это человек мужского пола, имеющий таких же родителей:

$мужчина(X), родитель(P, X), родитель(P, Y), разныеЛюди(X, Y) \rightarrow$
 $брат(X, Y).$

Предикат *разныеЛюди*(X, Y) обозначает, что X и Y – это два разных человека; предполагается, что подобные факты хранятся в базе данных. Конечно, каждая прикладная система логического вывода предлагает удобные способы выражения идентичности и неидентичности сущностей, но мы остановимся на этом абстрактном решении, чтобы сохранить общность рассуждений.

Аналогичным образом определяется предикат *сестра*:

$женщина(X), родитель(P, X), родитель(P, Y), разныеЛюди(X, Y) \rightarrow$
 $сестра(X, Y).$

Дядя – это брат одного из родителей:

$брат(X, P), родитель(P, Y) \rightarrow дядя(X, Y).$

Бабушка – это мать одного из родителей:

$мать(X, P), родитель(P, Y) \rightarrow бабушка(X, Y).$

Предок – это либо родитель, либо предок родителя:

$родитель(X, Y) \rightarrow предок(X, Y),$
 $предок(X, P), родитель(P, Y) \rightarrow предок(X, Y).$

5.3. Монотонные правила: синтаксис

Рассмотрим простое правило: все постоянные клиенты старше 60 лет имеют право на специальную скидку:

$постоянныйКлиент(X), возраст(X) > 60 \rightarrow скидка(X).$

Выделим некоторые элементы правил:

- *переменные* – величины, которые могут принимать различные значения, например X;
- *константы* – величины, которые обозначают фиксированные значения, например 60;
- *предикаты*, которые связывают объекты, например *постоянныйКлиент*, >;

- *функциональные символы*, которые обозначают функции, применяемые к определенным аргументам, например *возраст*.

В случае если функциональные символы не используются, говорят о логике Хорна без функциональных символов.

5.3.1. Правила

Правило имеет вид

$$B_1, \dots, B_n \rightarrow A,$$

где $A, B_1, \dots, B_n$ – атомарные формулы. A – это *голова* правила, $B_1, \dots, B_n$ – это *посылки* правила. Множество $\{B_1, \dots, B_n\}$ также называется *телом* правила.

Запятые в теле правила читаются как конъюнкция: если формулы B_1 и B_2 и ... и B_n истинны, то формула A также истинна (что эквивалентно утверждению: чтобы доказать формулу A , достаточно доказать все формулы $B_1, \dots, B_n$).

Обратите внимание, что переменные могут входить в формулы $A, B_1, \dots, B_n$. Например:

$$\text{постоянныйКлиент}(X), \text{возраст}(X) > 60 \rightarrow \text{скидка}(X).$$

Это правило применяется для любого клиента, удовлетворяющего данным условиям: если клиент является постоянным и ему больше 60 лет, то он получает скидку. Другими словами, переменная X неявно связана квантором всеобщности ($\forall X$). В общем случае все переменные, входящие в правило, неявно связаны квантором всеобщности.

Таким образом, правило r

$$B_1, \dots, B_n \rightarrow A$$

интерпретируется как следующая формула, обозначаемая $pl(r)$:

$$\forall X_1 \dots \forall X_k ((B_1 \wedge \dots \wedge B_n) \rightarrow A)$$

или, что эквивалентно:

$$\forall X_1 \dots \forall X_k (A \vee \neg B_1 \vee \dots \vee \neg B_n),$$

где $X_1, \dots, X_k$ – переменные, входящие в формулы $A, B_1, \dots, B_n$.

5.3.2. Факты

Факт – это атомарная формула; например факт *постоянныйКлиент(a345678)* означает, что клиент с ID = a345678 является постоянным клиентом. Переменные фактов неявно связаны квантором всеобщности.

5.3.3. Логическая программа

Логическая программа P – это конечное множество фактов и правил. Ее трансляция в логику предикатов $pl(P)$ представляет множество всех интерпретаций логики предикатов для правил и фактов из P .

5.3.4. Цели

Цель обозначает запрос G к логической программе. Цель имеет следующий вид:

$$B_1, \dots, B_n \rightarrow .$$

Если $n = 0$, то цель называется пустой.

Интерпретируем цель в терминах логики предикатов. Используя принятые выше предположения (интерпретация запятых как конъюнкции, неявное связывание квантором всеобщности), мы получим следующую интерпретацию цели:

$$\forall X_1 \dots \forall X_k (\neg B_1 \vee \dots \vee \neg B_n).$$

Эта формула фактически совпадает с $pl(r)$, с той лишь разницей, что A – голова правила – опущена.

Эквивалентным представлением цели в логике предикатов является следующее:

$$\neg \exists X_1 \dots \exists X_k (B_1 \wedge \dots \wedge B_n),$$

где $X_1, \dots, X_k$ – переменные, входящие в формулы $B_1, \dots, B_n$. Дадим краткое объяснение этой формулы. Предположим, что известен факт

$$p(a)$$

и рассматривается цель

$$p(X) \rightarrow .$$

Фактически нас интересует, существует ли значение, при котором предикат p истинен. Мы ожидаем получить положительный ответ, поскольку существует факт $p(a)$. Таким образом, $p(X)$ связан квантором существования. Но зачем тогда записывать отрицание формулы? Это объясняется тем, что мы будем использовать метод доказательства, который в математике называется доказательством «от противного». То есть, для того чтобы доказать, что высказывание A следует из высказывания B , мы предполагаем, что A ложно, и приходим к противоречию, учитывая, что B истинно. Таким образом, делаем вывод, что A должно следовать из B .

В логическом программировании доказывается, что цель может быть достигнута именно таким образом: цель отрицается¹, и с помощью логической программы доказывается, что мы в этом случае придём к противоречию. Например, рассмотрим логическую программу

$$p(a)$$

и цель

$$\neg \exists X p(X).$$

Мы получаем логическое противоречие: вторая формула означает, что ни один элемент не обладает свойством p , а первая формула означает, что значение a имеет свойство p . Таким образом, высказывание $\exists X p(X)$ следует из факта $p(a)$.

5.4. Монотонные правила: семантика

5.4.1. Семантика логики предикатов

Один из способов обработки запросов – использование интерпретации логики предикатов для фактов, правил и запросов и применение хорошо изученной семантики логики предикатов. Уточним данный подход. Пусть даны логическая программа P и запрос

$$B_1, \dots, B_n \rightarrow$$

с переменными $X_1, \dots, X_k$. Данная цель достижима тогда и только тогда, когда

$$pl(P) \models \exists X_1 \dots \exists X_k (B_1 \wedge \dots \wedge B_n), \quad (1)$$

или, что эквивалентно, когда формула

$$pl(P) \cup \{\neg \exists X_1 \dots \exists X_k (B_1 \wedge \dots \wedge B_n)\} \quad (2)$$

невыполнима.

Другими словами, цель достижима, если представление программы P в логике предикатов совместно с интерпретацией запроса невыполнимо (то есть существует противоречие).

Формальное определение семантики логики предикатов вы можете найти в соответствующей литературе. Здесь же приводится только неформальное описание. Компоненты логического языка (сигнатура) могут иметь любой смысл. Модель логики предикатов фиксирует определенный смысл. В частности, модель $\mathcal{A}$ включает:

¹ То есть предполагается, что цель недостижима. – *Прим. перев.*

- домен (область определения) $dom(\mathcal{A})$ – непустое множество объектов, о которых делаются высказывания в виде формул;
- элемент из домена для каждой константы;
- конкретную функцию, определённую на множестве $dom(\mathcal{A})$, для каждого функционального символа;
- конкретное отношение, определенное на множестве $dom(\mathcal{A})$, для каждого предиката.

Если символ $=$ используется для обозначения равенства (то есть его интерпретация зафиксирована), то мы имеем дело с *логикой Хорна с символом равенства*. Значения логических связок $\neg$, $\vee$, $\wedge$, $\rightarrow$, $\forall$, $\exists$ определяются в соответствии с их интуитивным смыслом: не, или, и, следует, для всех, существует. Таким образом, мы определяем, когда формула ϕ истинна в модели $\mathcal{A}$, что обозначается как $\mathcal{A} \models \phi$.

Формула ϕ следует из множества формул M , если ϕ истинна во всех моделях $\mathcal{A}$, в которых M истинно (то есть все формулы из M истинны в модели $\mathcal{A}$).

Теперь мы можем объяснить формулы (1) и (2). Вне зависимости от того, как интерпретируются константы, предикаты и функциональные символы в программе P и в запросе, если интерпретация программы P в логике предикатов истинна, то и формула $\exists X_1 \dots \exists X_k (B_1 \wedge \dots \wedge B_n)$ должна быть также истинной. То есть существуют такие значения для переменных $X_1, \dots, X_k$, что все атомарные формулы B_i являются истинными.

Например, предположим, что дана программа P следующего вида:

$p(a)$,
 $p(X) \rightarrow q(X)$.

Рассмотрим запрос

$q(X) \rightarrow$.

Очевидно, $q(a)$ следует из $pl(P)$. Таким образом, формула $\exists X q(X)$ следует из $pl(P)$, и, следовательно, $pl(P) \cup \{\neg \exists X q(X)\}$ невыполнима и рассматриваемая цель достижима. Но если мы рассмотрим запрос

$q(b) \rightarrow$,

то мы получим отрицательный ответ, потому что $q(b)$ не следует из $pl(P)$.

5.4.2. Семантика модели Эрбрана

Другой вид семантики логических программ, семантика модели Эрбрана, требует больше математических выкладок и описан в стандартных учебниках по логике (см. список рекомендуемой литературы).

5.4.3. Базовые и параметризованные свидетельства

До сих пор рассматривались запросы, ответ на которые представлялся в виде «истина» или «ложь». Однако подобные ответы не обязательно являются оптимальными. Предположим, что имеются факт

$$p(a)$$

и запрос

$$p(X) \rightarrow .$$

Положительный ответ на данный запрос является правильным, но недостаточным. Это напоминает старую шутку: вас спрашивают: «Вы знаете, сколько времени?» — а вы смотрите на часы и отвечаете: «Да».

В нашем примере удовлетворительным ответом является подстановка

$$\{X/a\},$$

что означает положительный ответ и указывает удовлетворяющий данному запросу конкретный экземпляр для переменной X . Константа a называется *базовым свидетельством* (англ. *ground witness*). Пусть даны факты:

$$p(a),$$

$$p(b).$$

Тогда для того же запроса будут существовать два базовых свидетельства: a и b . Что равнозначно возвращению в ответ на запрос подстановок вида:

$$\{X/a\},$$

$$\{X/b\}.$$

Несмотря на значимость базовых свидетельств, они также не всегда представляют собой оптимальные ответы на запросы. Рассмотрим логическую программу:

$$\text{сложить}(X, 0, X),$$

$$\text{сложить}(X, Y, Z), \rightarrow \text{сложить}(X, s(Y), s(Z)).$$

Эта программа вычисляет сумму, если s трактуется как «функция следования», которая возвращает значение аргумента, увеличенное на единицу. Третий аргумент предиката *сложить* вычисляет сумму первых двух аргументов. Рассмотрим следующий запрос:

$$\text{сложить}(X, s^s(0), Z) \rightarrow .$$

Возможные базовые свидетельства для данного запроса определяются следующими подстановками:

$$\{X/0, Z/s^8(0)\},$$

$$\{X/s(0), Z/s^9(0)\},$$

$$\{X/s(s(0)), Z/s^{10}(0)\}.$$

Однако *параметризованное свидетельство* $Z = s^8(X)$ представляет собой наиболее общий способ выразить, что

$$\exists X \exists Z \text{ сложить}(X, s^8(0), Z),$$

так как оно означает, что предикат *сложить*($X, s^8(0), Z$) принимает значение «истина», если значение переменной Z равно сумме значений переменной X и константы 8.

Вычисление наиболее общих свидетельств является целью систем доказательств, называемых SLD-резолюциями, описание которых выходит за рамки этой книги.

5.5. OWL2 RL: дескрипционные логики и правила

Как отмечалось в начале этой главы, логика Хорна и дескрипционные логики ортогональны. Самый простой подход к их интеграции в рамках одной структуры заключается в рассмотрении возможности пересечения этих логик, то есть возможности перевода некоторых конструкций одного языка без потери семантики в конструкции другого языка, и наоборот. По существу, профиль OWL2 RL покрывает фрагмент языка OWL, обладающий такими свойствами. Перечислим преимущества данного подхода:

- для решения конкретной задачи моделирования разработчик свободен использовать либо язык OWL, либо правила (и соответствующие инструменты и методологии) в зависимости от своего опыта и предпочтений;
- для реализации могут быть использованы либо машины логического вывода на основе дескрипционных логик, либо дедуктивные системы правил. Таким образом, можно проводить моделирование знаний с помощью одной структуры, например языка OWL, а использовать машину вывода для другой структуры, например правил. Эта особенность обеспечивает дополнительную гибкость и совместимость с различными инструментами.

Далее в этом разделе будет показано, каким образом различные конструкции языков RDF Schema и OWL2 RL могут быть выражены

с помощью логики Хорна, а также рассмотрены некоторые конструкции этих языков, которые, в общем случае, не могут быть выражены подобным образом. При этом основное внимание уделено базовым конструкциям, на которых демонстрируются связи и различия между правилами и дескрипционными логиками. Для получения более подробной информации о конструкциях языка OWL2 RL и их связи с логикой рекомендуем обратиться к разделу 5.6.4 и рекомендуемой литературе, указанной в конце этой главы.

Начнем рассмотрение с конструкций языков RDF и RDF Schema. RDF-триплет вида (a, P, b) может быть выражен как факт

$$P(a, b).$$

Объявление экземпляра в виде $type(a, C)$, означающее, что a – это экземпляр класса C , может быть выражено следующим образом:

$$C(a).$$

Тот факт, что класс C является подклассом класса D , легко выражается как правило вида

$$C(X) \rightarrow D(X).$$

Аналогичным образом записывается правило для отношения «свойство–подсвойство». Ограничения домена и диапазона свойств также может быть выражено в логике Хорна. Например, следующее правило означает, что класс C является доменом свойства P :

$$P(X, Y) \rightarrow C(X).$$

Теперь рассмотрим конструкции языка OWL. Конструкция **equivalentClass(C, D)** может быть выражена с помощью двух правил:

$$C(X) \rightarrow D(X),$$

$$D(X) \rightarrow C(X).$$

Аналогично записываются правила для конструкции **equivalentProperty**. Транзитивность свойства P может быть представлена правилом

$$P(X, Y), P(Y, Z) \rightarrow P(X, Z).$$

Перейдем к рассмотрению булевых операций. Тот факт, что пересечение классов C_1 и C_2 является подклассом класса D , записывается следующим образом:

$$C_1(X), C_2(X) \rightarrow D(X).$$

И наоборот, утверждение, что класс C является пересечением классов D_1 и D_2 , записывается как:

$$C(X) \rightarrow D_1(X), \\ C(X) \rightarrow D_2(X).$$

Утверждение о том, что объединение классов C_1 и C_2 является подклассом класса D , записывается с помощью двух следующих правил:

$$C_1(X) \rightarrow D(X), \\ C_2(X) \rightarrow D(X).$$

Однако обратное утверждение невозможно записать с помощью логики Хорна. Действительно, чтобы выразить тот факт, что класс C является подклассом объединения классов D_1 и D_2 , потребуются дизъюнкция в качестве головы соответствующего правила, что недопустимо в логике Хорна. Но для частных случаев можно выразить подобный факт. Например, в случае если класс D_1 является подклассом класса D_2 , то правила $C(X) \rightarrow D_2(X)$ достаточно, чтобы выразить, что класс C является подклассом объединения классов D_1 и D_2 . Однако не существует правила, которое выражало бы этот факт в общем случае.

Наконец, кратко рассмотрим некоторые формы ограничений в языке OWL. OWL-утверждение

```
:C rdfs:subClassOf [ rdf:type owl:Restriction ;
  owl:onProperty :P ;
  owl:allValuesFrom :D ] .
```

может быть выражено в логике Хорна следующим образом:

$$C(X), P(X, Y) \rightarrow D(Y).$$

Однако обратное утверждение в общем случае не может быть выражено в виде правила.

OWL-утверждение вида

```
[ rdf:type          owl:Restriction ;
  owl:onProperty  :P ;
  owl:someValuesFrom :D ] rdfs:subClassOf :C .
```

может быть выражено с помощью логики Хорна следующим образом:

$$P(X, Y), D(Y) \rightarrow C(X).$$

Обратное утверждение также не может быть в общем случае выражено в виде правила.

Кроме того, ограничения мощности и дополнение классов в общем случае не могут быть выражены с помощью логики Хорна.

5.6. Формат обмена правилами: RIF

5.6.1. Обзор

Технологии, основанные на правилах, развиваются на протяжении нескольких десятилетий и имеют различные реализации (например, правила действий, правила логики первого порядка, логическое программирование). Исходя из этого, цель рабочей группы W3C Rule Interchange Format Working Group заключалась не в разработке нового языка правил, который будет решать все задачи семантического веба, а в создании механизма для обмена между различными системами правил в вебе (существующими или будущими). Общий подход рабочей группы состоит в разработке семейства языков, называемых *диалектами*; формат RIF определяет два вида диалектов:

- 1) диалекты, основанные на логике. Данные диалекты предназначены для языков правил, которые основаны на каком-либо виде логики, например логика первого порядка и различные подходы логического программирования с различными интерпретациями отрицания (программирование множества ответов, обоснованные семантики и т. д.). На настоящий момент к конкретным диалектам данного вида относятся следующие диалекты:
 - *Core RIF* – ядро RIF, по существу, соответствует логике Хорна без функциональных символов;
 - *RIF Basic Logic Dialect (BLD)* – основанный логический диалект RIF по существу, соответствует логике Хорна с символом равенства;
- 2) правила с действиями. Данные диалекты предназначены для продукционных систем и реактивных правил. На настоящий момент конкретным диалектом этого типа является диалект:
 - *Production Rule Dialect (RIF-PRD)* – диалект продукционных правил.

Семейство диалектов RIF обладает свойствами единообразия и расширяемости. Единообразие достигается за счет того, что синтаксис и семантика всех RIF-диалектов разработаны на основе общих принципов. Расширяемость означает возможность добавления в семейство RIF диалектов, которые будут разработаны в будущем. Для поддержки единообразия и расширяемости логических диалектов рабочая группа RIF разработала соответствующий фреймворк – *Framework for Logic Dialects (RIF-FLD)*, который позволяет задавать различные языки правил с помощью указания значений разнообраз-

ных параметров. Создание этого фрейворка можно считать крупным достижением, однако его описание выходит за рамки этой книги. Далее будут представлены основные идеи диалекта RIF-BLD.

Но, прежде чем перейти к описанию диалекта RIF-BLD, отметим, что многие (а может, и большинство) работы рабочей группы RIF посвящены семантическим аспектам обмена правилами. Конечно, обмен правилами осуществляется и на синтаксическом уровне с помощью отображения между различными синтаксическими конструкциями логических систем и формата RIF (например, с использованием языка XML). Но главная цель состоит в обмене правилами с сохранением семантики.

5.6.2. RIF-BLD

Основной логический диалект RIF в основном соответствует логике Хорна с символом равенства, а также содержит:

- типы данных и встроенные конструкции;
- фреймы.

Диалект RIF-BLD, как и все диалекты языка RIF, поддерживает единый набор общих типов данных, предикатов и функций. Этот набор включает в себя типы данных (такие как целочисленный тип `integer`, логический тип `boolean`, строки `string`, даты `data`), встроенные предикаты (например, «число больше, чем» – `numeric-greater-than`, «начать с» – `starts-with`, «дата раньше, чем» – `date-less-than`) и функции (вычитание чисел – `numeric-subtract`, замена – `replace`, часы из времени – `hours-from-time`), определенные для этих типов данных.

В качестве примера рассмотрим, каким образом в виде правил можно выразить следующие утверждения: «Актер является звездой, если он снялся более чем в трех успешных фильмах, произведенных с временным промежутком, по крайней мере, в 5 лет», «Фильм считается успешным, если он получил признание критиков (скажем, рейтинг выше 8 из 10) или был финансово успешным (принес более 100 млн долларов от продажи билетов)». Правила должны работать на наборе данных проекта DBpedia.

Данные правила могут быть выражены на диалекте RIF-BLD следующим образом:

```
Document(
  Prefix(func <http://www.w3.org/2007/rif-builtin-function#>
  Prefix(pred <http://www.w3.org/2007/rif-builtin-predicate#>
  Prefix(rdfs <http://www.w3.org/2000/01/rdf-schema#>
  Prefix(imdbrel <http://example.com/imdbrelation#>
```

```

Prefix(dbpedia <http://dbpedia.org/ontology/>
Prefix(ibdbrel <http://example.com/ibdbrelation#>
Group(
  Forall ?Актёр ?Фильм ?Год (
    If And( dbpedia:starring(?Фильм ?Актёр)
            dbpedia:dateOfFilm(?Фильм ?Год)
    Then dbpedia:starredInYear(?Фильм ?Актёр ?Год)
  )
  Forall ?Актёр (
    If (Exists ?Фильм1 ?Фильм2 ?Фильм3 ?Год1 ?Год2 ?Год3
        And (dbpedia:starredInYear(?Фильм1 ?Актёр ?Год1)
            dbpedia:starredInYear(?Фильм2 ?Актёр ?Год2)
            dbpedia:starredInYear(?Фильм3 ?Актёр ?Год3)
            External (pred:numeric-greater-than(
                External(func:numeric-subtract ?Год1 ?Год3) 5)))
        dbpedia:successful(?Фильм1)
        dbpedia:successful(?Фильм2)
        dbpedia:successful(?Фильм3)
        External (pred:literal-not-identical(?Фильм1 ?Фильм2))
        External (pred:literal-not-identical(?Фильм1 ?Фильм3))
        External (pred:literal-not-identical(?Фильм2 ?Фильм3))
      )
    Then dbpedia:movieStar(?Актёр)
  )
  Forall ?Фильм (
    If Or (
      External(pred:numeric-greater-than(
        dbpedia:criticalRating(?Фильм 8))
      External(pred:numeric-greater-than(
        dbpedia:boxOfficeGross(?Фильм 100000000)))
    Then dbpedia:successful(?Фильм)
  )
)
)

```

Этот пример демонстрирует использование типов данных и встроенных конструкций диалекта RIF-BLD. Обратите внимание, что при использовании предикатов указывается ключевое слово `External`, а для группировки правил – ключевое слово `Group`.

Синтаксис RIF довольно простой, хотя и очень подробный (конечно, существует и синтаксис на основе языка XML для поддержки обмена между системами правил). Имена переменных начинаются с вопросительного знака. Символы `=`, `#`, и `##` используются для выражения отношений эквивалентности, принадлежности к классу и отношения «класс–подкласс» соответственно.

Использование *фреймов* имеет богатую историю в области объектно-ориентированного программирования и в области представления знаний, а также фреймы занимают значительное место в языках правил (например, в языке FLogic). Основная идея заключается в представлении объектов в виде фреймов, а свойств объектов – в виде *слотов*. Например, можно описать класс «Профессор» со слотами «Имя», «Офис», «Телефон», «Факультет» и т. д. Подобная информация выражается на диалекте RIF-BLD следующим образом:

```
oid[slot1 -> value1 ... slotn -> valuen].
```

5.6.3. Совместимость с языками RDF и OWL

Главной особенностью формата RIF является его совместимость со стандартами RDF и OWL. То есть можно осуществлять логический вывод на основе комбинации RIF-, RDF- и OWL-документов. Таким образом, формат RIF обеспечивает обмен не только правилами, но и RDF-графами и/или OWL-аксиомами.

Основная идея совместимости формата RIF с языком RDF базируется на представлении RDF-триплетов в виде фреймовых формул в формате RIF: триплет (s p o) представляется как s [p -> o]. С точки зрения семантики, RDF-триплет справедлив тогда и только тогда, когда справедлива соответствующая фреймовая RIF-формула. Например, если истинен RDF-триплет

```
ex:УнесенныеВетром ex:ГодВыпускаФильма ex:1939,
```

то истинен и RIF-факт

```
ex:УнесенныеВетром [ex:ГодВыпускаФильма -> ex:1939].
```

Учитывая RIF-правило (которое означает, что кодекс Хейса¹ действовал с 1930 по 1968 г.)

```
Group(
  Forall ?Фильм (
    If And( ?Фильм[ex:Год -> ?Год]
      External(pred:dateGreaterThan(?Год 1930)
      External(pred:dateGreaterThan(1968 ?Год)))
    Then ?Фильм[ex:КодексХейса -> ex:Истина]
  )
)
```

¹ Кодекс Хейса – этический кодекс производства фильмов в Голливуде. – Прим. перев.

можно сделать вывод, что справедливы факт

`ex:УнесенныеВетром[ex:КодексХейса -> ex:Истина]`

и соответствующий RDF-триплет.

Подобные методы используются и для достижения совместимости между языками OWL и RIF. Укажем ее основные особенности:

- семантика языков OWL и RIF совместима;
- можно осуществлять логический вывод на основе определённых комбинаций OWL-аксиом и RIF-знаний;
- профиль OWL2 RL может быть реализован в формате RIF (см. следующий раздел).

5.6.4. Реализация профиля OWL2 RL в формате RIF

Профиль OWL2 RL частично описывается набором правил логики первого порядка, что можно считать основой для его реализации с использованием технологии правил. Для обеспечения совместимости между системами правил и онтологиями на языке OWL2 RL аксиомы OWL2 RL записываются с помощью RIF-правил (на диалекте BLD и даже на более простом диалекте Core RIF).

Правила профиля OWL2 RL можно разделить на четыре (непересекающиеся) категории: правила шаблонов триплетов, правила противоречивости, правила списков и правила типов данных.

Правила шаблонов триплетов. Эти правила позволяют получить определенные RDF-триплеты на основе конъюнкции шаблонов RDF-триплетов. Перевод этих правил в формат RIF (а именно во фреймовые формулы) прост. Для этого используются правила вида¹:

```
Group(
  Forall ?V1 ... ?Vn(
    s[p->o] :- And(s1[p1->o1]... sn[pn->on]))
)
```

Правила противоречивости. Данные правила указывают на противоречия в исходном RDF-графе (конечно, относительно существующих OWL-знаний). Они могут быть представлены в виде RIF-правил со следствием `rif:error`; это предикатный символ, определённый в пространстве имен RIF, который используется для выражения противоречивости утверждений. Например, противоречивость воз-

¹ Для улучшения читабельности эти правила приводятся в нотации языка Prolog (обратной нотации), а не в нотации типа «ЕСЛИ – ТО» (прямой нотации), которая использовалась выше.

никнет, если два предиката объявлены как непересекающиеся, но при этом связывают одни и те же сущности. Это может быть выражено на диалекте RIF следующим образом:

```
Group(
  Forall ?P1 ?P2 ?X ?Y(
    rif:error :- And(
      ?P1[owl:propertyDisjointWith ?P2] ?X[?P1->?Y] ?X[?P2->?Y]))
)
```

Правила списков. Некоторые правила на языке OWL2 RL включают обработку OWL-выражений, содержащих RDF-списки (например, `owl:AllDifferent`). Для записи таких правил в формате RIF можно использовать два подхода. Первый подход заключается в использовании рекурсивных правил для обхода RDF-графа во время обработки, что позволяет получить единообразное представление. Вторым подходом подразумевает, что на этапе предварительной обработки происходит непосредственное создание правил для списков, содержащихся в исходном RDF-графе, которые на практике могут выполняться лучше. Для получения более подробной информации читатели могут обратиться к соответствующим документам (см. рекомендуемую литературу).

Правила типов данных. Эти правила обеспечивают проверку типов данных и проверку значений типизированных литералов на равенство/неравенство для поддерживаемых типов данных. Например, такие правила могут вывести существования отношения `owl:sameAs` между литералами с одинаковыми значениями одного типа данных (например, литералов 1 и 1.0) или обнаружить противоречие, если литерал объявлен как экземпляр определенного типа данных, но его значение не принадлежит этому типу данных. Перевод правил этого типа в формат RIF довольно прост.

5.7. Язык правил семантического веба SWRL

Язык *SWRL* (Semantic Web Rules Language) – это язык семантического веба, объединяющий язык OWL DL с логикой Хорна без функциональных символов, который записан с помощью нотации Datalog RuleML (см. раздел 5.11). Таким образом, язык позволяет комбинировать правила логики Хорна с онтологиями на языке OWL DL.

Правило в языке SWRL имеет следующий вид:

$$B_1, \dots, B_n \rightarrow A_1, \dots, A_m,$$

где запятые обозначают конъюнкции с обеих сторон стрелки, а $A_1, \dots, A_m, B_1, \dots, B_n$ могут иметь вид $C(x)$, $P(x, y)$, $sameAs(x, y)$ или $differentFrom(x, y)$, где C – OWL-описание, P – OWL-свойство, x, y – переменные языка Datalog, OWL-индивиды, или значения OWL-типов данных.

Если голова правила содержит более чем один атом (то есть если голова – это конъюнкция атомов без общих переменных), то правило может быть простым способом преобразовано в эквивалентное множество правил с одним атомом в качестве головы.

Главная проблема языка SWRL связана с тем, что произвольные OWL-выражения, такие как ограничения, могут входить как в голову, так и в тело правила. Это добавляет значительную выразительную мощность к выразительности языка OWL за счет потери разрешимости; то есть не может существовать машины логического вывода, которая осуществляет вывод, в точности соответствующий семантике языка SWRL.

По сравнению с профилем OWL2 RL, язык SWRL реализует совсем иной подход к интеграции дескрипционных логик и правил без функциональных символов. Если OWL2 RL является выражением очень консервативного подхода, в соответствии с которым объединяются преимущества обоих языков в общий подязык, то SWRL реализует более максималистский подход, заключающийся в объединении выразительности этих языков. С практической точки зрения, задача заключается в том, чтобы определить подязыки SWRL, которые реализуют правильный баланс между выразительной мощностью и вычислительной сложностью. В качестве такого подязыка может рассматриваться расширение языка OWL DL с безопасными DL-правилами (DL-safe rules), в которых каждая переменная должна входить в атомарную формулу недескрипционной логики в теле правила; см. статьи об интеграции правил с дескрипционными логиками из списка рекомендуемой литературы.

Заметим: результат, сходный с языком SWRL, может быть достигнут путем объединения диалекта RIF-BLD с профилем OWL2 RL (см. рекомендуемую литературу).

5.8. Правила в языке SPARQL: SPIN

Правила могут быть выражены в языке SPARQL с использованием конструкции CONSTRUCT. Например, правило

$$дедушка(X, Z) \leftarrow родитель(Y, Z), отец(X, Y)$$

может быть выражено следующим образом:

```
CONSTRUCT {
  ?X дедушка ?Z.
} WHERE {
  ?V родитель ?Z.
  ?X отец ?V.
}
```

Недавно предложенная нотация SPIN использует эту идею в качестве основы языка моделирования, являющегося надмножеством языка SPARQL. Перечислим его основные характеристики:

- используются идеи объектно-ориентированного моделирования для связывания правил и классов; таким образом, правила представляют поведение классов и не могут существовать сами по себе (хотя могут быть определены глобальные правила);
- правила выражаются через такие конструкции языка SPARQL, как CONSTRUCT, DELETE, INSERT, а ограничения – с помощью конструкции ASK;
- обеспечиваются механизмы абстракции для правил, использующих шаблоны, которые в сущности представляют параметризованные SPARQL-запросы; определяемые пользователями SPIN-функции используются как механизм для построения правил высокого уровня (сложных SPARQL-запросов) на основе более простых правил.

Проиллюстрируем, каким образом правила OWL2 RL записываются на языке SPIN. Например, правило

$$C_2(X) \leftarrow C_1(X), \text{equivalentClass}(C_1, C_2)$$

может быть представлено на языке SPARQL следующим образом:

```
CONSTRUCT {
  ?X a ?C2.
}
WHERE {
  ?X a ?C1.
  ?C1 equivalentClass ?C2.
}
```

Кроме того, для класса owl:Thing нужно создать экземпляр spin:rules; это позволит применять данное правило ко всем возможным экземплярам.

Следует отметить, что SPARQL и, следовательно, SPIN являются языками правил, но не реализованной системой правил; например,

конструкция **CONSTRUCT** выражает только один шаг вывода (переход от одного RDF-графа к другому). Система правил на основе языка SPARQL (например, машина логического вывода SPIN) должна, среди прочего, выполнять конструкцию **CONSTRUCT** итеративно и управлять рекурсией в случае рекурсивных правил.

5.9. Немонотонные правила: назначение и синтаксис

5.9.1. Неформальное описание

Перейдем к рассмотрению систем немонотонных правил. Если доказаны посылки монотонного правила, то это правило может быть применено и в качестве следствия получена его голова. В системах немонотонных правил правило не может быть применено, даже если все посылки известны, потому что должны быть рассмотрены все взаимоисключающие цепочки логического вывода. В общем случае рассматриваемые далее правила называются *отменяемыми*, так как они могут быть отменены другими правилами. Чтобы позволить правилам конфликтовать между собой, допускается, что отрицание атомарных формул может входить как в голову, так в и тело правила. Например, можно записать правила вида:

$$\begin{aligned} p(X) &\rightarrow q(X), \\ r(X) &\rightarrow \neg q(X). \end{aligned}$$

Для того чтобы отличать отменяемые правила от стандартных монотонных правил, будем использовать в них стрелку другого вида:

$$\begin{aligned} p(X) &\Rightarrow q(X), \\ r(X) &\Rightarrow \neg q(X). \end{aligned}$$

С помощью этих правил, если даны факты

$$\begin{aligned} p(a), \\ r(a) \end{aligned}$$

невозможно получить ни следствие $q(a)$, ни следствие $\neg q(a)$. Это типичный пример двух правил, блокирующих друг друга. Данный конфликт может быть разрешен с помощью *приоритетов* для правил. Предположим, известно, что первое правило сильнее второго (то есть имеет более высокий приоритет); тогда можно получить следствие $q(a)$.

Приоритеты естественным образом определяются практическими задачами, а их выбор может быть основан на различных принципах:

- источник одного правила может быть более надежным, чем источник второго правила, или иметь более высокий приоритет. Например, федеральный закон перекрывает закон штата. А в области делового администрирования топ-менеджер имеет больше полномочий, чем менеджер среднего звена;
- одно правило может иметь более высокий приоритет, чем другое, на основании того, что оно новее (то есть сформировано позднее);
- одно правило может иметь более высокий приоритет, потому что оно является более конкретным. Типичный пример – общее правило с некоторыми исключениями; в этом случае исключения сильнее, чем правило.

Конкретность (исключительность) правила часто может быть вычислена на основе заданных правил, но первые два принципа не могут быть определены с помощью логической формализации. Поэтому мы абстрагируемся от конкретного принципа приоритетности и предположим существование *внешнего отношения приоритетов* на множестве правил. Чтобы выразить это отношение синтаксически, расширим синтаксис правил, введя понятие уникальной метки для правила. Например:

$$\begin{aligned} r_1 : p \neg(X) &\Rightarrow q(X), \\ r_2 : r(X) &\Rightarrow \neg q(X). \end{aligned}$$

Тогда можно явно указать

$$r_1 > r_2,$$

что означает, что правило r_1 сильнее, чем правило r_2 .

На отношение приоритета $>$ не накладываются какие-либо жесткие условия. Даже не требуется, чтобы правила указывались по порядку. Требуется только, чтобы отношение приоритета было ациклическим. То есть невозможно наличие циклов следующего вида:

$$r_1 > r_2 > \dots > r_n > r_1.$$

Отметим, что приоритеты предназначены для разрешения конфликтов между *конкурирующими правилами*. В простых случаях два правила являются конкурирующими, только если голова одного правила представляет собой отрицание главы второго правила. Однако в приложениях часто возникает следующая ситуация: если получен предикат p , то некоторые другие предикаты исключаются из рассмотрения. Например, консультант по инвестициям может давать рекомендации, учитывая три уровня риска для инвесторов: низкий,

средний и высокий. Однако для каждого инвестора в любой момент времени реализуется только один уровень риска. Технически эти ситуации моделируются путем поддержания множества конкурирующих правил $C(L)$ для каждого литерала L . $C(L)$ всегда содержит отрицание L , но может содержать и другие литералы.

5.9.2. Определение синтаксиса

Отменяемое правило имеет вид:

$$r : L_1, \dots, L_n \Rightarrow L,$$

где r – метка, $\{L_1, \dots, L_n\}$ – тело (или посылки) правила, L – голова правила, $L, L_1, \dots, L_n$ – положительные или отрицательные литералы (литерал – это атомная формула $p(t_1, \dots, t_m)$ или ее отрицание $\neg p(t_1, \dots, t_m)$). В правилах не могут использоваться функциональные символы¹. Иногда голову правила с меткой r обозначают как *head*(r), а тело – как *body*(r). Для сокращения записи будем писать «правило r » вместо «правило с меткой r ».

Отменяемая логическая программа – это триплет вида $(F, P, >)$, состоящий из множества фактов F , конечного множества отменяемых правил R и ациклического бинарного отношения $>$ на множестве R (точнее, набора пар, $r > r'$, где r и r' – метки правил из множества R).

5.10. Пример немонотонных правил: посреднические сделки

Этот пример показывает, каким образом немонотонные правила могут быть использованы в приложении электронной торговли (которое в идеале будет работать в семантическом вебе). Посреднические сделки осуществляются с участием независимой третьей стороны – брокера. Брокер сопоставляет требования покупателя и возможности продавца и предлагает сделку, которая может удовлетворить обе стороны путем торговли.

В качестве конкретного приложения рассмотрим приложение для сдачи квартир в аренду². Поиск квартиры – известная всем процедура, которая часто утомительна и отнимает много времени. Соответствующие веб-сервисы позволяют значительно уменьшить усилия. Начнем с представления требований потенциального арендатора.

¹ Это ограничение налагается по техническим причинам, обсуждение которых выходит за рамки этой главы.

² В этом случае роль абстрактного продавца играет арендодатель.

Карлос ищет квартиру с площадью не менее 45 кв. м и, по крайней мере, с двумя комнатами. Если квартира находится на третьем этаже или выше, то в доме должен быть лифт. Кроме того, в квартире должны быть разрешены домашние животные.

Карлос готов платить 300 долл. за расположенную в центре города квартиру с площадью 45 кв. м и 250 долл. за аналогичную квартиру на окраине. Кроме того, он готов платить 5 долл. за каждый дополнительный квадратный метр площади и 2 долл. за квадратный метр сада.

Карлос не в состоянии платить больше, чем 400 долл. в общей сложности. Если есть выбор, он предпочтет самый дешевый вариант. Его вторым приоритетом является наличие сада; самый низкий приоритет для него – наличие дополнительного пространства.

5.10.1. Формализация требований Карлоса

Для описания свойств квартиры будем использовать следующие предикаты с соответствующими значениями:

<i>квартира</i> (x)	x это квартира,
<i>площадь</i> (x, y)	квартира x имеет площадь y (в кв. м),
<i>комнаты</i> (x, y)	квартира x имеет y комнат,
<i>цена</i> (x, y)	квартира x имеет цену y ,
<i>этаж</i> (x, y)	квартира x находится на этаже y ,
<i>сад</i> (x, y)	квартира x имеет сад с площадью y ,
<i>лифт</i> (x)	в доме x есть лифт,
<i>домашние Животные</i> (x)	в квартире x разрешены домашние животные,
<i>центр</i> (x)	квартира x находится в центре города.

Кроме того, будем использовать предикаты:

<i>приемлема</i> (x)	квартира x удовлетворяет требованиям Карлоса,
<i>предлагаемая Цена</i> (x, y)	Карлос готов заплатить y долл. за квартиру x .

Теперь запишем требования Карлоса. Любая квартира априори является приемлемой:

$$r_1 : \text{квартира}(x) \Rightarrow \text{приемлема}(x)$$

Однако квартира y неприемлема, если одно из требований Карлоса не удовлетворяется.

r_2 : комнаты (X, Y) , $Y < 2 \Rightarrow \neg \text{приемлема}(X)$,
 r_3 : площадь (X, Y) , $Y < 45 \Rightarrow \neg \text{приемлема}(X)$,
 r_4 : $\neg \text{домашниеЖивотные}(X) \Rightarrow \neg \text{приемлема}(X)$,
 r_5 : этаж (X, Y) , $Y > 2$, $\neg \text{лифт}(X) \Rightarrow \neg \text{приемлема}(X)$,
 r_6 : цена (X, Y) , $Y > 400 \Rightarrow \neg \text{приемлема}(X)$.

Правила r_2 – r_6 являются исключениями для правила r_1 , поэтому добавляем следующие отношения приоритетов:

$r_2 > r_1, r_3 > r_1, r_4 > r_1, r_5 > r_1, r_6 > r_1$.

Затем мы вычислим цену, которую Карлос готов платить за квартиру.

r_7 : площадь (X, Y) , $Y \geq 45$, $\text{сад}(X, Z)$, $\text{центр}(X) \Rightarrow$
 $\text{предлагаемаяЦена}(X, 300 + 2Z + 5(Y - 45))$,
 r_8 : площадь (X, Y) , $Y \geq 45$, $\text{сад}(X, Z)$, $\neg \text{центр}(X) \Rightarrow$
 $\text{предлагаемаяЦена}(X, 250 + 2Z + 5(Y - 45))$.

Квартира приемлема только в том случае, если сумма, которую готов платить Карлос, не меньше, чем цена, указанная арендодателем (предполагается, что торг не возможен).

r_9 : $\text{предлагаемаяЦена}(X, Y)$, $\text{цена}(X, Z)$, $Y < Z \Rightarrow \neg \text{приемлема}(X)$,
 $r_9 > r_1$.

5.10.2. Представление информации о квартирах для сдачи в аренду

Каждой квартире, предназначенной для сдачи в аренду, дается уникальное имя, а ее свойства представляются как факты. Например, квартира a_1 может быть описан следующим образом:

$\text{комнаты}(a_1, 1)$,
 $\text{размер}(a_1, 50)$,
 $\text{центр}(a_1)$,
 $\text{этаж}(a_1, 1)$,
 $\neg \text{лифт}(a_1)$,
 $\text{домашниеЖивотные}(a_1)$,
 $\text{сад}(a_1, 0)$,
 $\text{цена}(a_1, 300)$.

Описания квартир для сдачи в аренду приведены в табл. 5.1. На практике информация о таких квартирах может храниться в реляционной базе данных или, если говорить о семантическом вебе, в хранилище RDF-триплетов.

Таблица 5.1. Квартиры для сдачи в аренду

Квар- тира	Комнаты	Площадь	Центр	Этаж	Лифт	Домашние животные	Сад	Цена
a_1	1	50	да	1	нет	да	0	300
a_2	2	45	да	0	нет	да	0	335
a_3	2	65	нет	2	нет	да	0	350
a_4	2	55	нет	1	да	нет	15	330
a_5	3	55	да	0	нет	да	15	350
a_6	2	60	да	3	нет	нет	0	370
a_7	3	65	да	1	нет	да	12	375

Если сопоставить требования Карлоса и квартиры для сдачи в аренду, то можно заметить, что

- квартира a_1 является неприемлемой, так как имеет только одну комнату (правило r_2);
- квартиры a_4 и a_6 являются неприемлемыми, потому что в них не разрешаются домашние животные (правило r_4);
- за квартиру a_2 Карлос готов платить 300 долл., но ее цена выше (правила r_7 и r_9);
- квартиры a_3 , a_5 , и a_7 являются приемлемыми (правило r_1).

5.10.3. Выбор квартиры

Выше были определены квартиры, приемлемые для Карлоса. Такой подбор подходящих квартир ценен сам по себе, поскольку сокращает количество квартир, которые Карлосу имеет смысл осматривать. Но это число можно еще уменьшить вплоть до одной квартиры, если принять во внимание предпочтения Карлоса. Напомним приоритеты Карлоса в порядке убывания: самая дешевая цена, самая большая площадь сада, дополнительная площадь квартиры. Представим их следующим образом:

- $r_{10} : \text{приемлема}(X) \Rightarrow \text{самаяДешевая}(X),$
 $r_{11} : \text{приемлема}(X), \text{цена}(X, Z), \text{приемлема}(Y), \text{цена}(Y, W),$
 $W < Z \Rightarrow \neg \text{самаяДешевая}(X),$
 $r_{12} : \text{самаяДешевая}(X) \Rightarrow \text{самыйБольшойСад}(X),$
 $r_{13} : \text{самаяДешевая}(X), \text{сад}(X, Z), \text{самаяДешевая}(Y),$
 $\text{сад}(Y, W), W > Z \Rightarrow \neg \text{самыйБольшойСад}(X),$
 $r_{14} : \text{самыйБольшойСад}(X) \Rightarrow \text{аренда}(X),$
 $r_{15} : \text{самыйБольшойСад}(X), \text{площадь}(X, Z), \text{самыйБольшойСад}(Y),$
 $\text{площадь}(Y, W), W > Z \Rightarrow \neg \text{аренда}(X)$
 $r_{11} > r_{10}, r_{13} > r_{12}, r_{15} > r_{14}.$

Правило r_{10} означает, что каждая приемлемая квартира является самой дешевой по умолчанию. Тем не менее если существует приемлемая квартира дешевле, чем квартира x , то в соответствии с правилом r_{11} (которое сильнее правила R_{10}) будет сделан вывод, что квартира x не самая дешевая.

Аналогичным образом правила r_{12} и r_{13} позволяют выбрать квартиры с самым большим садом среди самых дешевых квартир. А из них правила r_{14} и r_{15} позволяют выбрать квартиры в зависимости от их площади.

В нашем примере квартиры a_3 и a_5 самые дешевые. Из них квартира a_5 имеет сад с большей площадью. Обратите внимание, что в этом случае критерий размера квартиры не играет роли: правило r_{14} выполняется только для квартиры a_5 , а правило r_{15} не применяется. Таким образом, выбор квартиры a_5 сделан, и Карлос скоро переедет в нее.

5.11. Язык разметки правил RuleML

Язык RuleML (Rule Markup Language) – это результат долгосрочного проекта по разработке разметки правил в вебе. На самом деле это не один язык, а семейство языков разметки правил, соответствующих различным видам языков правил: правил вывода, ограничений целостности, реактивных правил и т. д. Ядром семейства языков RuleML является язык *Datalog*, который соответствует логике Хорна без функциональных символов.

RuleML является достаточно экспериментальным языком в том смысле, что он учитывает особенности языков правил, еще далеких от стандартизации (например, немонотонных правил). Идея заключается в том, что эти особенности в будущем могут служить основой для новых стандартов. Так, элементы языка RuleML играли большую роль при разработке стандарта формата RIF.

Таблица 5.2. Ключевые слова языка RuleML

Элемент правила	Обозначение в RuleML
факт	Asserted Atom
правило	Asserted Implies
голова	then
тело	if
атом	Atom
конъюнкция	And
предикат	Rel
константа	Ind
переменная	Var

RuleML обеспечивает описание языков разметки правил на языке XML, в формате RELAX NG или в виде XML-схем (или форматах более ранних версий). Элементы правил записываются достаточно просто. В табл. 5.2 представлены ключевые слова языка Datalog RuleML.

Правила с использованием ключевых слов языка RuleML также записываются просто. Например, правило «Предоставить клиенту скидку 7,5 процента при покупке продукта, если клиент имеет статус “премиум” и продукт относится к эксклюзивным» может быть представлено на языке RuleML 1.0 следующим образом:

```
<Implies>
  <then>
    <Atom>
      <Rel>скидка</Rel>
      <Var>клиент</Var>
      <Var>продукт</Var>
      <Ind>7.5 процента</Ind>
    </Atom>
  </then>
  <if>
    <And>
      <Atom>
        <Rel>премиум</Rel>
        <Var>клиент</Var>
      </Atom>
      <Atom>
        <Rel>эксклюзивный</Rel>
        <Var>продукт</Var>
      </Atom>
    </And>
  </if>
</Implies>
```

Язык SWRL, введенный в разделе 5.7, является расширением языка RuleML, и его использование также является простым. В качестве примера рассмотрим следующее правило:

брат(X, Y), *ребенок*(Z, Y) $\rightarrow$ *дядя*(X, Z).

Это правило в XML-синтаксисе языка SWRL на основе RuleML 1.0 записывается следующим образом:

```
<ruleml:Implies>
  <ruleml:then>
    <swrlx:individualPropertyAtom swrlx:property="дядя">
      <ruleml:Var>X</ruleml:Var>
      <ruleml:Var>Z</ruleml:Var>
    </swrlx:individualPropertyAtom>
```



```

</ruleml:then>
<ruleml:if>
  <ruleml:And>
    <swrlx:individualPropertyAtom swrlx:property="брат">
      <ruleml:Var>X</ruleml:Var>
      <ruleml:Var>Y</ruleml:Var>
    </swrlx:individualPropertyAtom>
    <swrlx:individualPropertyAtom swrlx:property="ребенок">
      <ruleml:Var>Z</ruleml:Var>
      <ruleml:Var>Y</ruleml:Var>
    </swrlx:individualPropertyAtom>
  </ruleml:And>
</ruleml:if>
</ruleml:Implies>

```

5.12. Выводы

- Правила в (семантическом) вебе реализуются с помощью разнообразных и разнородных технологий.
- Логика Хорна представляет подмножество логики предикатов, для которого возможен эффективный логический вывод. Она ортогональна дескрипционным логикам и является основой монотонных правил.
- RIF – это новый стандарт правил в вебе. Его логический диалект BLD основан на логике Хорна.
- Профиль OWL2 RL, который, по сути, представляет собой объединение дескрипционных логик и логики Хорна, может быть встроен в формат RIF.
- SWRL – гораздо более богатый язык, который объединяет дескрипционные логики с определенными типами правил.
- Использование немонотонных правил необходимо в ситуациях, когда доступная информация не является полной. Немонотонные правила – это правила, которые могут быть отменены противоречащими им данными (другими правилами).
- Приоритеты используются для разрешения конфликтов между немонотонными правилами.
- Представление правил с помощью XML-синтаксиса таких языков, как RIF и RuleML, достаточно просто.

Рекомендуемая литература

Логика Хорна – стандартный раздел курсов по логике. Дополнительную информацию по этой теме можно найти в следующих учебниках:

- Burke E. and Foxley E. Logic and Its Applications. Upper Saddle River, N. J.: Prentice Hall, 1996.
- Covington M. A., Nute D., and Vellino A. Prolog Programming in Depth, 2nd ed. Upper Saddle River, N. J.: Prentice Hall, 1997.
- Nerode A. and Shore R. A. Logic for Applications. New York: Springer, 1997.
- Nilsson U. and Maluszynski J. Logic, Programming and Prolog, 2nd ed. New York: Wiley, 1995.
- Nissanke N. Introductory Logic and Sets for Computer Scientists. Boston: Addison-Wesley, 1998.

Немонотонные правила – это достаточно новая тема. Информацию по ней можно найти в указанных ниже двух учебниках и следующих статьях:

- Antoniou G., Billington D., Governatori G., and Maher M. J. Representation Results for Defeasible Logic. ACM Transactions on Computational Logic 2 (April 2001): 255–287.
- Bassiliades N., Antoniou G., and Vlahavas I. A Defeasible Logic Reasoner for the Semantic Web. International Journal on Semantic Web and Information Systems 2,1 (2006): 1–41.
- Eiter T., Lukasiewicz T., Schindlauer R., and Tompits H. Combining Answer Set Programming with Description Logics for the Semantic Web. In Proceedings of the 9th International Conference on Principles of Knowledge Representation and Reasoning (KR'04), AAAI Press 2004, 141–151.
- Nute D. Defeasible Logic. In Handbook of Logic in Artificial Intelligence and Logic Programming Vol. 3, ed. D. M. Gabbay, C. J. Hogger, and J. A. Robinson. New York: Oxford University Press, 1994.

Информацию о формате RIF и его совместимости с языками RDF и OWL можно найти в следующих источниках:

- <http://www.w3.org/2005/rules/wiki/Primer>;
- <http://www.w3.org/TR/rif-overview/>;
- <http://www.w3.org/TR/rif-bld/>;
- <http://www.w3.org/TR/rif-rdf-owl/>.

Введение в формальные основы диалекта RIF-BLD представлено

в книге:

- Kifer M. Knowledge Representation and Reasoning on the Semantic Web: RIF. In Handbook of Semantic Web Technologies, eds. J. Domingue, D. Fensel, and J. Hendler. Springer 2011.

Информация о профиле OWL2 RL и его встраивании в формат RIF:

- <http://www.w3.org/TR/owl2-profiles/>;
- <http://www.w3.org/TR/rif-owl-rl/>.

Информация о языке SPIN и кодировании с помощью него части профиля OWL2 RL:

- <http://www.w3.org/Submission/spin-overview/>;
- <http://topbraid.org/spin/owlrl-all.html>.

Информация о языке SWRL:

- www.w3.org/Submission/SWRL/.

Другие важные работы по интеграции правил и дескрипционных логик:

- Horrocks, Patel-Schneider P., Bechhofer S., and Tsarkov D. OWL Rules: A Proposal and Prototype Implementation. *Journal of Web Semantics* 3,1 (2005): 23–40 // www.websemanticsjournal.org/ps/pub/2005-2.
- Motik B., Sattler U., and Studer R. Query Answering for OWL-DL with Rules. *Journal of Web Semantics* 3,1 (2005): 41–60 // www.websemanticsjournal.org/ps/pub/2005-3.
- Rosati R. On the Decidability and Complexity of Integrating Ontologies and Rules. *Journal of Web Semantics* 3,1 (2005): 61–73 // www.websemanticsjournal.org/ps/pub/2005-4.

Общая информация о языках разметки правил и его использовании в семантическом вебе представлена на сайте RuleML:

- www.ruleml.org/.

Упражнения и проекты

Следующие проекты используют технологию правил. Предполагается, что читатель имеет некоторые знания в области семантического веба и опыт работы с основными технологиями семантического веба. Если это не так, то лучше сначала выполнить базовые проекты, предложенные в главе 7.

Семантическая посредническая деятельность

1. Этот базовый проект может быть выполнен двумя людьми в течение двух или трех недель. Цель проекта заключается в реализации приложения, подобного приложению для аренды квартир, рассмотренного в качестве примера в разделе 5.10. Необходимо выполнить следующие задачи:
 - а) выберите область деятельности, в которой могут осуществляться посреднические сделки. Посредническая деятельность – это деятельность по сопоставлению предложений продавца и требований покупателя;
 - б) постройте RDFS-онтологию для выбранной предметной области;

- в) заполните онтологию предложениями о продаже, записанными на языке RDF;
 - г) запишите критерии подбора предложений, используя немонотонные правила;
 - д) выполните правила для ваших данных в формате RDF/RDFS, используя такие машины логического вывода, как DEVICE¹ или DR-Prolog². Для этого надо записать правила в формате, поддерживаемом выбранной системой.
2. Этот проект повышенной сложности может быть осуществлен двумя или тремя людьми в течение семестра. Цель проекта заключается в реализации сценария посреднической деятельности в мультиагентной среде. Помимо выполнения шагов, описанных в проекте 5.1, участники проекта должны, среди прочего, выполнить следующие задачи:
- а) разработайте концепцию посреднической деятельности в условиях мультиагентной среды на основе изучения соответствующей литературы:
 - Sycara K., Widoff S., Klusch M., and Lu J. Larks: Dynamic Matchmaking among Heterogeneous Software Agents in Cyberspace. *Autonomous Agents and Multi-Agent Systems* 5, 2 (2002): 173–203;
 - Antoniou G., Skylogiannis T., Bikakis A., and Bassiliades N. A Deductive Semantic Brokering System. In *Proceedings of the 9th International Conference on Knowledge-Based Intelligent Information and Engineering Systems*. LNCS 3682, Springer 2005, 746–752;
 - б) выберите мультиагентную систему и самостоятельно ознакомьтесь с ней. У нас был положительный опыт работы с системой JADE³;
 - в) решите, какими именно сообщениями будут обмениваться агенты;
 - г) выясните, каким образом можно удаленно использовать машину логического вывода.

Уровень доказательств

Целью приведенных ниже проектов является реализация слоя доказательств, который был кратко представлен на концептуальном уровне в главе 1. Заметим, не что существует единого слоя доказательств;

¹ lpis.csd.auth.gr/systems/dr-device.html.

² www.csd.uoc.gr/~bikakis/DR-Prolog/.

³ jade.tilab.com/.

для каждой системы логического вывода в семантическом вебе существует свой слой доказательств. Тем не менее некоторые соображения являются общими для всех подобных систем. Слой доказательств можно реализовать для двух типов логических систем:

- простые языки монотонных правил, такие как Datalog (логика Хорна без функциональных символов), для которых может быть использовано средство логического вывода Mandarax¹;
 - языки немонотонных правил, описанные в этой главе, для которых могут быть использованы машины логического вывода DR-DEVICE² или DR-Prolog³.
3. Этот базовый проект может быть выполнен двумя или тремя людьми примерно в течение двух месяцев. Целью проекта является разработка интерактивной системы, которая предоставляет пользователям объяснение логического вывода. Необходимо выполнить следующие задачи:
- а) решите, каким образом извлечь релевантную информацию из полного описания шагов доказательства. Для поиска идей вы можете использовать литературу по автоматизированным рассуждениям и логическому программированию;
 - б) определите уровень детализации представления доказательств. Должно ли доказательство отображаться полностью или только его меташаги? Может ли быть детализирован определенный шаг доказательства по желанию пользователя?
 - в) в качестве «листьев» доказательств используйте RDF-факты, правила или условия вывода.
4. Этот проект повышенной сложности может быть выполнен четырьмя или пятью людьми в течение семестра. Цель проекта заключается в реализации слоя доказательств в мультиагентной среде, то есть агенты должны обмениваться запросами и доказательствами. В качестве дополнительных задач требуется решить следующие задачи:
- а) выберите мультиагентную систему и самостоятельно ознакомьтесь с ней. У нас был положительный опыт работы с системой JADE⁴;
 - б) запишите доказательства на языке XML, в идеале с помощью расширения языка RuleML;
 - в) решите, какими именно сообщениями будет обмениваться агенты.

¹ mandarax.sourceforge.net/.

² lpis.csd.auth.gr/systems/dr-device.html.

³ www.csd.uoc.gr/~bikakis/DR-Prolog/.

⁴ jade.tilab.com/.

Глава 6

Приложения

В первые годы появления (2000–2007 гг.) технологии семантического веба использовались в веб-приложениях для промышленных и научных проектов, в которых не предполагалось широкомасштабного производства. Однако в последние годы наблюдается резкий рост количества приложений, использующих технологии семантического веба; с 2008 года в различных секторах бизнеса и коммерции начали внедряться полномасштабные промышленные системы, которые используют все преимущества данных технологий. В этой главе мы кратко опишем некоторые из таких приложений.

Обзор главы

В главе описывается, каким образом внедрение онтологии GoodRelations влияет на изменение онлайн-сектора розничной торговли (раздел 6.1), как британская широковеб-корпорация BBC использует технологии семантического веба для поддержания и публикации своих архивов об артистах и музыке (раздел 6.2), а также для репортажей о широко освещаемых в СМИ спортивных событиях (раздел 6.3), как правительства стран используют семантические технологии для публикации государственных данных (раздел 6.4) и как эти технологии использует издательский бизнес на примере газеты New York Times (раздел 6.5). В конце главы обсуждается будущее веб-поиска и кратко описываются сервисы Sig.ma (раздел 6.6), OpenCalais (раздел 6.7) и Schema.org (раздел 6.8).

6.1. GoodRelations

6.1.1. Общая информация

Одним из главных факторов, способствующих быстрому внедрению всемирной паутины WWW в повседневную жизнь, стала электронная коммерция и, в частности, коммерция типа «бизнес для потребителя» (B2C). Сейчас повсеместно можно увидеть веб-адреса, ука-

занные на витринах магазинов и грузовом транспорте, перевозящем товары. Только в Великобритании В2С-рынок вырос с 87 млн фунтов стерлингов в апреле 2000 года до 68,4 млрд фунтов стерлингов к концу 2009 года, то есть более чем в тысячу раз за одно десятилетие.

Эта большая площадка электронной торговли страдает от всех недостатков традиционного веба: сайты электронной коммерции, как правило, генерируются на основе таких данных структурированных информационных систем, как списки цен, наличие и типы продуктов, варианты доставки и т. д. Но прежде чем эта информация появляется на веб-странице компании, она преобразуется в HTML-формат и, следовательно, теряет свою машинно интерпретируемую структуру, в результате чего машины уже не могут отличить цену продукта от кода продукта. Поисковые системы страдают от этой неспособности интерпретировать страницы электронной коммерции, которые они пытаются сканировать и индексировать, поисковые роботы не в состоянии правильно различать типы товаров или производить имеющую смысл группировку товаров.

GoodRelations¹ представляет собой онтологию на языке OWL, описывающую предметную область электронной коммерции. Эта онтология может быть использована для того, чтобы представить предложение продукта, указать цену, описать бизнес и т. п. Синтаксис RDFa онтологии GoodRelations позволяет встраивать эту информацию в существующие веб-страницы, и, следовательно, она может быть обработана другими компьютерами. Основное преимущество и главный фактор, определяющий быстро растущую популярность GoodRelations, заключается в повышении качества поиска. Добавление терминов онтологии GoodRelations в веб-страницы улучшает видимость предложений товаров для современных поисковых и рекомендательных систем. GoodRelations позволяет создавать аннотации для продуктов и услуг в вебе таким образом, что они могут быть использованы поисковыми системами с целью улучшения поиска необходимой пользователям информации. На базе этих аннотаций поисковые системы позволяют строить специальные поисковые запросы и получать очень точные ответы.

Кроме информации о продукции и поставщиках, онтология GoodRelations также позволяет выразить коммерческие и функциональные детали транзакций электронной коммерции, такие как страны, в которые осуществляется доставка, способы оплаты и доставки, оптовые скидки, часы работы и т. д.

¹ <http://www.heppnetz.de/projects/goodrelations/>.

Онтология GoodRelations¹ содержит огромное число классов и свойств, например классы `gr:ProductOrServiceModel`, `gr:PriceSpecification`, `gr:OpeningHoursSpecification` и `gr:DeliveryChargeSpecification` и свойства `gr:typeOfGood`, `gr:acceptedPaymentMethods`, `gr:hasCurrency` и `gr:eligibleRegions`.

6.1.2. Пример

Приведем простой, но реалистичный пример, взятый с сайта магазина Karneval Alarm, который торгует маскарадными костюмами в Германии. Отдельная страница этого сайта² описывает костюм супермена, который имеет размер 48/50 и продается по цене 59,90 евро. Этот продукт (с номером 935 в каталоге магазина) представляется в виде RDF-сущности `offering_935`. Страница данного продукта представлена с помощью синтаксиса RDFa и содержит, среди прочего, следующие RDF-утверждения:

```
offering_935 gr:name "Superman Kostum 48/50" ;
  gr:availableAtOrFrom http://www.karneval-alarm.de/#shop ;
  gr:hasPriceSpecification UnitPriceSpecification_935 .
UnitPriceSpecification_935 gr:hasCurrency "EUR" ;
  gr:hasCurrencyValue "59.9" ;
  gr:valueAddedTaxIncluded "true" .
```

Эти описания также могут быть просмотрены в вебе³ с помощью инспектора RDFa поисковой системы Sindice⁴. Полный граф представлен на рис. 6.1, кроме того, его можно изучить онлайн с возможностью увеличения масштаба⁵.

Конечно, в подобной аннотации GoodRelation необходимо указать тип или категорию товара. Для этого используется онтология The Product Types Ontology⁶, которая описывает товары на основе информации из Википедии.

Приведем пример описания класса маскарадных костюмов:

```
pto:Party_costume a owl:Class;
  rdfs:subClassOf gr:ProductOrService;
  rdfs:label "Party Costume"@en;
  rdfs:comment ""A party costume is clothing..."@en
```

¹ <http://www.heppnetz.de/ontologies/goodrelations/v1>.

² <http://www.karneval-alarm.de/superman-m.html>.

³ <http://www.karneval-alarm.de/superman-m.html>.

⁴ <http://inspector.sindice.com/>.

⁵ <http://inspector.sindice.com/inspect?url=http%3A//www.karneval-alarm.de/superman-m.html#GRAPH>.

⁶ <http://www.productontology.org/>.

которое позволяет сформулировать следующее утверждение:

`offering_935 pr:typeOfGood pto:Party_Costume.`



Рис. 6.1 ❖ Триплеты, извлеченные из RDFS-разметки страницы, описывающей костюм супермена

Данная онтология товаров содержит несколько сотен тысяч определений классов товаров на языке OWL DL. Эти определения классов тесно связаны с Википедией: редакционные изменения в Википедии отражаются в онтологии товаров. Это означает, что если поставщик продает продукт, который не представлен в онтологии товаров, то он может создать страницу для своего продукта в Википедии, и в течение 24 часов в онтологии появится соответствующий класс.

6.1.3. Использование

Первой компанией, начавшей использовать онтологию GoodRelations в больших масштабах, была компания BestBuy, крупный американский ритейлер бытовой техники и электроники. BestBuy сообщила о 30-процентном увеличении поискового трафика для страниц, аннотированных с помощью GoodRelation, и значительном повышении коэффициента кликабельности. В настоящее время Google рекомендует использовать GoodRelations для семантической разметки страниц электронной коммерции, и как Google, так и Yahoo сканируют RDFS-утверждения и используют их для улучшения представления результатов поиска. На данный момент рассматриваемую онтологию используют ритейлер Overstock.com, издательство O'Reilly Media, сеть магазинов одежды Peek & Cloppenburg и более мелкие предприятия, такие как Robinson Outdoors и вышеупомянутый магазин Karneval Alarm. На момент написания книги семантическая поис-

ковая система Sindice возвращает 273 000 страниц, аннотированных с помощью словаря GoodRelations.

6.1.4. Публикация

Hepp Martin. GoodRelations: An Ontology for Describing Products and Services Offers on the Web. In Proceedings of the 16th International Conference on Knowledge Engineering and Knowledge Management (EKAW2008). Acitrezza, Italy. September 29 – October 3, 2008. Springer LNCS, Vol 5268, 332–347.

6.2. BBC Artists

6.2.1. Общая информация

Проект BBC Music Beta¹ представляет собой инициативу BBC, направленную на разработку семантически связанных и аннотированных веб-страниц об артистах и певцах, чьи песни звучат на радиостанциях BBC. На этих страницах коллекции данных описаны и взаимосвязаны с помощью семантических метаданных, что позволяет меломанам исследовать связи между артистами, о существовании которых они, возможно, еще не знали.

До появления проекта редакторы BBC должны были самостоятельно создавать (и обновлять) интересный и актуальный контент для каждой публикуемой страницы об артистах. Вместо этого в рамках проекта Music Beta BBC автоматически собирает информацию с внешних сайтов, таких как MusicBrainz и Википедия, и агрегирует ее для создания собственных веб-страниц.

Для этих целей BBC внедрила стандарт RDF и построила отображение между собственной схемой данных и схемой, опубликованной MusicBrainz, для того чтобы использовать уникальные идентификаторы, которые предоставляет данный проект. Это позволяет сайту BBC максимально использовать общедоступный контент рассматриваемой предметной области, пополняя свои страницы найденной таким образом информацией. MusicBrainz является открытой «метабазой данных», содержащей музыкальный контент, в том числе информацию о более чем 400 000 артистах; информация, содержащаяся в статьях Википедии, публикуется в виде наборов данных проекта DBpedia.

¹ <http://www.bbc.co.uk/music/artists>.

6.2.2. Пример

В качестве примера рассмотрим страницу сайта BBC Artist, посвященную Джону Леннону¹. На первый взгляд, эта веб-страница выглядит так же, как и любая обычная веб-страница с такой информацией об артисте, как биографические сведения, профессиональная информация (артисты, с которыми выступал Джон Леннон, группы, в которых он играл), рецензии на альбомы и т. д. Но страница также содержит большой набор RDF-триплетов, в которых используется идентификатор MusicBrainz 4d5447d7 для ресурса, обозначающего Джона Леннона. Приведем некоторые из этих триплетов:

```
5c014631#artist foaf:name "John Lennon"
```

```
4d5447d7#artist bio:event _:node15vknin3hx2
_:node15vknin3hx2 rdf:type bio:Death
_:node15vknin3hx2 bio:date "1980-12-08"
```

```
4d5447d7#artist foaf:made _:node15vknin3hx7
_:node15vknin3hx7 dc:title "John Lennon/Plastic Ono Band"
4d5447d7#artist owl:sameAs dbpedia:John_Lennon
```

Данные триплеты означают, что ~~художник~~ по имени Джон Леннон умер 12 декабря 1980 года, что он сделал запись под названием «John Lennon/Plastic Ono Band» и что он также может быть идентифицирован с помощью URI `dbpedia: John_Lennon`.

Страница сайта BBC Artist, посвященная Джону Леннону, в общем, содержит 60 триплетов, на основе которых можно получить еще 300 триплетов из 40 различных онтологий, описывающих семейные отношения, музыку, время и даты, географическую информацию, социальные отношения и другие предметные области.

В общем сайт содержит порядка 400 000 страниц об артистах, 160 000 внешних ссылок и 100 000 отношений типа «художник–художник».

Использование технологий семантического веба, а именно использование URI в качестве идентификаторов ресурсов и совмещение их с внешними источниками семантических данных, означает, что веб-страницы могут создаваться и поддерживаться лишь с помощью малой части от того количества работников, которые были необходимы для решения этих задач в прошлом. Интересно отметить, что ВСС не только использует внешние информационные ресурсы, но также

¹ <http://www.bbc.co.uk/music/artists/4d5447d7-c61c-4120-ba1b-d7f471d385b9>.

повышает их доступность для всех пользователей. Простое добавление расширения .rdf к URI любой веб-страницы проекта BBC Artist, созданной в формате RDF, на самом деле служит распространению RDF. Публикуя свои данные в RDF-формате, BBC обеспечит их доступность третьим лицам, желающим использовать их где-либо.

Конечно, при использовании общедоступной информации в качестве входных данных всегда есть риск, что эта информация содержит ошибки. В таких случаях BBC не исправляет эти ошибки у себя на сайте. Вместо этого ошибки исправляются на внешних источниках, таких как MusicBrainz и DBpedia. Это не только приводит к устранению ошибок на сайте BBC (так как он основан на данных из этих источников), но и к исправлению ошибок, видимых любым пользователям этих проектов, что способствует повышению качества общедоступных источников данных.

6.2.3. Использование

Проект BBC Artist является «частью общего движения, которое осуществляется на BBC, заключающего в переходе от публикации веб-страниц, созданных на основе традиционных систем управления информацией, к публикации данных, которые можно использовать в вебе более динамичным образом». BBC разрабатывает и другие части своего сайта с использованием стандартов связанных данных. Например, информация о программах передач¹ публикуется с использованием URI из онтологии программ BBC², а информация о дикой природе, представленная в большом количестве соответствующих передач BBC высокого качества, – с использованием собственной онтологии BBC WildLife ontology³.

6.3. Сайт BBC World Cup 2010

6.3.1. Общая информация

Кроме обеспечения вещания радио- и телепрограмм, BBC прилагает значительные усилия для создания веб-сайтов, представляющих новости и другую мн^дийную информацию. Например, при создании сайта BBC World Cup 2010, посвященного чемпионату мира по футболу 2010 года, были использованы семантические технологии. Это

¹ <http://www.bbc.co.uk/programmes>.

² <http://www.bbc.co.uk/ontologies/programmes/2009-09-07.shtml>.

³ <http://www.bbc.co.uk/ontologies/wildlife/2010-02-22.shtml>.

позволяет автоматизировать публикацию контента, увеличить количество публикуемых страниц и при этом сократить численность поддерживающих их сотрудников, обеспечить семантическую навигацию и персонализацию представления информации.

6.3.2. Пример

Вид типичной страницы игрока чемпионата представлен на рис. 6.2¹. Похожие страницы существуют для сотен игроков, десятков команд, всех групп, всех матчей и т. д., и, конечно, все они тесно взаимосвязаны.

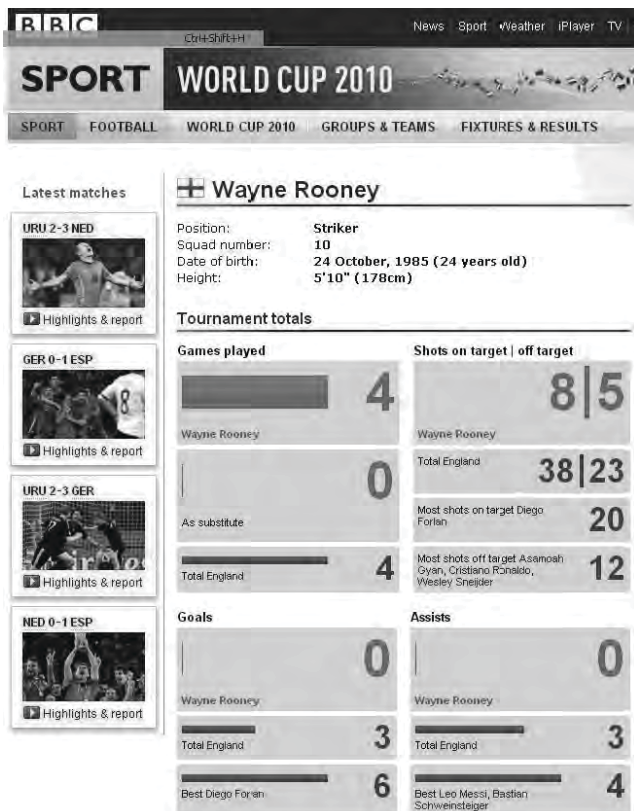


Рис. 6.2 ❖ Типовая страница сайта BBC World Cup

¹ http://www.bbc.co.uk/sport/football/world_cup_2010/groups_and_teams/team/england/wayne_rooney.

BBC разработала небольшие онтологии для описания предметной области футбола, в том числе предметно-ориентированных понятий, касающихся футбольных команд и турниров, а также самых общих понятий для описания событий и географических мест, используя такие хорошо известные онтологии, как FOAF¹ и GeoNames².

На рис. 6.3 показан небольшой фрагмент онтологии BBC World Cup. Эта онтология используется для логического вывода дополнительной информации, предоставляемой пользователю, например в каких национальных соревнованиях участвует игрок, членом какой команды является игрок и т. п.

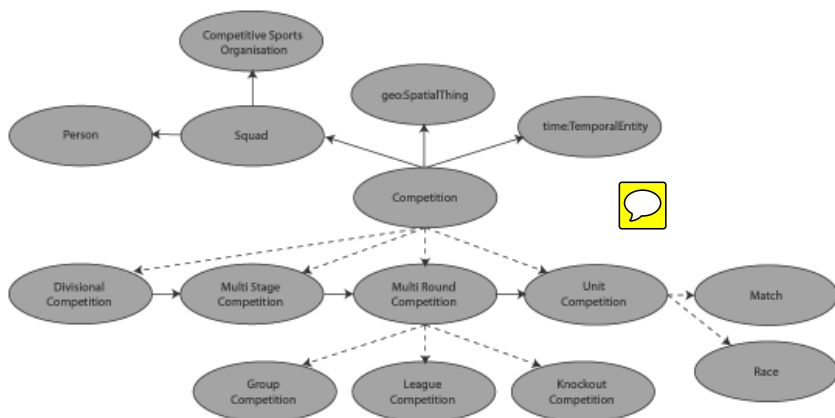


Рис. 6.3 ❖ Часть онтологии BBC, посвященной спортивным соревнованиям

6.3.3. Использование

Система, содержащая данные сайта BBC World Cup, имеет классическую трехуровневую архитектуру: (I) вся информация сохранена в хранилище RDF-триплетов, (II) это хранилище триплетов организовано на основе онтологий, что позволяет делать запросы к нему, (III) пользовательский интерфейс отображает для пользователей информацию, которая получена с помощью запросов к хранилищу триплетов, выполняемых на уровне онтологий.

BBC утверждает, что использование такой модели публикации данных позволило в значительной степени увеличить возможности по-

¹ <http://xmlns.com/foaf/spec/>.

² <http://www.geonames.org/>.

вторного использования контента и использования его в различных целях, снизить численность журналистов, необходимых для поддержки сайта, и улучшить пользовательский интерфейс за счет оформления страниц в соответствии с семантикой данных и наличием разнообразных «точек входа» на сайт (игроки, матчи, группы и т. д.).

Система использует хранилище триплетов коммерческого вендора, максимальная производительность которого характеризуется следующими возможностями: обработка тысяч SPARQL-запросов в минуту (в целом более миллиона запросов в день), вставка и обновление сотен RDF-утверждений каждую минуту, а также реализация полного OWL-вывода.

ВВС планирует применять тот же подход для веб-сайта, посвященного Олимпиаде 2012, который должен содержать информацию о более чем 10 000 спортсменов из более чем 200 стран, страницы, посвященные каждой дисциплине и каждому событию, отображающие в реальном режиме времени статистику игр, и хранить 58 000 часов видеоконтента. На рис. 6.4 показана лишь небольшая часть общей спортивной онтологии, разрабатываемой ВВС¹.

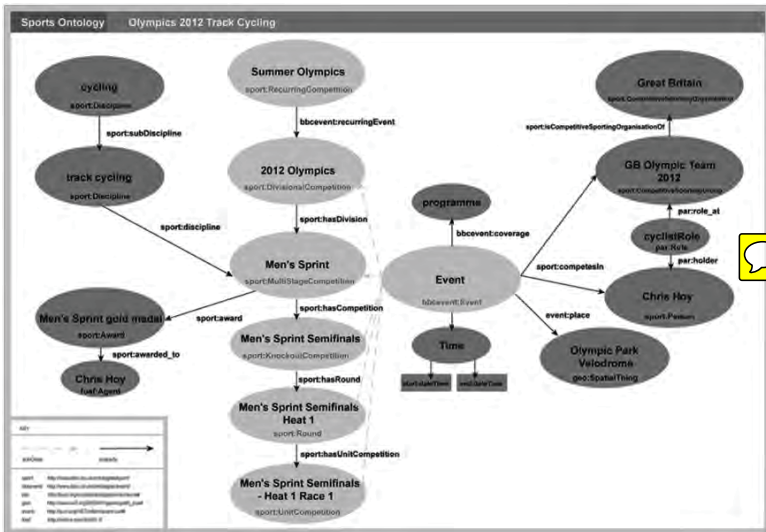


Рис. 6.4 ❖ Фрагмент онтологии BCC Sport Ontology

¹ На момент перевода книги компанией BBC уже разработана версия Sport Ontology 3.2. См. <http://www.bbc.co.uk/ontologies/sport>. – *Прим. перев.*

6.4. Правительственные данные

6.4.1. Общая информация

Один из первых в мире крупномасштабных проектов, использующих технологии семантического веба, можно считать результатом компании администрации президента США Обамы, направленной на увеличение прозрачности деятельности правительства. Публикация источников данных, которые традиционно использовались исключительно в правительственных учреждениях и не были доступны общественности, рассматривается как важный шаг к увеличению прозрачности работы правительства. И представляется, что именно технологии семантического веба являются одними из ключевых технологий для публикации этих данных. На сайте <http://data.gov> уже сейчас опубликовано 250 000 наборов данных из 250 различных государственных учреждений, начиная от экономических показателей и заканчивая статистикой медицинских услуг и данными об экологии. Не все эти данные (или даже немногие из них) представлены в формате *связанных данных* (*Linked Data*), то есть используют RDF-модель, URI, онтологии и связи с другими наборами данных. Здесь уместно рассмотреть 5-звездочную шкалу классификации связанных данных, предложенную консорциумом W3C, которая позволяет начинать публикацию данных с помощью простых технологий. Кратко опишем эту шкалу:

** Размещение любых данных в вебе в любом формате, в любом месте.* Например, просто размещение PDF-документа с таблицами (или даже отсканированных документов) на каком-либо веб-сайте уже будет оценено одной звездой по данной шкале связанных данных. Хотя с технической точки зрения это тривиальная задача, но ее выполнение уже означает преодоление социальных, организационных и бюрократических препятствий.

*** Использование машиночитаемых форматов.* Это означает, что следует избегать форматов, подобных PDF. Публикация таблиц Excel является наиболее типичным примером двухзвездной публикации данных.

**** Использование открытых форматов.* Это означает, что следует избегать проприетарных форматов, таких как Excel, и вместо этого использовать открытые форматы, такие как CSV или форматы Open Office. Обычно достаточно просто преобразовать формат данных, чтобы перейти от уровня двух звезд к уровню трех звезд.

**** *Использование URL для каждого элемента данных.* Это первый шаг, на котором появляется веб-семантика: использовать URI необходимо для обозначения каждого типа данных, элемента данных и свойства. Это позволяет другим пользователям ссылаться на опубликованный набор данных.

***** *Использование общих словарей.* Например, можно использовать термин «city» из DBpedia, когда речь идет о классе всех городов, и другие подобные термины для обозначения сущностей и свойств. Такое использование внешних словарей позволяет установить взаимосвязи между наборами данных различных сайтов.

6.4.2. Использование

Хотя большая часть данных сайта <http://data.gov> опубликована (и до сих пор представлена) только в формате двух звезд (по вышеупомянутой системе ранжирования), оказалось, что несложно преобразовать эти данные в форматы более высокого уровня. В течение восьми месяцев студентам политехнического института Ренсселера (США) удалось преобразовать 6,4 млрд элементов данных с сайта <http://data.gov> и, что более существенно, создать приложения для таких разнообразных проектов, как отображение зависимостей: расходов почтовой службы от ее производительности, лесных пожаров от бюджета ведомств (для измерения эффективности), межгосударственной миграции от информации из налоговых форм, семейного дохода от работы программы медицинской помощи; а также сравнения бюджетов ведомств по трем наборам данных, содержащих общедоступную информацию о бюджетах, и отображения социальных сетей людей, посещающих Белый дом. Хотя невозможно установить точную причинно-следственную связь, сообщается, что количество обращений граждан по поводу соблюдения закона о свободе информации существенно снизилось с момента запуска проекта <http://data.gov>.

Воодушевленные таким успехом, другие правительства начали следовать этому примеру, в том числе правительства многих государств, например Канады, Ирландии, Норвегии, Австралии и Новой Зеландии, администрации крупных городов в США; в таких странах, как Франция, Италия, Дания, Австрия, Германия, и многих других начали действовать гражданские инициативы в этой области. Возможно, самый значительный проект был реализован в Великобритании, где сайт <http://data.gov.uk> получил серьезную поддержку от правительства. Данный проект рассматривается не только как шаг к обеспечению большей прозрачности деятельности правительства, но и как

механизм снижения расходов. Вместо того чтобы разрабатывать и поддерживать дорогостоящие веб-сайты с государственной информацией, правительство просто публикует основные источники данных и призывает третьи стороны (будь то граждане или коммерческие компании) разрабатывать сервисы на основе этих опубликованных источников. Только в США были зарегистрированы более 200 приложений, разработанных третьими сторонами. К известным во всем мире приложениям такого типа можно отнести следующие проекты: карту безопасности велосипедных маршрутов, региональную карту расходов, сайты, предоставляющие информацию для покупателей домов об их новых соседях, информацию о повышении уровня загрязнения окружающей среды, сервисы поиска школы и детского сада, проект WhereDidMyTaxGo («На что потрачены мои налоги»)¹.

Межправительственные учреждения идут по тому же пути, например Всемирный банк (data.worldbank.org), веб-сайт Европейской комиссии для проведения процедур конкурсного размещения заказов (ted.europa.eu) и статистическая служба Европейского союза Eurostat (ec.europa.eu/eurostat/).

6.5. New York Times

Начиная с 1913 года, газета New York Times поддерживает каталог всех тем, на которые были публикации. Этот каталог вырос в целую коллекцию, содержащую почти 30 000 «предметных рубрик», описывающих местоположения, людей, организации и мероприятия. Каждая статья в New York Times, с момента первого выхода газеты в 1851 году, размечена с помощью этих предметных рубрик, и данные метки используются для обеспечения работы новостной службы, для автоматизации процесса редактирования и для создания «тематических страниц», которые объединяют всю информацию, опубликованную в газете, по заданной теме.

В 2009 году New York Times начала конвертировать этот каталог предметных рубрик в формат семантического веба. В формате RDF был опубликован весь список рубрик, кроме того, термины из этих рубрик связываются ссылками с известными наборами семантических данных, таких как DBpedia, Freebase и GeoNames. К 2010 году этот процесс был завершён примерно для 30 процентов всех предметных рубрик.

¹ <http://www.wheredidmytaxgo.co.uk/>.

New York Times сообщила, что связывание их предметных рубрик с облаком связанных данных помогает им в сборе географической информации (благодаря связи с GeoNames), в публикации статей с учетом других источников данных, которыми часто пользуется газета, например данных Библиотеки Конгресса, и позволяет быстро разрабатывать нестандартные мэшапы для архивного материала, например для поиска всей информации о выпускниках определенного университета.

6.6. Sig.ma и Sindice

Такие приложения, как GoodRelations, помогают традиционным поисковым системам улучшить результаты поиска, при этом конечный пользователь и не догадывается об используемых RDF-графах. По-другому работают поисковая система Sindice¹ и ее клиентское приложение Sig.ma²: Sindice работает непосредственно с RDF-графом, который образует глобальный семантический веб, а Sig.ma представляет пользователю бинарные отношения, обнаруженные таким образом.

Sindice имеет классическую для поисковой системы архитектуру: поисковый робот (краулер), очень большое индексированное хранилище и интерфейс поиска. Но, в отличие от классической поисковой системы, Sindice извлекает и индексирует не слова и фразы, а RDF-графы. На момент написания книги системой Sindice были проиндексированы 400 млн RDF-документов, содержащих в общей сложности 12 млрд RDF-утверждений. Эти утверждения проиндексированы и доступны для запросов через SPARQL-точку доступа. Таким образом, доступ к данным семантического веба похож на доступ к базе данных, что может быть использовано разработчиками и бизнесом для продвижения продуктов и услуг. Интересно, что поисковый робот Sindice не ограничивается поиском по RDF-графам, но сканирует и другие формы структурированных данных, например такие, которые используют формат микроданных HTML5 со словарем **schema.org** или другие микроформаты.

Sig.ma представляет собой пользовательский интерфейс, который был создан в качестве демонстрации непосредственной возможности осуществлять запросы к вебу данных. Пользователь вводит ключевое слово, Sig.ma определяет идентификаторы URL в вебе данных, кото-

¹ <http://sindice.com>.

² <http://sigm.ma>.

рые соответствуют этому ключевому слову, и находит все триплеты, описывающие эти URL. С помощью интерактивного пользовательского интерфейса пользователь может изучить, из каких источников данных получены триплеты, выбрать или исключить определенные источники данных, «обойти» граф, кликая на URL, которые ассоциируются с заданным ключевым словом, а затем переходя на URL, связанные с ними триплетными, и т. д.

С помощью поисковой системы Sindice приложение Sig.ma позволяет пользователю получить реальное представление о том, какая информация находится в вебе данных по любой теме. Благодаря оригинальному пользовательскому интерфейсу и очень полной базе данных Sindice, вероятно, можно считать, что Sig.ma представляет собой лучшее место, где пользователи-непрофессионалы могут понять структуру современного семантического веба и осознать, какое количество знаний в нем представлено.

6.7. OpenCalais

Информационное агентство Thompson Reuters каждый день производит тысячи новостей в области науки, бизнеса, политики и спорта. Хотя изначально новости предназначены для людей, Reuters сделало вывод, что значительное число потребителей их новостей **больше** составляют не люди, а компьютеры. Компьютеры читают поток новостей, выпускаемых Reuters, анализируют их и формируют выводы, дают советы по финансовым инвестициям и т. д.

Это приводит к парадоксальной ситуации: компьютеры производят информацию для компьютеров, используя неструктурированный естественный язык, который представляется совсем не подходящим для этого средством передачи данных. Сервис OpenCalais агентства Reuters предназначен помочь компьютерам обрабатывать данные, извлеченные из текстов на естественном языке. Это осуществляется путем анализа текста, распознавания имен сущностей, таких как люди, места, организации и т. д., и аннотирования текста с помощью RDF для идентификации этих именованных сущностей.

В качестве простого примера рассмотрим следующий текст¹, после анализа которого будут распознаны сущности и отношения, представленные на рис. 6.5.

¹ Пример приводится на английском языке, так как сервис OpenCalais не работает с текстами на русском языке. — *Прим. перев.*



Рис. 6.5 ❖ Объекты и отношения, распознанные сервисом OpenCalais

After studying mathematics and computer science in Amsterdam, Frank van Harmelen moved to the Department of AI of the University of Edinburgh. While in Edinburgh, he worked with Prof. Alan Bundy on proof planning for inductive theorem proving. After his PhD research, he moved back to Amsterdam where he worked from 1990 to 1995 in the SWI Department under Prof. Bob Wielinga. In 1995 he joined the AI research group at the Vrije Universiteit Amsterdam.

Для идентификации этих объектов Reuters использует свои собственные URI, многие из которых связаны свойством `owl:sameAs` с более широко используемыми URI для этих же объектов, взятых из наборов данных GeoNames, DBpedia, FreeBase, Internet Movie Database и др. Это позволяет нам в дальнейшем получать информацию об этих сущностях из базы данных агентства Reuters, например финансовую информацию, имена членов совета директоров и т. д. Таким образом, с помощью сервиса OpenCalais можно перейти от термина, упоминаемого в тексте, на естественном языке текста к структурированным данным семантического веба. Сообщается, что в 2011 году Calais обрабатывал пять миллионов документов в день.

6.8. Schema.org

Schema.org – это совместная инициатива разработчиков поисковых систем (Google, Yahoo, Microsoft, Яндекс), направленная на помощь создателям сайтов в семантической разметке страниц. Для этого

определена общая схема или словарь, доступный на сайте schema.org. Словарь описывает общие сущности, которые обычно ищут люди, например продукты, работу и события. Используя общий словарь, поисковые системы могут лучше индексировать веб-страницы, а также выдавать более содержательную информацию в качестве результатов поиска.

Schema.org поддерживает разметку, представленную как в формате микроданных, так и в формате RDFa 1.1 Lite. Таким образом, можно использовать словарь Schema.org совместно с другими словарями в вебе, например ранее рассмотренной онтологией GoodRelations. Schema.org демонстрирует, каким образом приложения могут внедрять машиночитаемую разметку.

6.9. Выводы

Технологии семантического веба используются в самых различных приложениях, позволяя эффективно поддерживать их, упрощая процесс понимания содержания веб-страниц для поисковых систем и т. д. Ключевая идея, лежащая в основе всех этих приложений, заключается в том, что общие онтологии обеспечивают основу для интеграции и понимания знаний из различных источников. В данной главе были описаны лишь некоторые основные приложения семантического веба, хотя в настоящее время существует бесчисленное множество ежедневно используемых приложений подобного типа. Рекомендуем вам самостоятельно исследовать веб в поисках таких приложений.

ОНТОЛОГИЧЕСКИЙ ИНЖИНИРИНГ

7.1. Введение

В предыдущих главах основное внимание было уделено технологиям семантического веба, а именно языкам представления знаний, языкам запросов, технологиям преобразования данных и логического вывода и соответствующим программным средствам. Очевидно, что многообразие данных технологий и инструментов заставляет задуматься о решении целого ряда методологических вопросов. Какие инструменты и методы являются лучшими с точки зрения их практического применения? Для решения какого типа задач подходят те или иные языки и инструменты и в какой последовательности их использовать? Как решаются вопросы контроля качества и управления ресурсами?

Обзор главы

Способы решения аналогичных задач хорошо изучены, например, в области разработки программного обеспечения, объектно-ориентированного проектирования и инженерии знаний. Их подробное обсуждение выходит за рамки этой книги. Тем не менее в этой главе мы кратко обсудим некоторые методологические проблемы, которые возникают при создании онтологий, в частности ручную разработку онтологий (раздел 7.2), повторное использование онтологий (раздел 7.3) и использование полуавтоматических методов разработки онтологий (раздел 7.4). В разделе 7.5 рассматривается процесс отображения онтологий. В разделе 7.6 описывается, каким образом можно конвертировать данные из реляционных баз данных в экземпляры онтологий. Наконец, в разделе 7.7 объясняется, как инструменты семантического веба могут быть интегрированы в единую архитектуру для разработки приложений.

7.2. Ручная разработка онтологий

Материал данного раздела опирается в основном на учебное пособие Натальи Ной и Дэборы Л. МакГиннесс «Разработка онтологий 101: руководство по созданию Вашей первой онтологии»¹. Дополнительные источники информации приведены в списке рекомендуемой литературы.

Можно выделить следующие основные этапы в процессе разработки онтологии:

- 1) определение области и масштаба онтологии;
- 2) рассмотрение возможности повторного использования существующих онтологий;
- 3) перечисление основных терминов онтологии;
- 4) определение таксономии;
- 5) определение свойств;
- 6) определение фасетов;
- 7) определение экземпляров;
- 8) проверка онтологии на аномалии.

Создание онтологии, как и любой другой процесс разработки, на практике реализуется не линейно. Этапы разработки могут повторяться, и на любой стадии может возникнуть необходимость возврата на более ранние этапы. В наши задачи не входит рассмотрение вопросов управления этим сложным процессом. Однако рассмотрим отдельные шаги разработки, приведенные выше.

7.2.1. Определение области и масштаба онтологии

Создание онтологии предметной области не является самоцелью. Разработка онтологии, как и разработка любой структуры данных, необходима для использования ее другими приложениями. Другими словами, онтология является моделью конкретной предметной области, которая разрабатывается для конкретной цели. Следовательно, не существует правильной онтологии определенной предметной области. Онтология представляет собой абстракцию конкретной области и всегда имеет жизнеспособные альтернативы. Что именно будет включено в эту абстракцию, определяется в зависимости от задач приложения, для которого она разрабатывается, и от будущих потенциальных вариантов ее использования. На данном шаге должны быть даны ответы на следующие

¹ Русскоязычный вариант пособия доступен на сайте ifets.ieee.org/russian/depositary/ontology101_rus.doc. – *Прим. перев.*

щие основные вопросы: Для какой предметной области разрабатывается онтология? Для чего мы собираемся использовать онтологию? На какие типы вопросов должна давать ответы информация в онтологии? Кто будет использовать и поддерживать онтологию?

7.2.2. Рассмотрение возможности повторного использования онтологий

С распространением технологий семантического веба многие онтологии, особенно для общих предметных областей, таких как социальные сети, медицина, география, стали доступны для использования. Таким образом, определение онтологии редко приходится начинать с нуля. Почти всегда можно найти уже существующую онтологию, которую можно рассматривать, по крайней мере, как отправную точку при разработке собственной онтологии (см. раздел 7.3).

7.2.3. Перечисление терминов онтологии

Первым шагом непосредственно при определении онтологии является создание неструктурированного списка, который содержит все возможные понятия, которыми должна оперировать онтология. Как правило, имена существительные в этом списке образуют основу для имен классов, а глаголы (или выражения с глаголами) – для имен свойств (например, «является частью», «имеет компонент»).

На данном этапе могут быть продуктивно использованы традиционные инструменты инженерии знаний, такие как лэддеринг и грид-анализ. Данные методы позволяют получить как сам список терминов, так и их первоначальную структуру.

7.2.4. Определение таксономии

После идентификации всех терминов онтологии их необходимо организовать в таксономическую иерархию. Не существует однозначного мнения по поводу того, какой метод проектирования иерархии (метод «снизу вверх» или метод «сверху вниз») является более эффективным и надежным.

Очень важно убедиться, что построенная иерархия представляет из себя таксономию (то есть классы связаны в ней отношением «класс–подкласс»). Другими словами, надо проверить, что если класс А является подклассом класса В, то каждый экземпляр класса А должен быть и экземпляром В. Только подобная проверка гарантирует, что мы не нарушаем семантику примитива моделирования `rdfs:subClassOf`.

7.2.5. Определение свойств

Этот шаг часто выполняется параллельно с предыдущим: естественно определять свойства, которые связывают классы, именно при определении классов и организации этих классов в иерархию.

Напомним, что отношение `subClassOf` имеет следующую семантику: если класс А является подклассом класса В, то каждое утверждение, которое справедливо для экземпляров класса В, также справедливо и для экземпляров класса А. Из-за этой особенности наследования имеет смысл относить свойства к самому старшему классу в иерархии, к которому их только можно отнести.

При определении свойств и соотнесении их с определенными классами имеет смысл сразу указывать их домен и диапазон. Здесь возникает некоторое методологическое противоречие между стремлением к общности и стремлением к точности. С одной стороны, привлекательно определить в качестве домена и диапазона свойства как можно более общие классы. Благодаря наследованию такое свойство может быть использовано для всех подклассов. С другой стороны, полезно определить в качестве домена и диапазона свойства как можно более узкие классы. Это позволит выявлять потенциальную несогласованность онтологии путем нахождения противоречий в соответствующих доменах и диапазонах.

7.2.6. Определение фасетов¹

Отметим, что после всех предыдущих шагов онтология имеет выразительность, предоставленную языком RDF Schema, и не использует каких-либо дополнительных примитивов языка OWL. На этом шаге увеличим выразительность онтологии за счет доопределения свойств следующими фасетами:

- кардинальность. Для всех свойств, для которых это возможно, следует указать ограничение кардинальности, то есть количество различных значений, которые они могут или должны иметь. На практике чаще всего ограничение кардинальности имеет тип «по крайней мере, одно значение» (то есть обязательные свойства) или «не более одного значения» (то есть однозначные свойства);
- ограничения на значения свойств. Часто классы определяются как совокупность индивидов, имеющих конкретные значения

¹ Следуя терминологии языка OWL, этот шаг вернее назвать «определение аксиом классов и свойств». – *Прим. перев.*

определенного свойства. Эти значения в языке OWL можно указать с помощью примитива `owl:hasValue`. Иногда требования к значениям свойств являются менее жесткими, то есть свойство должно иметь несколько значений из заданного класса (но не обязательно определенные значения). Для описания таких свойств используется примитив `owl:someValuesFrom`;

- относительные характеристики свойств. В заключение следует определить фасеты, связанные собственно с характеристиками свойств, такими как симметричность, транзитивность, функциональность и обратные свойства¹.

После выполнения этого шага в процессе разработки онтологии можно проверить онтологию на внутреннюю согласованность. (До реализации этого шага такая проверка была не возможна, так как язык RDF Schema не позволяет выразить противоречивость знаний.) Примером часто встречающихся типов противоречий являются несовместимые определения доменов и диапазонов для транзитивных, симметричных или обратных свойств. Также причиной противоречивости онтологии могут быть ограничения кардинальности свойств. Наконец, ограничения на значения свойств могут вступать в конфликт с ограничениями доменов и диапазонов, что является еще одним источником возможных несоответствий в онтологии.

7.2.7. Определение экземпляров

Создание онтологии редко является самоцелью. На практике онтологии используются для организации связей между конкретными сущностями, то есть экземплярами классов. И одним из шагов разработки онтологии является ее наполнение такими экземплярами. Как правило, число экземпляров в онтологии на несколько порядков больше, чем число классов. Размер онтологий варьируется от нескольких сотен до десятков тысяч классов и от сотен до сотен тысяч и более экземпляров.

Учитывая большое количество экземпляров, наполнение онтологии, как правило, осуществляется не вручную. Часто экземпляры извлекаются из унаследованных источников данных, например баз данных (см. раздел 7.6). Другим часто используемым методом наполнения онтологий является автоматизированное извлечение экземпляров из корпусов текстов.

¹ Сюда же следует отнести характеристики асимметричности, обратной функциональности, рефлексивности, иррефлексивности, а также эквивалентные и непересекающиеся свойства. – *Прим. перев.*

7.2.8. Проверка онтологии на аномалии

Важным преимуществом использования языка OWL, по сравнению с языком RDF Schema, является возможность обнаружения противоречий как в онтологии в целом, так и во множестве ее экземпляров. Примерами часто встречающихся аномалий являются противоречивые определения доменов и диапазонов транзитивных, симметричных или обратных свойств. Также причиной противоречивости онтологии могут быть ограничения кардинальности свойств. Наконец, ограничения на значения свойств могут вступать в конфликт с ограничениями доменов и диапазонов, что является еще одним источником возможных несоответствий в онтологии.

7.3. Повторное использование существующих онтологий

Разработку онтологии следует по возможности начинать на базе уже существующей онтологии. На данный момент в вебе представлено широкое разнообразие таких онтологий.

7.3.1. Систематизированные своды экспертных знаний

Некоторые онтологии кропотливо разрабатывались большими коллективами экспертов в течение многих лет.

Примером подобной онтологии в медицинской области является онтология Национального института рака (National Cancer Institute) в США¹. Примерами онтологий данного типа в области культуры являются тезаурус искусства и архитектуры (Art and Architecture Thesaurus, AAT)², содержащий 125 000 терминов, каталог художников (Union List of Artist Names, ULAN)³, содержащий 220 000 записей о художниках, словарь Iconclass⁴, включающий в себя 28 000 терминов для описания культурных образов. Примером экспертной онтологии в области географии является тезаурус географических названий Getty (Getty Thesaurus of Geographic Names, TGN)⁵, содержащий более 1 млн записей.

¹ www.mindswap.org/2003/CancerOntology/.

² www.getty.edu/research/tools/vocabulary/aat.

³ www.getty.edu/research/conducting_research/vocabularies/ulan.

⁴ www.iconclass.nl/.

⁵ www.getty.edu/research/conducting_research/vocabularies/tgn.

7.3.2. Интегрированные словари

Также предпринимались попытки интеграции независимо разработанных словарей в один большой ресурс. Ярким примером подобного ресурса является система унифицированного медицинского языка (Unified Medical Language System, UMLS)¹, которая объединяет 100 биомедицинских словарей и классификаторов. Только метатезаурус этой системы содержит 750 000 понятий, между которыми существует более 10 млн связей. Неудивительно, что семантика такого ресурса, объединяющего множество независимо разработанных словарей, является довольно сложной для понимания. Тем не менее он оказался очень полезным во многих приложениях, по крайней мере в качестве отправной точки при их разработке.

7.3.3. Онтологии верхнего уровня

Рассмотренные выше онтологии описывают знания из конкретных предметных областей. Но также предпринимались попытки разработать онтологии, формализующие знания, общие для всех предметных областей. Такие онтологии называют *онтологиями верхнего уровня*. Двумя основными примерами онтологий верхнего уровня являются онтология Сус², содержащая 60 000 утверждений о 6000 понятиях, и Стандартная онтология верхнего уровня (Standard Upperlevel Ontology, SUO)³.

7.3.4. Тематические иерархии

Кроме того, существуют коллекции терминов, объединенных в специализированные иерархии. Такие коллекции трудно назвать онтологиями в строгом смысле этого слова. Подобные иерархии, как правило, не являются строгими таксономиями, то есть понятия в них связаны не только отношениями «класс–подкласс», но и другими отношениями, такими как «является частью» или «содержится в». Тем не менее подобные ресурсы часто очень полезны в качестве отправной точки при разработке. Примером подобного ресурса, доступного в формате RDF, является иерархия Открытый словарь (Open Directory)⁴, которая содержит более 400 000 иерархически организованных категорий.

¹ umlsinfo.nlm.nih.gov/.

² www.openencyc.org/.

³ suo.ieee.org/.

⁴ dmoz.org/.

7.3.5. Лингвистические ресурсы

Некоторые проекты представляют собой скорее лингвистические ресурсы, чем абстракции конкретной предметной области. Такие проекты также полезно использовать в качестве отправной точки при разработке онтологий. Ярким примером в этой категории является проект WordNet¹, содержащий более чем 90 000 определений значений английских слов.

7.3.6. Энциклопедические знания

Проект Википедия представляет собой генерируемую веб-сообществом энциклопедию, содержащую огромное количество информации по различным темам. Проект DBpedia² извлекает знания из Википедии и представляет их как связанные данные с помощью языков RDF и OWL. Широта знаний, представленных в Википедии, позволяет использовать данные проекта DBpedia практически при разработке любой онтологии. Еще одним проектом подобного рода является база знаний Yago³, которая также использует Википедию, но, кроме того, содержит информацию из проекта WordNet и географических ресурсов, таких как GeoNames⁴.

7.3.7. Библиотеки онтологий

Также существуют интернет-хранилища онтологий. Примерами таких хранилищ являются репозиторий онтологий TONES⁵, проект BioPortal⁶ и хранилище онтологий, разработанных в редакторе Protégé⁷. Возможно, самым лучшим хранилищем онлайн-онтологий на данный момент является проект Swoogle⁸, в котором каталогизированы более 10 000 документов семантического веба, проиндексированы метаданные об описанных в них классах, свойствах и индивидах, а также отношениях между ними. В проекте Swoogle также определены ранжированные свойства документов семантического веба, которые используются для сортировки результатов поиска. Аналогичный про-

¹ <http://wordnet.princeton.edu/>, доступен в RDF: <http://semanticweb.cs.vu.nl/lod/wn30/>.

² <http://dbpedia.org>.

³ <http://www.mpi-inf.mpg.de/yago-naga/yago/>.

⁴ <http://www.geonames.org/>.

⁵ <http://owl.cs.manchester.ac.uk/repository/browser>.

⁶ <http://bioportal.bioontology.org/>.

⁷ <http://protege.stanford.edu/download/ontologies.html>.

⁸ <http://swoogle.umbc.edu/>.

ект – поисковый индекс *Sindice*¹ – поддерживает индексы почти всех RDF-документов, опубликованных в семантическом вебе. Проект *Prefix.cc*² содержит перечень наиболее часто используемых префиксов пространств имен, используемых в семантическом вебе. При этом имя префикса prefix.cc связано гиперссылкой с соответствующей онтологией.

Заметим, что только в редких случаях существующие онтологии могут быть повторно использованы без изменений. Как правило, существующие концепты и свойства должны быть уточнены (с помощью примитивов `owl:subClassOf` и `owl:subPropertyOf`). Кроме того, имеет смысл ввести альтернативные имена классов и свойств, которые лучше подходят для конкретной предметной области (например, с помощью примитивов `owl:equivalentClass` и `owl:equivalentProperty`). Также можно использовать такую возможность языков RDF и OWL, как скрытое уточнение классов, определенных в других онтологиях.

В целом вопрос импорта онтологий и создания отображения между ними по-прежнему остается открытым и рассматривается как одна из самых серьезных проблем семантического веба.

7.4. Полуавтоматическое построение онтологий

Внедрение концепции семантического веба связано с двумя основными проблемами.

Во-первых, для построения веба метаданных необходимо иметь методы и инструменты решения такой задачи реинжиниринга, как семантическое пополнение этих метаданных. Успех концепции семантического веба в значительной степени зависит от скорости распространения онтологий и реляционных метаданных. Это, в свою очередь, требует высокой скорости и низкой стоимости разработки онтологий. Данные требования могут быть выполнены за счет применения методов машинного обучения для обеспечения семантической совместимости при слиянии и выравнивании онтологий.

Во-вторых, необходимо обеспечить средства внедрения и поддержки машинно обрабатываемых данных, которые являются основой семантического веба. Таким образом, нужны механизмы, которые поддерживают динамическую природу веба.

¹ <http://sindice.com>.

² <http://prefix.cc>.

Хотя за последнее десятилетие инструменты онтологического инжиниринга значительно усовершенствовались, ручная разработка онтологий остается трудоемкой задачей, требующей существенных затрат времени, ресурсов и привлечения высококвалифицированных специалистов. При этом основной проблемой при ручной разработке онтологий является проблема сбора (приобретения) знаний.

С данной проблемой инженеры по знаниям сталкивались на протяжении двух последних десятилетий, работая над методологиями приобретения знаний и инструментальными средствами разработки баз знаний. Интеграция методов сбора знаний с методами машинного обучения оказалась эффективным способом ее решения.

Область исследований, связанная с машинным обучением, имеет богатую историю и решает как задачи приобретения или извлечения знаний, так и задачи обновления или поддержки знаний, а также предоставляет различные методы решения данных задач.

С помощью методов машинного обучения могут быть решены следующие задачи:

- извлечение онтологий из существующих в вебе данных;
- извлечение реляционных данных и метаданных из существующих в вебе данных;
- слияние и отображение онтологий путем анализа возможностей расширения концептов;
- поддержка онтологий на основе анализа данных экземпляров;
- улучшение приложений семантического веба с помощью наблюдений за пользователями.

Для решения этих задач в области машинного обучения могут быть использованы следующие методы:

- кластеризация;
- инкрементное обновление онтологий;
- поддержка инженера по знаниям;
- улучшение больших онтологий естественного языка;
- обучение онтологий в предметной области.

Б. Омелаенко (см. рекомендованные источники) выделяет следующие три типа онтологий, которые могут поддерживаться с помощью методов машинного обучения.

7.4.1. Онтологии естественного языка

Онтологии естественного языка (Natural language ontologies, NLOs) содержат лексические отношения между понятиями языка; они имеют большие размеры и не требуют частого обновления. Обычно онто-

логии этого типа представляют собой базовые знания системы и используются для расширения пользовательских запросов. Состояние разработок в области обучения подобных онтологий выглядит достаточно оптимистично: существуют не только стабильные онтологии естественного языка общего назначения, но и методы автоматического или полуавтоматического построения и пополнения предметно-ориентированных онтологий данного типа.

7.4.2. Онтологии предметных областей

Онтология предметной области охватывает знания из одной конкретной области, например фармакологические знания или знания о принтерах. Онтологии этого типа обеспечивают детальное описание понятий ограниченной предметной области. Как правило, они строятся вручную, но инженеру по знаниям (особенно неопытному) могут помочь различные методы машинного обучения. Методы обучения онтологий предметных областей гораздо менее развиты, чем соответствующие методы для онтологий естественного языка. Сбор знаний для онтологий предметных областей все еще осуществляется преимущественно инженерами, и автоматизированные методы обучения занимают здесь незначительное место. Необходимо найти статистически достоверные зависимости в текстах о данной предметной области и предложить их инженерам по знаниям.

7.4.3. Экземпляры онтологии

Экземпляры онтологии могут генерироваться автоматически и постоянно обновляться (например, профиль компании в Желтых страницах будет часто обновляться), в то время как сама онтология остается неизменной. Задача обучения экземпляров онтологии также вписывается в рамки машинного обучения. Для решения этой задачи уже существует несколько успешных приложений, основанных именно на алгоритмах машинного обучения. Но эти приложения либо жестко привязаны к онтологии предметной области, либо заполняют онтологию экземплярами вообще без связи с какой-либо теорией предметной области. До сих пор не разработана общая методика для извлечения экземпляров онтологии из текстов, описывающих предметную область.

Кроме различных типов онтологий, которые могут поддерживаться с помощью методов машинного обучения, существуют также различные варианты использования обучения онтологий. Первые три

задачи в следующем списке (также предложенном Б. Омелаенко) относятся в инженерии знаний к задачам сбора знаний для онтологий, а последние три – к задачам сопровождения онтологий:

- создание онтологии с нуля инженером по знаниям. В рамках решения данной задачи методы машинного обучения помогают инженеру по знаниям, а именно выделяют самые важные отношения в предметной области и осуществляют проверку и верификацию построенной базы знаний;
- извлечение схемы онтологии из веб-документов. Для решения данной задачи системы машинного обучения на основе входных данных и метазнаний (например, метаонтологий) генерируют готовые к использованию онтологии; при этом в процессе генерации могут использоваться знания инженера;
- извлечение экземпляров онтологии из заданной онтологической схемы или из веб-документов. Эта задача сходна с задачами извлечения информации и аннотирования веб-страниц, и поэтому для ее решения могут применяться методы, используемые в этих областях;
- интеграция онтологий и навигация по онтологиям подразумевают реконструкцию и навигацию по большим и, возможно, машинно обучаемым базам знаний. Например, подобная задача может заключаться в преобразовании машинно обучаемой декларативной базы знаний в базу знаний первого порядка;
- задача сопровождения онтологии заключается в обновлении некоторых (изначально предназначенных для обновления) ее частей (например, тегов форматирования, которые должны быть синхронизированы с изменениями, внесенными в макет страницы);
- пополнение онтологии (или оптимизация онтологии) включает в себя автоматизированную модификацию вспомогательных отношений в существующей онтологии. В результате решения данной задачи основные концепты и структура онтологии не меняются, но онтология становится более точной.

Машинное обучение предлагает широкое разнообразие методов, алгоритмов и инструментов. Однако для представления онтологий важно, чтобы сами онтологии были символьными, человекочитаемыми и понятными. Это требование позволяет использовать исключительно символьные алгоритмы обучения со способностью обобщения. Другие методы машинного обучения, такие как нейронные сети и генетические алгоритмы, не подходят для решения рассмотренных

задач. Ниже приведены некоторые потенциально применимые для онтологий алгоритмы:

- алгоритмы обучения, основанные на исчислении высказываний; извлекают ассоциативные правила или другие виды правил типа «атрибут–значение»;
- байесовское обучение, в основном представленное наивным байесовским классификатором. Данный классификатор основан на применении теоремы Байеса и предположении об условной независимости между атрибутами обучаемых сущностей, классификатор генерирует вероятностные правила вида «атрибут–значение»;
- алгоритмы обучения, основанные на логике предикатов первого порядка; извлекают правила, которые содержат переменные, называемые фразами Хорна;
- алгоритмы кластеризации группируют экземпляры на основе подобия или мер расстояния между каждой парой экземпляров, определенных значениями их атрибутов.

В заключение отметим, что, несмотря на большой потенциал методов машинного обучения в области семантического веба, это еще далеко не изученная область.

7.5. Отображение онтологий

Повторное использование существующих онтологий на сегодняшний день стало стандартным этапом в процессе разработки новой онтологии. Практически никто не ведет разработку онтологий «с нуля». В связи с этим задача интеграции онтологий становится все более актуальной. Причем при разработке конкретного приложения чаще всего используется не одна существующая онтология, а их комбинация. Это обостряет проблему интеграции онтологий, которую также называют проблемой *выравнивания*, или *отображения*, онтологий. В последние годы решению данной проблемы посвящено множество научных работ.

Современные подходы к отображению онтологий подразумевают использование различных методов из самых разных областей знания. Выделим лингвистические, статистические, структурные и логические методы.

7.5.1. Лингвистические методы

Основные лингвистические методы предполагают использование языковых меток, которые прикрепляются к понятиям исходной и це-

левой онтологии, для того чтобы обнаружить возможные совпадения. При этом могут применяться методы стемминга¹, расчет расстояния Хэмминга² или использоваться специализированные знания предметной области. Например, знания предметной области определяют существенную разницу между понятиями «сахарный диабет типа I» и «сахарный диабет типа II», хотя расстояние Хемминга между этими фразами очень мало.

7.5.2. Статистические методы

Статистические методы для определения соответствия между понятиями онтологий используют, в отличие от лингвистических методов, не языковые метки, а данные экземпляров этих онтологий. Если существует значительная статистическая корреляция между экземплярами концептов исходной и целевой онтологии, значит, можно предположить, что эти понятия тесно связаны (отношением «класс–подкласс» или, возможно, даже отношением эквивалентности). Эти подходы предполагают наличие достаточно большого корпуса экземпляров, которые классифицированы как в исходной, так и в целевой онтологии.

7.5.3. Структурные методы

Любая онтология имеет внутреннюю структуру. Поэтому имеет смысл использовать эту структуру, а именно структуру графа исходной и целевой онтологий, для определения сходства между понятиями онтологии. Данные методы часто используют с комбинации с другими, например лингвистическими, методами. Если исходный и целевой концепты имеют сходные лингвистические метки, но фрагменты графа, в которых они представлены, различаются, то следует предположить, что данные понятия являются *омонимами*. Использование только лингвистических методов в подобных случаях приведет к ошибочным результатам – омонимы будут определены как подобные понятия.

7.5.4. Логические методы

Логические методы, возможно, являются наиболее характерными для решения задачи отображения онтологий. Не стоит забывать, что по определению Р. Штудера (R. Studer) онтология – это «формальная

¹ Стемминг – это процесс нахождения основы заданного слова. – *Прим. перев.*

² Расстояние Хэмминга – число позиций, в которых соответствующие символы двух строк одинаковой длины различны. – *Прим. перев.*

спецификация совместно используемой концептуализации», и имеет смысл использовать эту формализацию исходной и целевой структуры для установления соответствия между ними. На практике применение данного подхода ограничивается тем, что многие существующие онтологии предназначены для представления самой простой семантики предметной области и не поддерживают мощных логических формализмов.

7.5.5. Реализации отображений онтологий

Для отображения онтологий существует несколько фреймворков, например R2R Framework¹ и LIMEs². Сервис sameas.org собирает экземпляры онтологий из различных источников и выявляет между ними отображения вида owl:sameAs. Несмотря на успех этих проектов, создание отображений остается открытой актуальной задачей. Научное сообщество инициировало проект «Инициатива по оцениванию методов выравнивания онтологий» (Ontology Alignment Evaluation Initiative)³, действующий в течение последних девяти лет, основной целью которого является содействие созданию более точных и полных отображений.

7.6. Использование реляционных баз данных

Большинство современных веб-сайтов не предоставляют пользователю набор статических страниц, хранящихся на веб-сервере, а генерируются динамически на основе информации, хранящейся в реляционных базах данных. Например, сайт о недвижимости генерируется на основе базы данных, содержащей информацию о различных домах и квартирах, их цене, местоположении и удобствах. Именно эта база данных используется для заполнения веб-страниц. Так как большое количество информации хранится именно в базах данных, то можно рассматривать их в качестве источника данных для создания экземпляров онтологий.

Приведем краткое описание процесса конвертации баз данных в онтологии.

¹ <http://www4.wiwiw.fu-berlin.de/bizer/r2r/>.

² <http://aksw.org/Projects/LIMEs?v=z11>.

³ <http://oaei.ontologymatching.org/>.

7.6.1. Терминология

Прежде всего напомним основную терминологию, используемую в теории базы данных, и рассмотрим, каким образом эту терминологию можно сопоставить с терминологией языков RDFS и OWL. Ниже приведен пример таблицы из базы данных, содержащей информацию о недвижимости (табл. 7.1). Таблицы в теории баз данных также называют отношениями. Приведенная ниже таблица состоит из столбцов ДомID, Город, Цена. Столбцы таблиц в теории баз данных называются атрибутами, а строки – кортежами.

Таблица 7.1. Таблица «Дома» базы данных недвижимости

ДомID	Город	Цена (евро)
1	Амстердам	100 000
2	Утрехт	200 000

На основании этой терминологии можно предложить следующий простой подход к отображению схемы реляционной базы данных в документ на языке RDFS или OWL. Каждую таблицу в базе данных можно рассматривать как класс, каждый атрибут – как свойство и каждый кортеж – как экземпляр. Таким образом, в нашем примере описаны два экземпляра класса Дом. Каждый экземпляр имеет свойства Город и Цена, а также соответствующие значения (например, для экземпляра с идентификатором ДомID, равным 1, значением свойства Город является Амстердам).

При выполнении отображения для каждой сущности необходимо создать URI. Это может быть сделано с помощью добавления префикса пространства имен в начало имени атрибута или таблицы. Для URI каждого экземпляра можно использовать первичный ключ. Подчеркнем, что основное различие между реляционными базами данных и RDF заключается в том, что в RDF-документах для идентификации сущностей используются URI, представляющие собой глобальные уникальные идентификаторы, а в реляционных базах данных идентификаторы являются уникальными только в пределах конкретной базы данных.

7.6.2. Инструменты конвертации данных

Большую часть процесса конвертации реляционных баз в RDF-документы можно автоматизировать на основе описанного выше формального метода. Существует несколько инструментов подобной

конвертации, представленных в отчете одной из рабочих групп консорциума 3WC – W3C Relational Database to RDF Incubator Group. Данный отчет в целом описывает существующие подходы к отображению реляционных баз в RDF-документы, ссылку на него вы можете найти в списке рекомендованной литературы. Большинство из этих инструментов работают на основе алгоритмов анализа структуры реляционной базы данных и генерируют почти завершенные RDF-документы. После этого пользователю необходимо изменить файлы конфигурации, указав более подходящие URI и добавив ссылки на существующие онтологии. Например, в приведенном выше примере вместо автоматически сгенерируемого URI для сущности Амстердам лучше использовать соответствующий URI из проекта DBpedia.

Инструменты конвертации могут использоваться двумя способами. Первый способ предполагает преобразования реляционной базы данных непосредственно при выполнении запроса в точке доступа SPARQL. Второй способ конвертации заключается в преобразовании всей базы данных в RDF-документы, которые затем можно загрузить в хранилище триплетов. Этот способ часто используется для интеграции экземпляров в существующую онтологию. Хорошим инструментом для начинающих разработчиков является D2R Server¹, поскольку он в довольно простом пакете предоставляет оба способа конвертации.

7.7. Архитектура приложений семантического веба

Реализация концепции семантического веба подразумевает не только использование новых языков, описанных в этой книге, но также несколько иной стиль разработки приложений и другой подход к интеграции приложений. Чтобы проиллюстрировать это, опишем процесс интеграции независимо разработанных инструментов, поддерживающих стандарты семантического веба, в одно легковесное приложение, общая архитектура которого представлена на рис. 7.1.

7.7.1. Сбор знаний

В нижней части рис. 7.1 представлены инструменты, которые используют методы анализа существующих документов с целью получения

¹ <http://www4.wiwiw.fu-berlin.de/bizer/d2r-server/>.

из них знаний. В качестве таких документов могут выступать неструктурированные документы на естественном языке или структурированные и слабоструктурированные документы (такие как базы данных, таблицы HTML и электронные таблицы).

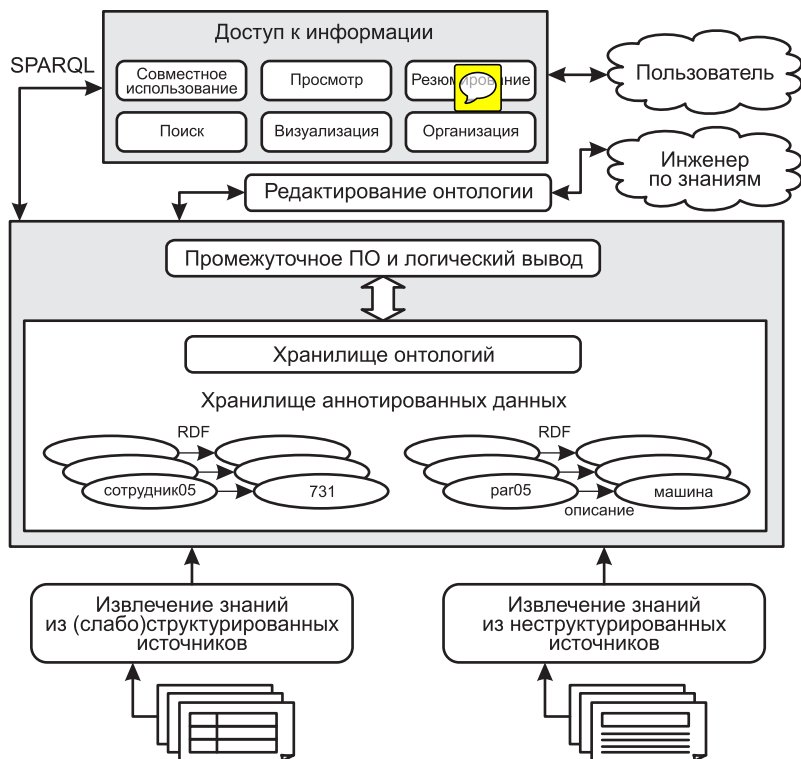


Рис. 7.1 ❖ Архитектура системы управления знаниями в семантическом вебе

Для анализа неструктурированных документов с целью извлечения ключевых понятий инструменты сбора знаний обычно используют комбинацию статистических и лингвистических методов.

Для структурированных и слабоструктурированных документов можно использовать инструменты конвертации данных, описанные выше. Для извлечения содержимого из слабоструктурированных документов могут быть также использованы методы распознавания образов и индуктивный метод.

7.7.2. Хранение знаний

Инструменты анализа и сбора знаний позволяют получить на выходе множество концептов, организованных в иерархию. Обычно такие иерархии являются неглубокими и включают в себя в лучшем случае небольшое число отношений вида «класс–подкласс», однако в совокупности с RDF-данными являются достаточно выразительными, чтобы представлять полученную информацию. Подчеркнем, что эта информация включает себя не только иерархию классов, но и данные экземпляров этих классов.

Репозитории (хранилища) онтологий и RDF-данных должны обеспечивать не только возможность хранения знаний, полученных на предыдущем этапе, но и возможность обращения к этим знаниям, предпочтительно с использованием структурированного языка запросов SPARQL. Кроме того, подобные RDFS-хранилища должны поддерживать теорию RDF-модели, которая подразумевает возможность определения принадлежности индивида к определенному классу на основе домена и диапазона свойств и получение транзитивных замыканий отношения `subClassOf`.

Обратите внимание, что репозитории должны обеспечивать хранение как онтологий (то есть иерархии классов и свойств), так и экземпляров онтологий (конкретных индивидов, принадлежащих определенным классам, и пар индивидов, связанных определёнными свойствами).

7.7.3. Сопровождение знаний

Основными функциями хранилищ знаний в семантическом вебе являются хранение знаний и обеспечение возможности доступа к этим знаниям. Кроме того, подобные репозитории должны предоставлять функциональные возможности для управления и сопровождения онтологий, а именно управления изменениями, управления правами доступа и владения, а также управления транзакциями.

Кроме поддержки легковесных онтологий, которые автоматически генерируются из неструктурированных и слабоструктурированных данных, необходимо также обеспечить поддержку разработки гораздо более сложных онтологий, которая осуществляется инженерами по знаниям. Сложные среды редактирования онтологий позволяют инженеру по знаниям извлечь онтологию из хранилища, осуществить ее преобразование и загрузить обратно в хранилище.

7.7.4. Использование знаний

Онтологии и данные, хранящиеся в репозиториях, используются в приложениях, предназначенных конечным пользователям. Такие приложения уже рассматривались в этой книге. Добавим, что внешние приложения могут получить доступ к этим знаниям, если они размещены в вебе. Причем для размещения RDF-данных в вебе можно использовать один из следующих подходов (или их комбинации): организация SPARQL-точки доступа к данным, публикация данных в облаке связанных данных, публикация данных в формате RDFa.

7.7.5. Пример реализации архитектуры

В проекте On-To-Knowledge¹ архитектура приложения, представленная на рис. 7.1, была реализована с очень слабыми связями между компонентами. Синтаксическая совместимость проекта обеспечивается за счет того, что все компоненты поддерживают RDF-модель данных, семантическая совместимость – за счет того, что компоненты для выражения семантики используют язык RDF Schema. Наконец, физическая совместимость достигается за счет того, что коммуникация между компонентами проекта осуществляется с помощью простых HTTP-соединений и все компоненты (кроме редактора онтологий) реализованы в виде удаленных сервисов. При работе с системой On-To-Knowledge в Амстердаме инструменты извлечения онтологий, работающие на сервере в Норвегии, могут анализировать документ, размещенный на сервере в Лондоне; полученные в результате анализа RDF- и RDF Schema-данные загружаются в хранилище, размещенное на сервере в Амерсфорте (Нидерланды). Затем эти данные выгружаются в локально установленный редактор онтологий и после редактирования инженером по знаниям снова загружаются на сервер в Амерсфорте. На основе этих данных работают генератор веб-сайтов, размещенный на сервере в Швеции, и поисковая система в Великобритании. Результаты работы этих приложений отображаются в браузере компьютера в Амстердаме.

Таким образом, инструменты проекта запускаются удаленно. Подчеркнем, что все инструменты были разработаны независимо друг от друга, а высокая степень их совместимости достигается только за счет использования протокола HTTP и языка RDF.

¹ <http://www.ontoknowledge.org/>.

7.7.6. Фреймворки

Для реализации рассмотренной выше архитектуры можно использовать различные фреймворки. Например, популярная система управления контентом Drupal¹ поддерживает семантическое представление данных. Также широко используются два хорошо поддерживаемых фреймворка с открытым кодом: Jena² и Sesame³. Наконец, такие компании, как Kasabi и Dydra, предоставляют доступ к размещенным на хостинге фреймворкам для разработки семантических веб-приложений.

Рекомендуемая литература

Некоторые ключевые публикации, которые были использованы в качестве основы для этой главы:

- Noy N. and McGuinness D. *Ontology Development 101: A Guide to Creating Your First Ontology* // www.ksl.stanford.edu/people/dlm/papers/ontology101/ontology101-noy-mcguinness.html.
- Omelayenko B. *Learning of Ontologies for the Web: The Analysis of Existing Approaches*. In *Proceedings of the International Workshop on Web Dynamics, 8th International Conference on Database Theory (ICDT 2001)*. 2001 // www.cs.vu.nl/~borys/papers/Web-Dyn01.pdf.
- Satya S. Sahoo, Wolfgang Halb, Sebastian Hellmann, Kingsley Idehen, Ted Thibodeau Jr, Sören Auer, Juan Sequeda, and Ahmed Ezzat. *A Survey of Current Approaches for Mapping of Relational Databases to RDF (PDF)*. W3CW3C RDB2RDF Incubator Group 2009-01-31 // http://www.w3.org/2005/Incubator/rdb2rdf/RDB2RDF_SurveyReport.pdf.
- Uschold M. and Gruninger M. *Ontologies: Principles, Methods and Applications*. *Knowledge Engineering Review*, Volume 11, 2 (1996): 93–155.

Две часто цитируемые книги:

- Davies J., Fensel D., and Harmelen F. van. *Towards the: Ontology-Driven Knowledge Management*. New York: Wiley, 2003.
- Maedche A. *Ontology Learning for the Семантического веба*. New York: Springer 2002.

¹ <http://semantic-drupal.com/>.

² <http://jena.sourceforge.net/>.

³ <http://openrdf.org>.

Упражнения и проекты

Предлагаемый проект имеет среднюю сложность и может быть выполнен двумя или тремя людьми примерно за две-три недели. Все необходимое для его реализации программное обеспечение находится в свободном доступе. Указанное ниже программное обеспечение было успешно использовано нами в аналогичных проектах, однако, учитывая очень активное состояние разработок в этой области, его список может достаточно быстро изменяться. Кроме того, если определенное программное обеспечение не упоминается, это не значит, что мы не одобряем его использования.

Задание состоит из трех частей.

1. Первая часть задания заключается в разработке онтологии, которая описывает некоторую предметную область и содержит информацию, необходимую для создания конкретного приложения. Термины, определенные в онтологии, в дальнейшем должны быть использованы для описания конкретных данных. На данном этапе следует применять методологию разработки онтологий, изложенную в первой части этой главы. В качестве языка представления знаний необходимо использовать язык OWL (см. главу 4).
2. Вторая часть работы заключается в разработке различных представлений данных на основе онтологии. Для каждого предполагаемого представления данных необходимо извлечь информацию из онтологии с помощью соответствующих запросов. Для выполнения этой части задания потребуется использование RDF-хранилищ и средств построения запросов (см. главу 2).
3. В третьей части проекта необходимо с помощью веб-технологий разработать различные графические представления изъятых из онтологии данных.

Часть I. Создание онтологии

Во-первых, необходимо выбрать предметную область, для которой будет строиться онтология. Желательно, что у вас были достаточные знания в этой области либо свободный доступ к эксперту, обладающему такими знаниями.

В этом описании проекта в качестве предметной области рассматривается теле- и радиовещание, а именно передачи радио и телевидения, теле- и радиопрограммы вещания, каналы, жанры передач, знаменитости радио и телевидения. Вы можете заменить эту предметную

область на любую другую. В наших собственных курсах каждый год мы предлагаем студентам различные предметные области, начиная с рейтингов университетов и заканчивая фильмами и рейтингом крупнейших компаний США Fortune 500.

Во-вторых, онтология, которую необходимо разработать для выбранной предметной области (например, вашего факультета), должна быть записана на языке OWL. При этом онтология должна содержать, по крайней мере, несколько десятков классов, но не обязательно, чтобы она покрывала всю предметную область. Уделите особое внимание качеству онтологии (ее ширине и глубине) и стремитесь максимально использовать выразительность языка OWL. На этом этапе работы можно использовать различные программные средства. Возможно, лучшим редактором онтологий является Protégé¹, но также у нас есть положительный опыт с программой TopBraid Composer².

Если вы амбициозны, то можете начать разработку с использования инструментов извлечения данных из текста или поэкспериментировать с инструментами импорта онтологий из слабоструктурированных источников данных, таких как листы Excel или файлы с разделителями табуляцией (см., например, Excel2RDF, ConvertToRDF, Any23 или XLWrap). Также вы можете использовать уже существующие в этой области онтологии.

Для валидации онтологии и проверки онтологии на наличие аномалий желательно использовать машину логического вывода. Если вы ведете разработку в редакторе онтологий Protégé, то можете использовать доступные для него плагины, например различные плагины для визуализации онтологии или машины вывода Pellet или HermiT.

В-третьих, необходимо наполнить онтологию конкретными индивидами и указать их свойства. Это может быть сделано, допустим, с помощью выбранного инструмента редактирования онтологий, если он поддерживает создание индивидов (например, Protégé). Учитывая простую синтаксическую структуру индивидов в языке RDF, вы можете также описать экземпляры вручную. Кроме того, можно запрограммировать некоторые простые скрипты для извлечения данных об экземплярах из доступных источников. Например, можно конвертировать данные из реляционной базы данных в RDF-утверждения или разработать скрейпер – программу для извлечения информации

¹ <http://protege.stanford.edu/>.

² http://topquadrant.com/products/TB_Composer.html.

с различных сайтов, содержащих информацию о программах теле- и радиовещания, конкретных передачах, их жанрах и знаменитостях. Телеканал BBC даже предлагает собственный удобный интерфейс прикладного программирования для реализации запросов непосредственно к его программе передач¹. Вы можете использовать сервис для валидации RDF-синтаксиса, предлагаемый консорциумом W3C². Данный сервис не только позволяет проверить файлы на синтаксическую правильность, но и обеспечивает визуализацию RDF-триплетов. Кроме того, на данном этапе вы также можете поэкспериментировать с инструментами импорта данных из слабоструктурированных источников.

После выполнения этого этапа вы должны предоставить:

- полную OWL-онтологию;
- экземпляры онтологии, описанные на языке RDF;
- отчет, в котором описываются предметная область онтологии и ее масштаб, а также основные проектные решения, принятые при ее моделировании.

Часть II. Разработка профилей с помощью SPARQL-запросов

На данном этапе необходимо использовать средства построения запросов к онтологиям с целью извлечения из нее определенных частей и данных. Для этого нужно разместить онтологию в репозитории, который поддерживает как средства построения запросов, так и средства логического вывода. Вы можете использовать для хранения онтологии и осуществления запросов к ней RDF-хранилище Sesame RDF³, которое поддерживает машину вывода OWLIM. Мы также обнаружили, что SPARQL-сервер Joseki является хорошей отправной точкой для разработки, поскольку он обеспечивает встроенный веб-сервер.

Прежде всего необходимо загрузить онтологию (в формате RDF/XML или Turtle) и связанные экземпляры в хранилище. При этом могут потребоваться некоторые усилия по его установке.

Затем с помощью языка запросов SPARQL следует определить различные профили пользователей и использовать SPARQL-запросы для извлечения из онтологии данных, соответствующих каждому из этих профилей.

¹ <http://www.bbc.co.uk/programmes/developers>.

² www.w3.org/RDF/Validator/.

³ www.openrdf.org/.

Например, онтологию телевизионных программ можно использовать для разработки следующих профилей: программы для людей с определенными предпочтениями (например, спорт, новости), программы для зрителей определенных возрастных групп (например, несовершеннолетних), сбор данных о передачах нескольких телевизионных станций (или даже разных стран), получение представлений, предназначенных для доступа через широкополосное подключение или через более медленные мобильные подключения и т. д.

Результаты выполнения запросов, которые определяют профиль, как правило, представлены в формате XML или JSON (JavaScript Object Notation).

Часть III. Представление информации, основанной на профиле

Используйте результаты выполнения запросов, полученные в части II данного проекта, для генерации удобочитаемого представления различных профилей. Для различных языков существуют удобные библиотеки для реализации запросов к SPARQL-точкам доступа: SPARQLWrapper¹ в языке Python, CM2² в языке PHP, ARQ (популярная часть библиотеки Jena³) в языке Java. Есть даже Javascript-библиотеки для визуализации результатов SPARQL-запросов, например sgvizler⁴.

Задача этой части проекта заключается в разработке тесно взаимосвязанных представлений данных (созданных и выбранных в ходе выполнения частей I и II данного проекта), доступных для просмотра в браузере.

Выбор альтернативной предметной области

В качестве предметной области для данного проекта вместо области радио- и телевидения можно выбрать деятельность факультета университета с данными о профессорско-преподавательском составе, кафедрах и читаемых на факультете дисциплинах. В этом случае для генерации онтологии и экземпляров данных вы можете использовать онлайн-источники информации, такие как телефонный справочник факультета, учебные планы, расписание и т. п. В качестве примеров

¹ <http://sparql-wrapper.sourceforge.net/>.

² <http://incubator.apache.org/jena/documentation/query/index.html>.

³ <http://incubator.apache.org/jena/>.

⁴ code.google.com/p/sgvizler/.

профилей для этой предметной области можно привести профили для студентов разных курсов, профили для иностранных студентов, профили для студентов и преподавателей и т. д.

Заключение

После завершения всех частей данного проекта вы эффективно реализуете основные части архитектуры, представленной на рис. 7.1. В ходе выполнения проекта вы используете языки, описанные в этой книге (RDF, RDF Schema, SPARQL, OWL2), и в результате разработаете полноценное приложение семантического веба: создадите онтологию как модель конкретной предметной области, используете запросы для определения пользовательских представлений, основанных на онтологии, и с помощью веб-технологий разработаете доступные для просмотра презентации конкретных пользовательских представлений.

Глава 8

Заключение

Семантический веб – это реальность. Как отмечалось в главе 6, растет число компаний, правительств и пользователей, которые внедряют и используют стандарты, технологии и подходы, описанные в этой книге. Мы рассмотрели, каким образом использование в вебе гибкой модели данных, RDF, позволяет осуществлять обмен информацией между приложениями и ее повторное использование. Постепенное добавление к этой модели данных выразительных семантических структур позволяет реализовать все более интересные логические выводы (глава 2). Язык OWL2 (глава 4) обеспечивает выразительное представление знаний, а языки правил (глава 5) позволяют закодировать вывод новых знаний с ориентацией на конкретные приложения. Доступ ко всей этой информации осуществляется с помощью веб-интегрированного языка запросов SPARQL (глава 3). Наконец, мы рассмотрели, каким образом могут применяться методы онтологического инжиниринга, для того чтобы использовать эти технологии при создании более интеллектуальных, более функциональных приложений (глава 6).

На протяжении всей книги подчеркивалось, что технологии и методы семантического веба основаны на знаниях, полученных из различных компьютерных дисциплин, включая искусственный интеллект, теорию баз данных и распределенных систем. Также указывались источники, которые могут быть использованы для дальнейшего изучения всех рассмотренных в книге тем. Читателю предлагается изучить эти области глубже.

8.1. Принципы

Знакомство с семантическим вебом заключается не только в изучении его приложений или технологий, но и в изучении общих принципов, которые являются полезными при проектировании систем, даже если эти системы не используют подходов семантического веба. Поэтому здесь данные принципы.

8.1.1. От простых технологий к сложным

Разработчики и пользователи нуждаются, с одной стороны, в простых технологиях на первых шагах их изучения и внедрения, а с другой – в механизме усложнения этих технологий по мере увеличения потребности в этом. Такая концепция постепенного пути усложнения технологий была успешно применена в семантическом вебе. Например, можно начать изучение и внедрение технологий с простой модели данных RDF, а затем перейти к более сложному и выразительному языку OWL. Даже в рамках самого языка OWL есть выбор – от простых версий правил, которые позволяют осуществлять быстрый вывод, до сложных описаний диалекта OWL2 Full.

8.1.2. Стандарты экономят время

Развитию веба в целом и семантического веба в частности способствует наличие стандартов. Эти стандарты позволяют потребителям информации не беспокоиться об адаптации к каждому новому производителю информации, а производителям, в свою очередь, быть уверенными в том, что они предоставляют потребителям именно нужную им информацию. При этом нет необходимости заново изобретать синтаксис или модели. В семантическом вебе стандарты обеспечивают возможность повторного использования информации. Если в разрабатываемом приложении необходимо представить информацию об Амстердаме, то нет необходимости собирать ее самостоятельно, так как можно использовать информацию, доступную в Википедии (в ее RDF-представлении, DBpedia). Эта возможность повторного использования данных экономит время при разработке и интеграции приложений. Сообщество семантического веба продолжает осваивать преимущества стандартов. Все чаще стандартизируются не только форматы и технологии, но также онтологии и другой контент.

8.1.3. Связи имеют жизненно важное значение

Сила семантического веба, как и веба в целом, заключается в связях между данными. Именно наличие связей позволяет сделать знания распределенными между организациями, людьми и системами; позволяет лицам, ответственным за знания, поддерживать их, а другим пользователям находить эти знания и получать к ним доступ. С одной стороны, связи между данными создаются достаточно легко, с другой – обеспечивают мощный механизм интеграции знаний. Мы можем наблюдать это на примере облака связанных данных LOD,

в котором сотни специализированных наборов данных используют другие специализированные наборы данных, для того чтобы обеспечить (простым способом) наличие большого количества детальной информации о некоторой теме или некотором понятии. Поэтому утверждение о том, что успех или провал любой крупной информационной системы определяется ее способностью реализовать и поддерживать связи, кажется вполне оправданным.

8.1.4. Немного семантики имеет большое значение

Джим Хендлер, один из создателей семантического веба, сказал: «Немного семантики имеет большое значение». Эта фраза подчеркивает важность возможности автоматического распознавания смысла терминов. Не достаточно реализовать обмен данными, нужно обеспечить обмен смыслом этих данных. Например, простое знание о том, что ключ – это музыкальный знак, а не инструмент для открывания и закрывания замков, имеет огромное значение для функционирования сложных систем. На протяжении всей книги мы описывали методы ~~для~~ добавления семантики к данным и использования этой семантики в приложениях. Применяются ли эти конкретные технологии в конкретной системе – это отдельный вопрос; самое важное – осознавать, что семантика данных может быть отличительным компонентом систем.

Принципы, описанные выше, обеспечили успех концепции семантического веба, но что еще более важно – они могут быть применены при разработке любой крупномасштабной программной системы или набора технологий.

8.2. Что дальше?

Семантический веб все еще развивается и меняется. Совершенствуются существующие технологии, и разрабатываются новые. Если отвлечься от частных случаев, можно выделить несколько крупных научных направлений, которые активно развиваются:

- логические выводы в зависимости от контекста. Это умение делать выводы в зависимости не только от конкретных знаний, но и от ситуации в той области, откуда эти знания получены (их происхождения). Например, вы можете доверять некоторому источнику данных о квартирах, когда вы только приступили к поиску квартиры, но не тогда, когда дело доходит до принятия окончательного решения;

- масштабные логические выводы. В связи с увеличением количества данных в семантическом вебе появились трудности в осуществлении логического вывода. Такие проекты, как Large Knowledge Collidor (<http://www.larkc.eu>), демонстрируют возможность осуществлять простой вывод, основанный на правилах, на миллиардах триплетов. Однако, так как растет сложность семантики и продолжает лавинообразно увеличиваться количество данных и знаний, появляется потребность в более быстрых и качественных машинах логического вывода;
- распределенные запросы. На сегодняшний день стандартная практика использования распределенных источников знаний в семантическом вебе заключается в централизации знаний с помощью их сбора из этих источников. Это делается в целях эффективности. Однако по-прежнему остается открытым вопрос о том, как использовать эти распределенные источники информации без необходимости ее централизации, в том числе как осуществлять распределенные запросы;
- потоковые знания. Технологии семантического веба прежде всего предназначены для обработки статической или медленно изменяющейся информации. Однако ситуация быстро меняется, так как информация производится с помощью датчиков, микроблогов и других источников с постоянно возрастающей скоростью. Соответственно, вопросы о том, как извлекать, обрабатывать и использовать семантику таких потоковых источников данных, являются открытыми.

Таковы лишь некоторые из текущих направлений исследований, проводимых в области семантического веба. Сама область и применяемые в ней технологии быстро меняются. В этой книге описаны лишь основы семантического веба, но сама история его развития далека от завершения. Чтобы ознакомиться с текущим положением дел, обратитесь к онлайн-приложению книги: <http://www.semanticwebprimer.org>.

Приложение А

Основы XML

А.1. Язык XML

XML-документ состоит из пролога, элементов и необязательного эпилога (который здесь не обсуждается).

А.1.1. Пролог

Пролог состоит из XML-декларации и необязательных ссылок на внешние структурирующие документы¹. Приведем пример XML-декларации:

```
<?xml version="1.0" encoding="UTF-16"?>
```

Данная декларация указывает, что текущий документ является XML-документом, и определяет версию и кодировку, используемую в конкретной системе (например, UTF-8, UTF-16 и ISO 8859-1). Указание кодировки необязательно, но считается хорошей практикой. В декларации можно также указать, является ли документ автономным, то есть содержит ли он ссылки на внешние структурирующие документы:

```
<?xml version="1.0" encoding="UTF-16" standalone="no"?>
```

Ссылка на внешние структурирующие документы выглядит следующим образом:

```
<!DOCTYPE book SYSTEM "book.dtd">
```

В данном примере структурирующая информация находится в локальном файле с именем `book.dtd`. Вместо имени файла может быть использован идентификатор URL. Если известно только локальное имя или только URL, то указывается метка `SYSTEM`. Если указывается и локальное имя, и URL, то данная метка должна быть заменена меткой `PUBLIC`.

¹ Имеются в виду документы, содержащие средства разметки, с помощью которых структурируют исходный документ. – *Прим. перев.*

А.1.2. Элементы

XML-элементы представляют собой «вещи» – сущности, о которых идет речь в XML-документе, например книги, авторы и издатели. Элементы представляют основную концепцию XML-документов. Элемент состоит из открывающего тега, содержимого и закрывающего тега. Например:

```
<lecturer>Дэвид Биллингтон</lecturer>.
```

Имена тегов могут быть выбраны почти произвольно; есть только несколько ограничений. Наиболее важные из них: первый символ должен быть буквой, знаком подчеркивания или двоеточием; имя не может начинаться со строки «xml», записанной любым способом (например, как «Xml» и «XML»).

Содержимое элемента может представлять собой текст, другие элементы или быть пустым. Например:

```
<lecturer>
  <name>Дэвид Биллингтон</name>
  <phone>+61-7-3875 507</phone>
</lecturer>
```

Если содержимое отсутствует, то элемент называется пустым. Пустой элемент, например

```
<lecturer></lecturer>,
```

может быть сокращенно записан следующим образом:

```
<lecturer/>.
```

А.1.3. Атрибуты

Пустой элемент необязательно является бессмысленным, так как он может иметь некоторые свойства, указанные в качестве *атрибутов*. Атрибут представляет пару «имя–значение» внутри открывающего тега элемента:

```
<lecturer name="Дэвид Биллингтон" phone="+61-7-3875 507"/>
```

Приведем пример атрибутов для непустого элемента:

```
<order orderNo="23456" customer="Джон Смит" date="15.10.2002">
  <item itemNo="a528" quantity="1"/>
  <item itemNo="c817" quantity="3"/>
</order>
```

Эта же информация может быть записана следующим образом, если заменить атрибуты вложенными элементами:

```
<order>
  <orderNo>23456</orderNo>
  <customer>Джон Смитт</customer>
  <date>15.10.2002</date>
  <item>
    <itemNo>a528</itemNo>
    <quantity>1</quantity>
  </item>
  <item>
    <itemNo>c817</itemNo>
    <quantity>3</quantity>
  </item>
</order>
```

Выбор между использованием элементов и атрибутов часто является делом вкуса. Однако обратите внимание, что атрибуты не могут быть вложенными.

А.1.4. Комментарии

Комментарий – это фрагмент текста, который должен быть проигнорирован парсером. Комментарии имеют следующий вид:

```
<!-- Это комментарий -->
```

А.1.5. Инструкции обработки

Инструкции обработки (англ. *Processing Instructions*, PIs) обеспечивают механизм передачи информации приложению о том, как обрабатывать элементы. Общая форма инструкции обработки имеет вид:

```
<?ЦелевоеПриложение Инструкция?>
```

Например:

```
<?stylesheet type="text/css" href="mystyle.css"?>
```

Инструкции обработки предоставляют процедурные возможности другой объявленной среды.

А.1.6. Корректные XML-документы

Корректный (англ. *well-formed*) XML-документ – это синтаксически правильный документ. Ниже приведены некоторые синтаксические правила, которым должен удовлетворять корректный документ:

- в документе существует только один элемент верхнего уровня (так называемый *корневой* элемент);
- каждый элемент содержит открывающий и соответствующий закрывающий теги;
- теги не могут перекрываться, например следующим образом:

```
<author><name>Ли Хонг</author></name>;
```

- атрибуты внутри элемента имеют уникальные имена;
- имена элементов и тегов должны быть правильными.

A.1.7. Модель «дерево» для представления XML-документов

Корректные XML-документы можно представить в виде деревьев. Таким образом, именно дерево является формальной моделью данных в языке XML. Представление данных в виде дерева обеспечивает их наглядность. В качестве примера рассмотрим следующий документ:

```
<?xml version="1.0" encoding="UTF-16"?>
<!DOCTYPE email SYSTEM "email.dtd">
<email>
  <head>
    <from name="Михаил Махер" address="michaelmaher@cs.gu.edu.au"/>
    <to name="Григорис Антониу" address="grigoris@cs.unibremen.de"/>
    <subject>Где проект статьи?</subject>
  </head>
  <body>
    Григорис, где проект статьи, который ты мне обещал на прошлой неделе?
  </body>
</email>
```

На рис. A.1 изображено представление этого XML-документа в виде дерева. Это упорядоченное, помеченное дерево, обладающее следующими свойствами:

- существует точно один корень дерева;
- дерево не содержит циклов;
- каждый узел, кроме корня, имеет точно один родительский узел;
- каждый узел имеет метку;
- порядок XML-элементов имеет важное значение.

Однако если порядок элементов важен, то порядок атрибутов не имеет значения. Таким образом, следующие два элемента эквивалентны:


```
<person lastname="Иванов" firstname="Петр"/>
<person firstname="Петр" lastname="Иванов"/>
```

Эта особенность не может быть корректно представлена в описанном дереве. В общем случае требуется уточнение понятия дерева, например необходимо различать типы узлов (узел элемента, узел атрибута и т. д.). Однако здесь граф используется в качестве иллюстрации, поэтому мы не будем углубляться в детали.

На рис. А.1 также видна разница между *корнем* (представляющим XML-документ) и *корневым элементом*, в нашем случае элементом email. Это различие будет играть роль при рассмотрении адресации и запросов к XML-документам.

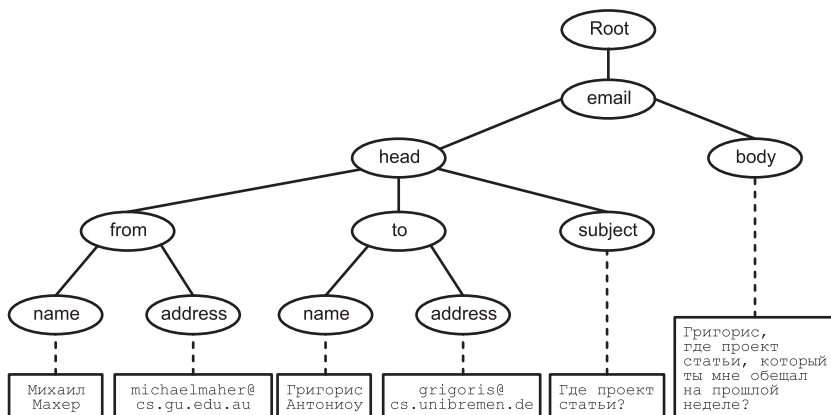


Рис. А.1 ❖ Представление XML-документа в виде дерева

А.2. Структурирование информации

Корректный XML-документ удовлетворяет определенным синтаксическим правилам. Однако эти правила не говорят нам ничего определенного о структуре документа. Пусть необходимо осуществлять обмен данными между двумя приложениями на основе одного словаря. Для этого нужно определить все имена элементов и атрибутов, которые могут быть использованы. Кроме того, должна быть определена структура документов: значения, которые могут принимать атрибуты, допустимая и требуемая вложенность элементов и т. д.

Наличие подобной информации о структуризации документа позволяет усовершенствовать процедуру проверки документа. Мы го-

ворим, что XML-документ является *валидным*, если он корректен, использует структурирующую информацию и удовлетворяет ей.

Структура XML-документов может быть определена двумя способами: с помощью деклараций типов документов – DTD (это более старый способ, имеющий ряд ограничений) и с помощью XML-схем, которые предоставляют расширенные возможности, в основном для определения типов данных.

A.2.1. DTD

Внешние и внутренние DTD

Компоненты DTD могут быть определены в отдельном файле (внешняя DTD) или в самом XML-документе (внутренняя DTD). Обычно лучше использовать внешние DTD, так как их определения могут быть использованы в нескольких документах; в противном случае неизбежно дублирование, а следовательно, поддержка непротиворечивости данных со временем станет проблематичной.

Элементы

Рассмотрим элемент из предыдущего раздела:

```
<lecturer>
  <name>Дэвид Биллингтон</name>
  <phone>+61-7-3875 507</phone>
</lecturer>
```

DTD для данного типа элемента¹ выглядит так:

```
<!ELEMENT lecturer (name,phone)>
<!ELEMENT name (#PCDATA)>
<!ELEMENT phone (#PCDATA)>
```

Смысл этой декларации DTD заключается в следующем:

- в данном документе могут быть использованы типы элементов `lecturer`, `name` и `phone`;
- элемент `lecturer` содержит элемент `name` и элемент `phone`, именно в таком порядке;
- элемент `name` и элемент `phone` могут иметь любое содержимое. В DTD `#PCDATA` используется для обозначения только атомарных типов элементов.

¹ Различие между типом элемента `lecturer` и конкретным элементом этого типа, например Дэвид Биллингтон, очевидно. Все конкретные элементы типа (называемые элементами `lecturer`) имеют структуру, аналогичную той, которая определяется здесь.

Для того чтобы выразить тот факт, что элемент `lecturer` содержит либо элемент `name`, либо элемент `phone`, используется следующая запись:

```
<!ELEMENT lecturer (nameTphone)>
```

Сложнее выразить тот факт, что элемент `lecturer` содержит элемент `name` и элемент `phone`, причем порядок этих элементов не имеет значения. Для этого мы можем использовать следующий способ:

```
<!ELEMENT lecturer ((name,phone)|(phone,name))>
```

Однако этот способ является неудобным, если требуется указать большое количество элементов в любом порядке (например, десять элементов).

Атрибуты

Рассмотрим элемент из предыдущего раздела:

```
<order orderNo="23456" customer="Джон Смитт" date="15.10.2002">
  <item itemNo="a528" quantity="1"/>
  <item itemNo="c817" quantity="3"/>
</order>
```

DTD для него выглядит следующим образом:

```
<!ELEMENT order (item+)>
<!ATTLIST order
  orderNo    ID      #REQUIRED
  customer   CDATA   #REQUIRED
  date       CDATA   #REQUIRED>
<!ELEMENT item EMPTY>
<!ATTLIST item
  itemNo ID #REQUIRED
  quantity CDATA #REQUIRED
  comments CDATA #IMPLIED>
```

По сравнению с предыдущим примером, тут появились две новые конструкции. Во-первых, тип элемента `item` определяется как пустой. Во-вторых, после элемента `item` в определении типа элемента `order` указан оператор `+`. Это один из *операторов кардинальности*:

- `?`: означает, что элемент может встречаться нуль или один раз;
- `*`: означает, что элемент может встречаться нуль или более раз;
- `+`: означает, что элемент может встречаться один или более раз.

Отсутствие оператора кардинальности означает, что должен быть указан точно один элемент.

Кроме элементов, также необходимо определить атрибуты. Это делается с помощью *списка атрибутов* (тег `<!ATTLIST>`). Первым компонентом списка является имя типа элемента, к которому применяется список, а затем указывается сам список, состоящий из троек вида «имя атрибута, тип атрибута, тип значения». Имя атрибута представляет собой имя, допустимое в XML-документе, использующем DTD.

Типы атрибутов

Типы атрибутов схожи с предопределенными типами данных, но их количество очень ограничено. Перечислим наиболее важные типы атрибутов:

- CDATA, строка (последовательность символов);
- ID, имя, которое является уникальным во всем XML-документе;
- IDREF, ссылка на другой элемент с атрибутом ID, имеющим то же значение, что и атрибут IDREF;
- IDREFS, последовательность IDREF;
- $(v_1 | \dots | v_n)$, перечисление всех возможных значений.

Очевидно, что выбор типов очень невелик. Например, нет специальных типов для представления дат и чисел, они должны интерпретироваться как строки (CDATA). Таким образом, не может быть учтена их специфическая структура.

Типы значений атрибутов

Определены четыре типа значений атрибутов:

- #REQUIRED. Атрибут является обязательным для каждого элемента рассматриваемого типа в XML-документе. В предыдущем примере атрибуты `itemNo` и `quantity` должны быть указаны для любого элемента `item`;
- #IMPLIED. Атрибут не является обязательным. В нашем примере атрибут `comments` не является обязательным;
- #FIXED "значение". Атрибут является обязательным, причем его значение всегда равно значению, указанному после слова #FIXED. Значение, заданное в XML-документе, не будет иметь смысла, потому что оно по умолчанию перекрывается этим фиксированным значением;
- "значение". Таким образом определяется значение атрибута по умолчанию. Если в XML-документе будет указано конкретное значение данного атрибута, то оно перекроет значение по умолчанию. Например, для системы электронной почты по умолча-

нию может использоваться кодировка "mime", но пользователь может явно задать кодировку "binhex".

Ссылки

Приведем пример использования атрибутов типов IDREF и IDREFS. Во-первых, зададим следующую декларацию DTD:

```
<!ELEMENT family (person*)>
<!ELEMENT person (name)>
<!ELEMENT name (#PCDATA)>
<!ATTLIST person
  id      ID      #REQUIRED
  mother  IDREF   #IMPLIED
  father  IDREF   #IMPLIED
  children IDREFS  #IMPLIED>
```

Приведем пример XML-элемента, удовлетворяющего этой DTD:

```
<family>

  <person id="bob" mother="mary" father="peter">
    <name>Bob Marley</name>
  </person>

  <person id="bridget" mother="mary">
    <name>Bridget Jones</name>
  </person>

  <person id="mary" children="bob bridget">
    <name>Mary Poppins</name>
  </person>

  <person id="peter" children="bob">
    <name>Peter Marley</name>
  </person>
</family>
```

Рекомендуем читателям самостоятельно изучить ссылки между людьми в этом элементе.

XML-сущности

XML-сущности могут использоваться разными способами: как обозначение именованной последовательности символов, которую требуется повторять (то есть в качестве сокращения), как раздел

внешних данных (например, XML-данных или других) или как часть декларации для элементов. Например, предположим, что документ содержит несколько ссылок на авторские права, которые относятся к текущему году. В этом случае имеет смысл объявить сущность

```
<!ENTITY thisyear "2007">.
```

Тогда в каждом месте, где нужно указать этот год, мы вместо этого можем использовать ссылку на сущность `&thisyear`. Таким образом, для того чтобы изменить значение года на "2008", во всем документе нужно будет только обновить декларацию сущности.

Заключительный пример

В качестве заключительного примера приведем DTD для элемента `email`, представленного выше:

```
<!ELEMENT email (head,body)>
<!ELEMENT head (from,to+,cc*,subject)>
<!ELEMENT from EMPTY>
<!ATTLIST from
  name      CDATA      #IMPLIED
  address   CDATA      #REQUIRED>
<!ELEMENT to EMPTY>
<!ATTLIST to
  name      CDATA      #IMPLIED
  address   CDATA      #REQUIRED>
<!ELEMENT cc EMPTY>
<!ATTLIST cc
  name      CDATA      #IMPLIED
  address   CDATA      #REQUIRED>
<!ELEMENT subject (#PCDATA)>
<!ELEMENT body (text,attachment*)>
<!ELEMENT text (#PCDATA)>
<!ELEMENT attachment EMPTY>
<!ATTLIST attachment
  encoding  (mime|binhex) "mime"
  file      CDATA      #REQUIRED>
```

Остановимся на наиболее значимых фрагментах этой DTD:

- элемент `head` содержит элемент `from`, по крайней мере один элемент `to`, ноль или более элементов `cc`, и элемент `subject` в указанном порядке;
- в элементах `from`, `to` и `cc` атрибут `name` является необязательным, а атрибут `address` — обязательным;

- элемент `body` содержит элемент `text` и затем, возможно, несколько элементов `attachment`;
- атрибут `encoding` элемента `attachment` должен принимать значение "mime" или "binhex", причем первое из них является значением по умолчанию.

Завершим описание DTD двумя замечаниями. Во-первых, DTD может интерпретироваться как расширенная форма Бэкуса-Наура (РФБН). Например, декларация

```
<!ELEMENT email (head,body)>
```

эквивалентна правилу

```
email ::= head body,
```

которое означает, что элемент `email` состоит из элемента `head` и элемента `body`. А во-вторых, в DTD допустимы рекурсивные определения. Например, DDT

```
<!ELEMENT bintree ((bintree root bintree)|emptytree)>
```

определяет бинарные деревья следующим образом: бинарное дерево – это либо пустое дерево, либо состоит из левого поддерева, корня и правого поддерева.

A.2.2. XML-схема

XML-схема представляет значительно более выразительный язык для определения структуры XML-документов. Одной из характеристик XML-схемы является то, что ее синтаксис основан собственно на языке XML. Это проектное решение обеспечивает значительное улучшение читабельности и, что более важно, позволяет значительно увеличить повторное использование технологии. Для того чтобы получить отдельный синтаксис, не нужно писать отдельные парсеры, редакторы, программы для печати в наглядной форме и т. д., как это требовалось для DTD; любой XML-документ обеспечивает синтаксис. Еще более важное преимущество – возможность повторного использования и уточнения схем. XML-схема позволяет определять новые типы за счет расширения или ограничения уже существующих. Эта особенность в сочетании с синтаксисом на основе XML позволяет строить схемы на базе уже существующих схем, тем самым сокращая затраты на разработку. Наконец, XML-схема обеспечивает большое разнообразие типов данных, которые могут использоваться

в XML-документах (в то время как типы данных в DDT представлены только строками).

XML-схема – это элемент с открывающим тегом следующего вида:

```
<xsd:schema
  xmlns:xsd="http://www.w3.org/2000/10/XMLSchema"
  version="1.0">
```

Приведенный элемент использует схему для XML-схемы, которую можно найти на сайте консорциума W3C. Это, образно говоря, фундамент, на основе которого могут быть построены новые схемы. Префикс `xsd` обозначает пространство имен данной схемы (подробнее пространства имен будут рассмотрены в следующем разделе). Если в атрибуте `xmlns` префикс опущен, то по умолчанию используются элементы именно из этого пространства имен:

```
<schema
  xmlns="http://www.w3.org/2000/10/XMLSchema"
  version="1.0">
```

В дальнейшем мы будем опускать префикс `xsd`.

Теперь перейдем к рассмотрению элементов схемы, а именно к способам определения элементов и типов атрибутов, которые задаются с помощью типов данных.

Типы элементов

Типы элементов задаются с использованием следующего синтаксиса:

```
<element name="..."/>.
```

Они могут иметь несколько дополнительных атрибутов следующего вида:

```
type="..."
```

или ограничения кардинальности:

- `minOccurs="x"`, где `x` – любое натуральное число (в том числе ноль);
- `maxOccurs="x"`, где `x` – любое натуральное число (в том числе ноль) или `unbounded` (неограниченное значение).

Ограничения `minOccurs` и `maxOccurs` являются обобщениями операторов кардинальности `?`, `*` и `+`, используемыми в DTD. Если ограничения кардинальности не указаны явным образом, то по умолчанию `minOccurs=1` и `maxOccurs=1`.

Приведем несколько примеров:

```
<element name="email"/>
<element name="head" minOccurs="1" maxOccurs="1"/>
<element name="to" minOccurs="1"/>
```

Типы атрибутов

Типы атрибутов задаются с использованием следующего синтаксиса:

```
<attribute name="..." />
```

Они могут иметь несколько дополнительных атрибутов следующего вида:

```
type="..."
```

или атрибутов наличия (что соответствует типам значений атрибутов #OPTIONAL и #IMPLIED в DTD):

use="x", где x может принимать значения optional (необязательный атрибут), required (обязательный атрибут) или prohibited (запрещенный атрибут)

или значение по умолчанию (что соответствует типу значений атрибутов #FIXED и значению по умолчанию в DTD).

Приведем примеры:

```
<attribute name="id" type="ID" use="required"/>
<attribute name="speaks" type="Language" use="optional" default="en"/>
```

Типы данных

Как уже отмечалось, основным недостатком DTD является очень ограниченный выбор типов данных. XML-схема предоставляет большие возможности для определения типов данных. Во-первых, определены *встроенные типы данных*. Перечислим некоторые из них:

- числовые типы данных, в том числе integer, short, Byte, long, float, decimal;
- строковые типы данных, в том числе string, ID, IDREF, CDATA, language;
- типы данных для определения даты и времени, в том числе time, date, gMonth, gYear.

Кроме того, существуют типы данных, определенные пользователем, делящиеся на *простые типы данных* – simpleType (элементы этих типов не могут содержать других элементов или атрибутов)

и сложные типы данных – complexType (элементы этих типов могут содержать другие элементы и атрибуты). Прежде всего рассмотрим сложные типы данных, отложив обсуждение простых до тех пор, пока не опишем процедуру ограничения типов данных. Сложные типы определяются на основе уже существующих типов данных посредством определения атрибутов (если таковые имеются) и с использованием следующих элементов:

- sequence, последовательность существующих элементов типов данных, в которой важен порядок элементов;
- all, совокупность элементов, в которой не важен порядок элементов;
- choice, совокупность элементов, из которых будет выбран один.

Приведем пример:

```
<complexType name="lecturerType">
  <sequence>
    <element name="firstname" type="string"
      minOccurs="0" maxOccurs="unbounded"/>
    <element name="lastname" type="string"/>
  </sequence>
  <attribute name="title" type="string" use="optional"/>
</complexType>
```

Любой элемент в XML-документе, который объявлен как элемент типа lecturerType, может иметь атрибут title, может включать в себя любое количество элементов name и должен включать в себя ровно один элемент lastname.

Расширение типов данных

Уже существующие типы данных могут быть расширены новыми элементами или атрибутами. В качестве примера рассмотрим расширение типа данных lecturerType.

```
<complexType name="extendedLecturerType">
  <complexContent>
    <extension base="lecturerType">
      <sequence>
        <element name="email" type="string" minOccurs="0" maxOccurs="1"/>
      </sequence>
      <attribute name="rank" type="string" use="required"/>
    </extension>
  </complexContent>
</complexType>
```

В этом примере тип данных `lecturerType` расширен за счет добавления элемента `email` и атрибута `rank`. В результате тип `extendedLecturerType` данных выглядит следующим образом:

```
<complexType name="extendedLecturerType">
  <sequence>
    <element name="firstname" type="string"
      minOccurs="0" maxOccurs="unbounded"/>
    <element name="lastname" type="string"/>
    <element name="email" type="string" minOccurs="0" maxOccurs="1"/>
  </sequence>
  <attribute name="title" type="string" use="optional"/>
  <attribute name="rank" type="string" use="required"/>
</complexType>
```

Между расширяемым (исходным) и расширенным типами данных существует связь иерархического типа. *Экземпляры расширенного типа также являются экземплярами расширяемого типа.* Они могут содержать дополнительную информацию, но не могут содержать меньше информации, чем экземпляры исходного типа данных, или информацию несоответствующих типов.

Ограничение типов данных

Существующий тип данных также может быть ограничен путем добавления ограничений на определенные значения. Например, могут быть добавлены новые атрибуты `type` и `use` или ужесточены численные ограничения на значения `minOccurs` и `maxOccurs`.

Важно понимать, что ограничение *не* является процессом, обратным к расширению. Ограничение не может осуществляться путем удаления элементов или атрибутов. Таким образом, следующие иерархические отношения остаются в силе: *экземпляры ограниченного типа также являются экземплярами ограничиваемого (исходного) типа.* Они удовлетворяют, по крайней мере, ограничениям исходного типа и некоторым новым ограничениям.

В качестве примера рассмотрим ограничение типа данных `lecturerType`:

```
<complexType name="restrictedLecturerType">
  <complexContent>
    <restriction base="lecturerType">
      <sequence>
        <element name="firstname" type="string" minOccurs="1" maxOccurs="2"/>
      </sequence>
```

```
        <attribute name="title" type="string" use="required"/>
    </restriction>
</complexContent>
</complexType>
```

Ужесточенные ограничения выделены в примере жирным шрифтом. Читателям рекомендуется сравнить их с исходными ограничениями.

Простые типы данных также могут быть определены путем ограничения существующих типов данных. Например, можно определить тип данных для обозначения дней месяца `DayOfMonth`, экземпляры которого должны принимать значения от 1 до 31, следующим образом:

```
<simpleType name="dayOfMonth">
  <restriction base="integer">
    <minInclusive value="1"/>
    <maxInclusive value="31"/>
  </restriction>
</simpleType>
```

Кроме того, можно определить тип данных путем перечисления всех возможных значений. Например, можно определить тип данных для обозначения дней недели `DayOfWeek` следующим образом:

```
<simpleType name="dayOfWeek">
  <restriction base="string">
    <enumeration value="Mon"/>
    <enumeration value="Tue"/>
    <enumeration value="Wed"/>
    <enumeration value="Thu"/>
    <enumeration value="Fri"/>
    <enumeration value="Sat"/>
    <enumeration value="Sun"/>
  </restriction>
</simpleType>
```

Заключительный пример

Определим XML-схему для элемента `email`; таким образом, можно сравнить эту схему с декларацией DTD, приведенной выше.

```
<element name="email" type="emailType"/>
<complexType name="emailType">
  <sequence>
    <element name="head" type="headType"/>
    <element name="body" type="bodyType"/>
```

```

    </sequence>
</complexType>

<complexType name="headType">
  <sequence>
    <element name="from" type="nameAddress"/>
    <element name="to" type="nameAddress"
      minOccurs="1" maxOccurs="unbounded"/>
    <element name="cc" type="nameAddress"
      minOccurs="0" maxOccurs="unbounded"/>
    <element name="subject" type="string"/>
  </sequence>
</complexType>

<complexType name="nameAddress">
  <attribute name="name" type="string" use="optional"/>
  <attribute name="address" type="string" use="required"/>
</complexType>

<complexType name="bodyType">
  <sequence>
    <element name="text" type="string"/>
    <element name="attachment" minOccurs="0"
      maxOccurs="unbounded">
      <complexType>
        <attribute name="encoding" use="optional" default="mime">
          <simpleType>
            <restriction base="string">
              <enumeration value="mime"/>
              <enumeration value="binhex"/>
            </restriction>
          </simpleType>
        </attribute>
        <attribute name="file" type="string" use="required"/>
      </complexType>
    </element>
  </sequence>
</complexType>

```

Следует отметить, что некоторые типы данных определяются отдельно и имеют имя, а другие определены анонимно внутри других типов (например, типы для элемента `attachment` и атрибута `encoding`). В общем случае, если тип данных используется только один раз, имеет смысл определить его анонимно для локального применения. Од-

нако применение этого подхода ограничивается, если вложенность элементов становится слишком глубокой.

А.3. Пространства имен

Одним из основных преимуществ использования XML как универсального языка разметки (метаязыка) является то, что он допускает использование информации из различных источников; с технической точки зрения это значит, что XML-документ может использовать более чем одну DTD или схему. Но так как подобные структурирующие документы разрабатываются независимо друг от друга, то неминуемы *конфликты имен*. Пусть DTD *A* и DTD *B* определяют тип элемента *e* по-разному. Тогда парсеру, анализирующему XML-документ, в котором содержится элемент *e*, необходимо сообщить, какую именно декларацию DTD использовать для валидации.

Для решения этой проблемы неоднозначности имен можно предложить простое техническое решение: для каждой DTD или схемы использовать собственный префикс. Префикс отделяется от локального имени двоеточием:

префикс:имя.

В качестве примера рассмотрим (воображаемый) совместный проект австралийского университета, скажем, университета Гриффит (Griffith University), и американского университета, скажем, университета Кентукки (University of Kentucky), в рамках которого необходимо разработать унифицированное представление информации для онлайн-студентов. Каждый университет использует свою собственную терминологию, которая в чем-то отличается от терминологии другого университета. Например, преподаватели (lecturer) в Соединенных Штатах не считаются постоянным профессорско-преподавательским составом (faculty), в то время как в Австралии это так (фактически преподаватели (lecturer) в Австралии соответствуют доцентам (assistant professor) в Соединенных Штатах). Следующий пример показывает, каким образом может быть достигнута однозначность в именовании элементов.

```
<?xml version="1.0" encoding="UTF-16"?>
<vu:instructors
  xmlns:vu="http://www.vu.com/empDTD"
  xmlns:gu="http://www.gu.au/empDTD"
  xmlns:uky="http://www.uky.edu/empDTD">
```

```

<uky:faculty
  uky:title="assistant professor"
  uky:name="John Smith"
  uky:department="Computer Science"/>
<gu:academicStaff
  gu:title="lecturer"
  gu:name="Mate Jones"
  gu:school="Information Technology"/>
</vu:instructors>

```

Таким образом, пространства имен объявляются внутри элемента и могут быть использованы в этом элементе и в любом из его дочерних элементов и атрибутов. Объявление пространства имен имеет следующий вид:

xmlns:префикс="расположение",

где расположение представляет собой адрес DTD или схемы. Если префикс не указан, например

xmlns:="расположение",

то указанный адрес используется по умолчанию. Допустим, предыдущий пример эквивалентен следующему документу:

```

<?xml version="1.0" encoding="UTF-16"?>
<vu:instructors
  xmlns:vu="http://www.vu.com/empDTD"
  xmlns="http://www.gu.au/empDTD"
  xmlns:uky="http://www.uky.edu/empDTD">
  <uky:faculty
    uky:title="assistant professor"
    uky:name="John Smith"
    uky:department="Computer Science"/>
  <academicStaff
    title="lecturer"
    name="Mate Jones"
    school="Information Technology"/>
</vu:instructors>

```

А.4. Адресация и запросы к XML-документам

В реляционных базах данных различные части информации могут быть выбраны и возвращены с помощью языков запросов, таких как SQL. То же самое можно сделать и с XML-документами, для кото-

рых существует целый ряд языков запросов, таких как XQL, XML-QL и XQuery.

Центральное понятие в языках запросов к XML – *выражение пути* (*path expression*), который определяет, каким образом может быть достигнут узел или набор узлов в дереве, представляющем XML-документ. Будем рассматривать выражения пути в форме выражений XPath, потому что они могут быть использованы не только для запросов, но и для других целей, а именно для преобразования XML-документов.

XPath является языком для организации доступа к частям XML-документа. Он работает на модели данных «дерево» для XML-документов и имеет синтаксис, отличный от принятого в XML. Основным понятием языка является выражение пути. Путь может быть:

- абсолютным (начинающимся с корня дерева); синтаксически выражение такого пути начинается с символа /, что указывает на корень документа, расположенного на один уровень выше корневого элемента данного документа;
- относительным (путь начинается с узла контекста).

Рассмотрим следующий XML-документ:

```
<?xml version="1.0" encoding="UTF-16"?>
<!DOCTYPE library PUBLIC "library.dtd">
<library location="Бремен">
  <author name="Генри Вайз">
    <book title="Искусственный интеллект"/>
    <book title="Современные веб-сервисы"/>
    <book title="Теория вычислений"/>
  </author>
  <author name="Вильям Смарт">
    <book title="Искусственный интеллект"/>
  </author>
  <author name="Синтия Синглтон">
    <book title="Семантический веб"/>
    <book title="Технологии браузеров"/>
  </author>
</library>
```

Его представление в виде дерева изображено на рис. А.2.

Ниже проиллюстрированы возможности XPath несколькими примерами выражений путей.

1. Обращение ко всем элементам author:

```
/library/author
```

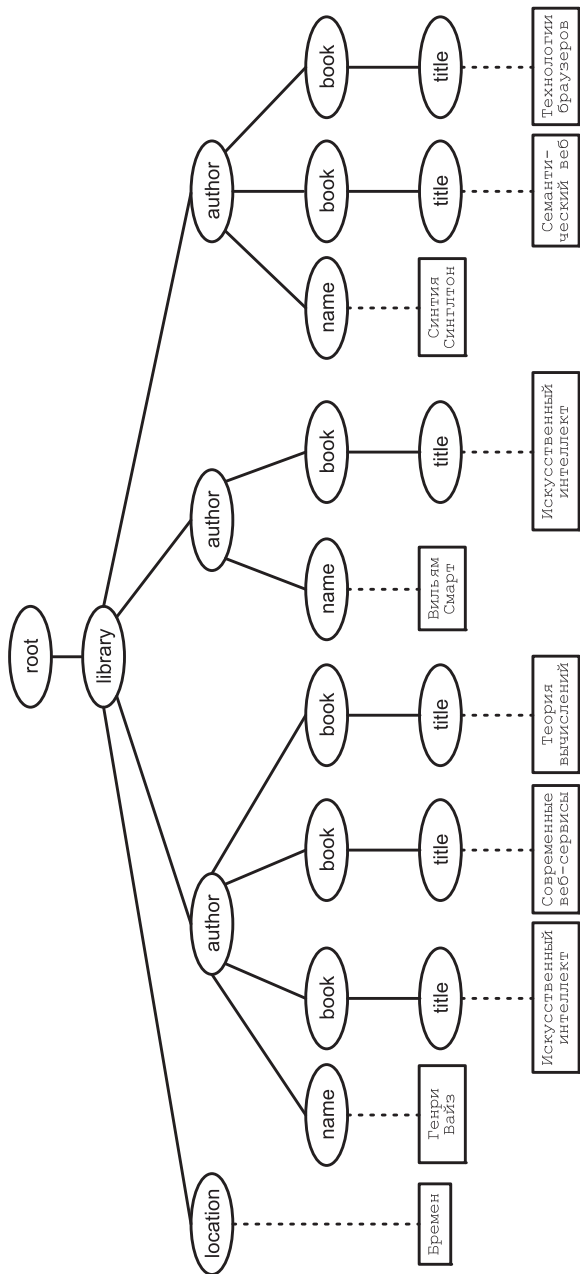



Рис. А.2 ❖ Дерево, представляющее документ library

Это выражение пути позволяет осуществить доступ ко всем элементам `author`, которые являются дочерними для узла – элемента `library`, который находится непосредственно под корнем. Путь в дереве определяется с помощью последовательности $/t_1/.../t_n$, где каждый узел t_{i+1} является дочерним для узла t_i .

2. Альтернативное решение для предыдущего примера:

```
//author
```

Здесь символы `//` означают, что необходимо рассмотреть все элементы документа и проверить, являются ли они элементами типа `author`. Другими словами, это выражение пути обеспечивает доступ ко всем элементам `author`, расположенным в любом месте документа. Учитывая структуру конкретного рассматриваемого XML-документа, это выражение и предыдущее выражение приведут к одному и тому же результату; однако в общем случае они могут давать разные результаты.

3. Обращение к узлу `location`, являющемуся атрибутом элемента `library`:

```
/library/@location
```

Символ `@` используется для обозначения узлов атрибутов.

4. Обращение ко всем узлам `title`, являющимся атрибутами элементов `book`, которые имеют значение "Искусственный интеллект" и расположены в любом месте документа (см. рис. А.3):

```
//book[@title=."Искусственный интеллект"]
```

5. Обращение ко всем книгам с названием "Искусственный интеллект" (см. рис. А.4):

```
//book[@title="Искусственный интеллект"]
```

Условие в квадратных скобках называется *выражением фильтра* (*filter expression*). Фильтр ограничивает набор узлов, к которым осуществляется доступ. Обратите внимание на разницу между этим выражением и выражением в запросе 4. Здесь мы обращаемся к элементам `book`, у которых атрибут `title` удовлетворяет определенному условию. А в запросе 4 осуществлялся доступ к атрибутам `title` элементов `book`. Разница между этими запросами продемонстрирована на рис. А.3 и А.4.

6. Обращение к первому в XML-документе узлу элемента `author`:

```
//author[1]
```

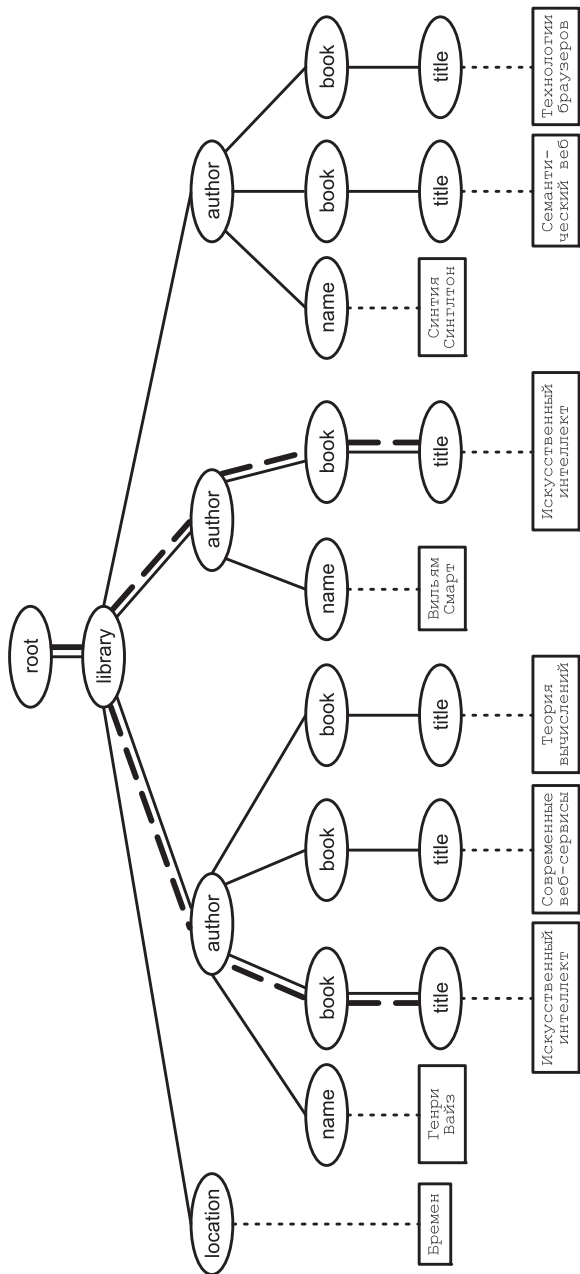


Рис. А.3 ❖ Дерево, представляющее запрос 4

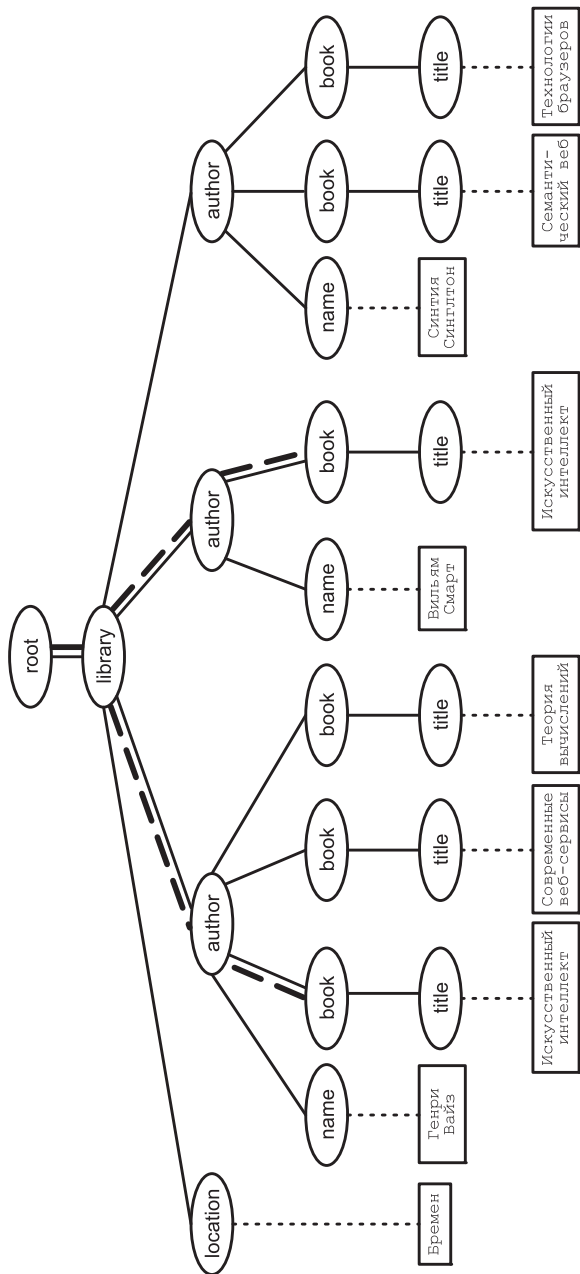


Рис. А.4 ❖ Дерево, представляющее запрос 5

7. Обращение к последнему узлу элемента `book`, вложенного в первый в XML-документе элемент `author`.

```
//author[1]/book[last()]
```

8. Обращение ко всем узлам элементов `book` без атрибута `title`:

```
//book[not (@title)]
```

Эти примеры приведены для того, чтобы дать читателю представление о выразительной силе выражений путей. В общем случае выражение пути состоит из последовательности шагов, разделенных косыми чертами. *Шаг* состоит из спецификатора оси, условия проверки узлов и необязательного предиката.

- Спецификатор оси определяет отношение в виде дерева между узлами, к которым осуществляется доступ, и узлом контекста. Примерами являются родительские узлы, узлы-предки, дочерние узлы (по умолчанию), одноуровневые узлы и узлы атрибутов. Символы `//` также являются спецификаторами оси; они означают узел-потомок или собственно узел контекста.
- *Условие проверки узлов* определяет узлы, к которым осуществляется доступ. Чаще всего они представляют собой имена элементов (для которых могут использоваться префиксы пространств имен), но могут быть и другого вида. Например, символ `*` используется для обращения ко всем узлам элементов, `comment()` – ко всем узлам комментариев и т. д.
- *Предикаты (или выражения фильтра)* являются необязательными и используются для уточнения множества узлов, к которым обеспечивается доступ. Например, выражение `[1]` позволяет выбрать первый узел, выражение `[position()=last()]` – последний узел, выражение `[position() mod 2 = 0]` – четные узлы и т. д.

В данном разделе был представлен только сокращенный синтаксис; полный синтаксис языка XPath сложнее. Список рекомендуемой литературы приводится в конце данного приложения.

A.5. Обработка

До сих пор мы ничего не говорили о способах отображения XML-документов. Однако это очень важно, так как, в отличие от HTML-документов, XML-документы не содержат информации о форматирова-

нии. Поэтому один и тот же XML-документ может быть представлен различными способами, если к нему применяются разные *таблицы стилей*. Несомненно, это является преимуществом. Например, рассмотрим XML-элемент:

```
<author>
  <name>Григорис Антониу </name>
  <affiliation>Бременский университет</affiliation>
  <email>ga@tzi.de</email>
</author>
```

Если используется некоторая таблица стилей, то данный документ может отображаться следующим образом:

Григорис Антониу

Бременский университет

ga@tzi.de

Если использовать иную таблицу стилей, то можно получить, например, следующее его отображение:

Григорис Антониу

Бременский университет

ga@tzi.de

Таблицы стилей могут быть написаны на различных языках, таких как CSS2 (каскадные таблицы стилей второго уровня). Кроме того, можно использовать расширяемый язык таблиц стилей XSL (eXtensible Stylesheet Language).

XSL включает в себя как язык преобразования (XSLT), так и язык форматирования. Каждый из них основан на языке XML. XSLT определяет правила, в соответствии с которыми исходный XML-документ преобразуется в другой XML-документ, HTML-документ или обычный текст. Полученный документ может использовать ту же DTD или схему, что и исходный документ, но может использовать и совершенно иной словарь.

Язык XSLT (XSL Transformation) может быть использован независимо от языка форматирования. Его способность передавать данные и метаданные из одного XML-представления в другое делает его самым ценным инструментом для разработки приложений на основе XML. В общем случае XSLT применяется для обмена данными между приложениями, использующими различные DTD или схемы. XSLT представляет собой инструмент, который может быть использован

для машинной обработки информации без какого-либо учета человекочитаемого отображения информации. Несмотря на этот факт, в дальнейшем мы будем использовать только XSLT для отображения XML-документов.

Один из способов определить представление XML-документа заключается в том, чтобы преобразовать его в HTML-документ. Приведем пример. Определим XSLT-документ, который будет применяться к рассмотренному выше элементу `author`:

```
<?xml version="1.0" encoding="UTF-16"?>
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/
Transform">

  <xsl:template match="/author">
    <html>
      <head><title>Автор< /title></head>
      <body bgcolor="white">
        <b><xsl:value-of select="name"/></b><br></br>
        <xsl:value-of select="affiliation"/><br></br>
        <i><xsl:value-of select="email"/></i>
      </body>
    </html>
  </xsl:template>
</xsl:stylesheet>
```

Применение данной таблицы стилей к предыдущему XML-документу позволит получить следующий HTML-документ (который и определяет представление XML-документа):

```
<html>
<head><title>Автор< /title></head>
<body bgcolor="white">
  <b>Григорис Антониу</b><br>
  Бременский университет<br>
  <i>ga@tzi.de</i>
</body>
</html>
```

Сделаем несколько замечаний. XSLT-документы являются XML-документами. Таким образом, язык XSLT основан на языке XML (то есть это приложение языка XML). XSLT-документ определяет *шаблон*; в нашем примере это HTML-документ с некоторыми полями для заполнения, куда будет вставлена информация (см. рис. А.5).

```

<html>
<head><title>Автор</title></head>
<body bgcolor="white">
  <b>...</b><br>
  ...<br>
  <i>...</i>
</body>
</html>

```

Рис. А.5 ❖ Шаблон

В рассмотренном XSLT-документе `xsl:value-of` извлекает значение указанного элемента и копирует его в итоговый документ. То есть таким образом информация вставляется в шаблон.

Теперь предположим, что у нас есть XML-документ с подробной информацией о нескольких авторах. Очевидно, что создавать отдельный шаблон для каждого элемента `author` нецелесообразно. В таких случаях определяется специальный шаблон для элементов `author`, который используется в основном шаблоне. Проиллюстрируем этот подход на примере следующего исходного документа:

```

<authors>
  <author>
    <name>Григорис Антониу</name>
    <affiliation>Бременский университет</affiliation>
    <email>ga@tzi.de</email>
  </author>
  <author>
    <name>Дэвид Биллингтон</name>
    <affiliation>Университет Гриффита</affiliation>
    <email>david@gu.edu.net</email>
  </author>
</authors>

```

Определим XSLT-документ:

```

<?xml version="1.0" encoding="UTF-16"?>
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/
Transform">

  <xsl:template match="/">
    <html>
      <head><title>Авторы</title></head>
      <body bgcolor="white">
        <xsl:apply-templates select="authors"/>
        <!--Применение шаблонов для дочерних узлов AUTHORS -->

```



```

    </body>
  </html>
</xsl:template>

<xsl:template match="authors">
  <xsl:apply-templates select="author"/>
</xsl:template>

<xsl:template match="author">
  <h2><xsl:value-of select="name"/></h2>
  Место работы:<xsl:value-of select="affiliation"/><br>
  Email: <xsl:value-of select="email"/>
  <p>
</xsl:template>
</xsl:stylesheet>

```

Полученный документ будет иметь следующий вид:

```

<html>
<head><title>Авторы</title></head>
<body bgcolor="white">
  <h2>Григорис Антониу</h2>
  Место работы: Бременский университет<br>
  Email: ga@tzi.de
  <p>
  <h2>Дэвид Биллингтон</h2>
  Место работы: Университет Гриффита <br>
  Email: david@gu.edu.net
  <p>
</body>
</html>

```

Элемент `xsl:apply-templates` использует все дочерние элементы узла контекста с выражением пути, указанным как значение атрибута `select`. Например, если текущий шаблон применяется к `/` (то есть если текущий узел контекста является корнем документа), то элемент `xsl:apply-templates` применяется к корневому элементу, которым в данном примере является элемент `authors` (напомним, что `/` обозначает корень документа, а корень документа расположен выше корневого элемента). Если же текущий узел – это элемент `authors`, то элемент `xsl:apply-templates select="author"` применяет шаблон элемента `author` ко всем узлам типа `author`, являющимся дочерними для узла `authors`.

Рекомендуется определять шаблон для каждого типа элемента в документе. То есть элемент `xsl:apply-templates` следует определить, даже если не предполагается специальная обработка определенных elemen-

тов, как в нашем примере. Таким образом, мы рассматриваем все пути от корня к листьям дерева, и все шаблоны действительно применяются.

Теперь рассмотрим атрибуты. Предположим, что необходимо с помощью XSLT обработать следующий элемент:

```
<person firstname="Петр" lastname="Иванов"/>
```

Пусть надо выполнить самую простую из возможных задач – преобразование элемента самого к себе. Может возникнуть соблазн написать следующий шаблон:

```
<xsl:template match="person">
  <person
    firstname="<xsl:value-of select="@firstname">"
    lastname="<xsl:value-of select="@lastname">" />
</xsl:template>
```

Однако этот XML-документ не является корректным, так как не допускается указание тегов в качестве значений атрибутов. Но он демонстрирует намерение добавить значения атрибутов в шаблон. В XSLT-шаблонах данные, заключенные в фигурные скобки, заменяют элемент `xsl:value-of`. Правильный способ определения шаблона для этого примера приведен ниже:

```
<xsl:template match="person">
  <person
    firstname="{@firstname}"
    lastname="{@lastname}" />
</xsl:template>
```

В заключение приведем пример преобразования одного XML-документа в другой без определения способа отображения. В качестве исходного документа используем все тот же документ `authors` и определим для него XSLT-документ следующим образом:

```
<?xml version="1.0" encoding="UTF-16"?>
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/
Transform">

  <xsl:template match="/">
    <authors>
      <xsl:apply-templates select="authors"/>
    </authors>
  </xsl:template>

  <xsl:template match="authors">
    <xsl:apply-templates select="author"/>
```

```

</xsl:template>
<xsl:template match="author">
  <author>
    <name><xsl:value-of select="name"/></name>
    <contact>
      <institute>
        <xsl:value-of select="affiliation"/>
      </institute>
      <email><xsl:value-of select="email"/></email>
    </contact>
  </author>
</xsl:template>
</xsl:stylesheet>

```

На рис. А.6 изображено преобразование дерева исходного документа в дерево конечного документа с помощью приведенного выше XSLT-документа.

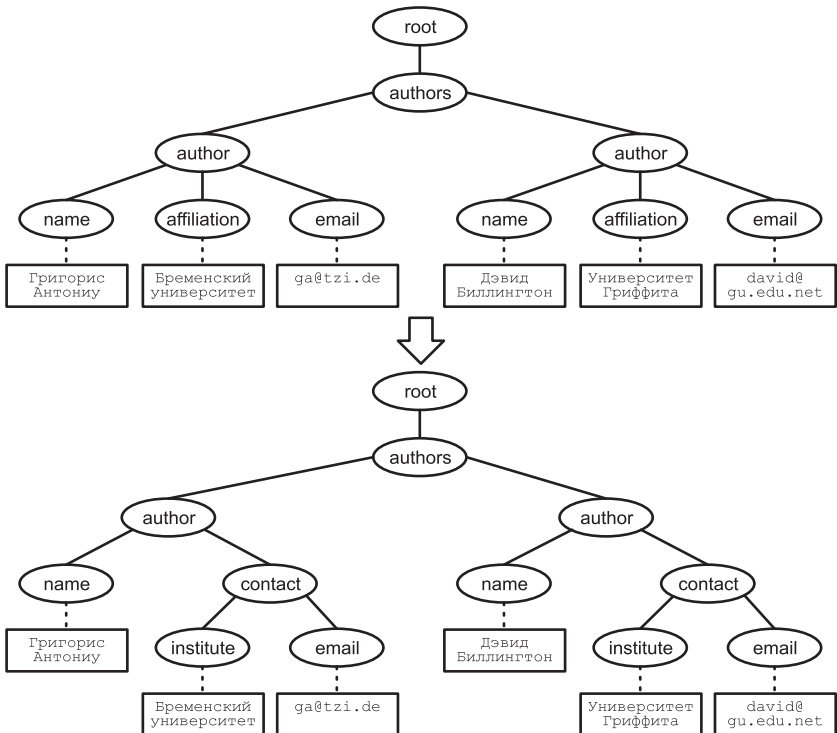


Рис. А.6 ❖ XSLT-преобразование дерева документа

Предметный указатель

Символы

#PCDATA, 210

А

Агрегатная функция, 80

Аксиома, 59, 96

Аксиоматическая семантика, 59

Аннотация, 100

В

Веб данных, 14

Восходящая частичная
согласованность, 27

Выражение, 96

пути, 224

фильтра, 226

Г

Граф, 36

именованный, 38

по умолчанию, 44

Графовый шаблон, 72

Д

Дескрипционные логики, 88, 94

Диапазон, 103

Домен, 103

И

Имя уточненное, 41

Индивид, 96

Искусственный интеллект, 26

К

Каламбур, 106, 107

Класс, 21, 47, 97

дополнение, 108

перечисление, 109

Классы

непересекающееся объединение, 108

непересекающиеся, 107

объединение, 108

эквивалентные, 106

Ключ, 114

Константа, 128

Концепт, 21

Корень дерева, 208, 209

Корневой элемент, 209

Корректный XML-документ, 207

Л

Литерал, 35

Логика, 23, 122

первого порядка, 122

предикатов, 59, 122

Хорна, 123, 132

Хорна без функциональных
символов, 129

Логическая программа, 123, 130

Логический вывод, 92

М

Машинное обучение, 183

Метаданные, 18, 20

Метамоделирование, 107

Модель данных, 32

независимая от предметной
области, 33

Модель логики предикатов, 131

Н

Надкласс, 48

Наследование, 48

Нисходящая совместимость, 27

О

Объект, 35

Объектная модель документа, 32

Объяснения, 25

Ограничение, 96

кардинальности, 111

на диапазон значений и типы
данных, 113

на значения, 111

самоограничение, 114

существования, 110

универсальное, 110

Омоним, 35, 188

Онтологии

выравнивание, 187

интеграция, 187

конвертация в базы данных, 189

отображение, 187

Онтология, 20, 89, 98

верхнего уровня, 181

выразительность, 90

импорт, 99
ручная разработка, 176
Отношение класс–подкласс, 21

П

Переменная, 128
Перечисление, 107
Подкласс, 48
Подсвойство, 50
Правило, 129
 голова, 129
 дедуктивное, 124
 конкурирующие, 146
 монотонное, 125, 127, 128
 немонотонное, 125
 отменяемое, 145, 147
 посылки, 129
 приоритеты, 145
 реактивное, 124
 тело, 129
Правительственные данные, 168
Предикат, 39, 128
Предположение об открытости мира, 78
Представление знаний, 16, 122
Принцип «Следуй за своим носом», 85
Принципы проектирования, 14
Программа отменяемая, 147
Программа дескрипционной логики, 117
Пространство имен, 28, 41, 98, 222
Профили OWL2, 116
Пути свойств, 75

Р

Разрешимость, 93
Реификация, 38, 53
Реляционные базы данных, 189
Репозитории онтологий, 193
Ресурс, 35

С

Свойства
 непересекающиеся, 104
 цепочки, 104
 эквивалентные, 104
Свойство, 35
 Bottom, 101
 Top, 101
 аннотации, 100
 асимметричное, 102
 диапазон, 48
 домен, 48
 значение, 35
 иррефлексивное, 103

 композиционное, 101, 105
 обратное, 104
 обратно-функциональное, 102
 объектное, 99
 рефлексивное, 103
 симметричное, 102
 типа данных, 99
 транзитивное, 101
 функциональное, 100, 102
Связанные данные, 37, 194
 модель 5 звезд, 168
Связка, 73
Семантика, 32, 59
 формальная, 89
Семантическая совместимость, 22
Семантический веб, 14
Синтаксис, 32
 OWL/XML, 97
 манчестерский, 98
 функциональный, 97
Система вывода, 64
Системы доказательства, 123
Системы правил, 123
Словарь, 33
Слой, 26
 доверия, 29
 доказательства, 29
 логики, 29
Стандарты, 27
Субъект, 35

Т

Таблица стилей, 230
Типы атрибутов, 212, 217
Типы данных, 40, 217
 именованные, 113
 ограничение, 219
 расширение, 218
Типы элементов, 216
Триплет, 33, 35

У

Условия необходимые
и достаточные, 110
Утверждение, 33, 35, 96, 115
 об идентичности индивидов, 115
 о классах и свойствах, 115
 отрицательные, 116

Ф

Факт, 129
Функциональный символ, 129

Х

Хранилища онтологий, 193
Хранилище триплетов, 71

Ц

Цель, 130

Э

Экземпляр класса, 47

Я

Язык онтологий, 29, 49, 89

А

ААТ, 180
Art and Architecture
Thesaurus, 180

В

BBC Artists, 162
BBC World Cup, 164
BioPortal, 182

С

CDATA, 212
Core RIF, 137
CSS2, 230
Сус, 181

Д

data.gov, 168
Datalog, 151
DLP, 117
DTD, 210

Е

eXtensible Markup Language, 19

Г

GoodRelations, 158

И

Iconclass, 180
ID, 212
IDREF, 212
IDREFS, 212

Л

Large Knowledge Collidor, 204
Linked Data Cloud, 18
Linked Open Data, 18
LOD, 18

Н

New York Times, 170

О

On-To-Knowledge, 194
Ontology Alignment Evaluation
Initiative, 189
OpenCalais, 172
Open Directory, 181
OWL, 88
OWL2, 88
OWL2 DL, 94
OWL2 EL, 117
OWL2 Full, 94
OWL2 QL, 117
OWL2 RL, 117

Р

Prefix.cc, 183
Product Ontology, 160

Р

rdf:
Property, 52
Statment, 52
type, 52
RDF, 34
модель данных, 34
синтаксис, 34, 39
RDFa, 45, 194
rdfs:
Class, 52
comment, 54
Datatype, 52
domain, 53
isDefinedBy, 54
label, 54
Literal, 52
range, 53
Resource, 52
seeAlso, 54
subClassOf, 52
subPropertyOf, 52
RDFS, 34, 47, 51
RDF Schema, 47, 51
RDF/XML, 44
RIF, 125, 137
Basic Logic Dialect, 137
FLD, 137
Production Rule Dialect, 137
диалект, 137
RuleML, 151

S

Schema.org, 173
Sig.ma, 171
Sindice, 171
SLD-резолюция, 134
SPARQL, 70
 точка доступа, 71, 194
SPARQL Update, 83
SPIN, 143
Standard Upperlevel Ontology, 181
SUO, 181
SWRL, 142

T

TGN, 180
Thesaurus of Geographic Names, 180
Turtle, 39

U

ULAN, 180
UMLS, 181

Unified Medical Language System, 181
Uniform Resource Identifier, 15
Uniform Resource Locator, 34
Union List of Artist Names, 180
URI, 15
 разыменование, 35
URL, 34

W

WordNet, 182

X

XML, 19, 205
XML-атрибут, 206
XML-декларация, 205
XML-сущность, 213
XML-схема, 215
XML-элемент, 206
XPath, 224
XSLT, 230
XSLT-шаблон, 231

Книги издательства «ДМК Пресс» можно заказать
в торгово-издательском холдинге «Планета Альянс» наложенным платежом,
выслав открытку или письмо по почтовому адресу:

115487, г. Москва, 2-й Нагатинский пр-д, д. 6А.

При оформлении заказа следует указать адрес (полностью),

по которому должны быть высланы книги;

фамилию, имя и отчество получателя.

Желательно также указать свой телефон и электронный адрес.

Эти книги вы можете заказать и в интернет-магазине: **www.aliants-kniga.ru**.

Оптовые закупки: тел. **(499) 782-38-89**.

Электронный адрес: **books@aliants-kniga.ru**.

Григорис Антониоу, Паул Грос,
Фрэнк ван Хармелен, Ринке Хоекстра

Семантический веб. Учебник **Третье издание**

Главный редактор *Мовчан Д. А.*
dmkpress@gmail.com

Корректор *Синяева Г. И.*

Верстка *Чаннова А. А.*

Дизайн обложки *Мовчан А. Г.*

Формат 70×100 1/16 .

Гарнитура «Петербург». Печать офсетная.

Усл. печ. л. 16,5. Тираж 200 экз.

Веб-сайт издательства: www.dmk.ru